

spintent

SPANISH PARSE INTENTS

V0.99 2026-09-29*

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1	Descripción	1	3.3	El comando <code>\spmoney</code>	6
1.1	Carga del paquete	1	3.3.1	Llaves para <code>\spmoney</code>	7
1.2	El paquete <code>babel</code>	2	3.3.2	Monedas y alias para <code>\spmoney</code>	7
1.3	Fuentes tipográficas	2	3.4	El comando <code>\spshort</code>	8
1.4	El paquete <code>hyperref</code>	2	3.4.1	Llaves para <code>\spshort</code>	8
2	El comando <code>\setspintent</code>	3	3.4.2	Argumentos para <code>\spshort</code>	8
3	The big four	3	4	Utilidades generales	10
3.1	El comando <code>\spnum</code>	3	5	Utilidades para símbolos	11
3.1.1	Llaves para <code>\spnum</code>	3	6	Utilidades para números	13
3.2	El comando <code>\spunit</code>	4	7	Utilidades para geometría	17
3.2.1	Llaves para <code>\spunit</code>	4	8	Referencias	33
3.2.2	Argumentos para <code>\spunit</code>	4	9	Change history	34
3.2.3	El comando <code>\spnewunit</code>	6	10	Índice de la Documentación	35
3.2.4	El comando <code>\spunitalias</code>	6	11	Implementation	38
			12	Índice de la Implementación	157

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con MathCAT[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de *The LaTeX Tagged PDF Project* y escucharlo con NVDA/MathCAT para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/MathCAT.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de MathML. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

*Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-09-29.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/luatex/spintent/
spintent.lua  ⇒ TDS:tex/luatex/spintent/
spintent.pdf  ⇒ TDS:doc/luatex/spintent/
README.md    ⇒ TDS:doc/luatex/spintent/
spintent.dtx  ⇒ TDS:source/luatex/spintent/
spintent.ins  ⇒ TDS:source/luatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves («*keys*») aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{comando}{key-uno = valor, key-dos = valor, key-tres = valor, ...}`

El comando `\setspintent` establece las llaves (*keys*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*keys*) definidas en el *argumento opcional* [*key = val*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*key = val*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*key = val*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como (*valor prohibido*) no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo (*valor prohibido*) se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{<número>}` `\spnum{<número,número>}` `\spnum{<número.número>}` `\spnum{<número;número>}`
`\spnum[key = val]{<número,número;número>}` `\spnum[key = val]{<número,número;número e exponente>}`
`\spnum[key = val]{<número.número;número>}` `\spnum[key = val]{<número.número;número e exponente>}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<número>}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma '; '.

Si no se proporciona el *argumento opcional* [*key = val*] y se ejecuta `\$ \spnum{+23} \$` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\$ \spnum{3,14} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\$ \spnum{3,1;4} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {<número>} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\$ \spnum{3,14e3} \$` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {<argumento>}. El {<argumento>} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {<true | false>}` default: true

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en `true` se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en `false`, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\$ \spnum{100000} \$` imprimirá '100 000' en la salida PDF y `\$ \spnum[cifras-sep=false]{10\,00\,00} \$` imprimirá '10 00 00' en la salida PDF.

`read-sign = {<terse | clear>}` default: terse

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en `terse`, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en `clear`, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\$ \spnum{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\$ \spnum[read-sign=clear]{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {<coma | punto>}` default: coma

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ';' de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ',' en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá '.' en la

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

salida PDF y NVDA/MathCAT verbalizará «punto». Por ejemplo $\$ \backslash \text{spnum}\{3;4\}$ imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma cuatro periódico» y $\$ \backslash \text{spnum}[\text{read-period-sep=punto}]\{3;4\}$ imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «tres punto cuatro periódico»

- Esta llave es necesaria SOLO cuando se utilizan «decimales periódicos puros», es decir, cuando no se ha incluido en el $\{ \langle \text{argumento} \rangle \}$ una coma ‘,’ o un punto ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = { \langle times \rangle | \langle cdot \rangle }`

default: `cdot`

Establece el *símbolo* que se utilizará para imprimir la «notación científica» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘×’ en la salida PDF y NVDA/MathCAT verbalizará «por»; si se establece en `cdot` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «por». Por ejemplo $\$ \backslash \text{spnum}\{3,14\text{e}3\}$ imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma catorce por diez al cubo» y $\$ \backslash \text{spnum}[\text{notation-sym=times}]\{3,14\text{e}3\}$ imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «tres coma catorce por diez al cubo».

- Decir «notación científica» aquí es un poco exagerado, en realidad es «multiplicación por potencias de 10», no se verifica si la entrada cumple o no con la condición de estar escrita en «notación científica».

3.2 El comando `\spunit`

<code>\spunit</code>	<code>\spunit\{ \langle unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad * unidad \rangle \}</code>
	<code>\spunit[\langle key = val \rangle]\{ \langle unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad * unidad / unidad \rangle \}</code>
	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad \rangle \}</code>	<code>\spunit[\langle key = val \rangle]\{ \langle número unidad * unidad / unidad * unidad \rangle \}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle unidad \rangle \}$, el cual puede contener una «parte numérica» opcional la cual se procesa internamente por el comando `\spnum` al inicio del $\{ \langle argumento \rangle \}$ seguido de una $\{ \langle unidad \rangle \}$ o un $\{ \langle alias \rangle \}$ valido descritos en §3.2.2 o una $\{ \langle unidad \rangle \}$ creada por el comando `\spnewunit` (§3.2.3) o un $\{ \langle alias \rangle \}$ creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$ y se ejecuta $\$ \backslash \text{spunit}\{23 \text{ m}\}$ se imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros».

La multiplicación entre las $\{ \langle unidades \rangle \}$ se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^ $\{ \langle exponente \rangle \}$ ’. Por ejemplo si se ejecuta $\$ \backslash \text{spunit}\{23 \text{ m} * \text{s}^{\{2\}}\}$ se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros segundos al cuadrado».

Se pueden usar *fracciones lineales* con las $\{ \langle unidades \rangle \}$ utilizando una «única barra» ‘/’ para la separación entre las $\{ \langle unidades \rangle \}$ del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta $\$ \backslash \text{spunit}\{23 \text{ m} / \text{s}^{\{2\}}\}$ se imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «veintitrés metros por segundos al cuadrado».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El $\{ \langle argumento \rangle \}$ puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm\{ \langle \#1 \rangle \}` o `\mathbf\{ \langle \#1 \rangle \}` por ejemplo, NO puede contener más de una «única barra» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para `\spunit`

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = { \langle true \rangle | \langle false \rangle }`

default: `false`

Establece si se *imprime* `\cdot` ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de $\{ \langle unidades \rangle \}$. Si se establece en `true`, se imprimirá ‘·’ entre las $\{ \langle unidades \rangle \}$ en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las $\{ \langle unidades \rangle \}$ en la salida PDF.

`read-cdot = { \langle true \rangle | \langle false \rangle }`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de $\{ \langle unidades \rangle \}$. Si se establece en `true`, NVDA/MathCAT verbalizará «por» entre las $\{ \langle unidades \rangle \}$; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las $\{ \langle unidades \rangle \}$.

`read-slash = { \langle por \rangle | \langle partido-por \rangle }`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «barra» ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «partido por».

3.2.2 Argumentos para `\spunit`

Resumen de $\{ \langle unidades \rangle \}$ y $\{ \langle alias \rangle \}$ soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»

Unidades directas (sin alias):

L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{\nueva unidad}\{verbalización NVDA}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\langle \textit{nueva unidad} \rangle$ que establece lo que se imprimirá en la salida PDF y $\langle \textit{verbalización NVDA} \rangle$ que establece como NVDA/MathCAT verbalizará la $\langle \textit{nueva unidad} \rangle$.

La `{\nueva unidad}` no debe estar registrada como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2. Si la `{\nueva unidad}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\langle\text{nueva unidad}\rangle$ es «global» y se almacena en el módulo `spintent.lua` como una $\langle\text{unidad}\rangle$ válida para pasar como $\langle\text{argumento}\rangle$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\langle\text{nueva unidad}\rangle$ ‘px’ y cuando se utilice `\spunit{100 px}` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias \spunitalias{⟨nuevo alias⟩}{⟨expresión de unidades⟩}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \text{nuevo alias} \rangle\}$ que establece es el nuevo $\{\langle \text{alias} \rangle\}$ de $\{\langle \text{argumento} \rangle\}$ para `\spunit` y $\{\langle \text{expresión de unidades} \rangle\}$ que puede contener multiplicación o división de $\{\langle \text{unidades} \rangle\}$ ya registradas que se imprimirán en la salida PDF.

El `{\nuevo alias}` no debe estar registrado como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2 o estar definido como `{\unidad}` por medio del comando `\spnewunit`. Si el `{\nuevo alias}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{\nuevo alias}` es «global» y se almacena en el módulo `spintent`. Lua como una `{\alias}` valido para pasar como `{\argumento}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^{\textstyle 3}}/s}` creará el `{\alias}` ‘caudal’ y cuando se utilice `\$ \spunit{50 caudal} \$` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

<u>\spmoney</u>	\spmoney{ <i>(cifra)</i> }	\spmoney[<i>(key = val)</i>]{ <i>(cifra)</i> }
	\spmoney{ <i>(cifra símbolo moneda)</i> }	\spmoney[<i>(key = val)</i>]{ <i>(cifra símbolo moneda)</i> }
	\spmoney{ <i>(cifra alias moneda)</i> }	\spmoney[<i>(key = val)</i>]{ <i>(cifra alias moneda)</i> }

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \text{cifra} \rangle \}$ que contiene la «cifra numérica» positiva o negativa la cual se procesa internamente por el comando `\spnum` acompañada de «símbolo moneda» o «alias moneda» opcionales descritos en §3.3.2.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\$500` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase «*símbolo moneda*» o «*alias moneda*» en el argumento *obligatorio* donde tienen prioridad por sobre los valores por defecto. Por ejemplo: `\spmoney{500 dolares}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos dolares*» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estándar ISO. Por ejemplo: `$\spmoney{500 CLP}` imprimirá ‘CLP500’ en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*».

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país, pero, NO se modifica la salida PDF imprimiendo el «*símbolo de moneda*» correspondiente. Por ejemplo: `\$ \spmoney{500 clp} \$` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo moneda*» como '\$', '€', '£' o un «*alias moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del «*símbolo moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un «*símbolo moneda*» poco habitual que esté soportada por `spintent`. El `{(argumento)}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. La coma ',' o el punto '.' como *separador decimal* solo pueden aparecer una «*única vez*». Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {(moneda | alias)}` default: peso

Establece el *símbolo moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{(argumento)}` al comando `\spmoney`. El «*símbolo moneda*» o el «*alias moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {(true | false)}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares} \$` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cien dólares con cincuenta centavos*».

`dolar-math = {(true | false)}` default: true

Establece si el *símbolo '\$'* se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir '\$' en la salida PDF; si se establece en `false`, se usará la la fuente tipográfica definida para el *modo texto* para imprimir '\$' en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los «*símbolos moneda*» y «*alias moneda*» soportados en el `{(argumento)}` de `\spmoney`.

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	« <i>peso / pesos</i> »
€	euro, euros	euro	€	« <i>euro / euros</i> »
£	libra, libras	libra	£	« <i>libra / libras</i> »
¥	yen, yenes	yen	¥	« <i>yen / yenes</i> »
¥	yuan, yuanes	yuan	¥	« <i>yuan / yuanes</i> »
₩	won, wons	won	₩	« <i>won / wons</i> »
₪	shekel, shekels, sequel, sequeles	séquel	₪	« <i>séquel / séqueles</i> »
₹	rupia, rupias	rupia	₹	« <i>rupia / rupias</i> »
₲	rublo, rublos	rublo	₲	« <i>rublo / rublos</i> »
₺	lira, liras	lira	₺	« <i>lira / liras</i> »
₴	grivna, grivnas	grivna	₴	« <i>grivna / grivnas</i> »
¢	centavo, centavos	centavo	¢	« <i>centavo / centavos</i> »

Divisas en formato ISO (Mayúsculas)				
CLP	—	peso chileno	CLP	« <i>peso chileno / pesos chilenos</i> »
MXN	—	peso mexicano	MXN	« <i>peso mexicano / pesos mexicanos</i> »
USD	—	dólar	USD	« <i>dólar estadounidense / dólares estadounidenses</i> »
EUR	—	euro	EUR	« <i>euro / euros</i> »
GBP	—	libra	GBP	« <i>libra / libras</i> »
JPY	—	yen	JPY	« <i>yen / yenes</i> »
CNY	—	yuan	CNY	« <i>yuan / yuanes</i> »
KRW	—	won	KRW	« <i>won / wons</i> »
ILS	—	séquel	ILS	« <i>séquel / séqueles</i> »
INR	—	rupia	INR	« <i>rupia / rupias</i> »
RUB	—	rublo	RUB	« <i>rublo / rublos</i> »

Continúa en la siguiente página...

Cuadro 2 – Continuación de la página anterior

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»
Códigos ISO en minúsculas				
clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	dolar, dolares	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4 El comando \spshort

```
\spshort{<abreviaturas | acrónimos | números ordinales>}
\spshort[<key = val>]{<abreviaturas | acrónimos | números ordinales>}
\spshort[<read-arg={<verbalización NVDA>}]{<comandos LATEX>}
```

El comando `\spshort` trabaja *solo en modo texto* y recibe *un argumento obligatorio* del tipo `{<abreviaturas>}`, `{<acrónimos>}` o `{<números ordinales>}` descritos en §3.4.2.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spshort{dr.a}` se imprimirá ‘Dr.^a’ en la salida PDF y NVDA/MathCAT verbalizará «doctora»; si se ejecuta `\spshort{onu}` se imprimirá ‘ONU’ en la salida PDF y NVDA/MathCAT verbalizará «organización de las naciones unidas» y si se ejecuta `\spshort{mineduc}` se imprimirá ‘Mineduc’ en la salida PDF y NVDA/MathCAT verbalizará «ministerio de educación».

Para la escritura de `{<números ordinales>}` se puede ejecutar `\spshort{1.a}` que imprimirá ‘1.^a’ en la salida PDF y NVDA/MathCAT verbalizará «primera»; si se ejecuta `\spshort{1.o}` imprimirá ‘1.^o’ en la salida PDF y NVDA/MathCAT verbalizará «primero».

Para crear un nueva `{<abreviatura>}`, `{<acrónimo>}` o `{<número ordinal>}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener `{<comandos LATEX>}` *en modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de `{<números ordinales>}` se debe tener cuidado y no confundir la «letra o voladita» ‘o’ con el símbolo de grado ‘°’. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «letra voladita», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1.o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete `babel` en español compatible con *tagged PDF*. Es de las pocas utilidades que provee el paquete `spintent` que trabaja *solo en modo texto*. Internamente utiliza la llave `E` (*expansion text*) proveída por el paquete `tagpdf`[29].

3.4.1 Llaves para \spshort

```
read-arg = {<verbalización NVDA>} default: no usado
```

Establece la *verbalización del argumento* `{<comandos LATEX>}` de la nueva `{<abreviatura>}`, `{<acrónimo>}` o `{<número ordinal>}` para NVDA. El valor de esta llave *siempre* debe pasarse entre ‘{ }’. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva `{<abreviatura>}` que imprimirá ‘8.^{avo}’ en la salida PDF y NVDA verbalizará «octavo».

```
small-caps = {<true | false>} default: false
```

Establece si se usarán *versalitas* para imprimir `{<acrónimos>}` y para la «letra volada» en `{<abreviaturas>}` y `{<números ordinales>}` respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán *versalitas*. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^A’ en la salida PDF y NVDA verbalizará «doctora».

3.4.2 Argumentos para \spshort

El cuadro 3 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 4 detalla las siglas y acrónimos registrados en el módulo Lua **spintent.lua**.

Cuadro 4: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	ongs	ONG	«organización no gubernamental»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 5 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 5: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete `spintent` provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo {<siglo>} \spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el argumento obligatorio `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘XXI’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El argumento obligatorio `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete `spintent` que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete `tagpdf` en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {⟨ac⟩ | ⟨dc⟩ | none}
```

default: none

Establece si se *imprime* o no la abreviatura después de imprimir el argumento $\{\langle siglo \rangle\}$. Si se establece en *ac*, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xI a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en *dc*, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xI d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en *none*, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xI’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

```
\sptime \sptime{⟨horas⟩ : ⟨minutos⟩} \sptime{⟨horas⟩ : ⟨minutos⟩ : ⟨segundos⟩}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle horas \rangle : \langle minutos \rangle\}$ o $\{\langle horas \rangle : \langle minutos \rangle : \langle segundos \rangle\}$. Por ejemplo: `\sptime{23:45}` imprimirá en «modo texto» la salida PDF ‘23:45’ y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

```
\spdate \spdate{⟨DD/MM/YYYY⟩} \spdate{⟨YYYY/MM/DD⟩} \spdate{⟨YYYY-MM-DD⟩} \spdate{⟨DD-MM-YYYY⟩}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle DD/MM/YYYY \rangle\}$, $\{\langle DD-MM-YYYY \rangle\}$ o $\{\langle YYYY-MM-DD \rangle\}$. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en «modo texto» la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el argumento $\{\langle YYYY/MM/DD \rangle\}$, internamente se convertirá en $\{\langle DD/MM/YYYY \rangle\}$ y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete `spintent` provee cuatro comandos `\spnewread`, `\spread`, `\spdsym` y `\spmsym` para trabajar con «símbolos matemáticos» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «símbolo» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por `spintent` (que analizan y dan formato a la salida), estos no procesan los «símbolos matemáticos», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spnewread`

```
\spnewread \spnewread{⟨símbolo⟩}{⟨verbalización NVDA⟩}
```

El comando `\spnewread` trabaja en *modo texto* y recibe dos argumentos obligatorios: $\{\langle símbolo \rangle\}$ y $\{\langle verbalización NVDA \rangle\}$ los cuales se almacenan de forma interna para ser utilizados como como $\{\langle argumento \rangle\}$ en `\spread`. Solo se establece como NVDA/MathCAT verbalizará el $\{\langle símbolo \rangle\}$ al utilizarlo con `\spread` sin modificar la impresión de este.

El argumento $\{\langle símbolo \rangle\}$ no debe estar registrado. Si ya está registrado se devolverá un “error” y se detendrá la compilación del documento. La declaración es «global» y se almacena en el módulo `spintent.lua`. Por ejemplo: `\spnewread{\cong}{es-congruente-con}` registrará ‘\cong’ y cuando se utilice `\spread{\cong}` se imprimirá ‘≅’ en la salida PDF y NVDA/MathCAT verbalizará «es congruente con».

El comando `\spread`

```
\spread \spread{⟨símbolo⟩} \spread[⟨key = val⟩]{⟨símbolo⟩}
\spread[⟨read-arg={⟨verbalización NVDA⟩}⟩]{⟨comandos LATEX⟩}
```

El comando `\spread` trabaja en *modo matemático* y recibe el argumento obligatorio de la forma $\{\langle símbolo \rangle\}$ previamente registrado por el comando `\spnewread` o $\{\langle comandos L^A T_E X \rangle\}$ para ser usado junto a la llave *read-arg*.

Si el argumento es $\{\langle símbolo \rangle\}$, `\spread{\cong}` imprimirá ‘≅’ en la salida PDF y el argumento $\{\langle verbalización NVDA \rangle\}$ establecido al registrar $\{\langle símbolo \rangle\}$ con `\spnewread` se verbalizará en NVDA/MathCAT.

Si el argumento es $\{\langle comandos L^A T_E X \rangle\}$ y no se establece la llave *read-arg* se devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spread`

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Establece la *verbalización* del argumento $\{\langle\text{comandos L}\text{a}\text{T}\text{E}\text{X}\rangle\}$. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y sobrescribe la verbalización del $\{\langle\text{símbolo}\rangle\}$ si está registrado previamente. Por ejemplo: `\spread[read-arg={es-paralela-a}]{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela a».

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del argumento $\{\langle\text{símbolo}\rangle\}$ o $\{\langle\text{comandos L}\text{a}\text{T}\text{E}\text{X}\rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

- Al utilizar `silent-cmd` no es necesario que el argumento $\{\langle\text{símbolo}\rangle\}$ esté registrado o usar la llave `read-arg` con $\{\langle\text{comandos L}\text{a}\text{T}\text{E}\text{X}\rangle\}$. Dependiendo de que contenga $\{\langle\text{argumento}\rangle\}$ puede que `silent-cmd` no sea suficiente, en ese caso utilice `read-arg={_}`.

El comando `\spusep`

`\spusep` `\spusep{\{[ ( ) ] \}}` `\spusep[key = val]{\{[ ( ) ] \}}`

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\{[()] \}\}$, el cual debe ser «solo uno» de los paréntesis soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’ o ‘\’.

Si no se proporciona el *argumento opcional* $[key = val]$, `\spusep{}` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «abrir paréntesis» acorde a su propia heurística.

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por Lua $\text{\TeX}$ crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

`size = {⟨none | big | Big | bigg | Bigg⟩}`

default: *none*

Establece el *tamaño del paréntesis* pasado como $\{\langle\text{argumento}\rangle\}$ que se imprimirá en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *none*, se imprime el $\{\langle\text{argumento}\rangle\}$ en el tamaño por defecto del modo matemático en curso; si se establece en *big*, se usa `\bigl/\bigr` antes del $\{\langle\text{argumento}\rangle\}$; si se establece en *Big*, se usa `\Bigl/\Bigr` antes del $\{\langle\text{argumento}\rangle\}$; si se establece en *bigg*, se usa `\biggl/\biggr` antes del $\{\langle\text{argumento}\rangle\}$; y si se establece en *Bigg*, se usa `\Biggl/\Biggr` antes del $\{\langle\text{argumento}\rangle\}$.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* en NVDA/MathCAT la verbalización del $\{\langle\text{argumento}\rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spdsym`

`\spdsym` `\spdsym[key = val]`

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* $[key = val]$ mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* $[key = val]$, `\spdsym` imprimirá ‘:’ en la salida PDF y NVDA/MathCAT verbalizará «dividido».

Llaves para `\spdsym`

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será *verbalizado* por NVDA/MathCAT «antes» de *verbalizar* el $\{\langle\text{valor}\rangle\}$ establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`suffix = {⟨sufijo⟩}`

default: *no usado*

Establece el *sufijo* que será *verbalizado* por NVDA/MathCAT «después» de *verbalizar* el $\{\langle\text{valor}\rangle\}$ establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`set-sym = {⟨old-style | two-dots | slash⟩}`

default: *two-dots*

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en *old-style*, imprimirá ‘÷’; si se establece en *two-dots*, imprimirá ‘:’ y si se establece *slash*, imprimirá ‘/’.

`read-sym = {⟨dividido | dividido-por | dividido-entre⟩}`

default: *dividido*

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en *dividido*, NVDA/MathCAT verbalizará «dividido»; si se establece en *dividido-por*, NVDA/MathCAT verbalizará «dividido por» y si se establece en *dividido-entre*, NVDA/MathCAT verbalizará «dividido entre».

El comando `\spmsym`

<code>\spmsym</code>	<code>\spmsym</code>	<code>\spmsym[⟨key = val⟩]</code>
	El comando <code>\spmsym</code> trabaja en <i>modo matemático</i> y recibe el <i>argumento opcional</i> <code>[⟨key = val⟩]</code> mediante el cual se establece la «<i>forma</i>» en la que se imprimirá el <i>símbolo de multiplicación</i> y como se verbalizará en NVDA/MathCAT. Si no se proporciona el <i>argumento opcional</i> <code>[⟨key = val⟩]</code>, <code>\$\spmsym\$</code> imprimirá ‘$\cdot$’ en la salida PDF y NVDA/MathCAT verbalizará «<i>por</i>». Llaves para <code>\spmsym</code> <code>prefix = {⟨prefijo⟩}</code> default: <i>no usado</i> Establece el <i>prefijo</i> que será <i>verbalizado</i> por NVDA/MathCAT «<i>antes</i>» de <i>verbalizar</i> el <code>{⟨valor⟩}</code> establecido por la llave <code>read-sym</code>. El valor de esta llave <i>siempre</i> debe pasarse entre llaves ‘<code>{ }</code>’. <code>suffix = {⟨sufijo⟩}</code> default: <i>no usado</i> Establece el <i>sufijo</i> que será <i>verbalizado</i> por NVDA/MathCAT «<i>después</i>» de <i>verbalizar</i> el <code>{⟨valor⟩}</code> establecido por la llave <code>read-sym</code>. El valor de esta llave <i>siempre</i> debe pasarse entre llaves ‘<code>{ }</code>’. <code>set-sym = {⟨cross dot asterisk⟩}</code> default: <i>dot</i> Establece el <i>símbolo de multiplicación</i> que se imprimirá en la salida PDF. Si se establece en <code>cross</code>, imprimirá ‘$\times$’; si se establece en <code>dot</code>, imprimirá ‘$\cdot$’ y si se establece en <code>asterisk</code>, imprimirá ‘$*$’. <code>read-sym = {⟨por multiplicado-por veces⟩}</code> default: <i>por</i> Establece la <i>forma</i> con la cual NVDA/MathCAT verbalizará el <i>símbolo de multiplicación</i>. Si se establece en <code>por</code>, NVDA/MathCAT verbalizará «<i>por</i>»; si se establece en <code>multiplicado-por</code>, NVDA/MathCAT verbalizará «<i>multiplicado por</i>» y si se establece en <code>veces</code>, NVDA/MathCAT verbalizará «<i>veces</i>». <code>not-print = {⟨true false⟩}</code> default: <i>false</i> Deshabilita la <i>impresión</i> del <i>símbolo de multiplicación</i>. Cuando se establece esta llave, NVDA/MathCAT verbalizará el <i>símbolo de multiplicación</i> acorde al valor establecido por la llave <code>read-sym</code>, pero, no se imprimirá este en la salida PDF.	

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

<code>\spdiv</code>	<code>\spdiv{⟨dividendo⟩}{⟨divisor⟩}</code>	<code>\spdiv[⟨key = val⟩]{⟨dividendo⟩}{⟨divisor⟩}</code>
	El comando <code>\spdiv</code> trabaja en <i>modo matemático</i> y recibe dos <i>argumentos obligatorios</i> <code>{⟨dividendo⟩}</code> y <code>{⟨divisor⟩}</code>. Si no se proporciona el <i>argumento opcional</i> <code>[⟨key = val⟩]</code>, <code>\$\spdiv{20}{26}\$</code> imprimirá ‘20 : 26’ en la salida PDF y NVDA/MathCAT verbalizará «<i>veinte dividido veinte y seis</i>» Llaves para <code>\spdiv</code> El comando <code>\spdiv</code> posee las <i>meta-keys</i> <code>cifras-sep</code>, <code>read-sign</code>, <code>read-period-sep</code> y <code>notation-sym</code> pasadas al comando <code>\spnum</code> (§3.1.1) junto a las <i>meta-keys</i> <code>set-sym</code> y <code>read-sym</code> pasadas al comando <code>\spdsym</code> (pág. 12). <code>wrap-parens = {⟨true false⟩}</code> default: <i>false</i> Establece si se <i>imprimen</i> paréntesis redondos ‘<code>(...)</code>’ alrededor de los <i>argumentos</i> <code>{⟨dividendo⟩}</code> y <code>{⟨divisor⟩}</code> pasados a <code>\spdiv</code>. Si se establece en <code>true</code>, se imprimirán paréntesis redondos alrededor de ambos <i>argumentos</i>; si se establece en <code>false</code>, no se imprimirán los paréntesis. <code>read-parens = {⟨true false⟩}</code> default: <i>true</i> Establece si NVDA/MathCAT <i>verbalizará</i> los paréntesis redondos ‘<code>(...)</code>’ alrededor de los <i>argumentos</i> <code>{⟨dividendo⟩}</code> y <code>{⟨divisor⟩}</code> pasados a <code>\spdiv</code>. Si se establece en <code>true</code>, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en <code>false</code>, NVDA/MathCAT no los verbalizará.	

El comando `\spmul`

<code>\spmul</code>	<code>\spmul{⟨factor⟩}{⟨factor⟩}</code>	<code>\spmul[⟨key = val⟩]{⟨factor⟩}{⟨factor⟩}</code>
	El comando <code>\spmul</code> trabaja en <i>modo matemático</i> y recibe dos <i>argumentos obligatorios</i> <code>{⟨factor⟩}</code> y <code>{⟨factor⟩}</code>. Si no se proporciona el <i>argumento opcional</i> <code>[⟨key = val⟩]</code>, <code>\$\spmul{20}{26}\$</code> imprimirá ‘20 · 26’ en la salida PDF y NVDA/MathCAT verbalizará «<i>veinte por veinte y seis</i>».	

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens = {⟨true | false⟩}` default: false

Establece si se imprimen paréntesis redondos ‘(...)’ alrededor de los *argumentos* {⟨factor⟩} y {⟨factor⟩} pasados a `\spmul`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(...)’ alrededor de los *argumentos* {⟨factor⟩} y {⟨factor⟩} pasados a `\spmul`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

`\spnD` `\spnD` `\spnD(⟨número⟩)` `\spnD[⟨key = val⟩](⟨número⟩)` `\spnD(⟨letra⟩)` `\spnD[⟨key = val⟩](⟨letra⟩)`

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma `(⟨número⟩)` que se procesa internamente por `\spnum` o `(⟨letra⟩)` que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`] ni el *argumento opcional delimitado por paréntesis*, `$\spnD$` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de ene».

Si se usa el *argumento* `(⟨número⟩)`, `$\spnD(6)$` imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Si se usa el *argumento* `(⟨letra⟩)`, `$\spnD(m)$` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de eme».

Llaves para `\spnD`

`concept = {⟨true | false⟩}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «divisores de» y el `(⟨argumento⟩)`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `(⟨argumento⟩)`.

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «divisores de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}` default: false

Establece si se *imprime* el resultado de calcular los *divisores* del *argumento* `(⟨número⟩)`. Si se establece en `true`, `$\spnD[result=true](6)$` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis son uno, dos, tres y seis»; si se establece en `false`, no se imprimirá ni verbalizará el resultado de calcular los *divisores*.

`result-num = {⟨número natural⟩}` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`silent-cmd = {⟨true | false⟩}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» y del `(⟨argumento⟩)` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnM`

`\spnM` `\spnM` `\spnM(⟨número⟩)` `\spnM[⟨key = val⟩](⟨número⟩)` `\spnM(⟨letra⟩)` `\spnM[⟨key = val⟩](⟨letra⟩)`

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma `(⟨número⟩)` que se procesa internamente por `\spnum` o `(⟨letra⟩)` que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`] ni el *argumento opcional delimitado por paréntesis*, `$\spnM$` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de ene».

Si se usa el *argumento* `(⟨número⟩)`, `$\spnM(6)$` imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Si se usa el *argumento* `(⟨letra⟩)`, `$\spnM(m)$` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de eme».

Llaves para \spnM

<code>concept = {⟨true false⟩}</code>	default: <i>true</i>
Establece si se <i>silencia</i> la verbalización en NVDA/MathCAT del concepto « <i>múltiplos de</i> » al ejecutar el comando sin modificar la impresión de este. Si se establece en <i>true</i> , NVDA/MathCAT verbalizará « <i>múltiplos de</i> » y el (<i>argumento</i>); si se establece en <i>false</i> , NVDA/MathCAT solo verbalizará el (<i>argumento</i>).	
<code>prefix = {⟨prefijo⟩}</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será verbalizado por NVDA/MathCAT <i>antes</i> de verbalizar el concepto « <i>múltiplos de</i> ». El valor de esta llave <i>siempre</i> debe pasarse entre llaves '{ }'.	
<code>result = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>imprime</i> el resultado de calcular los <i>múltiplos</i> del argumento (<i>número</i>). Si se establece en <i>true</i> , <code>\spnM[result=true](6)</code> imprimirá 'M(6) = {6, 12, 18, 24, 30, ...}' en la salida PDF y NVDA/MathCAT verbalizará « <i>múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos</i> »; si se establece en <i>false</i> , no se imprimirá ni verbalizará el resultado de calcular los <i>múltiplos</i> .	
<code>result-num = {⟨número natural⟩}</code>	default: <i>5</i>
Establece la <i>cantidad</i> de números que se mostrarán en la salida PDF al establecer <code>result=true</code> . Si se establece en <i>2</i> , solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo <i>5</i> valores y luego se añadirá '...' en la salida PDF.	
<code>silent-cmd = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>silencia</i> la verbalización en NVDA/MathCAT del concepto « <i>multiplos de</i> » y del (<i>argumento</i>) pasado al comando sin modificar la impresión de este. Si se establece en <i>true</i> , NVDA/MathCAT NO verbalizará la salida del comando; si se establece en <i>false</i> , NVDA/MathCAT sí verbalizará la salida del comando.	

El comando \spgcd

`\spgcd` `\spgcd` `\spgcd(⟨número, número, ...⟩)` `\spgcd(⟨letra, letra, ...⟩)` `\spgcd[⟨key = val⟩](⟨número, número, ...⟩)`

El comando `\spgcd` trabaja en *modo matemático* y recibe el el *argumento opcional delimitado por paréntesis* de la forma (*número, número, ...*) que se procesa internamente por `\spnum` o (*letra, letra, ...*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spgcd` imprimirá 'mcd' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor*».

Si se usa el argumento (*número, número, ...*), `\spgcd(2, 3, 6)` imprimirá 'mcd(2, 3, 6)' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor entre dos, tres y seis*».

Si se usa el argumento (*letra, letra, ...*), `\spgcd(m, n)` imprimirá 'mcd(*m*, *n*)' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor entre eme y ene*».

Llaves para \spgcd

<code>read-cmd = {⟨terse clear⟩}</code>	default: <i>clear</i>
Establece la <i>forma</i> en la que NVDA/MathCAT verbalizará al comando <code>\spgcd</code> . Si se establece en <i>terse</i> , NVDA/MathCAT verbalizará « <i>m c d</i> »; si se establece en <i>clear</i> , NVDA/MathCAT verbalizará « <i>máximo común divisor</i> ».	
<code>upper-case = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se usarán <i>mayúsculas</i> o <i>minúsculas</i> en la salida PDF para el comando <code>\spgcd</code> . Si se establece en <i>true</i> , se imprimirá 'MCD' en la salida PDF; si se establece en <i>false</i> , se imprimirá 'mcd' en la salida PDF.	
<code>prefix = {⟨prefijo⟩}</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será verbalizado por NVDA/MathCAT « <i>antes</i> » de verbalizar el valor establecido por la llave <code>read-cmd</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves '{ }'.	
<code>result = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>imprime</i> el resultado de calcular el « <i>máximo común divisor</i> » del (<i>argumento</i>) pasado a <code>\spgcd</code> . Si se establece en <i>true</i> : <code>\spgcd[result=true](2, 3, 6)</code> imprimirá 'mcd(2, 3, 6) = 1' en la salida PDF y NVDA/MathCAT verbalizará « <i>máximo común divisor entre dos, tres y seis es igual a uno</i> »; si se establece en <i>false</i> no se imprimirá en la salida PDF el resultado de calcular el « <i>máximo común divisor</i> ».	
<code>silent-cmd = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>silencia</i> la verbalización en NVDA/MathCAT de <code>\spgcd(⟨argumento⟩)</code> cuando se establece <code>sample-arg=true</code> o cuando <code>\spgcd</code> se usa sin (<i>argumento</i>). Si se establece en <i>true</i> NVDA/MathCAT NO lo verbalizará; si se establece en <i>false</i> , NVDA/MathCAT Sí lo verbalizará.	

El comando `\spmc`

```
\spmc \spmc \spmc(⟨números separados por coma⟩) \spmc[key = val](⟨números separados por coma⟩)
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (`⟨números separados por coma⟩`), el cual debe ser una lista de *números naturales*, por ejemplo: `(2, 3, 6)`. Si se usa sin (`⟨argumento⟩`) `\spmc` imprimirá ‘mcm’ en la salida PDF, y NVDA/MathCAT verbalizará «*mínimo común múltiplo*» o «*m c m*», acorde al `{⟨valor⟩}` establecido por la llave `read-cmd`.

Si se usa con (`⟨argumento⟩`) `\spmc(2, 3, 6)` imprimirá ‘mcm(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis*» o «*m c m entre dos, tres y seis*» acorde al `{⟨valor⟩}` establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = {⟨true | false⟩}` default: false

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true` se imprimirá ‘MCM’ en la salida PDF; si se establece en `false` se imprimirá ‘mcm’ en la salida PDF.

`read-cmd = {⟨terse | clear⟩}` default: clear

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c m*»; si se establece en `clear`, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «*antes*» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}` default: false

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del (`⟨argumento⟩`) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «*mínimo común múltiplo*».

`sample-arg = {⟨true | false⟩}` default: false

Desactiva la *validación* interna del (`⟨argumento⟩`) pasado a `\spmc`, permitiendo usar ejemplos «*no numéricos*» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcm(m, n)’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre m y n*»; si se establece en `false` el (`⟨argumento⟩`) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = {⟨true | false⟩}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc(⟨argumento⟩)` cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (`⟨argumento⟩`). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnZ`

```
\spnZ \spnZ{⟨número entero⟩} \spnZ[key = val]{⟨número entero⟩}
```

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio `{⟨número entero⟩}` el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las *meta-keys* `cifras-sep` y `read-sign` pasadas al comando `\spnum` (§3.1.1).

`wrap-parens = {⟨true | false⟩}` default: false

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del argumento `{⟨número entero⟩}` pasado a `\spnZ`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor del `{⟨número entero⟩}`; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}` default: false

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(…)’ alrededor del *argumento* `{⟨número entero⟩}` pasado a `\spnZ`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(…)»; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` $\text{\spmQ}\langle\langle\text{entero}\rangle\rangle\langle\langle\text{numerador}\rangle\rangle\langle\langle\text{denominador}\rangle\rangle$ $\text{\spmQ}[\langle\text{key} = \text{val}\rangle]\langle\langle\text{entero}\rangle\rangle\langle\langle\text{numerador}\rangle\rangle\langle\langle\text{denominador}\rangle\rangle$

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: $\langle\langle\text{número entero}\rangle\rangle$, $\langle\langle\text{numerador}\rangle\rangle$ y $\langle\langle\text{denominador}\rangle\rangle$, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* $[\langle\text{key} = \text{val}\rangle]$, $\text{\spmQ}\{2\}\{5\}\{7\}$ imprimirá $2\frac{5}{7}$ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de `spintent`. Por ejemplo, la entrada $\text{\$3}\text{\frac{5}{7}}\text{\$}$ con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word` = $\{\langle\text{entero} \mid \text{enteros} \mid \text{y}\rangle\}$ default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en *entero*, $\text{\$}\text{\spmQ}[\text{join-word}=\text{entero}]\{1\}\{5\}\{7\}\text{\$}$ imprimirá $1\frac{5}{7}$ en la salida PDF y NVDA/MathCAT verbalizará «*un entero cinco séptimos*»; si se establece en *enteros*, $\text{\$}\text{\spmQ}[\text{join-word}=\text{enteros}]\{2\}\{5\}\{7\}\text{\$}$ imprimirá $2\frac{5}{7}$ en la salida PDF y NVDA/MathCAT verbalizará «*dos enteros cinco séptimos*»; si se establece en *y*, $\text{\$}\text{\spmQ}[\text{join-word}=\text{y}]\{2\}\{5\}\{7\}\text{\$}$ imprimirá $2\frac{5}{7}$ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

`use-spnZ` = $\{\langle\text{true} \mid \text{false}\rangle\}$ default: *true*

Establece si el $\langle\langle\text{numerador}\rangle\rangle$ y el $\langle\langle\text{denominador}\rangle\rangle$ que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en *true*, $\langle\langle\text{numerador}\rangle\rangle$ y $\langle\langle\text{denominador}\rangle\rangle$ se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en *false*, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`print-dfrac` = $\{\langle\text{true} \mid \text{false}\rangle\}$ default: *false*

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en *true*, al usar $\text{\$}\text{\spmQ}[\text{print-dfrac}=\text{true}]\{2\}\{5\}\{7\}\text{\$}$ imprimirá la fracción en tamaño *display* $2\frac{5}{7}$ en la salida PDF; si se establece en *false*, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens` = $\{\langle\text{true} \mid \text{false}\rangle\}$ default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en *true*, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = $\{\langle\text{true} \mid \text{false}\rangle\}$ default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

7 Utilidades para geometría

El paquete `spintent` provee comandos para trabajar con *geometría* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y reciben como $\langle\langle\text{argumento}\rangle\rangle$ base una *única* letra (mayúscula en el caso de puntos) con un modificador del tipo subíndice/prima, un número o comando.

Algunos comandos reciben $\langle\langle\text{argumento}\rangle\rangle$ único (modo “*single*”) o más de uno (modo “*batch*”) donde opcionalmente se pueden separar por comas ‘,’.

- Las primas se debe pasar de forma directa ‘*’*’ y los subíndices siempre deben escribirse usando llaves ‘*A_{...}*’ y nunca sin estas ‘*A_n*’ que funciona solo por coincidencia. .

El comando `\spcoord`

`\spcoord` $\text{\spcoord}\langle\langle\text{abscisa}, \text{ordenada}\rangle\rangle$ $\text{\spcoord}[\langle\text{key} = \text{val}\rangle]\langle\langle\text{abscisa}, \text{ordenada}\rangle\rangle$
 $\text{\spcoord}\langle\langle\text{abscisa}; \text{ordenada}\rangle\rangle$ $\text{\spcoord}[\langle\text{key} = \text{val}\rangle]\langle\langle\text{abscisa}; \text{ordenada}\rangle\rangle$

El comando `\spcoord` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\langle\langle\text{abscisa}, \text{ordenada}\rangle\rangle$ o $\langle\langle\text{abscisa}; \text{ordenada}\rangle\rangle$ el cual se procesa internamente bajo el comando `\spnum` cuando es numérico no fraccionario.

Si no se proporciona el *argumento opcional* $[\langle\text{key} = \text{val}\rangle]$, $\text{\$}\text{\spcoord}\{-3, 4\}\text{\$}$ imprimirá ‘(-3, 4)’ en la salida PDF y NVDA/MathCAT verbalizará «*menos tres coma cuatro*»; $\text{\$}\text{\spcoord}\{x_{\{1\}}, y_{\{1\}}\}\text{\$}$ imprimirá ‘(x₁, y₁)’ y verbalizará «*exis subíndice uno y subíndice uno*».

- El comando `\spcoord` solo acepta *un separador* ‘;’ o ‘,’ para la abscisa y la ordenada cuando son valores numéricos no fraccionarios.

Llaves para `\spcoord`

`silent-cmd` = { `<true>` | `<false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoint`

<code>\spPoint</code>	<code>\spPoint{<punto>}</code>	<code>\spPoint[<key = val>]{<punto>}</code>
	<code>\spPoint{<punto>}{<abscisa, ordenada>}</code>	<code>\spPoint[<key = val>]{<punto>}{<abscisa, ordenada>}</code>
	<code>\spPoint{<punto>}{<abscisa; ordenada>}</code>	<code>\spPoint[<key = val>]{<punto>}{<abscisa; ordenada>}</code>

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto>` } el cual es una *única letra mayúscula* con prima ‘`'`’ o subíndice ‘`_...`’ y el *argumento opcional delimitado por paréntesis* (`<abscisa, ordenada>`) el cual se procesa internamente bajo el comando `\spcoord`.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spPoint{A}` imprimirá ‘*A*’ en la salida PDF y NVDA/MathCAT verbalizará «*punto A*»; `\spPoint{A_{1}}` imprimirá ‘*A₁*’ y verbalizará «*punto A sub uno*».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spPoint{A}(-3,4)` imprimirá ‘*A(-3,4)*’ en la salida PDF y NVDA/MathCAT verbalizará «*punto A menos tres coma cuatro*»; `\spPoint{A_{1}}(-3,4)` imprimirá ‘*A₁(-3,4)*’ y verbalizará «*punto A sub uno menos tres coma cuatro*».

Llaves para `\spPoint`

`read-cmd` = { `<school>` | `<mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento { `<punto>` }. Si se establece en `school`, NVDA/MathCAT verbalizará el argumento { `<punto>` } sin el prefijo «mayúscula», `\spPoint{A}` se verbaliza «*punto A*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «*punto*». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`concept` = { `<true>` | `<false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*punto*» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`silent-cmd` = { `<true>` | `<false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*punto*» y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spvec`

<code>\spvec</code>	<code>\spvec{<letra>}</code>	<code>\spvec{<punto, punto>}</code>
	<code>\spvec{<letra>}{<abscisa, ordenada>}</code>	<code>\spvec{<punto, punto>}{<abscisa, ordenada>}</code>
	<code>\spvec[<key = val>]{<letra>}{<abscisa, ordenada>}</code>	<code>\spvec[<key = val>]{<punto, punto>}{<abscisa, ordenada>}</code>

El comando `\spvec` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma { `<letra>` } (minúscula) o { `<punto, punto>` } junto al *argumento opcional delimitado por paréntesis* (`<abscisa, ordenada>`) el cual se procesa internamente bajo el comando `\spcoord`.

Si el *argumento* es de la forma { `<punto, punto>` } con prima(s) ‘`'`’ o subíndice(s) ‘`_...`’ opcionales se aplica `\overrightarrow` en la impresión; si el *argumento* es de la forma { `<letra>` } (minúscula) con subíndice ‘`_...`’ opcional se aplica `\vec` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spvec{a}` imprimirá ‘ *$\vec{a}$* ’ en la salida PDF y NVDA/MathCAT verbalizará «*vector a*»; `\spvec{AB}` imprimirá ‘ *$\overrightarrow{AB}$* ’ y verbalizará «*vector A B*».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spvec{a}(-3,4)` imprimirá ‘ *$\vec{a} = (-3,4)$* ’ en la salida PDF y NVDA/MathCAT verbalizará «*vector a igual a menos tres coma cuatro*»; `\spvec{AB}(-3,4)` imprimirá ‘ *$\overrightarrow{AB} = (-3,4)$* ’ y verbalizará «*vector A B igual a menos tres coma cuatro*».

Llaves para `\spvec`

`read-cmd` = { `<punto, punto>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento { `<punto, punto>` }. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spvec{AB}` se verbaliza «vector A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «vector». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`autofit` = { `<true | false>` }

default: *false*

Establece si se *utiliza* `\vphantom{b}` al ejecutar el comando cuando se utiliza el argumento { `<letra>` }. Si se establece en *true*, se aplicará `\vphantom{b}` al inicio del argumento { `<letra>` }; si se establece en *false*, no se aplicará `\vphantom{b}`.

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en *false*, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`silent-cmd` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprecta`

`\sprecta` `\sprecta{<punto, punto>}`

`\sprecta{<letra>}`

`\sprecta[<key = val>]{<punto, punto>}`

`\sprecta[<key = val>]{<letra>}`

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma { `<letra>` } o { `<punto, punto>` } con prima(s) ' ' o subíndice(s) '_{...}' opcionales. Si el *argumento* es de la forma { `<punto, punto>` } se aplica `\overleftrightharrow` en la impresión; si el *argumento* es de la forma { `<letra>` } (mayúscula o minúscula), se imprimirá solo la { `<letra>` } sin aplicar `\overleftrightharrow`.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\sprecta{AB}` imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «recta A B»; `\sprecta{m}` imprimirá 'm' en la salida PDF y NVDA/MathCAT verbalizará «recta m».

Llaves para `\sprecta`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento { `<punto, punto>` } y el argumento { `<letra>` } cuando está en mayúscula. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprecta{AB}` se verbaliza «recta A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «recta». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «recta» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en *false*, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`silent-cmd` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «recta» y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprayo`

`\sprayo` `\sprayo{<punto, punto>}`

`\sprayo[<key = val>]{<punto, punto>}`

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto>` } con prima(s) ' ' o subíndice(s) '_{...}' opcionales. Representa un *rayo* o *semirrecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\sprayo{AB}` imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».

Llaves para `\sprayo`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\sprayo{AB}$` se verbaliza «rayo A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<rayo | semirrecta>` } default: `rayo`

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `$\sprayo[read-sty=rayo]{AB}$`, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza `$\sprayo[read-sty=semirrecta]{AB}$`, NVDA/MathCAT verbalizará «semirrecta A B».

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

- La impresión ' $\overline{AB}$ ' en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

`\spside` `\spside{<punto, punto>}`
`\spside[<key = val>]{<punto, punto>}`

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto>` } con prima(s) ' ' o subíndice(s) ' _{...}' opcionales. Representa un *lado, segmento o trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `$\spside{AB}$` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\spside{AB}$` se verbaliza «lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `$\spside[read-sty=lado]{AB}$`, NVDA/MathCAT verbalizará «lado A B»; si se utiliza `$\spside[read-sty=segmento]{AB}$`, NVDA/MathCAT verbalizará «segmento A B» y si se utiliza `$\spside[read-sty=trazo]{AB}$`, NVDA/MathCAT verbalizará «trazo A B».

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `$\spside[read-arg={hipotenusa}]{AB}$` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «hipotenusa A B».

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmside`

`\spmside` `\spmside{⟨punto, punto⟩}`
`\spmside[⟨key = val⟩]{⟨punto, punto⟩}`

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨punto, punto⟩}` con prima(s) ‘*’* o subíndice(s) ‘*_*{...}’ opcionales. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* ‘ $\overline{AB}$ ’; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmside{AB}` imprimirá ‘*AB*’ en la salida PDF y NVDA/MathCAT verbalizará «*medida del lado A B*».

Llaves para `\spmside`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmside{AB}` se verbaliza «*medida del lado A B*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = {⟨lado | segmento | trazo⟩}`

default: *lado*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «*medida del*» al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spmside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «*medida del lado A B*»; si se utiliza `\spmside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «*medida del segmento A B*» y si se utiliza `\spmside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «*medida del trazo A B*».

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*medida del*» seguido por el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘*{ }*’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*» y del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «*medida del*», el concepto y el `{⟨argumento⟩}`; si se establece en *false*, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘*{ }*’ y se debe usar en conjunto con la llave `concept`.

Por ejemplo: `\spmside[concept=false,read-arg={medida de la cuerda}]{AB}` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*medida de la cuerda A B*».

`print-m = {⟨true | false⟩}`

default: *false*

Establece si se *imprime* la notación visual ‘*m*’ antes aplicar `\overline` al `{⟨argumento⟩}`. Si se establece en *true*, `\spmside[print-m=true]{AB}` imprimirá ‘*m* $\overline{AB}$ ’ en la salida PDF, si se establece en *false* imprimirá ‘*AB*’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «*medida del lado A B*».

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*», del «*concepto*» establecido por la llave `read-sty` y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spangle`

<code>\spangle</code>	<code>\spangle{⟨punto letra griega número⟩}</code>	<code>\spangle{⟨punto, punto, punto⟩}</code>
	<code>\spangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\angle` o `\rightanglesqr` en la impresión. Solo los argumentos de la forma `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}` admiten prima(s) ‘`’` o subíndice(s) ‘`_{\dots}`’ opcionales.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `\spangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa»; `\spangle{1}` imprimirá ‘ $\angle 1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo uno» y `\spangle{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

- Si no se tiene instalada la fuente matemática NewCMath-Regular.otf parte del paquete `newcomputermodern` en el sistema o no se provee una fuente matemática con soporte para `\rightanglesqr` se usará `\rightangle` en su reemplazo.

Llaves para `\spangle`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spangle{AOB}` se verbaliza «ángulo A O B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`recto = {⟨true | false⟩}` default: *false*

Establece si se *usará* `\rightanglesqr` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A»; `\spangle[recto=true]{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en `false`, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto «ángulo» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmangle`

<code>\spmangle</code>	<code>\spmangle{⟨punto letra griega número⟩}</code>	<code>\spmangle{⟨punto, punto, punto⟩}</code>
	<code>\spmangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spmangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. Solo los argumentos de la forma `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}` admiten prima(s) ‘`’` o subíndice(s) ‘`_{\dots}`’ opcionales.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa»; `\spmangle{1}` imprimirá ‘ $\angle 1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo uno» y `\spmangle{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

- La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Llaves para `\spmangle`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto>` } o { `<punto, punto, punto>` }. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{AOB}` se verbaliza «medida del ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`recto` = { `<true | false>` }

default: *false*

Establece si se *usará* `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spmangle[recto=true]{A}` imprimirá '⊥A' en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A», `\spmangle[recto=true]{AOB}` imprimirá '⊥AOB' en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B»; si se establece en *false*, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`print-m` = { `<true | false>` }

default: *false*

Establece *si se imprime* la notación visual 'm' antes del argumento pasado al comando sin modificar la verbalización de este. Si se establece en *true*, `\spmangle[print-m=true]{AOB}` imprimirá 'm∠AOB' en la salida PDF, si se establece en *false* imprimirá '∠AOB' en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo A O B».

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» seguido por el concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «medida del ángulo» y el { `<argumento>` }; si se establece en *false*, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`silent-cmd` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del concepto «ángulo» y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spang`

```
\spang \spang{<grados>}
\spang \spang{<grados : minutos>}
\spang \spang{<grados : minutos : segundos>}
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<grados>` }, { `<grados : minutos>` } o { `<grados : minutos : segundos>` }. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar. Por ejemplo, `\spang{45:30:15}` imprimirá '45°30'15"' en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

```
\sparc \sparc{<punto, punto>} \sparc{<punto, punto, punto>}
\sparc \sparc[<key = val>]{<punto, punto>} \sparc[<key = val>]{<punto, punto, punto>}
```

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma { `<punto, punto>` } o { `<punto, punto, punto>` } con prima(s) ' ' o subíndice(s) '_{...}' opcionales aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\sparc{AB}` imprimirá 'AB' en la salida PDF y NVDA/MathCAT verbalizará «arco A B».

- El acento `\widearc` es proporcionado por algunos paquetes como `fourier-otf` o `kpfonts-otf`, en caso de no estar disponible se utilizará una versión interna para este.

Llaves para `\sparc`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` } o { `<punto, punto, punto>` }. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{AB}` se verbaliza «arco A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept = {\true | false}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «arco» y el $\{\langle \text{argumento} \rangle\}$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {\true | false}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmarc`

`\spmarc` $\{\langle \text{punto}, \text{punto} \rangle\}$

`\spmarc[\langle \text{key} = \text{val} \rangle]\{\langle \text{punto}, \text{punto} \rangle\}`

`\spmarc{\langle \text{punto}, \text{punto}, \text{punto} \rangle\}`

`\spmarc[\langle \text{key} = \text{val} \rangle]\{\langle \text{punto}, \text{punto}, \text{punto} \rangle\}`

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma $\{\langle \text{punto}, \text{punto} \rangle\}$ o $\{\langle \text{punto}, \text{punto}, \text{punto} \rangle\}$ con prima(s) ‘*’* o subíndice(s) ‘*_*{...}’ opcionales colocando ‘*m*’ antes de aplicar `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, `\$ \spmarc{AB} \$` imprimirá ‘*m* $\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del arco A B».

- La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* ‘ $\widehat{AB}$ ’; `\spmarc` representa su *medida* (un valor en grados).

Llaves para `\spmarc`

`read-cmd = {\school | mathcat}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento $\{\langle \text{punto}, \text{punto} \rangle\}$ o $\{\langle \text{punto}, \text{punto}, \text{punto} \rangle\}$. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\$ \spmarc{AB} \$` se verbaliza «medida del arco A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\prefijo}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves ‘*{ }*’.

`concept = {\true | false}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y el concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «medida del arco» y el $\{\langle \text{argumento} \rangle\}$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {\true | false}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», el concepto «arco» y el $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoly`

`\spPoly` $\{\langle \text{punto}, \text{punto}, \text{punto}, \dots \rangle\}$

`\spPoly[\langle \text{key} = \text{val} \rangle]\{\langle \text{punto}, \text{punto}, \text{punto}, \dots \rangle\}`

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto}, \text{punto}, \text{punto}, \dots \rangle\}$, una lista de al menos tres $\{\langle \text{puntos} \rangle\}$ con prima(s) ‘*’* o subíndice(s) ‘*_*{...}’ opcionales. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determinan acorde al número de puntos en el argumento y la llave `concept`.

Si son tres puntos se aplica `\triangle` y añade el concepto «triángulo» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el concepto «cuadrado» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo ni se añade verbalización.

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, `\$ \spPoly{ABC} \$` imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; `\$ \spPoly{ABCD} \$` imprimirá ‘ $\square ABCD$ ’ y verbalizará «cuadrado A B C D»; `\$ \spPoly{ABCDE} \$` imprimirá ‘ $ABCDE$ ’ y verbalizará «A B C D E».

Llaves para `\spPoly`

`read-cmd` = {`<school | mathcat>`}

default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {`<punto, punto, punto, ...>`}. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{ABC}` se verbaliza «*triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = {`<prefijo>`}

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el {`<argumento>`} o los conceptos «*triángulo*» y «*cuadrado*» establecido por la llave `concept`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = {`<true | false>`}

default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos «*triángulo*» y «*cuadrado*» acorde al número de puntos en el {`<argumento>`} al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «*triángulo*» o «*cuadrado*» y el {`<argumento>`}; si se establece en `false`, NVDA/MathCAT solo verbalizará el {`<argumento>`} acorde al valor establecido por la llave `read-cmd`.

`print-sym` = {`<true | false>`}

default: `true`

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión sin modificar la verbalización de este. Si se establece en `true`, `\spPoly[print-sym=true]{ABC}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; `\spPoly[print-sym=true]{ABCD}` imprimirá ' $\square ABCD$ ' en la salida PDF y NVDA/MathCAT verbalizará «*cuadrado A B C*»; si se establece en `false`, solo se imprimirá el {`<argumento>`} en la salida PDF y la verbalización será la misma.

`silent-cmd` = {`<true | false>`}

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos establecido por la llave `concept` y el {`<argumento>`} pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAfig`

<code>\spAfig</code>	<code>\spAfig</code>	<code>\spAfig(<punto, punto, punto, ...>)</code>
<code>\spAfig[<key = val>]</code>	<code>\spAfig[<key = val>](<punto, punto, punto, ...>)</code>	
<code>\spAfig(<1 2 3 ... n>)</code>	<code>\spAfig(F P B L T)</code>	
<code>\spAfig[<key = val>](<1 2 3 ... n>)</code>	<code>\spAfig[<key = val>](F P B L T)</code>	

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma {`<1 | 2 | 3 | ... | n>`}, {`<punto, punto, punto, ...>`} o {`<F | P | B | L | T>`} aplicando `\mathcal{A}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determinan acorde al {`<argumento>`} y la llave `concept`.

Si se proporciona el *argumento* {`<1 | 2 | 3 | ... | n>`}, `\spAfig(1)` imprimirá ' $\mathcal{A}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área uno*»; `\spAfig(n)` imprimirá ' $\mathcal{A}_n$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área enes*».

Si se proporciona el *argumento* {`<punto, punto, punto, ...>`}, `\spAfig(ABC)` imprimirá ' $\mathcal{A}_{\triangle ABC}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área del triángulo A B C*»; `\spAfig(ABCD)` imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área del cuadrado A B C D*»;

Si se proporciona el *argumento* {`<F | P | B | L | T>`}, `\spAfig(F)` imprimirá ' $\mathcal{A}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área de la figura*»; `\spAfig(F_{1})` imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área de la figura uno*».

Si no se proporciona el *argumento opcional* [`<key = val>`] ni el *argumento opcional delimitado por paréntesis*, `\spAfig` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área*».

- El *argumento* {`<punto, punto, punto, ...>`} acepta prima(s) '`'`' o subíndice(s) '`_{...}`'. El *argumento* {`<F | P | B | L | T>`} solo acepta subíndices '`_{...}`' y tiene una impresión y verbalización fija para cada letra: F «*figura*», P «*polígono*», B «*basal*», L «*lateral*» y T «*total*».

Llaves para `\spAfig`

`read-cmd` = {`<school | mathcat>`}

default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spAfig(ABC)` se verbaliza «*área del triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = {`<prefijo>`}

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*área del*» y el concepto «*nombre de la figura*» acorde al {`<argumento>`} pasado al comando. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { *⟨true | false⟩* } default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del» y el concepto «nombre de la figura» acorde al (*⟨argumento⟩*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «área del», el concepto «nombre de la figura» y el (*⟨argumento⟩*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*⟨argumento⟩*) acorde al valor establecido por la llave `read-cmd`.

`read-arg` = { *⟨nombre⟩* } default: *no usado*

Sobrescribe el término «área del» y el concepto «nombre de la figura» cuando se utiliza el argumento (*⟨punto, punto, punto, ...⟩*) que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `$\spAfig[read-arg={trapezio}](ABCD)` imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapezio A B C D».

Esta llave no se puede utilizar con los argumentos (*⟨F | P | B | L | T⟩*) y (*⟨1 | 2 | 3 | ... | n⟩*).

`concept*` { *⟨valor prohibido⟩* } default: *no usado*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al (*⟨argumento⟩*) pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «área» y el (*⟨argumento⟩*) acorde al valor establecido por la llave `read-cmd`.

`read-sub` = { *⟨true | false⟩* } default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos (*⟨1 | 2 | 3 | ... | n⟩*) y (*⟨F | P | B | L | T⟩*) para NVDA/MathCAT. Si se utiliza (*⟨1 | 2 | 3 | ... | n⟩*) y se establece en *true*, `$\spAfig[read-sub=true](1)` imprimirá ' $\mathcal{A}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «área sub uno»; si se establece en *false*, la impresión será la misma y se verbaliza «área uno».

Si se utiliza (*⟨F | P | B | L | T⟩*) y se establece en *true*, `$\spAfig[read-sub=true](F_{1})` imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura sub uno»; si se establece en *false*, la impresión será la misma y se verbaliza «área de la figura uno».

`print-sym` = { *⟨auto | triangle | square | none⟩* } default: *auto*

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión para los argumentos (*⟨punto, punto, punto, ...⟩*) y (*⟨1 | 2 | 3 | ... | n⟩*) pasado al comando sin modificar la verbalización de este. Si se utiliza (*⟨punto, punto, punto, ...⟩*) se infiere el *símbolo* acorde al número de puntos; si establece en *auto* se imprime y si se establece en *none* se omite: `$\spAfig[print-sym=auto](ABC)` y `$\spAfig[print-sym=none](ABC)` imprimirán ' $\mathcal{A}_{\triangle ABC}$ ' y ' $\mathcal{A}_{ABC}$ ' en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «área del triángulo A B C».

Si se utiliza (*⟨1 | 2 | 3 | ... | n⟩*) los valores *auto* y *none* quedan sin efecto; si se establece en *triangle* o *square* anida el argumento bajo el *símbolo*: `$\spAfig[print-sym=triangle](1)` imprimirá ' $\mathcal{A}_{\triangle_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del triángulo uno»; `$\spAfig[print-sym=square](1)` imprimirá ' $\mathcal{A}_{\square_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del cuadrado uno».

Esta llave no se puede utilizar con el argumento (*⟨F | P | B | L | T⟩*).

`silent-cmd` = { *⟨true | false⟩* } default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del», el concepto «nombre de la figura» y el (*⟨argumento⟩*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPfig`

<code>\spPfig</code>	<code>\spPfig</code>	<code>\spPfig(⟨punto, punto, punto, ...⟩)</code>
<code>\spPfig[key = val]</code>	<code>\spPfig[key = val]</code>	<code>\spPfig[key = val](⟨punto, punto, punto, ...⟩)</code>
<code>\spPfig(⟨1 2 3 ... n⟩)</code>	<code>\spPfig(⟨F P B L T⟩)</code>	<code>\spPfig(⟨F P B L T⟩)</code>
<code>\spPfig[key = val](⟨1 2 3 ... n⟩)</code>	<code>\spPfig[key = val](⟨F P B L T⟩)</code>	

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*⟨1 | 2 | 3 | ... | n⟩*), (*⟨punto, punto, punto, ...⟩*) o (*⟨F | P | B | L | T⟩*) aplicando `\mathcal{P}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determinan acorde al (*⟨argumento⟩*) y la llave `concept`.

Si se proporciona el *argumento* (*⟨1 | 2 | 3 | ... | n⟩*), `$\spPfig(1)` imprimirá ' $\mathcal{P}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro uno»; `$\spPfig(n)` imprimirá ' $\mathcal{P}_n$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro ene».

Si se proporciona el *argumento* (*⟨punto, punto, punto, ...⟩*), `$\spPfig(ABC)` imprimirá ' $\mathcal{P}_{\triangle ABC}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del triángulo A B C»; `$\spPfig(ABCD)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del cuadrado A B C D»;

Si se proporciona el *argumento* (*⟨F | P | B | L | T⟩*), `$\spPfig(F)` imprimirá ' $\mathcal{P}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura»; `$\spPfig(F_{1})` imprimirá ' $\mathcal{P}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura uno».

Si no se proporciona el *argumento opcional* [*(key = val)*] ni el *argumento opcional delimitado por paréntesis*, $\$ \backslash \text{spPfig} \$$ imprimirá ‘P’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro».

- El *argumento* (*(punto, punto, punto, ...)*) acepta prima(s) ‘_’ o subíndice(s) ‘_{...}’. El *argumento* (*(F|P|B|L|T)*) solo acepta subíndices ‘_{...}’ y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para $\backslash \text{spPfig}$

`read-cmd = { (school | mathcat) }` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \backslash \text{spPfig} (ABC) \$$ se verbaliza «perímetro del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { (prefijo) }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro del» y el concepto «nombre de la figura» acorde al (*(argumento)*) pasado al comando. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`concept = { (true | false) }` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro del» y el concepto «nombre de la figura» acorde al (*(argumento)*) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «perímetro del», el concepto «nombre de la figura» y el (*(argumento)*); si se establece en `false`, NVDA/MathCAT solo verbalizará el (*(argumento)*) acorde al valor establecido por la llave `read-cmd`.

`read-arg = { (nombre) }` default: no usado

Sobrescribe el término «perímetro del» y el concepto «nombre de la figura» cuando se utiliza el argumento (*(punto, punto, punto, ...)*) que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \backslash \text{spPfig} [\text{read-arg}=\{\text{trapezio}\}] (ABCD) \$$ imprimirá ‘ $\mathcal{P}_{\square ABCD}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro del trapecio A B C D».

- Esta llave no se puede utilizar con los argumentos (*(F|P|B|L|T)*) y (*(1|2|3|...|n)*).

`concept* (valor prohibido)` default: no usado

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al (*(argumento)*) pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «perímetro» y el (*(argumento)*) acorde al valor establecido por la llave `read-cmd`.

`read-sub = { (true | false) }` default: false

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos (*(1|2|3|...|n)*) y (*(F|P|B|L|T)*) para NVDA/MathCAT. Si se utiliza (*(1|2|3|...|n)*) y se establece en `true`, $\$ \backslash \text{spPfig} [\text{read-sub}=\text{true}] (1) \$$ imprimirá ‘ $\mathcal{P}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «perímetro uno».

Si se utiliza (*(F|P|B|L|T)*) y se establece en `true`, $\$ \backslash \text{spPfig} [\text{read-sub}=\text{true}] (F_{1}) \$$ imprimirá ‘ $\mathcal{P}_{F_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «perímetro de la figura uno».

`print-sym = { (auto | triangle | square | none) }` default: auto

Establece si se *usará* $\backslash \text{triangle}$ o $\backslash \text{mdlgwhtsquare}$ en la impresión para los argumentos (*(punto, punto, punto, ...)*) y (*(1|2|3|...|n)*) pasado al comando sin modificar la verbalización de este. Si se utiliza (*(punto, punto, punto, ...)*) se infiere el *símbolo* acorde al número de puntos; si establece en `auto` se imprime y si se establece en `none` se omite: $\$ \backslash \text{spPfig} [\text{print-sym}=\text{auto}] (ABC) \$$ y $\$ \backslash \text{spPfig} [\text{print-sym}=\text{none}] (ABC) \$$ imprimirán ‘ $\mathcal{P}_{\triangle ABC}$ ’ y ‘ $\mathcal{P}_{ABC}$ ’ en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «perímetro del triángulo A B C».

Si se utiliza (*(1|2|3|...|n)*) los valores `auto` y `none` quedan sin efecto; si se establece en `triangle` o `square` anida el argumento bajo el *símbolo*: $\$ \backslash \text{spPfig} [\text{print-sym}=\text{triangle}] (1) \$$ imprimirá ‘ $\mathcal{P}_{\triangle 1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro del triángulo uno»; $\$ \backslash \text{spPfig} [\text{print-sym}=\text{square}] (1) \$$ imprimirá ‘ $\mathcal{P}_{\square 1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro del cuadrado uno».

- Esta llave no se puede utilizar con el argumento (*(F|P|B|L|T)*).

`silent-cmd = { (true | false) }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro del», el concepto «nombre de la figura» y el (*(argumento)*) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

Los comandos `\sprad` y `\spdiam`

<code>\sprad</code>	<code>\sprad</code>	<code>\sprad[⟨key = val⟩]</code>
<code>\spdiam</code>	<code>\spdiam</code>	<code>\spdiam[⟨key = val⟩]</code>

Los comandos `\sprad` y `\spdiam` trabajan en *modo matemático* y no reciben ningún argumento: imprimen y verbalizan la letra ‘r’/‘d’ (o ‘R’/‘D’, según `upper-case`) que representa el radio o el diámetro de una circunferencia.

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`], `\sprad` imprimirá ‘r’ en la salida PDF y NVDA/MathCAT verbalizará «radio»; `\spdiam` imprimirá ‘d’ en la salida PDF y NVDA/MathCAT verbalizará «diámetro».

Llaves para `\sprad` y `\spdiam`

`read-cmd` = {`⟨school | mathcat⟩`} default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará la letra cuando `concept` es *false*. Si se establece en *school*, NVDA/MathCAT verbaliza la letra deletreada; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas.

`upper-case` = {`⟨true | false⟩`} default: *false*

Establece si se imprime la letra en mayúscula sin modificar la verbalización de este. Si se establece en *true*, `\sprad[upper-case=true]` imprimirá ‘R’ en la salida PDF; `\spdiam[upper-case=true]` imprimirá ‘D’. En ambos casos la verbalización no cambia.

`print-sym` = {`⟨true | false⟩`} default: *false*

Establece si se usará `\diameter` antes de la letra en la impresión sin modificar la verbalización de este. Si se establece en *true*, `\spdiam[print-sym=true]` imprimirá ‘ $\varnothing$ ’ en la salida PDF y NVDA/MathCAT verbalizará «diámetro».

🟡 La llave `print-sym` no tiene ningún efecto en `\sprad`.

`sub-index` = {`⟨subíndice⟩`} default: *no usado*

Establece un *subíndice* sobre la letra. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`read-sub` = {`⟨true | false⟩`} default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el `sub-index` para NVDA/MathCAT. Si se establece en *true*, `\sprad[sub-index={1}, read-sub=true]` imprimirá ‘ r_1 ’ en la salida PDF y NVDA/MathCAT verbalizará «radio sub uno»; si se establece en *false*, se verbaliza «radio uno».

`concept` = {`⟨true | false⟩`} default: *true*

Establece si NVDA/MathCAT verbaliza «radio» (o «diámetro») en lugar de la letra misma. Si se establece en *true*, `\sprad` se verbaliza «radio»; si se establece en *false*, se verbaliza la letra acorde al valor establecido por la llave `read-cmd`.

`read-arg` = {`⟨verbalización NVDA⟩`} default: *no usado*

Sobrescribe «radio» (o «diámetro») que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\sprad[read-arg={medida del radio}]` imprimirá ‘r’ en la salida PDF y NVDA/MathCAT verbalizará «medida del radio».

`silent-cmd` = {`⟨true | false⟩`} default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de «radio» (o «diámetro») y de la letra al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spcirc`

<code>\spcirc</code>	<code>\spcirc</code>	<code>\spcirc[⟨key = val⟩]</code>
<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro⟩)</code>
<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro, radio⟩)</code>

El comando `\spcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma `(⟨centro⟩)` o `(⟨centro, radio⟩)`, donde `⟨radio⟩` es opcional y debe pasarse junto al argumento `(⟨centro⟩)` separado por una coma ‘,’ obligatoria.

La forma en la que se procesa el `⟨radio⟩` junto a su impresión y verbalización se establece de manera interna: Si es un `⟨número⟩` se usa `\spnum`; si es una `⟨medida⟩` se usa `\spunit` y si es un segmento `⟨punto punto⟩` se usa `\spside` o `\spmside` acorde al valor establecido por la llave `radio-sty`. En cualquier otro caso no se procesa de manera interna.

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`] ni el *argumento opcional delimitado por paréntesis*, `\spcirc` imprimirá ‘ $\odot$ ’ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia».

Si se proporciona el *argumento* `(⟨centro⟩)`, `\spcirc(0)` imprimirá ‘ $\odot(O)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en O».

Si se proporciona el *argumento* ($\langle \text{centro}, \text{radio} \rangle$), $\$ \backslash \text{spc} \text{circ}(0, 3) \$$ imprimirá ' $\odot(O, 3)$ ' en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en O y radio tres».

- El argumento ($\langle \text{centro} \rangle$) y el argumento ($\langle \text{radio} \rangle$) cuando tiene la forma ($\langle \text{punto punto} \rangle$) aceptan prima ' $'$ ' o subíndice ' $_{\dots}$ '.

Llaves para $\backslash \text{spc} \text{circ}$

El comando $\backslash \text{spc} \text{circ}$ posee las *meta-keys* spside pasada al comando $\backslash \text{spside}$ (v. pág. 20) y spmside pasada al comando $\backslash \text{spmside}$ (v. pág. 21).

`read-cmd = { $\langle \text{school} \mid \text{mathcat} \rangle$ }` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento ($\langle \text{centro} \rangle$). Si se establece en *school*, NVDA/MathCAT verbaliza el argumento ($\langle \text{centro} \rangle$) sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

- Esta llave tiene efecto sobre el argumento ($\langle \text{radio} \rangle$) cuando tiene la forma ($\langle \text{punto punto} \rangle$).

`prefix = { $\langle \text{prefijo} \rangle$ }` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave *read-sty*. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = { $\langle \text{usual} \mid \text{odot} \rangle$ }` default: *odot*

Establece si se usará $\backslash \text{bigodot}$ o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, $\$ \backslash \text{spc} \text{circ}[\text{print-sym}=\text{usual}](0) \$$ imprimirá ' $C(O)$ ' en la salida PDF; si se establece en *odot*, $\$ \backslash \text{spc} \text{circ}[\text{print-sym}=\text{odot}](0) \$$ imprimirá ' $\odot(O)$ ' en la salida PDF.

`read-sty = { $\langle \text{circun} \mid \text{circle} \rangle$ }` default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, $\$ \backslash \text{spc} \text{circ}[\text{read-sty}=\text{circun}](0) \$$ se verbalizará «circunferencia de centro en O » en NVDA/MathCAT; si se establece en *circle*, $\$ \backslash \text{spc} \text{circ}[\text{read-sty}=\text{circle}](0) \$$ se verbalizará «círculo de centro en O » en NVDA/MathCAT.

`diameter = { $\langle \text{true} \mid \text{false} \rangle$ }` default: *false*

Establece *como* se verbaliza el argumento ($\langle \text{radio} \rangle$) al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \backslash \text{spc} \text{circ}[\text{diameter}=\text{true}](0, 3) \$$ se verbalizará «circunferencia de centro en O y diámetro tres» en NVDA/MathCAT; si se establece en *false*, $\$ \backslash \text{spc} \text{circ}[\text{diameter}=\text{false}](0, 3) \$$ se verbalizará «circunferencia de centro en O y radio tres» en NVDA/MathCAT.

`radio-sty = { $\langle \text{side} \mid \text{mside} \rangle$ }` default: *mside*

Establece el *comando* que procesa internamente el argumento ($\langle \text{radio} \rangle$) cuando tiene la forma ($\langle \text{punto punto} \rangle$). Si se establece en *side* se usará $\backslash \text{spside}\{\langle \text{punto punto} \rangle\}$, $\$ \backslash \text{spc} \text{circ}[\text{radio-sty}=\text{side}](0, AB) \$$ imprimirá ' $\odot(O, \overline{AB})$ ' en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en O y radio lado $A B$ »; si se establece en *mside* se usará $\backslash \text{spmside}\{\langle \text{punto punto} \rangle\}$, $\$ \backslash \text{spc} \text{circ}[\text{radio-sty}=\text{mside}](0, AB) \$$ imprimirá ' $\odot(O, AB)$ ' en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en O y radio medida del lado $A B$ ».

- Las *meta-keys* spside y spmside permiten modificar la impresión y verbalización del argumento ($\langle \text{radio} \rangle$) cuando tiene la forma ($\langle \text{punto punto} \rangle$).

`concept = { $\langle \text{true} \mid \text{false} \rangle$ }` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave *read-sty* al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el «concepto» y el argumento ($\langle \text{centro} \rangle$) o ($\langle \text{centro}, \text{radio} \rangle$); si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento ($\langle \text{centro} \rangle$) o ($\langle \text{centro}, \text{radio} \rangle$) acorde al valor establecido por la llave *read-cmd*.

`sub-index = { $\langle \text{subíndice} \rangle$ }` default: *no usado*

Establece un *subíndice* sobre $\backslash \text{bigodot}$ o la letra 'C' establecida por la llave *print-sym*. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: $\$ \backslash \text{spc} \text{circ}[\text{sub-index}=\{1\}, \text{radio-sty}=\text{mside}](0, AB) \$$ imprimirá ' $\odot_1(O, AB)$ ' en la salida PDF y NVDA/MathCAT verbalizará «circunferencia uno de centro en O y radio medida del lado $A B$ ».

`read-sub = { $\langle \text{true} \mid \text{false} \rangle$ }` default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecido por la llave *sub-index* al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \backslash \text{spc} \text{circ}[\text{sub-index}=\{1\}, \text{read-sub}=\text{true}](0) \$$ se verbalizará «circunferencia sub uno de centro en O » en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «circunferencia uno de centro en O ».

`read-arg = { $\langle \text{verbalización NVDA} \rangle$ }` default: *no usado*

Sobrescribe el *concepto* establecido por la llave *read-sty* que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: $\$ \backslash \text{spc} \text{circ}[\text{read-arg}=\{\text{disco}\}](0) \$$ imprimirá ' $\odot(O)$ ' en la salida PDF y NVDA/MathCAT verbalizará «disco de centro en O ».

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty`, del argumento (`⟨centro⟩`) o (`⟨centro, radio⟩`) pasados al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAcirc`

<code>\spAcirc</code>	<code>\spAcirc</code>	<code>\spAcirc[⟨key = val⟩]</code>
<code>\spAcirc(⟨centro⟩)</code>	<code>\spAcirc(⟨centro⟩)</code>	<code>\spAcirc[⟨key = val⟩](⟨centro⟩)</code>
<code>\spAcirc(⟨1 2 3 ... n⟩)</code>	<code>\spAcirc(⟨1 2 3 ... n⟩)</code>	<code>\spAcirc[⟨key = val⟩](⟨1 2 3 ... n⟩)</code>

El comando `\spAcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (`⟨centro⟩`) o (`⟨1 | 2 | 3 | ... | n⟩`), aplicando `\mathcal{A}` a la impresión.

Si no se proporciona el *argumento opcional* [`⟨key = val⟩`] ni el *argumento opcional delimitado por paréntesis*, `\spAcirc` imprimirá ‘ $\mathcal{A}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia».

Si se proporciona el *argumento* (`⟨centro⟩`), `\spAcirc(0)` imprimirá ‘ $\mathcal{A}_{O(O)}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia con centro en O ».

Si se proporciona el *argumento* (`⟨1 | 2 | 3 | ... | n⟩`), `\spAcirc(1)` imprimirá ‘ $\mathcal{A}_{O_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno».

- El *argumento* (`⟨centro⟩`) acepta prima ‘`′`’ o subíndice ‘`_{\dots}`’ y no acepta un *radio*. Una coma ‘`,`’ en el argumento siempre produce un “error”.

Llaves para `\spAcirc`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento (`⟨centro⟩`). Si se establece en *school*, NVDA/MathCAT verbaliza el argumento (`⟨centro⟩`) sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área de ...» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`read-sty = {⟨circun | circle⟩}`

default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «área de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, `\spAcirc[read-sty=circun](0)` se verbalizará «área de la circunferencia con centro en O » en NVDA/MathCAT; si se establece en *circle*, `\spAcirc[read-sty=circle](0)` se verbalizará «área del círculo con centro en O » en NVDA/MathCAT.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término, el concepto y el argumento (`⟨centro⟩`) o (`⟨1 | 2 | 3 | ... | n⟩`); si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento (`⟨centro⟩`) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento (`⟨centro⟩`) o con el argumento (`⟨1 | 2 | 3 | ... | n⟩`) es equivalente a usar `silent-cmd=true`.

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spAcirc[read-arg={esfera}](0)` se verbalizará «área de la esfera con centro en O » en NVDA/MathCAT; `\spAcirc[concept=false, read-arg={esfera}](0)` se verbalizará «esfera con centro en O » en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento (`⟨1 | 2 | 3 | ... | n⟩`) es equivalente a usar `silent-cmd=true`.

`print-sym = {⟨usual | odot⟩}`

default: *odot*

Establece si se *usará* `\bigodot` o la letra ‘ C ’ en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, `\spAcirc[print-sym=usual](0)` imprimirá ‘ $\mathcal{A}_{C(O)}$ ’ en la salida PDF; si se establece en *odot*, `\spAcirc[print-sym=odot](0)` imprimirá ‘ $\mathcal{A}_{O(O)}$ ’ en la salida PDF.

`sub-index = {⟨subíndice⟩}`

default: *no usado*

Establece un *subíndice* sobre `\bigodot` o la letra ‘ C ’ establecida por la llave `print-sym` cuando se utiliza el argumento (`⟨centro⟩`). El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’. Por ejemplo: `\spAcirc[sub-index={1}](0)` imprimirá ‘ $\mathcal{A}_{O_1(O)}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno con centro en O ».

`read-sub = {⟨true | false⟩}`

default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento $(\langle 1 | 2 | 3 | \dots | n \rangle)$ al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \backslash \text{spPcCirc}[\text{read-sub=true}](1) \$$ se verbalizará «área de la circunferencia sub uno» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «área de la circunferencia uno».

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...», del concepto establecido por la llave `read-sty` y del argumento $(\langle \text{centro} \rangle)$ o $(\langle 1 | 2 | 3 | \dots | n \rangle)$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPcCirc`

<code>\spPcCirc</code>	<code>\spPcCirc</code>	<code>\spPcCirc[\langle key = val \rangle]</code>
<code>\spPcCirc(\langle centro \rangle)</code>	<code>\spPcCirc[\langle key = val \rangle](\langle centro \rangle)</code>	
<code>\spPcCirc(\langle 1 2 3 \dots n \rangle)</code>	<code>\spPcCirc[\langle key = val \rangle](\langle 1 2 3 \dots n \rangle)</code>	

El comando `\spPcCirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $(\langle \text{centro} \rangle)$ o $(\langle 1 | 2 | 3 | \dots | n \rangle)$, aplicando `\mathcal{P}` a la impresión.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$ ni el *argumento opcional delimitado por paréntesis*, $\$ \backslash \text{spPcCirc} \$$ imprimirá ‘P’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia».

Si se proporciona el *argumento* $(\langle \text{centro} \rangle)$, $\$ \backslash \text{spPcCirc}(O) \$$ imprimirá ‘ $\mathcal{P}_{O(O)}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia con centro en O».

Si se proporciona el *argumento* $(\langle 1 | 2 | 3 | \dots | n \rangle)$, $\$ \backslash \text{spPcCirc}(1) \$$ imprimirá ‘ $\mathcal{P}_{O_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia uno».

- El *argumento* $(\langle \text{centro} \rangle)$ acepta prima ‘ $\prime$ ’ o subíndice ‘ $_ \{ \dots \}$ ’ y no acepta un *radio*. Una coma ‘ $,$ ’ en el argumento siempre produce un “error”.

Llaves para `\spPcCirc`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento $(\langle \text{centro} \rangle)$. Si se establece en *school*, NVDA/MathCAT verbaliza el argumento $(\langle \text{centro} \rangle)$ sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro de ...» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’.

`read-sty = {⟨circun | circle⟩}`

default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «perímetro de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, $\$ \backslash \text{spPcCirc}[\text{read-sty=circun}](O) \$$ se verbalizará «perímetro de la circunferencia con centro en O» en NVDA/MathCAT; si se establece en *circle*, $\$ \backslash \text{spPcCirc}[\text{read-sty=circle}](O) \$$ se verbalizará «perímetro del círculo con centro en O» en NVDA/MathCAT.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro de ...» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término, el concepto y el argumento $(\langle \text{centro} \rangle)$ o $(\langle 1 | 2 | 3 | \dots | n \rangle)$; si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento $(\langle \text{centro} \rangle)$ acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento $(\langle \text{centro} \rangle)$ o con el argumento $(\langle 1 | 2 | 3 | \dots | n \rangle)$ es equivalente a usar `silent-cmd=true`.

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \backslash \text{spPcCirc}[\text{read-sty=circle}, \text{read-arg}=\{\text{disco}\}](O) \$$ se verbalizará «perímetro del disco con centro en O» en NVDA/MathCAT; $\$ \backslash \text{spPcCirc}[\text{concept=false}, \text{read-arg}=\{\text{disco}\}](O) \$$ se verbalizará «disco con centro en O» en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento $(\langle 1 | 2 | 3 | \dots | n \rangle)$ es equivalente a usar `silent-cmd=true`.

`print-sym = {⟨usual | odot⟩}`

default: *odot*

Establece si se *usará* `\bigodot` o la letra ‘C’ en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, $\$ \backslash \text{spPcCirc}[\text{print-sym=usual}](O) \$$ imprimirá ‘ $\mathcal{P}_{C(O)}$ ’ en la salida PDF; si se establece en *odot*, $\$ \backslash \text{spPcCirc}[\text{print-sym=odot}](O) \$$ imprimirá ‘ $\mathcal{P}_{O(O)}$ ’ en la salida PDF.

`sub-index = {\langle subíndice \rangle}`

default: *no usado*

Establece un *subíndice* sobre `\bigodot` o la letra ‘C’ establecida por la llave `print-sym` cuando se utiliza el argumento (`\langle centro \rangle`). El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: `\$ \spPcirc[sub-index={1}](O) \$` imprimirá ‘ $\mathcal{P}_{\odot_1(O)}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro de la circunferencia uno con centro en O*».

`read-sub = {\langle true | false \rangle}`

default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento (`\langle 1 | 2 | 3 | \dots | n \rangle`) al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\$ \spPcirc[read-sub=true](1) \$` se verbalizará «*perímetro de la circunferencia sub uno*» en NVDA/MathCAT; si se establece en `false`, NVDA/MathCAT verbalizará «*perímetro de la circunferencia uno*».

`silent-cmd = {\langle true | false \rangle}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*perímetro de ...*», del concepto establecido por la llave `read-sty` y del argumento (`\langle centro \rangle`) o (`\langle 1 | 2 | 3 | \dots | n \rangle`) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\TeX}$ Project. “The $\text{\TeX}$ ₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\TeX}$ Project. “The $\text{\TeX}$ _{2 ϵ} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\TeX}$ Project. “ $\text{\TeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\TeX}$ Project. “ $\text{\TeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\TeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\TeX}$ Project. “latex-lab – $\text{\TeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-09-29 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C		sub-index 30	
Document class:		Keys for \spAfig provide by spintent :	
article	1	concept*	26
book	1	concept	26
letter	1	prefix	25
report	1	print-sym	26
Commands provide by spintent :		read-arg	26
\setspintent	3	read-cmd	25
\spAcirc	30	read-sub	26
\spAfig	25	silent-cmd	26
\spPcirc	31	Keys for \spangle provide by spintent :	
\spPfig	26	concept	22
\spPoint	18	prefix	22
\spPoly	24	read-cmd	22
\spangle	22	recto	22
\spang	23	silent-cmd	22
\sparc	23	Keys for \sparc provide by spintent :	
\spcirc	28	concept	24
\spcoord	17	prefix	23
\spdate	11	read-cmd	23
\spdiam	28	silent-cmd	24
\spdiv	13	Keys for \spcirc provide by spintent :	
\spdsym	12	concept	29
\spmQ	17	diameter	29
\spmangle	22	prefix	29
\spmarc	24	print-sym	29
\spmcd	15	radio-sty	29
\spmcm	16	read-arg	29
\spmoney	6	read-cmd	29
\spmside	21	read-sty	29
\spmsym	13	read-sub	29
\spmul	13	silent-cmd	30
\spnD	14	spmside	29
\spnM	14	spside	29
\spnZ	16	sub-index	29
\spnewread	11	Keys for \spcoord provide by spintent :	
\spnewunit	6	silent-cmd	18
\spnum	3	Keys for \spdiv provide by spintent :	
\sprad	28	cifras-sep	13
\sprayo	19	notation-sym	13
\spread	11	read-parens	13
\sprecta	19	read-period-sep	13
\spshort	8	read-sign	13
\spside	20	read-sym	13
\spsiglo	10	set-sym	13
\sptime	11	wrap-parens	13
\spunitalias	6	Keys for \spdsym provide by spintent :	
\spunit	4	prefix	12
\spusep	12	read-sym	12
\spvec	18	set-sym	12
		suffix	12
K		Keys for \spmangle provide by spintent :	
Keys for \spAcirc provide by spintent :		concept	23
concept	30	prefix	23
prefix	30	print-m	23
print-sym	30	read-cmd	23
read-arg	30	recto	23
read-cmd	30	silent-cmd	23
read-sty	30	Keys for \spmarc provide by spintent :	
read-sub	31	concept	24
silent-cmd	31	prefix	24

read-cmd	24	read-sign	3
silent-cmd	24	Keys for \spnZ provide by spintent :	
Keys for \spmcd provide by spintent :		cifras-sep	16
prefix	15	read-parens	16
read-cmd	15	read-sign	16
result	15	wrap-parens	16
silent-cmd	15	Keys for \spPcirc provide by spintent :	
upper-case	15	concept	31
Keys for \spmcm provide by spintent :		prefix	31
prefix	16	print-sym	31
read-cmd	16	read-arg	31
result	16	read-cmd	31
sample-arg	16	read-sty	31
silent-cmd	16	read-sub	32
upper-case	16	silent-cmd	32
Keys for \spmney provide by spintent :		sub-index	32
cifras-sep	7	Keys for \spPfig provide by spintent :	
currency	7	concept*	27
dolar-math	7	concept	27
sub-units	7	prefix	27
Keys for \spmQ provide by spintent :		print-sym	27
join-word	17	read-arg	27
print-dfrac	17	read-cmd	27
read-parens	17	read-sub	27
use-spñZ	17	silent-cmd	27
wrap-parens	17	Keys for \spPoint provide by spintent :	
Keys for \spmside provide by spintent :		concept	18
concept	21	prefix	18
prefix	21	read-cmd	18
print-m	21	silent-cmd	18
read-arg	21	Keys for \spPoly provide by spintent :	
read-cmd	21	concept	25
read-sty	21	prefix	25
silent-cmd	21	print-sym	25
Keys for \spmssym provide by spintent :		read-cmd	25
not-print	13	silent-cmd	25
prefix	13	Keys for \sprad provide by spintent :	
read-sym	13	concept	28
set-sym	13	print-sym	28
suffix	13	read-arg	28
Keys for \spmñ provide by spintent :		read-cmd	28
cifras-sep	14	read-sub	28
notation-sym	14	silent-cmd	28
read-parens	14	sub-index	28
read-period-sep	14	upper-case	28
read-sign	14	Keys for \sprayo provide by spintent :	
read-sym	14	concept	20
set-sym	14	prefix	20
wrap-parens	14	read-cmd	20
Keys for \spñD provide by spintent :		read-sty	20
concept	14	silent-cmd	20
prefix	14	Keys for \spread provide by spintent :	
result-num	14	read-arg	12
result	14	silent-cmd	12
silent-cmd	14	Keys for \sprecta provide by spintent :	
Keys for \spñM provide by spintent :		concept	19
concept	15	prefix	19
prefix	15	read-cmd	19
result-num	15	silent-cmd	19
result	15	Keys for \spshort provide by spintent :	
silent-cmd	15	read-arg	8
Keys for \spñnum provide by spintent :		small-caps	8
cifras-sep	3	Keys for \spside provide by spintent :	
notation-sym	4	concept	20
read-period-sep	3	prefix	20

read-arg	20		
read-cmd	20		
read-sty	20		
silent-cmd	21		
Keys for \spsiglo provide by spintent :			
print-era	11		
Keys for \spunit provide by spintent :			
cifras-sep	4		
notation-sym	4		
print-cdot	4		
read-cdot	4		
read-period-sep	4		
read-slash	4		
Keys for \spusep provide by spintent :			
silent	12		
size	12		
Keys for \spvec provide by spintent :			
autofit	19		
concept	19		
prefix	19		
read-cmd	19		
silent-cmd	19		
		M	
		Lua module provide by spintent :	
		spintent.lua	6–11
		P	
		Packages :	
		babel-spanish	2
		babel	2, 8
		documentmetadata-support	1
		enumext	1
		expl3	3
		fontspec	2
		fourier-otf	2
		hyperref	2
		l3keys	3
		latex-lab-mathintent	1
		libertinus-otf	2
		luamml	1
		scontents	1
		sourcecodepro	2
		tagpdf	1, 8, 10
		unicode-math	1
		S	
		\sprad	28

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and add only two keys: `mathcal` and `AFghost`, which can be passed as package options.

```
13 \ProvidesExplPackage {spintent} {2026-09-29} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n    = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n  = true,
19   mathcal .value_required:n = true,
20   AFghost .usage:n    = load,
21   AFghost .bool_set:N = \l__spintent_debug_ghost_bool,
22   AFghost .initial:n  = false,
23   AFghost .value_required:n = true,
24 }
25 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_LTeX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
26 \IfDocumentMetadataTF
27 {
28   \sys_if_engine luatex:TF
29   {
30     \msg_new:nnn { spintent } { package-not-load }
31     {
32       The~'#1'~package~will~be~loaded~as~a~dependency.
33     }
34     \IfPackageLoadedF { unicode-math }
35     {
36       \IfPackageLoadedF { lua-unicode-math }
37       {
38         \msg_info:nnn { spintent } { package-not-load } { unicode-math }
39         \RequirePackage{unicode-math}
```

```

40     }
41   }
42   \msg_new:nnn { spintent } { env-success }
43   { Everything~looks~correct,~spintent~will~make~the~intent! }
44   \msg_info:nn { spintent } { env-success }
45 }
46 {
47   \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
48   { The~spintent~package~requires~LuaLaTeX. }
49   { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
50   \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
51 }
52 }
53 {
54   \msg_new:nnnn { spintent } { fatal-metadata-missing }
55   { \token_to_str:N \DocumentMetadata \c_space_tl is-missing. }
56   {
57     The~spintent~package~requires~tagged~PDF~active.\\
58     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
59   }
60   \msg_fatal:nn { spintent } { fatal-metadata-missing }
61 }

```

11.4 Some kernel variants

```

\str_if_eq:nVTF \str_if_eq:nVF
\str_uppercase:V \str_lowercase:V
\seq_set_from_clist:Ne
\cs_generate_variant:Nn \str_if_eq:nnTF { V }
\cs_generate_variant:Nn \str_if_eq:nnF { V }
\cs_generate_variant:Nn \str_uppercase:n { V }
\cs_generate_variant:Nn \str_lowercase:n { V }
\cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

Non-standard kernel variants.

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Variables for the key `\mathcal`, `\rightanglesqr` and `\attribute`

First, we declare the string variable `\l__spintent_luaset_cal_fam_ok_str` used by the function `\__spintent_luafun_classic_mathcal:nn` (controlled by the key `mathcal`), the string variable `\l__spintent_luafun_right_angle_sqr` used by the function `\__spintent_luafun_right_angle_sqr:` to copy `\rightanglesqr` alongside the string variable `\l__spintent_luaset_scope_attr_str` used by `\attribute` before loading the Lua module `spintent.lua`. This is necessary due to the implementation of the functions within the Lua module; otherwise, `expl3` would issue a warning regarding duplicate variable declarations.

```

67 \str_new:N \l__spintent_luaset_cal_fam_ok_str
68 \str_new:N \l__spintent_luaset_sym_fam_ok_str
69 \str_new:N \l__spintent_luaset_scope_attr_str

```

(End of definition for `\l__spintent_luaset_cal_fam_ok_str`, `\l__spintent_luaset_sym_fam_ok_str`, and `\l__spintent_luaset_scope_attr_str`.)

11.6 Loading the Lua module and functions

`spintent.lua` Loads the Lua module `spintent.lua`, where every `_luafun_` function used throughout the package is registered.

```

70 \lua_load_module:n { spintent.lua }

```

(End of definition for `spintent.lua`.)

`\__spintent_luafun_set_debug_ghost:n` Passes `AFghost` to Lua as a plain string. When `true`, both ghost characters are marked `sptmp="outer"` or `sptmp="inner"` in the file `-luamml-mathml.html`, so an external script can identify and strip them before generating the Associated~Files structure for `tagged PDF` — `math-AF` mode does not want either ghost present.

```

71 \bool_if:NTF \l__spintent_debug_ghost_bool
72 { \__spintent_luafun_set_debug_ghost:n { true } }
73 { \__spintent_luafun_set_debug_ghost:n { false } }

```

(End of definition for `\__spintent_luafun_set_debug_ghost:n`.)

`\l__spintent_scope_attr_int` A LuaTeX attribute number, allocated collision-free from Lua via `luatexbase.new_attribute` and exposed here as an integer. Every command that uses the shared function `\__spintent_luafun_mrow_intent_arg:nn` activates it, via `\attribute \l__spintent_scope_attr_int = 1`, inside its own group — marking every node typeset within as eligible for that queue to claim. Without this mark, the queue could claim content from an unrelated command sitting in the same formula.

```

74 \int_new:N \l__spintent_scope_attr_int
75 \int_set:Nn \l__spintent_scope_attr_int { \l__spintent_luaset_scope_attr_str }

```

(End of definition for `\__spintent_scope_attr_int`.)

`\__spintent_luafun_spunit_lookup_alias:V` Variants generated for `\__spintent_luafun_spunit_lookup_alias:n` (used by `\spunit`), `\__spintent_luafun_spmoney_lookup_data:V` (used by `\spmoney`), `\__spintent_luafun_spmQ_scale_int:nn` (used by `\spmQ`) and `\__spintent_luafun_clean_split_and_set:V` (used by `\spnum`).

```

76 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
77 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
78 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
79 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

11.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdf_{TeX} for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

`mathcal-fallback`
`right-angle-sqr-fallback`

First, we'll define two error messages in case our fallback fails.

```

80 \hook_gput_code:nnn { begindocument } { spintent }
81 {
82   \__spintent_luafun_init_math_fallbacks:
83 }
84 \msg_new:nnn { spintent } { mathcal-fallback }
85 {
86   Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~
87   (font~not~found~or~no~free~math~family~available).~\msg_line_context:
88 }
89 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
90 {
91   Using~\token_to_str:N \rightangle \c_space_tl as~fallback~
92   for~the~right-angle~ glyph.~\msg_line_context:
93 }

```

(End of definition for `mathcal-fallback` and `right-angle-sqr-fallback`.)

`\__spintent_mathcal:n`

Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

94 \cs_new_protected:Nn \__spintent_mathcal:n
95 {
96   \bool_if:NTF \__spintent_mathcal_bool
97   {
98     \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
99     { \__spintent_luafun_classic_mathcal:n {#1} }
100     {
101       \msg_warning:nn { spintent } { mathcal-fallback }
102       \mathcal {#1}
103     }
104   }
105   { \mathcal {#1} }
106 }

```

(End of definition for `\__spintent_mathcal:n`.)

`\__spintent_right_angle_sqr:`

The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

107 \cs_new_protected:Nn \__spintent_right_angle_sqr:
108 {
109   \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
110   { \__spintent_luafun_right_angle_sqr: }
111   {
112     \msg_warning:nn { spintent } { right-angle-sqr-fallback }
113     \rightangle
114   }
115 }

```

(End of definition for `\__spintent_right_angle_sqr:`.)

11.8 Shared annotation mechanisms: the queue and the ghosts

Almost all commands in the `spintent` package share three annotation functions, the mechanism of which is defined in the Lua module `spintent.lua`.

`\__spintent_luafun_mrow_intent_arg:nn`

The internal function `\__spintent_luafun_mrow_intent_arg:nn` — called directly, with no `expl3` wrapper — pushes one entry onto a queue: a concept `intent` (`#1`, possibly empty) and a Lua table literal `arg=/intent=` (`#2`). A `pre_mlist_to_hlist_filter` callback (installed once, before `luamml[2]`’s own) walks the formula’s top-level noads in order and, for each one whose nucleus or subscript was typeset inside a `spintent` command’s own group (marked via a LuaTeX `\attribute` activated by every command that uses this queue), claims it for the next pending entry: `arg`/a literal `intent` land on that leaf, and, when the concept `intent` is non-empty, the leaf is wrapped in a native `<mrow>` carrying it — no token, no ghost involved. Entries are consumed strictly in order; any left unclaimed when the formula ends are discarded rather than carried into the next one. A native `\MathMLarg/\MathMLintent` claim is invisible to this queue: mixing the two for the same content risks the queue claiming something already tagged natively, or never being claimed at all.

(End of definition for `\__spintent_luafun_mrow_intent_arg:nn`.)

The other two are “ghost” characters (an invisible U+2063, self-tagged `:silent` through their own `mathml_filter` — so the queue above never mistakes either for one of its own free targets) dropped in as an extra sibling so the resulting `<mrow>` is not the formula’s sole top-level content, which `luamml` would otherwise promote straight to `<math intent=...>` instead of leaving it nested (issue #26).

`\__spintent_invisible_sep:`

Placed once, before opening the formula’s own outer `\MathMLintent` — protects the *whole formula* from collapsing when that outer group is its only top-level content. Fires at most once per document: past the first call, in any command, it does nothing, since one sibling anywhere in the formula is already enough.

```
116 \cs_new_protected:Nn \__spintent_invisible_sep:
117 {
118   \__spintent_luafun_invisible_sep:
119 }
```

🔗 Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="outer"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_invisible_sep:`.)

`\__spintent_inner_sep:`

Placed just inside a native `\MathMLintent` group — protects *that specific group* when it too would otherwise hold only one child. Never deduplicates: called once per such group, however many appear, nested or sibling, in the same formula.

```
120 \cs_new_protected:Nn \__spintent_inner_sep:
121 {
122   \__spintent_luafun_inner_sep:
123 }
```

🔗 Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="inner"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_inner_sep:`.)

`\c__spintent_invisible_sep_int`

A raw, untagged copy of the same U+2063 character, via the LuaTeX primitive `\Umathchardef` — for use as ordinary content *inside* an explicit `\MathMLintent{<intent>}` call (typically `:silent`), not as a sibling on its own.

```
124 \hook_gput_code:nnn { begindocument } { spintent }
125 {
126   \Umathchardef \c__spintent_invisible_sep_int = "0 "0 "2063
127 }
```

🔗 Documented in `source3` on page 887 for `\int_const:Nn`: “As stated, most constants can be defined as `\chardef` or `\mathchardef` but that’s engine dependent.” —so this appears to be the correct option here.

(End of definition for `\c__spintent_invisible_sep_int`.)

11.9 Normalize argument and affixes

We normalize the $\{\langle argument \rangle\}$ to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

128 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
129 {
130   \tl_if_in:NnT #1 { -- }
131   {
132     \tl_replace_all:Nnn #1 { -- } { - }
133     \__spintent_collapse_hyphens:N #1
134   }
135 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn` The function `\__spintent_sanitize_affix:Nn` checks if the $\{\langle argument \rangle\}$ passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘ ’ to hyphens ‘-’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string `foo-bar-baz` inside `\l__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```

136 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
137 {
138   \tl_if_blank:nTF {#2}
139   { \tl_clear:N #1 }
140   {
141     \tl_set:Ne #1 {#2}
142     \tl_trim_spaces:N #1
143     \tl_replace_all:Nnn #1 { ~ } { - }
144     \__spintent_collapse_hyphens:N #1
145     \tl_put_left:Nn #1 { \q_mark }
146     \tl_put_right:Nn #1 { \q_stop }
147     \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
148     \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
149     \tl_remove_all:Nn #1 { \q_mark }
150     \tl_remove_all:Nn #1 { \q_stop }
151   }
152 }
153 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }
```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.10 The command `\setspintent`

The `\setspintent` command is used to configure $\langle keys \rangle$ “globally” for the utilities provided by `spintent`.

11.10.1 Internal variables and error messages for `\setspintent`

`\l__spintent_setspintent_input_arg_cmd_tl` The variable `\l__spintent_setspintent_input_arg_cmd_tl` will store the command or command name passed to the “first” $\{\langle argument \rangle\}$ ‘#1’ of the command `\setspintent`.

```
154 \tl_new:N \l__spintent_setspintent_input_arg_cmd_tl
```

Error message if the $\{\langle argument \rangle\}$ ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

155 \msg_new:nnnn { spintent } { unknown-command-setup }
156 {
157   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
158 }
159 {
160   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
161   command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
162 }
```

(End of definition for `\l__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

__spintent_setspintent_set_keys:nn
__spintent_setspintent_set_keys:en

```

The function `__spintent_setspintent_set_keys:nn` will check if the command passed as `{⟨argument⟩}` exists in ‘#1’ and set the `⟨keys⟩` passed as `{⟨argument⟩}` in ‘#2’ to the correct path; otherwise, it will return an error.

```

163 \cs_new_protected:Nn __spintent_setspintent_set_keys:nn
164 {
165   \cs_if_exist:cTF { #1 }
166   {
167     \keys_set:nn { spintent / #1 } { #2 }
168   }
169   {
170     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
171   }
172 }
173 \cs_generate_variant:Nn __spintent_setspintent_set_keys:nn { en }

```

(End of definition for `__spintent_setspintent_set_keys:nn`.)

```

__spintent_setspintent_parse_args:nn
__spintent_setspintent_parse_args:Vn

```

The function `__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `__spintent_setspintent_set_keys:nn`.

```

174 \cs_new_protected:Nn __spintent_setspintent_parse_args:nn
175 {
176   \tl_if_single:nTF { #1 }
177   {
178     \token_if_cs:NTF #1
179     {
180       __spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
181     }
182     {
183       __spintent_setspintent_set_keys:nn { #1 } { #2 }
184     }
185   }
186   {
187     __spintent_setspintent_set_keys:nn { #1 } { #2 }
188   }
189 }
190 \cs_generate_variant:Nn __spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `__spintent_setspintent_parse_args:nn`.)

`\setspintent`

Now we define the user command `\setspintent`.

```

191 \NewDocumentCommand \setspintent { m m }
192 {
193   \tl_if_blank:nTF { #1 }
194   {
195     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
196   }
197   {
198     \tl_set:Nn \l__spintent_setspintent_input_arg_cmd_tl { #1 }
199     \tl_trim_spaces:N \l__spintent_setspintent_input_arg_cmd_tl
200     __spintent_setspintent_parse_args:Vn \l__spintent_setspintent_input_arg_cmd_tl { #2 }
201   }
202 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.11 Handling unknown keys

```

\l__spintent_command_name_str
unknown-choice
cmd-key-unknown
cmd-key-value-unknown

```

We define the variable `\l__spintent_command_name_str` which will store the “name” of each command within the implementation along with the messages `unknown-choice` used by `⟨keys⟩` that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the `⟨key⟩` `unknown` in commands that use `[⟨key = val⟩]`.

```

203 \str_new:N \l__spintent_command_name_str
204 \msg_new:nnn { spintent } { unknown-choice }
205 {
206   The~value~'!#3'~for~'!#1'~key~is~invalid~use~('!#2').
207 }
208 \msg_new:nnnn { spintent } { cmd-key-unknown }
209 {
210   The~key~'!#1'~is~unknown~by~command~

```

```

211     \c_backslash_str \l__spintent_command_name_str \c_space_tl
212     and~is~being~ignored~\msg_line_context:.
213 }
214 {
215     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
216     does~not~have~a~key~called~'#1'.\\
217     Check~that~you~have~spelled~the~key~name~correctly.
218 }
219 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
220 {
221     The~key~'#1=#2'~is~unknown~by~command~
222     \c_backslash_str \l__spintent_command_name_str \c_space_tl
223     and~is~being~ignored~\msg_line_context:.
224 }
225 {
226     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
227     does~not~have~a~key~called~'#1'.\\
228     Check~that~you~have~spelled~the~key~name~correctly.
229 }

```

(End of definition for `\l__spintent_command_name_str` and others.)

```

\__spintent_unknown_keys:nn
\__spintent_unknown_keys:Vn
\__spintent_unknown_keys:n

```

The internal functions `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n` used by the `<key>` unknown.

```

230 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
231 {
232     \tl_if_blank:nTF {#2}
233     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
234     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
235 }
236 \cs_generate_variant:Nn \__spintent_unknown_keys_cmd:nn { Vn }
237 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
238 {
239     \__spintent_unknown_keys_cmd:Vn \l_keys_key_str {#1}
240 }

```

(End of definition for `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n`.)

11.12 Definition of core base variables

```

\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:V

```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the `{<argument>}` passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “units”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘_str’ or ‘_tl’ described below to `true`, `false`, `error` or `stores` information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```

\l__spintent_luaset_sign_tl
\l__spintent_luaset_part_int_tl
\l__spintent_luaset_dec_sep_tl
\l__spintent_luaset_part_dec_tl
\l__spintent_luaset_part_period_tl
\l__spintent_luaset_has_integer_str
\l__spintent_luaset_has_natural_str

```

The variable `\l__spintent_luaset_sign_tl` stores the explicit ‘+’ or ‘−’ sign if present. The variable `\l__spintent_luaset_part_int_tl` stores the raw digit sequence of the “integer part”. The variable `\l__spintent_luaset_dec_sep_tl` stores the matched ‘,’ or ‘.’ used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` stores the raw digits of the “finite decimal part” that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` stores the digits of the “infinite periodic decimal part” that follow a semicolon ‘;’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

241 \tl_new:N \l__spintent_luaset_sign_tl
242 \tl_new:N \l__spintent_luaset_part_int_tl
243 \tl_new:N \l__spintent_luaset_dec_sep_tl
244 \tl_new:N \l__spintent_luaset_part_dec_tl
245 \tl_new:N \l__spintent_luaset_part_period_tl
246 \str_new:N \l__spintent_luaset_has_integer_str
247 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

```

\l__spintent_luaset_only_part_int_str
\l__spintent_luaset_only_part_dec_str
\l__spintent_luaset_only_part_period_str
\l__spintent_luaset_format_part_int_str
\l__spintent_luaset_format_part_dec_str

```

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are *stored* in `\l__spintent_luaset_only_part_int_str` for the “*integer part*”, in `\l__spintent_luaset_only_part_dec_str` for the “*finite decimal part*”, and in `\l__spintent_luaset_only_part_period_str` for the “*infinite periodic decimal part*”.

Formatted versions (`cifras-sep=true`) are *stored* in `\l__spintent_luaset_format_part_int_str` for the “*integer part*” and `\l__spintent_luaset_format_part_dec_str` for the “*finite decimal part*”.

```

248 \str_new:N \l__spintent_luaset_only_part_int_str
249 \str_new:N \l__spintent_luaset_only_part_dec_str
250 \str_new:N \l__spintent_luaset_only_part_period_str
251 \str_new:N \l__spintent_luaset_format_part_int_str
252 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

```

\l__spintent_luaset_millions_str
\l__spintent_luaset_decimal_period_str
\l__spintent_luaset_not_decimal_period_str
\l__spintent_luaset_not_decimal_not_period_str
\l__spintent_luaset_dec_is_all_zeros_str
\l__spintent_luaset_has_exponent_str
\l__spintent_luaset_exponent_tl

```

The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “*finite decimal part*” is completely zero (useful for “*pure periodic decimals*”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “*scientific notation*” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

253 \str_new:N \l__spintent_luaset_millions_str
254 \str_new:N \l__spintent_luaset_decimal_period_str
255 \str_new:N \l__spintent_luaset_not_decimal_period_str
256 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
257 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
258 \str_new:N \l__spintent_luaset_has_exponent_str
259 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

```

\l__spintent_luaset_has_units_str
\l__spintent_luaset_status_str
\l__spintent_luaset_arg_above_tl
\l__spintent_luaset_arg_below_tl
\l__spintent_luaset_denom_has_number_str

```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.14) and `\spunit` (§11.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “*units*”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

260 \str_new:N \l__spintent_luaset_has_units_str
261 \str_new:N \l__spintent_luaset_status_str
262 \tl_new:N \l__spintent_luaset_arg_above_tl
263 \tl_new:N \l__spintent_luaset_arg_below_tl
264 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

11.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

265 \msg_new:nnnn { spintent } { need-math-mode }
266 {
267   Math~mode~required~for~'#1'~\msg_line_context:.
268 }
269 {
270   Use~inside~$...$~or~in~math~environment.
271 }
272 \msg_new:nnnn { spintent } { need-text-mode }
273 {
274   Text~mode~required~for~'#1'~\msg_line_context:.
275 }
276 {
277   Use~outside~of~$...$~or~math~environment.
278 }

```

11.14 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the *intent functions* passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.14.1 Definition of keys and variables for `\spnum`

The variable `\l__spintenc_spnum_read_terse_bool` handled by the key `read-sign`, controls the *intent* passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “*positivo*” or “*negativo*” after reading the number; if set to `terse`, it will say “*más*” or “*menos*” before reading the number.

The variable `\l__spintenc_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the *intent* to `\MathMLintent`. If set to `coma`, it will say “*coma*” and print ‘,’; if set to `punto` it will say “*punto*” and print ‘.’.

The variable `\l__spintenc_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘.’ when set to `cdot`.

The variable `\l__spintenc_spnum_intent_read_sign_str` will *store* the spoken translation used in the *intent* function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l__spintenc_spnum_intent_read_dec_sep_str` *stores* the spoken translation that will be used in the *intent* function passed to `\MathMLintent` for reading the decimal separation as a “*coma*” or a “*punto*” according to the value found in the variable `\l__spintenc_luaset_dec_sep_tl`.

```
279 \bool_new:N \l__spintenc_spnum_read_terse_bool
280 \str_new:N \l__spintenc_spnum_read_period_sep_str
281 \tl_new:N \l__spintenc_spnum_notation_sym_tl
282 \str_new:N \l__spintenc_spnum_intent_read_sign_str
283 \str_new:N \l__spintenc_spnum_intent_read_dec_sep_str
```

(End of definition for `\l__spintenc_spnum_read_terse_bool` and others.)

```
cifras-sep      Now we add the keys cifras-sep, read-sign, read-period-sep and notation-sym.
read-sign
read-period-sep
notation-sym

284 \keys_define:nn { spintenc/spnum }
285 {
286   cifras-sep          .bool_set:N = \l__spintenc_spnum_cifras_sep_bool,
287   cifras-sep          .initial:n = true,
288   cifras-sep          .value_required:n = true,
289   read-sign           .choices:nn =
290     { terse , clear }
291     {
292       \int_case:nn { \l_keys_choice_int }
293       {
294         {1} { \bool_set_true:N \l__spintenc_spnum_read_terse_bool }
295         {2} { \bool_set_false:N \l__spintenc_spnum_read_terse_bool }
296       }
297     },
298   read-sign           .initial:n = terse,
299   read-sign           .value_required:n = true,
300   read-sign / unknown .code:n =
301     \msg_error:nneee { spintenc } { unknown-choice }
302     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
303   read-period-sep     .choices:nn =
304     { coma , punto }
305     {
306       \int_case:nn { \l_keys_choice_int }
307       {
308         {1} { \str_set:Nn \l__spintenc_spnum_read_period_sep_str { coma } }
309         {2} { \str_set:Nn \l__spintenc_spnum_read_period_sep_str { punto } }
310       }
311     },
312   read-period-sep     .initial:n = coma,
313   read-period-sep     .value_required:n = true,
314   read-period-sep / unknown .code:n =
315     \msg_error:nneee { spintenc } { unknown-choice }
316     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
317   notation-sym        .choices:nn =
318     { times , cdot }
```

```

319     {
320       \int_case:nn { \l_keys_choice_int }
321       {
322         {1} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \times } }
323         {2} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \cdot } }
324       }
325     },
326     notation-sym .initial:n = cdot,
327     notation-sym .value_required:n = true,
328     notation-sym / unknown .code:n =
329     \msg_error:nneee { spintent } { unknown-choice }
330     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
331     unknown .code:n =
332     {
333       \str_set:Nn \__spintent_command_name_str { snum }
334       \__spintent_unknown_keys_cmd:n {#1}
335     },
336   }

```

(End of definition for *cifras-sep* and others.)

11.14.2 Internal functions for \snum

```

\__spintent_snum_render_part:n
\__spintent_snum_render_int:
\__spintent_snum_render_dec:

```

The function `\__spintent_snum_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_snum_render_int:` and `\__spintent_snum_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintenc`. It evaluates whether the key *cifras-sep* is active or not.

```

337 \cs_new_protected:Nn \__spintent_snum_render_part:n
338 {
339   \MathMLintenc { \str_use:c { l__spintenc_luaset_only_part_#1_str } :number }
340   {
341     \bool_if:NTF \__spintenc_snum_cifras_sep_bool
342     { \str_use:c { l__spintenc_luaset_format_part_#1_str } }
343     { \tl_use:c { l__spintenc_luaset_part_#1_tl } }
344   }
345 }
346 \cs_new_protected:Nn \__spintenc_snum_render_int:
347 {
348   \__spintenc_snum_render_part:n { int }
349 }
350 \cs_new_protected:Nn \__spintenc_snum_render_dec:
351 {
352   \__spintenc_snum_render_part:n { dec }
353 }

```

(End of definition for `\__spintenc_snum_render_part:n`, `\__spintenc_snum_render_int:`, and `\__spintenc_snum_render_dec:`.)

```

\__spintenc_snum_render_only_period:
\__spintenc_snum_print_period_bar:
\__spintenc_snum_render_period_sep:

```

The function `\__spintenc_snum_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* :number passed to `\MathMLintenc` which will be used by `\MathMLarg` in the function `\__spintenc_snum_print_period_bar:`.

```

354 \cs_new_protected:Nn \__spintenc_snum_render_only_period:
355 {
356   \MathMLintenc { \l__spintenc_luaset_only_part_period_str :number }
357   {
358     \l__spintenc_luaset_part_period_tl
359   }
360 }

```

The function `\__spintenc_snum_print_period_bar:` inserts the function `\__spintenc_snum_render_only_period:` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintenc`, generating the correct spoken for the “infinite periodic decimal part”.

```

361 \cs_new_protected:Nn \__spintenc_snum_print_period_bar:
362 {
363   \MathMLintenc { _periódico:postfix($dp) }
364   {
365     {
366       \overline
367       {
368         \MathMLarg { dp } { \__spintenc_snum_render_only_period: }
369       }
370     }
371   }

```

```
372 }
```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```
373 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
374 {
375   \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
376   {
377     \MathMLintent { _coma:prefix($pp) }
378     {
379       \mathord{,}
380       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
381     }
382   }
383   {
384     \MathMLintent { _punto:prefix($pp) }
385     {
386       \mathord{.}
387       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
388     }
389   }
390 }
```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

```
\__spintent_snum_render_decimal_sep:
```

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```
391 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
392 {
393   \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
394   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
395   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
396   \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
397   {
398     \mathord { \l__spintent_luaset_dec_sep_tl }
399     \MathMLarg { dp }
400     {
401       \__spintent_inner_sep:
402       \__spintent_snum_render_dec:
403     }
404   }
405   % period part
406   \str_if_eq:VnTF \l__spintent_luaset_decimal_period_str { true }
407   {
408     \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
409     {
410       \__spintent_inner_sep:
411       \__spintent_snum_print_period_bar:
412     }
413     {
414       \MathMLintent { _con:prefix($pnum) }
415       {
416         \__spintent_inner_sep:
417         \MathMLarg { pnum }
418         {
419           \__spintent_snum_print_period_bar:
420         }
421       }
422     }
423   }
424 }
```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

```
\__spintent_snum_print_part_dec:
\__spintent_snum_print_part_period:
```

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”).

It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintenc_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

425 \cs_new_protected:Nn \__spintenc_snum_print_part_dec:
426 {
427   \tl_if_empty:NF \l__spintenc_luaset_part_dec_tl
428   {
429     \__spintenc_snum_render_decimal_sep:
430   }
431 }
432 \cs_new_protected:Nn \__spintenc_snum_print_part_period:
433 {
434   \tl_if_empty:NF \l__spintenc_luaset_part_period_tl
435   {
436     \str_if_eq:VnTF \l__spintenc_luaset_not_decimal_period_str { true }
437     { \__spintenc_snum_render_period_sep: }
438     {
439       \tl_if_empty:NT \l__spintenc_luaset_part_dec_tl
440       {
441         \__spintenc_snum_render_period_sep:
442       }
443     }
444   }
445 }

```

(End of definition for `\__spintenc_snum_print_part_dec:` and `\__spintenc_snum_print_part_period:`.)

```

\__spintenc_snum_print_integer:
\__spintenc_snum_print_num_start:
\__spintenc_snum_print_number_core:

```

The function `\__spintenc_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintenc_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintenc_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

 Note for `\__spintenc_invisible_sep:`.

```

446 \cs_new_protected:Nn \__spintenc_snum_print_integer:
447 {
448   \str_if_eq:VnTF \l__spintenc_luaset_not_decimal_not_period_str { true }
449   { \__spintenc_snum_render_int: }
450   {
451     \MathMLintent { \l__spintenc_luaset_only_part_int_str :number:prefix($next) }
452     {
453       \bool_if:NTF \l__spintenc_snum_cifras_sep_bool
454       { \l__spintenc_luaset_format_part_int_str }
455       { \l__spintenc_luaset_part_int_tl }
456       \MathMLarg { next }
457       {
458         \tl_if_empty:NTF \l__spintenc_luaset_part_dec_tl
459         { \__spintenc_snum_print_part_period: }
460         { \__spintenc_snum_print_part_dec: }
461       }
462     }
463   }
464 }
465 \cs_new_protected:Nn \__spintenc_snum_print_num_start:
466 {
467   \tl_if_empty:NTF \l__spintenc_luaset_sign_tl
468   {
469     \__spintenc_invisible_sep: % need
470     \__spintenc_snum_print_integer:
471   }
472   {
473     \bool_if:NTF \l__spintenc_snum_read_terse_bool
474     {
475       \str_if_eq:VnTF \l__spintenc_luaset_sign_tl { + }

```

```

476         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _más:prefix($n)} }
477         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _menos:prefix($n)} }
478     }
479     {
480         \str_if_eq:VnTF \__spinttent_luaset_sign_tl { + }
481         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _positivo:postfix($n) } }
482         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _negativo:postfix($n) } }
483     }
484     \__spinttent_invisible_sep: % need
485     \MathMLintent { \__spinttent_snum_intent_read_sign_str }
486     {
487         \__spinttent_luaset_sign_tl
488         \MathMLarg { n } { \__spinttent_snum_print_integer: }
489     }
490 }
491 }
492 \cs_new_protected:Nn \__spinttent_snum_print_number_core:
493 {
494     \__spinttent_snum_print_num_start:
495     \str_if_eq:VnTF \__spinttent_luaset_has_exponent_str { true }
496     {
497         \MathMLintent { _por } { \__spinttent_snum_notation_sym_tl }
498         10^{\__spinttent_luaset_exponent_tl }
499     }
500 }

```

(End of definition for `\__spinttent_snum_print_integer:`, `\__spinttent_snum_print_num_start:`, and `\__spinttent_snum_print_number_core:`.)

11.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

501 \msg_new:nnnn { spinttent } { spnum-has-units }
502 {
503     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~(''#1'').
504 }
505 {
506     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
507 }
508 \msg_new:nnnn { spinttent } { spnum-no-digits }
509 {
510     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
511 }
512 {
513     You~provided~''#1',~which~contains~no~digits.
514 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.14.4 Implementation of the `\spum` command

`\spnum` The public user command `\spnum` accepts an optional $[\langle key = val \rangle]$ argument in ‘#1’ and a mandatory $\langle rational\ number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spinttent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spinttent_snum_print_number_core:` to render the final number, and finally closes the group.

```

515 \NewDocumentCommand \spnum { O{} m }
516 {
517     \mode_if_math:TF
518     {
519         \group_begin:
520         \keys_set:nn { spinttent/spnum } { #1 }
521         \__spinttent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
522         \str_if_eq:VnTF \__spinttent_luaset_has_units_str { true }
523         { \msg_error:nnn { spinttent } { spnum-has-units } { #2 } }
524         {
525             \tl_if_empty:NTF \__spinttent_luaset_part_int_tl
526             { \msg_error:nnn { spinttent } { spnum-no-digits } { #2 } }
527             { \__spinttent_snum_print_number_core: }

```

```

528         }
529     \group_end:
530 }
531 { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
532 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the $\langle argument \rangle$ to Lua module `spintent.lua` via `__spintent_luafun_clean_split_and_set:n` to separate and clean the $\langle argument \rangle$.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.15.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

\__spintent_spunit_luaset_canonical_str
\__spintent_spunit_luaset_lookup_status_str
\__spintent_spunit_luaset_read_str
\__spintent_spunit_luaset_is_compact_str
\__spintent_spunit_luaset_compact_exp_str
\__spintent_spunit_luaset_is_sexagesimal_str
\__spintent_spunit_luaset_status_str

533 \str_new:N \__spintent_spunit_luaset_canonical_str
534 \str_new:N \__spintent_spunit_luaset_lookup_status_str
535 \str_new:N \__spintent_spunit_luaset_read_str
536 \str_new:N \__spintent_spunit_luaset_is_compact_str
537 \str_new:N \__spintent_spunit_luaset_compact_exp_str
538 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
539 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

\__spintent_spunit_exponent_tl
\__spintent_spunit_unit_name_tl
\__spintent_spunit_read_slash_str
\__spintent_spunit_exp_seq
\__spintent_spunit_mult_seq
\__spintent_spunit_is_mult_bool
\__spintent_spunit_read_cdod_bool

540 \tl_new:N \__spintent_spunit_exponent_tl
541 \tl_new:N \__spintent_spunit_unit_name_tl
542 \str_new:N \__spintent_spunit_read_slash_str
543 \seq_new:N \__spintent_spunit_exp_seq
544 \seq_new:N \__spintent_spunit_mult_seq
545 \bool_new:N \__spintent_spunit_is_mult_bool
546 \bool_new:N \__spintent_spunit_read_cdod_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

11.15.2 Definition of error messages

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the $\langle argument \rangle$, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

spunit-invalid-unit
spunit-no-units
inline-numeric-denominator

547 \msg_new:nnnn { spintent } { spunit-invalid-unit }
548 {
549     The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
550 }
551 {
552     Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
553 }
554 \msg_new:nnnn { spintent } { spunit-no-units }
555 {
556     The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
557 }
558 {
559     Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
560 }
561 \msg_new:nnnn { spintent } { inline-numeric-denominator }
562 {
563     Illegal~numeric~coefficient~in~inline~denominator~'#1'.
564 }
565 {
566     The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
567     does~not~allow~numbers~after~the~slash~().
568 }

```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

569 \keys_define:nn { spintent/spunit }
570 {
571   print-cdot      .bool_set:N = \l__spintent_spunit_print_cdot_bool,
572   print-cdot      .initial:n   = false,
573   print-cdot      .value_required:n = true,
574   read-cdot       .bool_set:N = \l__spintent_spunit_read_cdot_bool,
575   read-cdot       .initial:n   = false,
576   read-cdot       .value_required:n = true,
577   read-slash      .choices:nn =
578     { por , partido-por }
579   {
580     \int_case:nn { \l_keys_choice_int }
581     {
582       {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
583       {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
584     }
585   },
586   read-slash      .initial:n   = por,
587   read-slash      .value_required:n = true,
588   read-slash / unknown .code:n =
589     \msg_error:nneee { spintent } { unknown-choice }
590     { read-slash } { por , ~partido-por } { \exp_not:n {#1} },
591   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
592   cifras-sep      .value_required:n = true,
593   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
594   read-period-sep .value_required:n = true,
595   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
596   notation-sym    .value_required:n = true,
597   unknown         .code:n =
598     {
599       \str_set:Nn \l__spintent_command_name_str { spunit }
600       \__spintent_unknown_keys_cmd:n {#1}
601     },
602 }

```

(End of definition for `read-cdot` and others.)

11.15.4 Internal functions for \spunit

`\__spintent_spunit_format_canonical:` The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘*ℓ*’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

```

603 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
604 {
605   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
606   {
607     { ℓ } { \ell }
608   }
609   { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
610 }

```

(End of definition for `\__spintent_spunit_format_canonical:`.)

`\__spintent_spunit_process_base_exp:n` The function `\__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent, ‘*#1*’. It first looks up the unit in the dictionary; if not found, flags it as invalid. Otherwise, unless this is the first unit in a product (`\__spintent_spunit_is_mult_bool`), it handles the spacing and multiplication dot: `\cdot` when `print-cdot` is `true` (read as `por` when `read-cdot` is also `true`, silent otherwise), or a thin space `\,` when `print-cdot` is `false` (still read as `por` under `read-cdot`).

```

611 \cs_new_protected:Nn \__spintent_spunit_process_base_exp:n
612 {
613   \tl_clear:N \l__spintent_spunit_exponent_tl
614   \__spintent_luafun_spunit_lookup_alias:n { #1 }
615   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
616   {
617     \str_set:Nn \l__spintent_luaset_status_str { invalid }
618   }
619   {
620     \bool_if:NF \l__spintent_spunit_is_mult_bool

```

```

621     {
622       \bool_if:NTF \l__spinttent_spunit_print_cdot_bool
623       {
624         \bool_if:NTF \l__spinttent_spunit_read_cdot_bool
625         { \MathMLintent { por } { \cdot } }
626         { \MathMLintent { :silent } { \cdot } }
627       }
628       {
629         \bool_if:NTF \l__spinttent_spunit_read_cdot_bool
630         { \MathMLintent { por } { \, } }
631         { \, }
632       }
633     }

```

Then renders the base unit and its exponent: either a pre-defined compact exponent from Lua (`\l__spinttent_spunit_lua`) or by manually splitting ‘#1’ at the ‘^’ symbol otherwise.

```

634     \str_if_eq:VnTF \l__spinttent_spunit_lua_set_is_compact_str { true }
635     {
636       {
637         \MathMLintent { \l__spinttent_spunit_lua_set_read_str }
638         {
639           \l__spinttent_spunit_format_canonical:
640         }
641       }
642       ^ { \l__spinttent_spunit_lua_set_compact_exp_str }
643     }
644     {
645       \seq_set_split:Nnn \l__spinttent_spunit_exp_seq { ^ } { #1 }
646       \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_unit_name_tl
647       \seq_if_empty:NF \l__spinttent_spunit_exp_seq
648       {
649         \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_exponent_tl
650       }
651       \MathMLintent { \l__spinttent_spunit_lua_set_read_str }
652       {
653         \l__spinttent_spunit_format_canonical:
654       }
655       \tl_if_empty:NF \l__spinttent_spunit_exponent_tl
656       {
657         ^ { \l__spinttent_spunit_exponent_tl }
658       }
659     }
660     \bool_set_false:N \l__spinttent_spunit_is_mult_bool
661   }
662 }

```

(End of definition for `\l__spinttent_spunit_process_base_exp:n`.)

```

\l__spinttent_spunit_process_mult:n
\l__spinttent_spunit_process_mult:V

```

The function `\l__spinttent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the $\{ \langle \textit{argument} \rangle \}$ at each “star symbol” and applies `\l__spinttent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

663 \cs_new_protected:Nn \l__spinttent_spunit_process_mult:n
664 {
665   \bool_set_true:N \l__spinttent_spunit_is_mult_bool
666   \seq_set_split:Nnn \l__spinttent_spunit_mult_seq { * } { #1 }
667   \seq_map_function:NN \l__spinttent_spunit_mult_seq \l__spinttent_spunit_process_base_exp:n
668 }
669 \cs_generate_variant:Nn \l__spinttent_spunit_process_mult:n { V }

```

(End of definition for `\l__spinttent_spunit_process_mult:n`.)

```
\l__spinttent_print_spunit_hierarchy:
```

The function `\l__spinttent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

670 \cs_new_protected:Nn \l__spinttent_print_spunit_hierarchy:
671 {
672   \tl_if_empty:NF \l__spinttent_lua_set_arg_above_tl
673   {
674     \l__spinttent_spunit_process_mult:V \l__spinttent_lua_set_arg_above_tl
675     \tl_if_empty:NF \l__spinttent_lua_set_arg_below_tl
676     {
677       \MathMLintent { \l__spinttent_spunit_read_slash_str } { / }

```

```

678         \__spinttent_spunit_process_mult:V \l__spinttent_luaset_arg_below_tl
679     }
680 }
681 }

```

(End of definition for `\__spinttent_print_spunit_hierarchy:`.)

`\__spinttent_spunit_safe_exec:n`

The function `\__spinttent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{⟨argument⟩}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “*without any space in between*”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

682 \cs_new_protected:Nn \__spinttent_spunit_safe_exec:n
683 {
684     \str_if_eq:VnTF \l__spinttent_luaset_has_units_str { false }
685     { \msg_error:nnn { spinttent } { spunit-no-units } { #1 } }
686     {
687         \str_if_eq:VnTF \l__spinttent_luaset_status_str { valid }
688         {
689             \str_if_eq:VnTF
690             \l__spinttent_luaset_denom_has_number_str { true }
691             {
692                 \msg_error:nnn { spinttent }
693                 { inline-numeric-denominator } { #1 }
694             }
695             {
696                 \__spinttent_luafun_spunit_lookup_alias:V \l__spinttent_luaset_arg_above_tl
697                 \str_if_eq:VnTF \l__spinttent_spunit_luaset_is_sexagesimal_str { true }
698                 {
699                     \tl_if_empty:NF \l__spinttent_luaset_part_int_tl
700                     {
701                         \__spinttent_spnum_print_number_core:
702                     }
703                     \MathMLintent { \l__spinttent_spunit_luaset_read_str }
704                     {
705                         \mathord
706                         {
707                             \str_case:Vn \l__spinttent_spunit_luaset_canonical_str
708                             {
709                                 { " } { \prime \! \prime }
710                                 { ' } { \prime
711                                 { ° } { °
712                             }
713                         }
714                     }
715                 }
716                 {
717                     \tl_if_empty:NF \l__spinttent_luaset_part_int_tl
718                     {
719                         \__spinttent_spnum_print_number_core: \,
720                     }
721                     \__spinttent_print_spunit_hierarchy:
722                 }
723             }
724         }
725         { \msg_error:nnn { spinttent } { spunit-invalid-unit } { #1 } }
726     }
727 }

```

(End of definition for `\__spinttent_spunit_safe_exec:n`.)

11.15.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional `[⟨key = val⟩]` argument in ‘#1’ and a mandatory `{⟨unit expression⟩}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{⟨argument⟩}` to Lua module for parsing via `\__spinttent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spinttent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

728 \NewDocumentCommand \spunit { 0{ } m }

```

```

729 {
730   \mode_if_math:TF
731   {
732     \group_begin:
733     \keys_set:nn { spintent/spunit } { #1 }
734     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
735     \__spintent_spunit_safe_exec:n { #2 }
736     \group_end:
737   }
738   { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
739 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit`.

During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to TeX using the `\l__spintent_spunit_luaset_status_str` variable. If a collision is detected or the $\langle argument \rangle$ is invalid, the package throws the corresponding “error” message.

11.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

740 \msg_new:nnnn { spintent } { spnewunit-duplicate }
741 {
742   The~unit~symbol~or~alias~'#1'~is~already~defined!
743 }
744 {
745   You~cannot~redefine~with~\token_to_str:N \spnewunit.
746 }
747 \msg_new:nnnn { spintent } { spunitalias-duplicate }
748 {
749   The~alias~'#1'~is~already~defined~or~reserved!
750 }
751 {
752   You~cannot~redefine~an~existing~unit~or~another~alias.
753 }
754 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
755 {
756   The~expression~'#2'~for~alias~'#1'~is~invalid.
757 }
758 {
759   It~must~contain~only~defined~units,~exponents~and~one~'/'~.
760 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

761 \NewDocumentCommand \spnewunit { m m }
762 {
763   \mode_if_math:TF
764   {
765     \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
766   }
767   {
768     \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
769     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
770     {
771       \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
772     }
773   }
774 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias` The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘`#1`’ and the target expression ‘`#2`’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

775 \NewDocumentCommand \spunitalias { m m }
776 {
777   \mode_if_math:TF
778   {
779     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
780   }
781   {
782     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
783     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
784     {
785       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
786     }
787     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
788     {
789       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
790     }
791   }
792 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{\argument}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

\l__spintent_spmoney_luaset_symbol_str
\l__spintent_spmoney_luaset_position_str
\l__spintent_spmoney_luaset_sing_str
\l__spintent_spmoney_luaset_plur_str
\l__spintent_spmoney_luaset_conde_str
\l__spintent_spmoney_luaset_subsing_str
\l__spintent_spmoney_luaset_subplur_str
\l__spintent_spmoney_luaset_print_iso_str
\l__spintent_spmoney_luaset_currency_arg_str
\l__spintent_spmoney_luaset_status_str

793 \str_new:N \l__spintent_spmoney_luaset_symbol_str
794 \str_new:N \l__spintent_spmoney_luaset_position_str
795 \str_new:N \l__spintent_spmoney_luaset_sing_str
796 \str_new:N \l__spintent_spmoney_luaset_plur_str
797 \str_new:N \l__spintent_spmoney_luaset_conde_str
798 \str_new:N \l__spintent_spmoney_luaset_subsing_str
799 \str_new:N \l__spintent_spmoney_luaset_subplur_str
800 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
801 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
802 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

```

\l__spintent_spmoney_sub_units_bool
\l__spintent_spmoney_intent_money_str
\l__spintent_spmoney_intent_sign_str
\l__spintent_spmoney_intent_sub_unit_str
\l__spintent_spmoney_res_main_voice_str
\l__spintent_spmoney_subnum_int
\l__spintent_spmoney_extracted_curr_str
\l__spintent_spmoney_extracted_num_tl

803 \bool_new:N \l__spintent_spmoney_sub_units_bool
804 \str_new:N \l__spintent_spmoney_intent_money_str
805 \str_new:N \l__spintent_spmoney_intent_sign_str
806 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
807 \str_new:N \l__spintent_spmoney_res_main_voice_str
808 \int_new:N \l__spintent_spmoney_subnum_int
809 \str_new:N \l__spintent_spmoney_extracted_curr_str
810 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

11.17.1 Definition of error messages

currency-no-subunits
invalid-currency

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```

811 \msg_new:nnnn { spintent } { currency-no-subunits }
812 {
813     The~currency~'#1'~does~not~support~subunits.~
814     The~option~'sub-units=true'~is~ignored.
815 }
816 {
817     Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
818     do~not~have~fractional~subunits~registered~in~Lua~module.~
819 }
820 \msg_new:nnnn { spintent } { invalid-currency }
821 {
822     Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
823 }
824 {
825     The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
826     the~Lua~module.
827 }
```

(End of definition for currency-no-subunits and invalid-currency.)

11.17.2 Auxiliary functions for \spmoney

__spintenc_spmoney_set_currency:n
__spintenc_spmoney_normalize_key:n
__spintenc_spmoney_normalize_key:V

The function `\__spintenc_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintenc_spmoney_normalize_key:n`, which interfaces with the Lua module `spintenc.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintenc_spmoney_luaset_currency_arg_str` for subsequent processing.

```

828 \cs_new_protected:Nn \__spintenc_spmoney_set_currency:n
829 {
830     \__spintenc_spmoney_normalize_key:n {#1}
831     \str_if_eq:VnTF \__spintenc_spmoney_luaset_status_str { notfound }
832     {
833         \msg_error:nnn { spintent } { invalid-currency } {#1}
834     }
835     { \str_set:Nn \__spintenc_spmoney_luaset_currency_arg_str {#1} }
836 }
837 \cs_new_protected:Nn \__spintenc_spmoney_normalize_key:n
838 {
839     \__spintenc_luafun_spmoney_normalize_key:n { #1 }
840 }
841 \cs_generate_variant:Nn \__spintenc_spmoney_normalize_key:n { V }
```

(End of definition for __spintenc_spmoney_set_currency:n and __spintenc_spmoney_normalize_key:n.)

11.17.3 Definition of keys for \spmoney

currency
sub-units
dolar-math
cifras-sep

Now we add the keys `currency`, `sub-units`, `dolar-math` and `cifras-sep`.

```

842 \keys_define:nn { spintenc / spmoney }
843 {
844     currency      .code:n      = { \__spintenc_spmoney_set_currency:n {#1} },
845     currency      .initial:n    = { pesos },
846     currency      .value_required:n = true,
847     sub-units     .bool_set:N   = \__spintenc_spmoney_sub_units_bool,
848     sub-units     .value_required:n = true,
849     dolar-math    .bool_set:N   = \__spintenc_spmoney_dolar_math_bool,
850     dolar-math    .initial:n    = { true },
851     dolar-math    .value_required:n = true,
852     cifras-sep    .meta:nn     = { spintenc/spnum } { cifras-sep = {#1} },
853     cifras-sep    .value_required:n = true,
854     unknown       .code:n      =
855     {
856         \str_set:Nn \__spintenc_command_name_str { spmoney }
857         \__spintenc_unknown_keys_cmd:n {#1}
858     },
859 }
```

(End of definition for currency and others.)

11.17.4 Internal functions for \spmoney

__spintenc_spmoney_validate_money:

The function `\__spintenc_spmoney_validate_money`: verifies if the parsed input contains an embedded currency identifier by checking `\l__spintenc_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\l__spintenc_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\l__spintenc_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintenc.lua` via `\__spintenc_luafun_spmoney_lookup_data:V`.

```

860 \cs_new_protected:Nn \__spintenc_spmoney_validate_money:
861 {
862   \str_if_eq:VnT \l__spintenc_luaset_has_units_str { true }
863   {
864     \__spintenc_spmoney_normalize_key:V \l__spintenc_luaset_arg_above_tl
865     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_status_str { notfound }
866     { \msg_error:nne { spintenc } { invalid-currency } { \l__spintenc_luaset_arg_above_tl } }
867     {
868       \str_set:NV
869         \l__spintenc_spmoney_luaset_currency_arg_str
870         \l__spintenc_luaset_arg_above_tl
871     }
872   }
873   \str_set:Ne \l__spintenc_spmoney_luaset_status_str { found }
874   \__spintenc_luafun_spmoney_lookup_data:V \l__spintenc_spmoney_luaset_currency_arg_str
875 }

```

(End of definition for __spintenc_spmoney_validate_money:.)

__spintenc_spmoney_print_symbol:

The function `\__spintenc_spmoney_print_symbol`: renders the “*currency symbol*”. If `\l__spintenc_spmoney_luaset_print_iso_bool` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”. For dollar signs ‘\$’, it checks `\l__spintenc_spmoney_dolar_math_bool` to determine whether to render it directly as a math character ‘\$’ or wrap it inside `\text`.

```

876 \cs_new_protected:Nn \__spintenc_spmoney_print_symbol:
877 {
878   \str_if_eq:VnTF \l__spintenc_spmoney_luaset_print_iso_str { true }
879   { \mathrm { \l__spintenc_spmoney_luaset_currency_arg_str } }
880   {
881     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_symbol_str { $ }
882     {
883       \bool_if:NTF \l__spintenc_spmoney_dolar_math_bool
884       { \$ } { \text { \$ } }
885     }
886     { \l__spintenc_spmoney_luaset_symbol_str }
887   }
888 }

```

(End of definition for __spintenc_spmoney_print_symbol:.)

__spintenc_spmoney_render_layout:

The function `\__spintenc_spmoney_render_layout`: determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\l__spintenc_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

889 \cs_new_protected:Nn \__spintenc_spmoney_render_layout:
890 {
891   \str_if_eq:VnTF \l__spintenc_spmoney_luaset_position_str { pre }
892   {
893     \__spintenc_spmoney_print_symbol:
894     \MathMLarg { n } { \__spintenc_spmoney_render_number_node: }
895   }
896   {
897     \MathMLarg { n } { \__spintenc_spmoney_render_number_node: }
898     \__spintenc_spmoney_print_symbol:
899   }
900 }

```

(End of definition for __spintenc_spmoney_render_layout:.)

__spintenc_spmoney_render_number_node:
 __spintenc_spmoney_render_only_integer_node:

The function `\__spintenc_spmoney_render_number_node`: renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintenc_spmoney_render_only_integer_node` outputs exclusively the integer portion, omitting any decimal data.

```

901 \cs_new_protected:Nn \__spintenc_spmoney_render_number_node:
902 {
903   \__spintenc_spmoney_render_int:

```

```

904     \__spintent_snum_print_part_dec:
905   }
906 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
907 {
908     \__spintent_snum_render_int:
909 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`.)

`\__spintent_spmoney_print_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

```

910 \cs_new_protected:Nn \__spintent_spmoney_print_int:
911 {
912     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
913     {
914         \bool_if:NTF \l__spintent_spmoney_sub_units_bool
915         { \__spintent_spmoney_set_and_print_sub_units: }
916         { \__spintent_spmoney_intent_int: }
917     }
918     {
919         \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
920         {
921             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
922         }
923         {
924             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
925         }
926         \__spintent_invisible_sep:
927         \MathMLintent { \l__spintent_spmoney_intent_sign_str }
928         {
929             \l__spintent_luaset_sign_tl
930             \MathMLarg { n }
931             {
932                 \bool_if:NTF \l__spintent_spmoney_sub_units_bool
933                 { \__spintent_spmoney_set_and_print_sub_units: }
934                 { \__spintent_spmoney_intent_int: }
935             }
936         }
937     }
938 }

```

(End of definition for `\__spintent_spmoney_print_int:`.)

`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix `MathML` intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

939 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
940 {
941     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
942     {
943         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
944         {
945             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
946         }
947         {
948             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
949         }
950         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
951         {
952             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
953         }
954     }
955     {
956         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
957         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
958         {
959             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
960             {

```

```

961         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_con
962     }
963     {
964         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plu
965     }
966 }
967 }
968 \__spintent_invisible_sep:
969 \MathMLint { \l__spintent_spmoney_intent_money_str :postfix($n) }
970 {
971     \__spintent_spmoney_render_layout:
972 }
973 }

```

(End of definition for `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`
`spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “*subunits*”. If the database returns empty subunit data, it triggers a warning and automatically disables the “*subunit*” rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “*subunits*” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘5’ to ‘50’), evaluates the correct singular or plural inflections for both the main “*currency*” and the “*subunits*”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

974 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
975 {
976     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
977     {
978         \msg_warning:nne { spintent } { currency-no-subunits }
979         { \l__spintent_spmoney_luaset_currency_arg_str }
980         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
981     }
982 }
983 \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
984 {
985     \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str == 1 }
986     {
987         \int_set:Nn \l__spintent_spmoney_subnum_int
988         { \l__spintent_luaset_only_part_dec_str * 10 }
989         \str_set:Ne \l__spintent_luaset_only_part_dec_str
990         { \l__spintent_luaset_only_part_dec_str 0 }
991     }
992     {
993         \int_set:Nn \l__spintent_spmoney_subnum_int
994         { \l__spintent_luaset_only_part_dec_str }
995     }
996     \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
997     {
998         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
999         { \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
1000     }
1001     {
1002         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
1003         { \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
1004     }
1005     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
1006     {
1007         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1008         \l__spintent_spmoney_luaset_sing_str
1009     }
1010     {
1011         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1012         \l__spintent_spmoney_luaset_plur_str
1013     }
1014     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
1015     {
1016         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1017         \l__spintent_spmoney_luaset_conde_str
1018     }
1019     \__spintent_spmoney_build_tree:
1020 }

```

(End of definition for `\__spintent_spmoney_check_sub_units:` and `\__spintent_spmoney_set_and_print_sub_units:`.)

```
\__spintent_spmoney_build_tree:
\__spintent_spmoney_print_sep_con:
\__spintent_spmoney_print_dec_sub_unit:
```

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “*currencies*”. It arranges the integer component, the decimal separator, and the “*subunit*” component based on the layout position (pre or post) of the “*currency symbol*”.

Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “*subunit*” intent to the decimal digits, ensuring accurate accessibility output.

```
1021 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1022 {
1023   \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
1024   {
1025     \MathMLintent { \__spintent_spmoney_res_main_voice_str :postfix($n) }
1026     {
1027       \__spintent_spmoney_print_symbol:
1028       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1029     }
1030     \__spintent_spmoney_print_sep_con:
1031     \__spintent_spmoney_print_dec_sub_unit:
1032   }
1033   {
1034     \MathMLintent { \__spintent_spmoney_res_main_voice_str :postfix($n) }
1035     {
1036       \__spintent_inner_sep:
1037       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1038     }
1039     \__spintent_spmoney_print_sep_con:
1040     \__spintent_spmoney_print_dec_sub_unit:
1041     \MathMLintent {:silent} { \__spintent_spmoney_print_symbol: }
1042   }
1043 }
1044 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1045 {
1046   \MathMLintent { _con }
1047   {
1048     \mathord { \__spintent_luaset_dec_sep_tl }
1049   }
1050 }
1051 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1052 {
1053   \MathMLintent { \__spintent_spmoney_intent_sub_unit_str }
1054   {
1055     \__spintent_inner_sep:
1056     \MathMLarg { sub }
1057     {
1058       \__spintent_spmoney_render_dec:
1059     }
1060   }
1061 }
```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`.)

```
\__spintent_spmoney_parse_and_route:n
```

The internal function `\__spintent_spmoney_parse_and_route:n` processes the `{\argument}` passed in ‘#1’ using Lua module to extract the numeric value and any “*currency*” identifier. If a “*currency*” is found (`\__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “*currency*”. If “*subunit*” rendering is enabled (`\__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```
1062 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1063 {
1064   \__spintent_luafun_spmoney_prepare_input:n { #1 }
1065   \__spintent_luafun_clean_split_and_set:V \__spintent_spmoney_extracted_num_tl
1066   \str_if_empty:NF \__spintent_spmoney_extracted_curr_str
1067   {
1068     \str_set:Ne \__spintent_luaset_has_units_str { true }
1069     \str_set:NV \__spintent_luaset_arg_above_tl \__spintent_spmoney_extracted_curr_str
1070   }
1071   \__spintent_spmoney_validate_money:
1072   \bool_if:NT \__spintent_spmoney_sub_units_bool
```

```

1073     { \__spintent_spmoney_check_sub_units: }
1074     \__spintent_spmoney_print_int:
1075 }

```

(End of definition for __spintent_spmoney_parse_and_route:n.)

\spmoney The public document command **\spmoney** formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local $\TeX$ group to safely isolate the key assignments, delegates the raw input string to **__spintent_spmoney_parse_and_route:n** for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1076 \NewDocumentCommand \spmoney { 0{} m }
1077 {
1078     \mode_if_math:TF
1079     {
1080         \group_begin:
1081         \keys_set:nn { spintent / spmoney } { #1 }
1082         \__spintent_spmoney_parse_and_route:n { #2 }
1083         \group_end:
1084     }
1085     { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1086 }

```

(End of definition for \spmoney. This function is documented on page 6.)

11.18 The command \spshort

The user command **\spshort** is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on **MathML** attributes, this command uses the PDF *tagging tools* provided by the **tagpdf**[29] package. Specifically, it uses the **E** (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module **spintent.lua**. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the **E** key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1087 \str_new:N \__spintent_spshort_luaset_status_str
1088 \str_new:N \__spintent_spshort_luaset_layout_str
1089 \str_new:N \__spintent_spshort_luaset_expanded_str
1090 \str_new:N \__spintent_spshort_luaset_output_str
1091 \str_new:N \__spintent_spshort_luaset_base_str
1092 \str_new:N \__spintent_spshort_luaset_suffix_str

```

(End of definition for __spintent_spshort_luaset_status_str and others.)

11.18.1 Definition of error messages for \spshort

unknown-abbreviation The internal “error” message **unknown-abbreviation** is triggered when the command **\spshort** encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided $\langle argument \rangle$ cannot be mapped to any registered database entry in **spintent.lua** nor successfully resolved through the grammatical ordinal expansion rules.

```

1093 \msg_new:nnnn { spintent } { unknown-abbreviation }
1094 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
1095 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for unknown-abbreviation.)

11.18.2 Accessibility interface

__spintent_spshort_tag_span:nn The internal function **__spintent_spshort_tag_span:nn** processes the $\langle argument \rangle$ passed in ‘#2’ using the tools provided by **tagpdf**. By utilizing the **tag** and **E** keys, it creates a native PDF /E (#1) structure and expands the $\langle argument \rangle$ passed in ‘#1’ for proper reading with NVDA.

```

1096 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1097 {
1098     \tag_mc_end_push:
1099     \tag_struct_begin:n { tag=Span, E={ #1 } }
1100     \tag_mc_begin:n { tag=Span }
1101     \tag_suspend:n { \spshort }
1102     #2
1103     \tag_resume:n { \spshort }
1104     \tag_mc_end:

```

```

1105     \tag_struct_end:
1106     \tag_mc_begin_pop:n {}
1107 }
1108 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for __spintent_spshort_tag_span:nn.)

11.18.3 Definition of keys for \spshort

Now we add the keys `small-caps` and `read-arg`.

```

small-caps read-arg 1109 \keys_define:nn { spintent / spshort }
1110 {
1111     small-caps .bool_set:N = \__spintent_spshort_smallcaps_bool,
1112     small-caps .initial:n = false,
1113     small-caps .value_required:n = true,
1114     read-arg .str_set:N = \__spintent_spshort_read_arg_str,
1115     read-arg .initial:n = {},
1116     read-arg .value_required:n = true,
1117     unknown .code:n =
1118     {
1119         \str_set:Nn \__spintent_command_name_str { spshort }
1120         \__spintent_unknown_keys_cmd:n {#1}
1121     },
1122 }

```

(End of definition for `small-caps` and `read-arg`.)

11.18.4 Internal functions for \spshort

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional `<keys>` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `{<argument>}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```

1123 \cs_new_protected:Nn \__spintent_spshort_print:nn
1124 {
1125     \str_clear:N \__spintent_spshort_read_arg_str
1126     \keys_set:nn { spintent / spshort } {#1}
1127     \str_if_empty:NTF \__spintent_spshort_read_arg_str
1128     {
1129         \__spintent_spshort_process_and_render:n {#2}
1130     }
1131     {
1132         \__spintent_spshort_render_bypass:n {#2}
1133     }
1134 }

```

(End of definition for `\__spintent_spshort_print:nn`.)

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{<argument>}` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine `success`, `fallback` or `error`. Any unrecognized status automatically defaults to the “error” handler.

```

1135 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1136 {
1137     \__spintent_luafun_spshort_process:n {#1}
1138     \str_case:VnF \__spintent_spshort_luaset_status_str
1139     {
1140         { success } { \__spintent_spshort_render_success: }
1141         { fallback } { \__spintent_spshort_render_fallback: }
1142         { error } { \__spintent_spshort_render_error:n {#1} }
1143     }
1144     { \__spintent_spshort_render_error:n {#1} }
1145 }

```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

```

\__spintent_spshort_render_success:
\__spintent_spshort_render_fallback:
\__spintent_spshort_render_bypass:n
\__spintent_spshort_render_error:n

```

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “*speech*” to the $\{\langle argument \rangle\}$, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_rende` triggers a structural exception while yielding the unformatted $\{\langle argument \rangle\}$ to prevent fatal compilation.

```

1146 \cs_new_protected:Nn \__spintent_spshort_render_success:
1147 {
1148   \__spintent_spshort_tag_span:en
1149   { \l__spintent_spshort_luaset_expanded_str }
1150   { \__spintent_spshort_execute_layout: }
1151 }
1152 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1153 {
1154   \__spintent_spshort_tag_span:en
1155   { \l__spintent_spshort_luaset_expanded_str }
1156   {
1157     \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1158     { \textsc{ \l__spintent_spshort_luaset_output_str } }
1159     { \l__spintent_spshort_luaset_output_str }
1160   }
1161 }
1162 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1163 {
1164   \__spintent_spshort_tag_span:en { \l__spintent_spshort_read_arg_str } { #1 }
1165 }
1166 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1167 {
1168   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1169 }

```

(End of definition for `\__spintent_spshort_render_success:` and others.)

```

\__spintent_spshort_execute_layout:

```

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\l__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\text{\LaTeX}$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional `SMALL CAPS` based on the active boolean variables.

```

1170 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1171 {
1172   \str_case:VnF \l__spintent_spshort_luaset_layout_str
1173   {
1174     { linear_regular }
1175     {
1176       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1177       { \textsc{ \l__spintent_spshort_luaset_output_str } }
1178       { \l__spintent_spshort_luaset_output_str }
1179     }
1180     { linear_caps }
1181     { \l__spintent_spshort_luaset_output_str }
1182     { linear_mixed }
1183     { \l__spintent_spshort_luaset_output_str }
1184     { superscript }
1185     {
1186       \l__spintent_spshort_luaset_base_str
1187       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1188       {
1189         \textsuperscript{
1190           \textsc{
1191             \str_lowercase:V \l__spintent_spshort_luaset_suffix_str
1192           }
1193         }
1194       }
1195       {
1196         \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
1197       }
1198     }
1199     { small_caps_pure }
1200     {
1201       \bool_if:NTF \l__spintent_spshort_smallcaps_bool

```

```

1202         {
1203             \textsc{
1204                 \str_lowercase:V \l__spintent_spsshort_luaset_output_str
1205             }
1206         }
1207         { \l__spintent_spsshort_luaset_output_str }
1208     }
1209     { acronym_long }
1210     { \l__spintent_spsshort_luaset_output_str }
1211 }
1212 { \l__spintent_spsshort_luaset_output_str }
1213 }

```

(End of definition for `\l__spintent_spsshort_execute_layout:`.)

`\spsshort` The public document command `\spsshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\l__spintent_spsshort_print:nn` for processing and printing.

```

1214 \NewDocumentCommand \spsshort { 0{} m }
1215 {
1216     \mode_if_math:TF
1217     {
1218         \msg_error:nnn { spintent } { need-text-mode } { \spsshort }
1219     }
1220     {
1221         \mode_leave_vertical:
1222         \group_begin:
1223         \l__spintent_spsshort_print:nn {#1} {#2}
1224         \group_end:
1225     }
1226 }

```

(End of definition for `\spsshort`. This function is documented on page 8.)

11.19 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as `{\argument}`, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible MathML intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

`\l__spintent_spsiglo_luaset_error_str` These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```

1227 \str_new:N \l__spintent_spsiglo_luaset_error_str
1228 \str_new:N \l__spintent_spsiglo_luaset_arabic_str
1229 \str_new:N \l__spintent_spsiglo_luaset_roman_min_str
1230 \str_new:N \l__spintent_spsiglo_print_era_str

```

(End of definition for `\l__spintent_spsiglo_luaset_error_str` and others.)

11.19.1 Definition of error messages for `\spsiglo`

`invalid-century-format` This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the `{\argument}` to an integer within the valid range of 1 to 4000, preventing incorrect speech synthesis.

```

1231 \msg_new:nnnn { spintent } { invalid-century-format }
1232 {
1233     Invalid~century~format~in~'#1'.
1234 }
1235 {
1236     The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1237     Ensure~the~value~is~between~1~and~4000.
1238 }

```

(End of definition for `invalid-century-format`.)

11.19.2 Definition of keys for \spsiglo

`print-era` Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1239 \keys_define:nn { spintent / spsiglo }
1240 {
1241   print-era           .choices:nn =
1242     { none , ac , dc }
1243     {
1244       \int_case:nn { \l_keys_choice_int }
1245       {
1246         {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1247         {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1248         {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1249       }
1250     },
1251   print-era           .initial:n = none,
1252   print-era           .value_required:n = true,
1253   print-era / unknown .code:n =
1254     \msg_error:nneee { spintent } { unknown-choice }
1255     { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1256   unknown             .code:n =
1257     {
1258       \str_set:Nn \l__spintent_command_name_str { spsiglo }
1259       \__spintent_unknown_keys_cmd:n {#1}
1260     },
1261 }

```

(End of definition for `print-era`.)

11.19.3 Internal functions for \spsiglo

`\__spintent_spsiglo_render_simple:nn` The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1262 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1263 {
1264   \MathMLintent { #1:number } { \text { \textsc {#2} } } }
1265 }
1266 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1267 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1268 {
1269   \str_if_eq:nnTF { #1 } { ac }
1270   {
1271     \MathMLintent { antes-de-cristo:postfix($z) }
1272     {
1273       \__spintent_inner_sep:
1274       \MathMLarg { z }
1275       {
1276         \MathMLintent { #2:number }
1277         {
1278           \text { \textsc {#3}~a.~C. }
1279         }
1280       }
1281     }
1282   }
1283   {
1284     \MathMLintent { después-de-cristo:postfix($z) }
1285     {
1286       \__spintent_inner_sep:
1287       \MathMLarg { z }
1288       {
1289         \MathMLintent { #2:number }
1290         {
1291           \text { \textsc {#3}~d.~C. }
1292         }
1293       }
1294     }
1295   }
1296 }

```

```
1297 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }
```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

```
\__spintent_spsiglo_tag_span:nn
\__spintent_spsiglo_tag_span:en
```

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (`#1`) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```
1298 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1299 {
1300   \tag_mc_end_push:
1301   \tag_struct_begin:n { tag=Span, alt={ #1 } }
1302   \tag_mc_begin:n { tag=Span }
1303   \tag_suspend:n { \spsiglo }
1304   #2
1305   \tag_resume:n { \spsiglo }
1306   \tag_mc_end:
1307   \tag_struct_end:
1308   \tag_mc_begin_pop:n {}
1309 }
1310 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }
```

(End of definition for `\__spintent_spsiglo_tag_span:nn`.)

```
\__spintent_spsiglo_print_math:n
\__spintent_spsiglo_print_text:n
```

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the `{⟨argument⟩}` is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```
1311 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1312 {
1313   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1314   {
1315     \msg_error:nnnn { spintent } { invalid-century-format }
1316     { \spsiglo } { #1 }
1317   }
1318   {
1319     \str_if_eq:VnTF \l__spintent_spsiglo_print_era_str { none }
1320     {
1321       \__spintent_spsiglo_render_simple:VV
1322       \l__spintent_spsiglo_luaset_arabic_str
1323       \l__spintent_spsiglo_luaset_roman_min_str
1324     }
1325     {
1326       \__spintent_spsiglo_render_with_era:VVV
1327       \l__spintent_spsiglo_print_era_str
1328       \l__spintent_spsiglo_luaset_arabic_str
1329       \l__spintent_spsiglo_luaset_roman_min_str
1330     }
1331   }
1332 }
1333 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1334 {
1335   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1336   {
1337     \msg_error:nnnn { spintent } { invalid-century-format }
1338     { \spsiglo } { #1 }
1339   }
1340   {
1341     \str_case:Vn \l__spintent_spsiglo_print_era_str
1342     {
1343       { none }
1344       {
1345         \__spintent_spsiglo_tag_span:en
1346         { \l__spintent_spsiglo_luaset_arabic_str }
1347         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
1348       }
1349       { ac }
```

```

1350         {
1351             \__spintent_spsiglo_tag_span:en
1352             { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1353             { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1354         }
1355         { dc }
1356         {
1357             \__spintent_spsiglo_tag_span:en
1358             { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1359             { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1360         }
1361     }
1362 }
1363 }

```

(End of definition for __spintent_spsiglo_print_math:n and __spintent_spsiglo_print_text:n)

\spsiglo The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional *(keys)* passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1364 \NewDocumentCommand \spsiglo { 0{} m }
1365 {
1366     \group_begin:
1367     \keys_set:nn { spintent / spsiglo } { #1 }
1368     \__spintent_luafun_spsiglo_parse:n { #2 }
1369     \mode_if_math:TF
1370     { \__spintent_spsiglo_print_math:n { #2 } }
1371     {
1372         \mode_leave_vertical:
1373         \__spintent_spsiglo_print_text:n { #2 }
1374     }
1375     \group_end:
1376 }

```

(End of definition for `\spsiglo`. This function is documented on page 10.)

11.20 The command \sptime

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\l__spintent_sptime_luaset_seconds_str
\l__spintent_sptime_luaset_has_seconds_str
\l__spintent_sptime_luaset_error_str
\l__spintent_sptime_luaset_is_fraction_str
\l__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1377 \str_new:N \l__spintent_sptime_luaset_seconds_str
1378 \str_new:N \l__spintent_sptime_luaset_has_seconds_str
1379 \str_new:N \l__spintent_sptime_luaset_error_str
1380 \str_new:N \l__spintent_sptime_luaset_is_fraction_str
1381 \str_new:N \l__spintent_sptime_luaset_base_str

```

(End of definition for `\l__spintent_sptime_luaset_seconds_str` and others.)

11.20.1 Definition of error messages

`invalid-time-format`

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1382 \msg_new:nnnn { spintent } { invalid-time-format }
1383 {
1384     Invalid~time~format~in~#1. \\\
1385     The~argument~'~#2'~does~not~match~any~supported~time~format.
1386 }
1387 {
1388     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1389     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1390 }

```

(End of definition for `invalid-time-format`.)

11.20.2 Internal functions for \sptime

__spintent_sptime_build_seconds:nn

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1391 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1392 {
1393   \MathMLintent { #1:prefix($z) }
1394   {
1395     \text { : }
1396     \MathMLarg { z }
1397     {
1398       \MathMLintent { _segundos:postfix($m) }
1399       {
1400         \__spintent_inner_sep:
1401         \MathMLarg { m }
1402         {
1403           \text { #2 }
1404         }
1405       }
1406     }
1407   }
1408 }

```

(End of definition for __spintent_sptime_build_seconds:nn.)

__spintent_sptime_build_tree:

The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\l__spintent_sptime_luaset_has_seconds_str` and `\l__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:` with the appropriate prefix (‘con’ or ‘minutos-con’), or simply appends the ‘minutos’ intent tag if no seconds are present.

```

1409 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1410 {
1411   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1412   {
1413     \MathMLintent { :time }
1414     {
1415       \text { \l__spintent_sptime_luaset_base_str }
1416     }
1417     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1418     {
1419       \__spintent_sptime_build_seconds:nn { con }
1420       { \l__spintent_sptime_luaset_seconds_str }
1421     }
1422     {
1423       \__spintent_sptime_build_seconds:nn { _minutos-con }
1424       { \l__spintent_sptime_luaset_seconds_str }
1425     }
1426   }
1427   {
1428     \MathMLintent { :time }
1429     {
1430       \text { \l__spintent_sptime_luaset_base_str }
1431     }
1432     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { false }
1433     {
1434       \MathMLintent { _minutos }
1435       {
1436         \c__spintent_invisible_sep_int
1437       }
1438     }
1439   }
1440 }

```

(End of definition for __spintent_sptime_build_tree:.)

\sptime

The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If

an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1441 \NewDocumentCommand \sptime { m }
1442 {
1443   \mode_if_math:TF
1444   {
1445     \group_begin:
1446     \__spintent_luafun_sptime_parse:n { #1 }
1447     \str_if_eq:VnTF \__spintent_sptime_luaset_error_str { true }
1448     {
1449       \msg_error:nnnn { spintent } { invalid-time-format }
1450       { \sptime } { #1 }
1451     }
1452     {
1453       \__spintent_sptime_build_tree:
1454     }
1455   \group_end:
1456 }
1457 { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1458 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.21 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as `DD/MM/YYYY` or `YYYY-MM-DD`) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

```

\__spintent_spdate_luaset_output_str
\__spintent_spdate_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1459 \str_new:N \__spintent_spdate_luaset_output_str
1460 \str_new:N \__spintent_spdate_luaset_error_str

```

(End of definition for `\__spintent_spdate_luaset_output_str` and `\__spintent_spdate_luaset_error_str`.)

11.21.1 Definition of error messages

```
invalid-date-format
```

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```

1461 \msg_new:nnnn { spintent } { invalid-date-format }
1462 {
1463   Invalid~date~or~unsupported~format~in~#1. \
1464   The~argument~'#2'~does~not~match~a~supported~format.
1465 }
1466 {
1467   Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \
1468   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1469 }

```

(End of definition for `invalid-date-format`.)

11.21.2 Internal functions for `\spdate`

```
\__spintent_spdate_process:n
```

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the MathML `intent` structure using the formatted date stored in `\__spintent_spdate_luaset_output_str`.

```

1470 \cs_new_protected:Nn \__spintent_spdate_process:n
1471 {
1472   \__spintent_luafun_spdate_parse:n { #1 }
1473   \str_if_eq:VnTF \__spintent_spdate_luaset_error_str { true }
1474   {
1475     \msg_error:nnnn { spintent } { invalid-date-format }
1476     { \spdate } { #1 }
1477   }
1478   {
1479     \MathMLintent { :date }
1480     {
1481       \text { \__spintent_spdate_luaset_output_str }
1482     }
1483   }
1484 }

```

(End of definition for `\__spintent_sdate_process:n`.)

`\sdate` The command `\sdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_sdate_process:n` to process the input.

```

1485 \NewDocumentCommand \sdate { m }
1486 {
1487   \mode_if_math:TF
1488   {
1489     \group_begin:
1490       \__spintent_sdate_process:n { #1 }
1491     \group_end:
1492   }
1493   { \msg_error:nnn { spintent } { need-math-mode } { \sdate } }
1494 }

```

(End of definition for `\sdate`. This function is documented on page 11.)

11.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two `(keys)` `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank `{(argument)}`, the second this is when an input is invalid.

```

1495 \msg_new:nnnn { spintent } { spusep-empty-arg }
1496 {
1497   Empty~argument~'#1'~for~\token_to_str:N \spusep.
1498 }
1499 {
1500   \token_to_str:N \spusep~requires~a~non-empty~argument.
1501 }
1502 \msg_new:nnn { spintent } { spusep-invalid-arg }
1503 {
1504   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1505 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

`\__spintent_spusep_arg_tl`
`\__spintent_spusep_left_tl`
`\__spintent_spusep_right_tl`

The variable `\__spintent_spusep_arg_tl` stores the `{(argument)}`, the variable `\__spintent_spusep_left_tl` and `\__spintent_spusep_right_tl` stores the parentheses.

```

1506 \tl_new:N \__spintent_spusep_arg_tl
1507 \tl_new:N \__spintent_spusep_left_tl
1508 \tl_new:N \__spintent_spusep_right_tl

```

(End of definition for `\__spintent_spusep_arg_tl`, `\__spintent_spusep_left_tl`, and `\__spintent_spusep_right_tl`.)

`size`
`silent`
`unknown`

We define the `(keys)` `size` and `silent`.

```

1509 \keys_define:nn { spintent / spusep }
1510 {
1511   size .choices:nn =
1512     { none , big , Big , bigg , Bigg }
1513     {
1514       \int_case:nn { \l_keys_choice_int }
1515       {
1516         { 1 }
1517         {
1518           \tl_clear:N \__spintent_spusep_left_tl
1519           \tl_clear:N \__spintent_spusep_right_tl
1520         }
1521         { 2 }
1522         {
1523           \tl_set:Nn \__spintent_spusep_left_tl { \bigl }
1524           \tl_set:Nn \__spintent_spusep_right_tl { \bigr }
1525         }
1526       }
1527     }

```

```

1526         { 3 }
1527         {
1528             \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1529             \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1530         }
1531         { 4 }
1532         {
1533             \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1534             \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1535         }
1536         { 5 }
1537         {
1538             \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1539             \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1540         }
1541     }
1542 },
1543 size .initial:n = none,
1544 size .value_required:n = true,
1545 size / unknown .code:n =
1546     \msg_error:nneee { spintent } { unknown-choice }
1547     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1548 silent .bool_set:N = \l__spintent_spusep_silent_bool,
1549 silent .initial:n = false,
1550 silent .value_required:n = true,
1551 unknown .code:n =
1552     {
1553         \str_set:Nn \l__spintent_command_name_str { spusep }
1554         \__spintent_unknown_keys_cmd:n {#1}
1555     },
1556 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintent_spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` and then check the state of the variable `\l__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1557 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1558 {
1559     \tl_put_right:cn { \l__spintent_spusep_ #1 _tl } { #2 }
1560     \bool_if:NTF \l__spintent_spusep_silent_bool
1561     {
1562         \MathMLint { :silent } { \tl_use:c { \l__spintent_spusep_ #1 _tl } }
1563     }
1564     {
1565         \tl_use:c { \l__spintent_spusep_ #1 _tl }
1566     }
1567 }

```

(End of definition for `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n`

The function `\__spintent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\l__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1568 \cs_new_protected:Nn \__spintent_spusep_convert:n
1569 {
1570     \tl_set:Nn \l__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1571     \tl_if_empty:NT \l__spintent_spusep_arg_tl
1572     {
1573         \msg_error:nnn { spintent } { spusep-empty-arg } {#1}
1574     }
1575     \str_case:VnF \l__spintent_spusep_arg_tl
1576     {
1577         { ( }
1578         {
1579             \__spintent_spusep_set_parens:nn { left } { \lparen }
1580         }
1581         { ) }
1582         {
1583             \__spintent_spusep_set_parens:nn { right } { \rparen }

```

```

1584     }
1585     { [ ] }
1586     {
1587         \__spintent_spusep_set_parens:nn { left } { \lbrack }
1588     }
1589     { ] }
1590     {
1591         \__spintent_spusep_set_parens:nn { right } { \rbrack }
1592     }
1593     { \{ }
1594     {
1595         \__spintent_spusep_set_parens:nn { left } { \lbrace }
1596     }
1597     { \} }
1598     {
1599         \__spintent_spusep_set_parens:nn { right } { \rbrace }
1600     }
1601 }
1602 {
1603     \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
1604 }
1605 }

```

(End of definition for __spintent_spusep_convert:n.)

\spusep The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1606 \NewDocumentCommand \spusep { O{} m }
1607 {
1608     \mode_if_math:TF
1609     {
1610         \group_begin:
1611         \keys_set:nn { spintent / spusep } {#1}
1612         \__spintent_spusep_convert:n {#2}
1613         \group_end:
1614     }
1615     { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1616 }

```

(End of definition for \spusep. This function is documented on page 12.)

11.23.1 The command \spread

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

The “error” `spnewread-empty-args` is raised when `\spnewread` receives an empty or blank symbol or reading. The “error” `spnewread-duplicate` is raised when it registers a symbol already in the dictionary. The “error” `spread-unknown-sym` is raised when `\spread` is given a symbol that is neither registered nor accompanied by `read-arg`.

```

1617 \msg_new:nnnn { spintent } { spnewread-empty-args }
1618 {
1619     Empty~argument~in~\token_to_str:N \spnewread.
1620 }
1621 {
1622     \token_to_str:N \spnewread~requires~two~non-empty~arguments.
1623 }
1624 \msg_new:nnnn { spintent } { spnewread-duplicate }
1625 {
1626     The~symbol~'~#1'~is~already~defined!
1627 }
1628 {
1629     You~cannot~redefine~with~\token_to_str:N \spnewread.
1630 }
1631 \msg_new:nnnn { spintent } { spread-empty-sym }
1632 {
1633     Empty~symbol~argument~in~\token_to_str:N \spread.
1634 }
1635 {
1636     \token_to_str:N \spread~requires~a~non-empty~symbol.
1637 }
1638 \msg_new:nnnn { spintent } { spread-unknown-sym }

```

```

1639 {
1640   The~symbol~'#1'~is~not~registered~and~no~'read-arg'~was~given.
1641 }
1642 {
1643   Register~it~first~with~\token_to_str:N \spnewread,~or~pass~'read-arg'.
1644 }

```

(End of definition for `spnewread-empty-args`, `spnewread-duplicate`, and `spread-unknown-sym`.)

`\l__spintent_spread_luaset_status_str` and `\l__spintent_spread_luaset_reading_str` are set by `\__spintent_luafun_spread_new:nn/\__spintent_luafun_spread_lookup:n`. `\l__spintent_spread_read_arg_tl` stores `read-arg`.

```

1645 \str_new:N \l__spintent_spread_luaset_status_str
1646 \str_new:N \l__spintent_spread_luaset_reading_str
1647 \tl_new:N \l__spintent_spread_read_arg_tl

```

(End of definition for `\l__spintent_spread_luaset_status_str`, `\l__spintent_spread_luaset_reading_str`, and `\l__spintent_spread_read_arg_tl`.)

`read-arg` We define the `(keys)` `read-arg` and `silent-cmd`.

```

1648 \keys_define:nn { spintent / spread }
1649 {
1650   read-arg .code:n =
1651     { \__spintent_sanitizе_affix:Nn \l__spintent_spread_read_arg_tl {#1} },
1652   read-arg .initial:n = ,
1653   read-arg .value_required:n = true,
1654   silent-cmd .bool_set:N = \l__spintent_spread_silent_cmd_bool,
1655   silent-cmd .initial:n = false,
1656   silent-cmd .value_required:n = true,
1657   unknown .code:n =
1658     {
1659       \str_set:Nn \l__spintent_command_name_str { spread }
1660       \__spintent_unknown_keys_cmd:n {#1}
1661     },
1662 }

```

(End of definition for `read-arg`, `silent-cmd`, and `unknown`.)

`\__spintent_spread_exec:nn` The function `\__spintent_spread_exec:nn` receives the optional `(keys)` in ‘#1’ and the `{(symbol)}` in ‘#2’. With `silent-cmd` true, the symbol is typeset with a `:silent intent`, nothing else evaluated. Otherwise, with `read-arg` given, it is used directly, overriding any registered reading for this call alone. With `read-arg` empty, calls `\__spintent_luafun_spread_lookup:n`, using the registered reading when found, reporting `spread-unknown-sym` otherwise.

```

1663 \cs_new_protected:Nn \__spintent_spread_exec:nn
1664 {
1665   \keys_set:nn { spintent / spread } { #1 }
1666   \tl_if_blank:nTF { #2 }
1667   { \msg_error:nn { spintent } { spread-empty-sym } }
1668   {
1669     \__spintent_invisible_sep:
1670     \bool_if:NTF \l__spintent_spread_silent_cmd_bool
1671     {
1672       \MathMLintent { :silent } { #2 }
1673     }
1674     {
1675       \tl_if_empty:NTF \l__spintent_spread_read_arg_tl
1676       {
1677         \__spintent_luafun_spread_lookup:n { \exp_not:n {#2} }
1678         \str_if_eq:VnTF \l__spintent_spread_luaset_status_str { found }
1679         {
1680           \MathMLintent { \l__spintent_spread_luaset_reading_str } { #2 }
1681         }
1682         {
1683           \msg_error:nne { spintent } { spread-unknown-sym } { \exp_not:n {#2} }
1684         }
1685       }
1686       {
1687         \MathMLintent { \l__spintent_spread_read_arg_tl } { #2 }
1688       }
1689     }
1690   }
1691 }

```

(End of definition for `\__spintent_spread_exec:nn`.)

\spread The public command `\spread` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spread_exec:nn`.

```
1692 \NewDocumentCommand \spread { O{} m }
1693 {
1694   \mode_if_math:TF
1695   {
1696     \group_begin:
1697     \__spintent_spread_exec:nn { #1 } { #2 }
1698     \group_end:
1699   }
1700   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1701 }
```

(End of definition for `\spread`. This function is documented on page 11.)

\spnewread The public command `\spnewread` registers a new symbol/reading pair. It passes the symbol ‘`#1`’ and its reading ‘`#2`’ to Lua via `\__spintent_luafun_spread_new:nn`, reporting `spnewread-duplicate` when the symbol is already registered.

```
1702 \NewDocumentCommand \spnewread { m m }
1703 {
1704   \mode_if_math:TF
1705   {
1706     \msg_error:nnn { spintent } { need-text-mode } { \spnewread }
1707   }
1708   {
1709     \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1710     { \msg_error:nn { spintent } { spnewread-empty-args } }
1711     {
1712       \__spintent_luafun_spread_new:nn { \exp_not:n {#1} } { #2 }
1713       \str_if_eq:VnT \__spintent_spread_luasets_status_str { duplicate }
1714       {
1715         \msg_error:nnn { spintent } { spnewread-duplicate } { #1 }
1716       }
1717     }
1718   }
1719 }
```

(End of definition for `\spnewread`. This function is documented on page 11.)

11.23.2 The command `\spdsym`

`\l__spintent_spdsym_read_sym_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```
\l__spintent_spdsym_symbol_tl
\l__spintent_spdsym_prefix_tl
\l__spintent_spdsym_suffix_tl
1720 \tl_new:N \l__spintent_spdsym_read_sym_tl
1721 \tl_new:N \l__spintent_spdsym_symbol_tl
1722 \tl_new:N \l__spintent_spdsym_prefix_tl
1723 \tl_new:N \l__spintent_spdsym_suffix_tl
```

(End of definition for `\l__spintent_spdsym_read_sym_tl` and others.)

prefix We define the `<keys>` prefix, suffix, set-sym and read-sym for `\spdsym`.

```
suffix
set-sym
read-sym
unknown
1724 \keys_define:nn { spintent / spdsym }
1725 {
1726   prefix .code:n =
1727   { \__spintent_sanitize_affix:Nn \l__spintent_spdsym_prefix_tl {#1} },
1728   prefix .initial:n = ,
1729   prefix .value_required:n = true,
1730   suffix .code:n =
1731   { \__spintent_sanitize_affix:Nn \l__spintent_spdsym_suffix_tl {#1} },
1732   suffix .initial:n = ,
1733   suffix .value_required:n = true,
1734   set-sym .choices:nn =
1735   { old-style , two-dots , slash }
1736   {
1737     \int_case:nn { \l_keys_choice_int }
1738     {
1739       {1} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { \div } }
1740       {2} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { : } }
1741       {3} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { / } }
1742     }
1743   }
```

```

1743     },
1744     set-sym          .initial:n = two-dots,
1745     set-sym          .value_required:n = true,
1746     set-sym / unknown .code:n =
1747       \msg_error:nneee { spintent } { unknown-choice }
1748       { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1749     read-sym         .choices:nn =
1750       { dividido , dividido-por , dividido-entre }
1751       {
1752         \int_case:nn { \l_keys_choice_int }
1753         {
1754           {1} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido } }
1755           {2} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-por } }
1756           {3} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-entre } }
1757         }
1758       },
1759     read-sym          .initial:n = dividido,
1760     read-sym          .value_required:n = true,
1761     read-sym / unknown .code:n =
1762       \msg_error:nneee { spintent } { unknown-choice }
1763       { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1764       { \exp_not:n {#1} },
1765     unknown           .code:n =
1766       {
1767         \str_set:Nn \l__spintent_command_name_str { spdsym }
1768         \__spintent_unknown_keys_cmd:n {#1}
1769       },
1770   }

```

(End of definition for prefix and others.)

\spdsym Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1771 \NewDocumentCommand \spdsym { 0{} }
1772 {
1773   \mode_if_math:TF
1774   {
1775     \group_begin:
1776     \keys_set:nn { spintent / spdsym } { #1 }
1777     \MathMLintent
1778     {
1779       \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1780       { \l__spintent_spdsym_prefix_tl - }
1781       \l__spintent_spdsym_read_sym_tl
1782       \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1783       { - \l__spintent_spdsym_suffix_tl }
1784     }
1785     { \l__spintent_spdsym_symbol_tl }
1786     \group_end:
1787   }
1788   { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1789 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.23.3 The command `\spmsym`

`\l__spintent_spmsym_read_sym_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1790 \tl_new:N \l__spintent_spmsym_read_sym_tl
1791 \tl_new:N \l__spintent_spmsym_symbol_tl
1792 \tl_new:N \l__spintent_spmsym_prefix_tl
1793 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l__spintent_spmsym_read_sym_tl` and others.)

prefix We define the `\keys` prefix, suffix, set-sym, read-sym and not-print for `\spmsym`.

```

suffix 1794 \keys_define:nn { spintent / spmsym }
set-sym 1795 {
read-sym 1796   prefix .code:n =
not-print 1797   { \__spintent_sanitizе_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
unknown 1798   prefix .initial:n = ,
1799   prefix .value_required:n = true,

```

```

1800     suffix                .code:n =
1801     { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
1802     suffix                .initial:n = ,
1803     suffix                .value_required:n = true,
1804     set-sym               .choices:nn =
1805     { cross , dot , asterisk }
1806     {
1807       \int_case:nn { \l_keys_choice_int }
1808       {
1809         {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
1810         {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
1811         {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
1812       }
1813     },
1814     set-sym               .initial:n = dot,
1815     set-sym               .value_required:n = true,
1816     set-sym / unknown    .code:n =
1817     \msg_error:nneee { spintent } { unknown-choice }
1818     { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1819     read-sym              .choices:nn =
1820     { por , multiplicado-por , veces }
1821     {
1822       \int_case:nn { \l_keys_choice_int }
1823       {
1824         {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
1825         {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
1826         {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
1827       }
1828     },
1829     read-sym              .initial:n = por,
1830     read-sym              .value_required:n = true,
1831     read-sym / unknown    .code:n =
1832     \msg_error:nneee { spintent } { unknown-choice }
1833     { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1834     not-print             .bool_set:N = \l__spintent_spmsym_not_print_bool,
1835     not-print             .initial:n = false,
1836     not-print             .value_required:n = true,
1837     unknown               .code:n =
1838     {
1839       \str_set:Nn \l__spintent_command_name_str { spmsym }
1840       \__spintent_unknown_keys_cmd:n {#1}
1841     },
1842   }

```

(End of definition for *prefix* and others.)

\spmsym Defines the `\spmsym` operator. Similar to `\spdsym`, it outputs the correct mathematical glyph based on the local *keys*, safely attaching prefixes and suffixes if present.

```

1843 \NewDocumentCommand \spmsym { 0{ } }
1844 {
1845   \mode_if_math:TF
1846   {
1847     \group_begin:
1848     \keys_set:nn { spintent / spmsym } { #1 }
1849     \MathMLintent
1850     {
1851       \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1852       { \l__spintent_spmsym_prefix_tl - }
1853       \l__spintent_spmsym_read_sym_tl
1854       \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1855       { - \l__spintent_spmsym_suffix_tl }
1856     }
1857     {
1858       \bool_if:NTF \l__spintent_spmsym_not_print_bool
1859       {
1860         \invisibletimes
1861       }
1862       { \l__spintent_spmsym_symbol_tl }
1863     }
1864     \group_end:
1865   }

```

```

1866 { \msg_error:nnn { spintenc } { need-math-mode } { \spmsym } }
1867 }

```

(End of definition for `\spmsym`. This function is documented on page 13.)

11.24 The command `\spdiv`

The user-level command `\spdiv` formats division representations. It takes two mandatory arguments: the $\langle dividend \rangle$ and the $\langle divisor \rangle$. Both $\langle arguments \rangle$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\langle dividend \rangle$ nor the $\langle divisor \rangle$ is *blank*.

11.24.1 Definition of keys for `\spdiv`

We define the $\langle keys \rangle$ `wrap-parens` and `read-parens` for `\spdiv` command. Number-related $\langle keys \rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

1868 \keys_define:nn { spintenc / spdiv }
1869 {
1870   cifras-sep      .meta:nn = { spintenc / spnum } { cifras-sep      = {#1} },
1871   read-sign       .meta:nn = { spintenc / spnum } { read-sign       = {#1} },
1872   read-period-sep .meta:nn = { spintenc / spnum } { read-period-sep = {#1} },
1873   notation-sym    .meta:nn = { spintenc / spnum } { notation-sym    = {#1} },
1874   set-sym         .meta:nn = { spintenc / spdsym } { set-sym         = {#1} },
1875   read-sym        .meta:nn = { spintenc / spdsym } { read-sym        = {#1} },
1876   wrap-parens     .bool_set:N = \l__spintenc_spdiv_wrap_parens_bool,
1877   wrap-parens     .initial:n = false,
1878   wrap-parens     .value_required:n = true,
1879   read-parens     .bool_set:N = \l__spintenc_spdiv_read_parens_bool,
1880   read-parens     .initial:n = true,
1881   read-parens     .value_required:n = true,
1882   unknown         .code:n =
1883   {
1884     \str_set:Nn \l__spintenc_command_name_str { spdiv }
1885     \__spintenc_unknown_keys_cmd:n {#1}
1886   },
1887 }

```

(End of definition for `cifras-sep` and others.)

11.24.2 Definition of error messages

`spdiv-empty-argument` Raised when either the $\langle dividend \rangle$ or the $\langle divisor \rangle$ is missing or *blank*.

```

1888 \msg_new:nnnn { spintenc } { spdiv-empty-argument }
1889 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1890 {
1891   You~must~provide~both~a~dividend~and~a~divisor.\\
1892 }

```

(End of definition for `spdiv-empty-argument`.)

11.24.3 Internal functions for `\spdiv`

`\__spintenc_spdiv_render_op:nn` The function `\__spintenc_spdiv_render_op:nn` parses both $\langle arguments \rangle$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1893 \cs_new_protected:Nn \__spintenc_spdiv_render_op:nn
1894 {
1895   \spnum {#1}
1896   \MathMLintenc { \l__spintenc_spdsym_read_sym_tl }
1897   {
1898     \l__spintenc_spdsym_symbol_tl
1899   }
1900   \spnum {#2}
1901 }

```

(End of definition for `\__spintenc_spdiv_render_op:nn`.)

`\__spintenc_spdiv_render_parens:nn` The function `\__spintenc_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1902 \cs_new_protected:Nn \__spintenc_spdiv_render_parens:nn
1903 {
1904   \bool_if:NTF \l__spintenc_spdiv_read_parens_bool
1905   {
1906     ( \__spintenc_spdiv_render_op:nn {#1} {#2} )

```

```

1907     }
1908     {
1909         \MathMLintent { :silent } { ( }
1910         \__spintent_spdiv_render_op:nn {#1} {#2}
1911         \MathMLintent { :silent } { ) }
1912     }
1913 }

```

(End of definition for __spintent_spdiv_render_parens:nn.)

__spintent_spdiv_process:nnn

The function `\__spintent_spdiv_process:nnn` sets $\langle keys \rangle$, validates $\{\langle arguments \rangle\}$, and decides on parenthesis wrapping.

```

1914 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1915 {
1916     \keys_set:nn { spintent / spdiv } { #1 }
1917     \tl_if_blank:nTF { #2 }
1918     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1919     {
1920         \tl_if_blank:nTF { #3 }
1921         { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1922         {
1923             \bool_if:NTF \__spintent_spdiv_wrap_parens_bool
1924             { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1925             { \__spintent_spdiv_render_op:nn {#2} {#3} }
1926         }
1927     }
1928 }

```

(End of definition for __spintent_spdiv_process:nnn.)

\spdiv

Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1929 \NewDocumentCommand \spdiv { O{} m m }
1930 {
1931     \mode_if_math:TF
1932     {
1933         \group_begin:
1934         \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1935         \group_end:
1936     }
1937     { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1938 }

```

(End of definition for \spdiv. This function is documented on page 13.)

11.25 The command \spmul

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the $\{\langle first\ factor \rangle\}$ and the $\{\langle second\ factor \rangle\}$. Both $\{\langle arguments \rangle\}$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\{\langle first\ factor \rangle\}$ nor the $\{\langle second\ factor \rangle\}$ is *blank*.

11.25.1 Definition of keys for \spmul

cifras-sep
read-sign
read-period-sep
notation-sym

We define the $\langle keys \rangle$ `wrap-parens` and `read-parens` for `\spmul` command. Number-related $\langle keys \rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1939 \keys_define:nn { spintent / spmul }
1940 {
1941     cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1942     read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1943     read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1944     notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1945     set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1946     read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1947     wrap-parens     .bool_set:N = \l__spintent_spmul_wrap_parens_bool,
1948     wrap-parens     .initial:n  = false,
1949     wrap-parens     .value_required:n = true,
1950     read-parens     .bool_set:N = \l__spintent_spmul_read_parens_bool,
1951     read-parens     .initial:n  = true,
1952     read-parens     .value_required:n = true,
1953     unknown         .code:n     =
1954     {

```

```

1955         \str_set:Nn \l__spintent_command_name_str { spmul }
1956         \__spintent_unknown_keys_cmd:n {#1}
1957     },
1958 }

```

(End of definition for *cifras-sep* and others.)

11.25.2 Definition of error messages

`spmul-empty-argument` Raised when either the $\langle\textit{first factor}\rangle$ or the $\langle\textit{second factor}\rangle$ is missing or *blank*.

```

1959 \msg_new:nnnn { spintent } { spmul-empty-argument }
1960 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1961 {
1962     You~must~provide~both~first~factor~and~second~factor.\\
1963 }

```

(End of definition for *spmul-empty-argument*.)

11.25.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn` The function `\__spintent_spmul_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1964 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1965 {
1966     \spnum {#1}
1967     \MathMLintent { \l__spintent_spmsym_read_sym_tl }
1968     {
1969         \l__spintent_spmsym_symbol_tl
1970     }
1971     \spnum {#2}
1972 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn` The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1973 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
1974 {
1975     \bool_if:NTF \l__spintent_spmul_read_parens_bool
1976     {
1977         ( \__spintent_spmul_render_op:nn {#1} {#2} )
1978     }
1979     {
1980         \MathMLintent { :silent } { ( }
1981         \__spintent_spmul_render_op:nn {#1} {#2}
1982         \MathMLintent { :silent } { ) }
1983     }
1984 }

```

(End of definition for `\__spintent_spmul_render_parens:nn`.)

`\__spintent_spmul_process:nnn` The function `\__spintent_spmul_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

1985 \cs_new_protected:Nn \__spintent_spmul_process:nnn
1986 {
1987     \keys_set:nn { spintent / spmul } { #1 }
1988     \tl_if_blank:NTF { #2 }
1989     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1990     {
1991         \tl_if_blank:NTF { #3 }
1992         { \msg_error:nnn { spintent } { spmul-empty-argument } }
1993         {
1994             \bool_if:NTF \l__spintent_spmul_wrap_parens_bool
1995             { \__spintent_spmul_render_parens:nn {#2} {#3} }
1996             { \__spintent_spmul_render_op:nn {#2} {#3} }
1997         }
1998     }
1999 }

```

(End of definition for `\__spintent_spmul_process:nnn`.)

```

\spmul Defines the user command \spmul. It strictly relies on math mode.

2000 \NewDocumentCommand \spmul { O{} m m }
2001 {
2002     \mode_if_math:TF
2003     {
2004         \group_begin:
2005         \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
2006         \group_end:
2007     }
2008     { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
2009 }

```

(End of definition for \spmul. This function is documented on page 13.)

11.26 The commands \spgcd and \spmcn

The user-level commands `\spgcd` and `\spmcn` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.26.1 Variables for \spgcd and \spmcn

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```

\__spintent_spmcn_spmcd_luaset_error_str
\__spintent_mc_luaset_types_str
\__spintent_mc_luaset_all_numeric_str
\__spintent_mc_args_seq
\__spintent_mc_raw_args_seq
\__spintent_spmcd_print_result_bool
\__spintent_spmcn_print_result_bool
\__spintent_spmcd_read_terse_bool
\__spintent_spmcn_read_terse_bool
\__spintent_spmcd_print_upper_bool
\__spintent_spmcn_print_upper_bool
\__spintent_spmcd_prefix_tl
\__spintent_spmcn_prefix_tl
\__spintent_spmcd_suffix_tl
\__spintent_spmcn_suffix_tl

2010 \str_new:N \__spintent_spmcn_spmcd_luaset_error_str
2011 \clist_new:N \__spintent_mc_luaset_types_clist
2012 \str_new:N \__spintent_mc_luaset_all_numeric_str
2013 \seq_new:N \__spintent_mc_args_seq
2014 \seq_new:N \__spintent_mc_raw_args_seq
2015 \seq_new:N \__spintent_mc_types_seq
2016 \int_new:N \__spintent_mc_idx_int
2017 \clist_map_inline:nn { mcd , mcn }
2018 {
2019     \tl_new:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
2020     \bool_new:c { \__spintent_sp #1 _read_terse_bool }
2021     \tl_new:c { \__spintent_sp #1 _prefix_tl }
2022 }

```

(End of definition for __spintent_spmcn_spmcd_luaset_error_str and others.)

11.26.2 Definition of error messages

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```

2023 \msg_new:nnnn { spintent } { mc-invalid-arguments }
2024 {
2025     Invalid~arguments~provided~to~#1.
2026 }
2027 {
2028     The~arguments~must~be~a~comma-separated~list. \\\
2029 }

```

(End of definition for mc-invalid-arguments.)

11.26.3 Keys for \spgcd and \spmcn

```

result We define the shared (keys) result, prefix, read-cmd, upper-case, concept and silent-cmd.
prefix
read-cmd
upper-case
concept
silent-cmd
unknown

2030 \clist_map_inline:nn { mcd , mcn }
2031 {
2032     \keys_define:nn { spintent / sp #1 }
2033     {
2034         result .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2035         result .initial:n = false,
2036         result .value_required:n = true,
2037         prefix .code:n =
2038             { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2039         prefix .initial:n = ,
2040         prefix .value_required:n = true,
2041         read-cmd .choices:nn =
2042             { terse , clear }
2043         {
2044             \int_case:nn { \_keys_choice_int }
2045             {
2046                 {1} { \bool_set_true:c { \__spintent_sp #1 _read_terse_bool } }
2047                 {2} { \bool_set_false:c { \__spintent_sp #1 _read_terse_bool } }

```

```

2048         }
2049     },
2050     read-cmd          .initial:n = clear,
2051     read-cmd          .value_required:n = true,
2052     read-cmd / unknown .code:n =
2053         \msg_error:nneee { spintent } { unknown-choice }
2054         { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
2055     upper-case        .bool_set:c = { l__spintent_sp #1 _print_upper_bool },
2056     upper-case        .initial:n = false,
2057     upper-case        .value_required:n = true,
2058     concept           .bool_set:c = { l__spintent_sp #1 _concept_bool },
2059     concept           .initial:n = true,
2060     concept           .value_required:n = true,
2061     silent-cmd        .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
2062     silent-cmd        .initial:n = false,
2063     silent-cmd        .value_required:n = true,
2064     unknown           .code:n =
2065         {
2066             \str_set:Nn \l__spintent_command_name_str { sp #1 }
2067             \__spintent_unknown_keys_cmd:n {##1}
2068         },
2069     }
2070 }

```

(End of definition for result and others.)

11.26.4 Internal functions for \spmc and \spmcm

__spintenc_mc_process_args:nn

The function `\__spintenc_mc_process_args:nn` parses the operation ‘#1’ and its raw argument list ‘#2’. Calls `\__spintenc_luafun_mc_classify_args:n`, splits ‘#2’ into `\l__spintenc_mc_raw_args_seq` and `\l__spintenc_mc_luaset_types_clist` into `\l__spintenc_mc_types_seq`, then walks the former by position, fetching the matching type from the latter through `\seq_item:Nn`, to build `\l__spintenc_mc_args_seq`: each argument wrapped through `\spnum` when its type is `number`, verbatim otherwise. Injects MathML intents around the already-formatted `\l__spintenc_mc_args_seq`.

```

2071 \cs_new_protected:Nn \__spintenc_mc_process_args:nn
2072 {
2073     \__spintenc_luafun_mc_classify_args:n {#2}
2074     \seq_set_from_clist:Nn \l__spintenc_mc_raw_args_seq {#2}
2075     \seq_set_from_clist:NN \l__spintenc_mc_types_seq \l__spintenc_mc_luaset_types_clist
2076     \seq_clear:N \l__spintenc_mc_args_seq
2077     \int_zero:N \l__spintenc_mc_idx_int
2078     \seq_map_inline:Nn \l__spintenc_mc_raw_args_seq
2079     {
2080         \int_incr:N \l__spintenc_mc_idx_int
2081         \str_if_eq:eeTF
2082         { \seq_item:Nn \l__spintenc_mc_types_seq { \l__spintenc_mc_idx_int } } { number }
2083         {
2084             \seq_put_right:Nn \l__spintenc_mc_args_seq { \spnum {##1} }
2085         }
2086         { \seq_put_right:Nn \l__spintenc_mc_args_seq {##1} }
2087     }

```

Injects MathML intents around the already-formatted `\l__spintenc_mc_args_seq`.

```

2088     \bool_if:cTF { l__spintenc_sp #1 _silent_cmd_bool }
2089     {
2090         \seq_set_map:NNn \l__spintenc_mc_args_seq \l__spintenc_mc_args_seq
2091         { \MathMLintent { :silent } { ##1 } }
2092         \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % prevent "por"
2093         \MathMLintent { :silent } { ( }
2094         \seq_use:Nnnn \l__spintenc_mc_args_seq
2095         { \MathMLintent { :silent } { , } }
2096         { \MathMLintent { :silent } { , } }
2097         { \MathMLintent { :silent } { , } }
2098     }
2099     {
2100         \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % prevent doble "de"
2101         \bool_if:cTF { l__spintenc_sp #1 _concept_bool }
2102         {
2103             \MathMLintent { _entre } { ( }
2104         }
2105         {
2106             \MathMLintent { :silent } { ( }

```

```

2107     }
2108     \seq_use:Nnnn \l__spintent_mc_args_seq
2109     { \MathMLintent { _y- } { , } }
2110     { \MathMLintent { :silent } { , } }
2111     { \MathMLintent { _y- } { , } }
2112   }
2113   \MathMLintent { :silent } { ) }
2114 }

```

(End of definition for `\__spintent_mc_process_args:nn`.)

`\__spintent_spmc_process:nnnn`

The internal function `\__spintent_spmc_process:nnnn` is the shared handler for both `\spmc` and `\spmc`. Takes the short identifier in ‘#1’, its full reading string in ‘#2’, the local $\langle keys \rangle$ in ‘#3’, and the optional arguments in ‘#4’.

The operator name is silenced with `silent-cmd=true` — `\__spintent_mc_process_args:nn` silences the arguments too, in that case — or with `concept=false` alone, leaving the arguments untouched. With `concept=true`, it carries the literal `intent` instead — #1 or #2 per `read-cmd`, prefixed with `prefix` when given.

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure, and, when `result` is `true`, printing the computed value. `\__spintent_mc_process_args:n` prints the argument list itself either way.

- We should use `\mathrm` instead of `\operatorname`, which generates unwanted spacing when `\c__spintent_invisible_sep_int` is applied.

```

2115 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
2116 {
2117   \keys_set:nn { spintent / sp #1 } { #3 }
2118   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
2119   {
2120     \MathMLintent { :silent }
2121     {
2122       \mathrm
2123       {
2124         \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2125         { \text_uppercase:n {#1} } { #1 }
2126       }
2127     }
2128   }
2129   {
2130     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2131     {
2132       \MathMLintent
2133       {
2134         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
2135         {
2136           \tl_use:c { l__spintent_sp #1 _prefix_tl } -
2137         }
2138         \bool_if:cTF { l__spintent_sp #1 _read_terse_bool } { #1 :nofix } { #2 :nofix }
2139       }
2140     }
2141     \mathrm
2142     {
2143       \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2144       { \text_uppercase:n {#1} } { #1 }
2145     }
2146   }
2147 }
2148 {
2149   \MathMLintent { :silent }
2150   {
2151     \mathrm
2152     {
2153       \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2154       { \text_uppercase:n {#1} } { #1 }
2155     }
2156   }
2157 }
2158 }

```

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting mc-invalid-arguments on failure — which should not happen, since every argument already passed classification — and, when `result` is `true`, printing the computed value. With any non-numeric argument, no calculation is attempted at all. Either way, `\__spintent_mc_process_args:n` prints the argument list itself.

```

2159 \tl_if_novalue:nF { #4 }
2160 {
2161   \__spintent_luafun_mc_classify_args:n {#4}
2162   \str_if_eq:VnT \__spintent_mc_luaset_all_numeric_str { true }
2163   {
2164     \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2165     \str_if_eq:VnT \__spintent_spmcm_spmcd_luaset_error_str { true }
2166     {
2167       \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2168     }
2169   }
2170   \__spintent_mc_process_args:nn {#1} {#4}
2171   \str_if_eq:VnT \__spintent_mc_luaset_all_numeric_str { true }
2172   {
2173     \bool_if:cT { \__spintent_sp #1 _print_result_bool }
2174     {
2175       \MathMLintent { _es-igual-a } { = }
2176       \tl_use:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
2177     }
2178   }
2179 }
2180 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmcd` The commands `\spmcd` and `\spmcm` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2181 \NewDocumentCommand \spmcd { 0{} } d() {
2182   {
2183     \mode_if_math:TF
2184     {
2185       \group_begin:
2186         \__spintent_spmc_process:nnnn { mcd } { _máximo-común-divisor } { #1 } { #2 }
2187       \group_end:
2188     }
2189     { \msg_error:nnn { spintent } { need-math-mode } { \spmcd } }
2190   }
2191 \NewDocumentCommand \spmcm { 0{} } d() {
2192   {
2193     \mode_if_math:TF
2194     {
2195       \group_begin:
2196         \__spintent_spmc_process:nnnn { mcm } { _mínimo-común-múltiplo } { #1 } { #2 }
2197       \group_end:
2198     }
2199     { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }
2200   }

```

(End of definition for `\spmcd` and `\spmcm`. These functions are documented on page 15.)

11.27 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.27.1 Definition of variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```

\__spintent_md_out_seq
\__spintent_spmcd_luaset_is_number_str
\__spintent_spmM_spmD_luaset_error_str
\__spintent_spmM_luaset_result_tl
\__spintent_spmD_luaset_result_tl
\__spintent_spmM_luaset_is_truncated_str
\__spintent_spmD_luaset_is_truncated_str
\__spintent_spmM_prefix_tl
\__spintent_spmD_prefix_tl
2201 \seq_new:N \__spintent_md_out_seq
2202 \str_new:N \__spintent_spmM_spmD_luaset_error_str
2203 \str_new:N \__spintent_spmcd_luaset_is_number_str
2204 \clist_map_inline:nn { nM , nD }
2205 {
2206   \tl_new:c { \__spintent_sp #1 _luaset_result_tl }

```

```

2207     \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2208     \tl_new:c { \__spintent_sp #1 _prefix_tl }
2209 }

```

(End of definition for __spintent_md_out_seq and others.)

11.27.2 Keys for \spnM and \spnD

We define the shared *(keys)* result, silent-cmd, result-num, prefix and concept.

```

result      \clist_map_inline:nn { nM , nD }
silent-cmd  2210
result-num  2211 {
prefix      2212   \keys_define:nn { spintent / sp #1 }
concept     2213   {
unknown     2214     result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
              2215     result      .initial:n = false,
              2216     result      .value_required:n = true,
              2217     silent-cmd .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
              2218     silent-cmd .initial:n = false,
              2219     silent-cmd .value_required:n = true,
              2220     result-num .tl_set:c = { \__spintent_sp #1 _print_result_num_tl },
              2221     result-num .initial:n = ,
              2222     prefix      .code:n =
              2223       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
              2224     prefix      .initial:n = ,
              2225     prefix      .value_required:n = true,
              2226     concept     .bool_set:c = { \__spintent_sp #1 _concept_bool },
              2227     concept     .initial:n = true,
              2228     concept     .value_required:n = true,
              2229     unknown     .code:n =
              2230       {
              2231         \str_set:Nn \__spintent_command_name_str { sp #1 }
              2232         \__spintent_unknown_keys_cmd:n {##1}
              2233       },
              2234     }
              2235   }

```

(End of definition for result and others.)

11.27.3 Definition of error messages for \spnM and \spnD

md-invalid-argument Exception triggered when the argument for multiples or divisors is invalid.

```

2236 \msg_new:nnnn { spintent } { md-invalid-argument }
2237 {
2238   Invalid~argument~provided~to~#1~\msg_line_context:.
2239 }
2240 {
2241   The~argument~must~be~a~single~positive~integer. \
2242 }

```

(End of definition for md-invalid-argument.)

11.27.4 Internal functions for \spnM and \spnD

__spintent_spnM_spnD_process:nnnnn The internal function `\__spintent_spnM_spnD_process:nnnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintent_spn_M_and_D_exec:nnnn`.

```

2243 \cs_new_protected:Nn \__spintent_spnM_spnD_process:nnnnn
2244 {
2245   \keys_set:nn { spintent / sp #1 } { #4 }
2246   \__spintent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2247 }

```

(End of definition for __spintent_spnM_spnD_process:nnnnn.)

__spintent_spnM_spnD_intent:nn The internal function `\__spintent_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined, normalizes accentuation for “*múltiplos*”, and appends ‘-de’.

```

2248 \cs_new:Nn \__spintent_spnM_spnD_intent:nn
2249 {
2250   \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
2251     { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
2252   \str_case:nnF { #2 } { { múltiplos } { _múltiplos } }

```

```

2253     { #2 }
2254   -de
2255 }

```

(End of definition for `\__spintent_spnM_spnD_intent:nn`.)

`\__spintent_spnM_spnD_silent:nn`

The internal function `\__spintent_spn_M_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules.

```

2256 \cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
2257 {
2258   \MathMLintent { :silent } { \operatorname { #1 } }
2259   \MathMLintent { :silent } { ( }
2260   \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2261   \MathMLintent { :silent } { #2 }
2262   \MathMLintent { :silent } { ) }
2263 }

```

(End of definition for `\__spintent_spnM_spnD_silent:nn`.)

`\__spintent_spnM_spnD_result:nn`

The internal function `\__spintent_spn_M_and_D_result:nn` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’ and the classified numeric argument ‘#2’. It invokes the corresponding Lua backend calculation, reporting `md-invalid-argument` if that fails — a safety net beyond `\__spintent_luafun_spnmd_classify:n`’s own check, since the Lua calculation never trusted that check blindly. On success, it maps the resulting clist into a sequence, and formats the final printed output utilizing custom intents for both separators and conjunctions.

```

2264 \cs_new_protected:Nn \__spintent_spnM_spnD_result:nn
2265 {
2266   \use:c { __spintent_luafun_calcular_ #1 :nn } { #2 }
2267   {
2268     \tl_use:c { l__spintent_sp #1 _print_result_num_tl }
2269   }
2270   \str_if_eq:VnTF \l__spintent_spnM_spnD_luaset_error_str { true }
2271   {
2272     \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2273   }
2274   {
2275     \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2276     \MathMLintent { _son } { = \{ }
2277     \seq_set_from_clist:Ne
2278       \l__spintent_md_out_seq { \tl_use:c { l__spintent_sp #1 _luaset_result_tl } }
2279     \seq_use:Nnnn \l__spintent_md_out_seq
2280       { \MathMLintent { _y- } { , } }
2281       { \MathMLintent { :silent } { , } }
2282       {
2283         \MathMLintent { :silent } { , }
2284         \str_if_eq:vnT { l__spintent_sp #1 _luaset_is_truncated_str } { false }
2285         {
2286           \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" final
2287         }
2288       }
2289     \str_if_eq:vnT { l__spintent_sp #1 _luaset_is_truncated_str } { true }
2290     {
2291       \MathMLintent { :silent } { , } ~ \dots
2292     }
2293     \MathMLintent { :silent } { \} }
2294   }
2295 }

```

(End of definition for `\__spintent_spnM_spnD_result:nn`.)

`\__spintent_spnM_spnD_exec:nnnn`

The function `\__spintent_spnM_spnD_exec:nnnn` takes the command name in ‘#1’, the symbol in ‘#2’, the base reading string in ‘#3’, and the optional argument in ‘#4’. The parentheses are always silenced.

With ‘#4’ absent, the placeholder `n` stands in for the argument. With ‘#4’ given, `\__spintent_luafun_spnmd_classify:n` decides whether it is wrapped through `\spnum` — and, when it is, whether `\__spintent_spnM_spnD_result:n` also prints the computed set, per `result` — or left verbatim, with no result attempted.

Either way, `silent-cmd` silences the whole thing through `\__spintent_spnM_spnD_silent:nn`; otherwise `concept=true` carries the literal `intent` from `\__spintent_spnM_spnD_intent:nn`, `false` silences only the operator symbol, leaving the argument to read normally.

```

2296 \cs_new_protected:Nn \__spintent_spnM_spnD_exec:nnnn

```

```

2297 {
2298   \tl_if_novalue:nTF { #4 }
2299   {
2300     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
2301     { \__spintent_spnM_spnD_silent:nn { #2 } { n } }
2302     {
2303       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2304       {
2305         \MathMLintent
2306         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2307         { \operatorname { #2 } }
2308       }
2309       { \MathMLintent { :silent } { \operatorname { #2 } } }
2310       \MathMLintent { :silent } { ( }
2311       \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2312       n
2313       \MathMLintent { :silent } { ) }
2314     }
2315   }
2316   {
2317     \__spintent_luafun_spnmd_classify:n { \exp_not:n { #4 } }
2318     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
2319     {
2320       \str_if_eq:VnTF \__spintent_spnmd_lua_set_is_number_str { true }
2321       { \__spintent_spnM_spnD_silent:nn { #2 } { \spnum { #4 } } }
2322       { \__spintent_spnM_spnD_silent:nn { #2 } { #4 } }
2323     }
2324     {
2325       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2326       {
2327         \MathMLintent
2328         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2329         { \operatorname { #2 } }
2330       }
2331       { \MathMLintent { :silent } { \operatorname { #2 } } }
2332       \MathMLintent { :silent } { ( }
2333       \str_if_eq:VnTF \__spintent_spnmd_lua_set_is_number_str { true }
2334       { \spnum { #4 } } { #4 }
2335       \MathMLintent { :silent } { ) }
2336     }
2337     \str_if_eq:VnT \__spintent_spnmd_lua_set_is_number_str { true }
2338     {
2339       \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2340       {
2341         \__spintent_spnM_spnD_result:nn { #1 } { #4 }
2342       }
2343     }
2344   }
2345 }

```

(End of definition for `\__spintent_spnM_spnD_exec:nnnn`.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\__spintent_spnM_spnD_process:nnnn`, passing along the optional *(keys)* ‘#1’ and the numeric *{(argument)}* ‘#2’.

```

2346 \NewDocumentCommand \spnM { O{} d() }
2347 {
2348   \mode_if_math:TF
2349   {
2350     \group_begin:
2351     \__spintent_spnM_spnD_process:nnnn { nM } { M } { multiplos } { #1 } { #2 }
2352     \group_end:
2353   }
2354   { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2355 }
2356 \NewDocumentCommand \spnD { O{} d() }
2357 {
2358   \mode_if_math:TF
2359   {
2360     \group_begin:

```

```

2361     \__spintent_spnM_spnD_process:nnnn { nD } { D } { _divisores } {#1} {#2}
2362   \group_end:
2363 }
2364 { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2365 }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 14.)

11.28 The command `\sprazon`

The user command `\sprazon` typesets a ratio $a:b$ as a fraction (`\dfrac` or `\frac`, per `use-dfrac`), read natively — no Lua parsing involved.

11.28.1 Definition of keys for `\sprazon`

We define the *(keys)* `silent-cmd`, `use-dfrac` and `inline`.

```

silent-cmd 2366 \keys_define:nn { spintent / sprazon }
use-dfrac   2367 {
inline      2368   silent-cmd .bool_set:N = \__spintent_sprazon_silent_cmd_bool,
unknown     2369   silent-cmd .initial:n = false,
            2370   silent-cmd .value_required:n = true,
            2371   use-dfrac .bool_set:N = \__spintent_sprazon_use_dfrac_bool,
            2372   use-dfrac .initial:n = false,
            2373   use-dfrac .value_required:n = true,
            2374   inline .bool_set:N = \__spintent_sprazon_inline_bool,
            2375   inline .initial:n = false,
            2376   inline .value_required:n = true,
            2377   unknown .code:n =
            2378   {
            2379     \str_set:Nn \__spintent_command_name_str { sprazon }
            2380     \__spintent_unknown_keys_cmd:n {#1}
            2381   },
            2382 }

```

(End of definition for `silent-cmd` and others.)

`\__spintent_sprazon_exec:nnn` The function `\__spintent_sprazon_exec:nnn` receives the optional *(keys)* in ‘#1’, the numerator in ‘#2’ and the denominator in ‘#3’. Under `silent-cmd`, the whole thing is silenced as one unit — `\frac/\dfrac` (per `use-dfrac`) normally, or `#2 : #3` under `inline`.

Otherwise, with `inline=false` (the default), numerator and denominator are each tagged through a native `\MathMLarg` (num/denom) inside `\frac/\dfrac`, and the outer `\MathMLintent` reads `_es-a:infix($num,$denom)`. With `inline=true`, the same two references are tagged side by side instead — no `\frac/\dfrac` involved — with the `:` between them carrying the `_es-a:infix($num,$denom)` intent on its own.

```

2383 \cs_new_protected:Nn \__spintent_sprazon_exec:nnn
2384 {
2385   \keys_set:nn { spintent / sprazon } { #1 }
2386   \__spintent_invisible_sep:
2387   \bool_if:NTF \__spintent_sprazon_silent_cmd_bool
2388   {
2389     \MathMLintent { _- }
2390     {
2391       \__spintent_inner_sep:
2392       \bool_if:NTF \__spintent_sprazon_inline_bool
2393       { #2 : #3 }
2394       {
2395         \bool_if:NTF \__spintent_sprazon_use_dfrac_bool
2396         { \dfrac { #2 } { #3 } }
2397         { \frac { #2 } { #3 } }
2398       }
2399     }
2400   }
2401   {
2402     \bool_if:NTF \__spintent_sprazon_inline_bool
2403     {
2404       \MathMLintent { _es-a:infix($num,$denom) }
2405       {
2406         \MathMLarg { num } { #2 }
2407         :
2408         \MathMLarg { denom } { #3 }
2409       }
2410     }
2411     {
2412       \MathMLintent { _es-a:infix($num,$denom) }

```

```

2413         {
2414             \__spintent_inner_sep:
2415             \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2416             {
2417                 \dfrac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } }
2418             }
2419             { \frac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } } }
2420         }
2421     }
2422 }
2423 }

```

(End of definition for __spintent_sprazon_exec:nnn.)

\sprazon The public command **\sprazon** enforces *math mode* and delegates execution to **__spintent_sprazon_exec:nnn**.

```

2424 \NewDocumentCommand \sprazon { O{} m m }
2425 {
2426     \mode_if_math:TF
2427     {
2428         \group_begin:
2429         \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2430         \group_end:
2431     }
2432     { \msg_error:nnn { spintent } { need-math-mode } { \sprazon } }
2433 }

```

(End of definition for \sprazon. This function is documented on page ??.)

11.29 The command \spProp

The user command **\spProp** typesets a proportion $a:b = c:d$ as two fractions joined by $=$, reusing **__spintent_sprazon_exec:nnn** directly (not the public **\sprazon** command) for each ratio.

\spProp The public command **\spProp** enforces *math mode*, and forwards the same *⟨keys⟩* in ‘#1’ to both ratios: a/b (‘#2’/‘#3’) and c/d (‘#4’/‘#5’), joined by a native **\MathMLintent** reading **_como** over the $=$ sign.

```

2434 \NewDocumentCommand \spProp { O{} m m m m }
2435 {
2436     \mode_if_math:TF
2437     {
2438         \group_begin:
2439         \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2440         \bool_if:NTF \l__spintent_sprazon_silent_cmd_bool
2441         {
2442             \MathMLintent { :silent } { = }
2443         }
2444         { \MathMLintent { _como } { = } }
2445         \__spintent_sprazon_exec:nnn { #1 } { #4 } { #5 }
2446         \group_end:
2447     }
2448     { \msg_error:nnn { spintent } { need-math-mode } { \spProp } }
2449 }

```

(End of definition for \spProp. This function is documented on page ??.)

11.30 The command \spang

The command **\spang** is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function **__spintent_luafun_spang_parse:n**. This separates the degrees, minutes, and seconds to construct a **MathML intent** tree with the correct postfixes for NVDA/MathCAT.

```

\l__spintent_spang_luaset_grado_str
\l__spintent_spang_luaset_minuto_str
\l__spintent_spang_luaset_segundo_str
\l__spintent_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module **spintent.lua**. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2450 \str_new:N \l__spintent_spang_luaset_grado_str
2451 \str_new:N \l__spintent_spang_luaset_minuto_str
2452 \str_new:N \l__spintent_spang_luaset_segundo_str
2453 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for \l__spintent_spang_luaset_grado_str and others.)

11.30.1 Definition of error messages for \spang

invalid-angle-format

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```
2454 \msg_new:nnnn { spintent } { invalid-angle-format }
2455 { Invalid~angle~format~in~#1. }
2456 {
2457   The~argument~'#2'~could~not~be~parsed. \\
2458 }
```

(End of definition for `invalid-angle-format`.)

11.30.2 Internal functions for \spang

__spintent_spang_process:n

The internal function `__spintent_spang_process:n` parses the angle input using `__spintent_luafun_spang_parse:n` after resetting the error flag in `\l__spintent_spang_luaaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\l__spintent_spang_luaaset_grado_str`, `\l__spintent_spang_luaaset_minuto_str`, and `\l__spintent_spang_luaaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```
2459 \cs_new_protected:Nn \__spintent_spang_process:n
2460 {
2461   \str_set:Nn \l__spintent_spang_luaaset_error_str { false }
2462   \__spintent_luafun_spang_parse:n { #1 }
2463   \str_if_eq:VnTF \l__spintent_spang_luaaset_error_str { true }
2464   {
2465     \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2466   }
2467   {
2468     \str_if_empty:NF \l__spintent_spang_luaaset_grado_str
2469     {
2470       \spunit{ \l__spintent_spang_luaaset_grado_str °}
2471     }
2472     \str_if_empty:NF \l__spintent_spang_luaaset_minuto_str
2473     {
2474       \MathMLintent { minutos : postfix($m) }
2475       {
2476         \MathMLarg { m }
2477         {
2478           \spunit{ \l__spintent_spang_luaaset_minuto_str ' }
2479         }
2480       }
2481     }
2482     \str_if_empty:NF \l__spintent_spang_luaaset_segundo_str
2483     {
2484       \MathMLintent { segundos : postfix($s) }
2485       {
2486         \MathMLarg { s }
2487         {
2488           \spunit { \l__spintent_spang_luaaset_segundo_str " }
2489         }
2490       }
2491     }
2492   }
2493 }
```

(End of definition for `__spintent_spang_process:n`.)

\spang

The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `__spintent_spang_process:n` to process the `{⟨argument⟩}`.

```
2494 \NewDocumentCommand \spang { m }
2495 {
2496   \mode_if_math:TF
2497   {
2498     \group_begin:
2499     \__spintent_spang_process:n { #1 }
2500     \group_end:
2501   }
2502   { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2503 }
```

(End of definition for `\spang`. This function is documented on page 23.)

11.31 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle \textit{argument} \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.31.1 Definition of keys for `\spnZ`

Now we define the $\langle \textit{keys} \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle \textit{keys} \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2504 \keys_define:nn { spintenc / spnZ }
2505 {
2506   cifras-sep .meta:nn = { spintenc / spnum } { cifras-sep = {#1} },
2507   read-sign .meta:nn = { spintenc / spnum } { read-sign = {#1} },
2508   wrap-parens .bool_set:N = \l__spintenc_spnz_wrap_parens_bool,
2509   wrap-parens .initial:n = false,
2510   wrap-parens .value_required:n = true,
2511   read-parens .bool_set:N = \l__spintenc_spnz_read_parens_bool,
2512   read-parens .initial:n = true,
2513   read-parens .value_required:n = true,
2514   unknown .code:n =
2515     {
2516       \str_set:Nn \l__spintenc_command_name_str { spnZ }
2517       \__spintenc_unknown_keys_cmd:n {#1}
2518     },
2519 }

```

(End of definition for `cifras-sep` and others.)

11.31.2 Definition of error messages

`spnz-not-integer` Raised when the $\langle \textit{argument} \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2520 \msg_new:nnnn { spintenc } { spnz-not-integer }
2521 { The~value~'#1'~is~not~a~valid~integer. }
2522 {
2523   Command~\token_to_str:N \spnZ~only~accepts~integers.\
2524 }

```

(End of definition for `spnz-not-integer`.)

11.31.3 Internal functions for `\spnZ`

`\__spintenc_spnz_render_parens:` The function `\__spintenc_spnz_render_parens:` first checks the state of the boolean variable `\l__spintenc_spnz` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2525 \cs_new_protected:Nn \__spintenc_spnz_render_parens:
2526 {
2527   \bool_if:NTF \l__spintenc_spnz_read_parens_bool
2528     {
2529       ( \__spintenc_spnum_print_num_start: )
2530     }
2531     {
2532       \MathMLintent { :silent } { ( }
2533       \__spintenc_spnum_print_num_start:
2534       \MathMLintent { :silent } { ) }
2535     }
2536 }

```

(End of definition for `\__spintenc_spnz_render_parens:.`)

`\__spintenc_spnz_process:nn` The function `\__spintenc_spnz_process:nn` first processes the $\langle \textit{keys} \rangle$ passed in ‘#1’ and then calls `\__spintenc_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\l__spintenc_luasets_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\l__spintenc_luasets_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\l__spintenc_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintenc_spnz_render_parens:` or `\__spintenc_spnum_print_num_start:`.

```

2537 \cs_new_protected:Nn \__spintenc_spnz_process:nn
2538 {

```

```

2539 \keys_set:nn { spintent / spnZ } {#1}
2540 \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2541 \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2542 {
2543   \tl_if_empty:NTF \__spintent_luaset_part_int_tl
2544   { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2545   {
2546     \bool_if:NTF \__spintent_spnz_wrap_parens_bool
2547     { \__spintent_spnz_render_parens: }
2548     { \__spintent_spnnum_print_num_start: }
2549   }
2550 }
2551 { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2552 }

```

(End of definition for `\__spintent_spnz_process:nn`.)

`\spnZ` Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2553 \NewDocumentCommand \spnZ { O{} m }
2554 {
2555   \mode_if_math:TF
2556   {
2557     \group_begin:
2558     \__spintent_spnz_process:nn { #1 } { #2 }
2559     \group_end:
2560   }
2561   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2562 }

```

(End of definition for `\spnZ`. This function is documented on page 16.)

11.32 The command `\spmQ`

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spintent.lua` via `\__spintent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

11.32.1 Definition of variables for `\spmQ`

```

\__spintent_spmQ_join_word_str
\__spintent_spnQ_intent_read_sign_str
\__spintent_spnQ_wrap_parens_l_tl
\__spintent_spnQ_wrap_parens_r_tl
\__spintent_saved_luamml_flag_int

```

The variable `\__spintent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\__spintent_spnQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘−’ are present in the integer part, used by the function `\__spintent_spmQ_print_scale_int:n`.

The variables `\__spintent_spnQ_wrap_parens_l_tl` and `\__spintent_spnQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spintent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The integer variable `\__spintent_saved_luamml_flag_int` temporarily stores the current value of `\__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the `sub_box` processed in isolation, before `\frac` has expanded.

```

2563 \str_new:N \__spintent_spmQ_join_word_str
2564 \str_new:N \__spintent_spnQ_intent_read_sign_str
2565 \tl_new:N \__spintent_spnQ_wrap_parens_l_tl
2566 \tl_new:N \__spintent_spnQ_wrap_parens_r_tl
2567 \int_new:N \__spintent_saved_luamml_flag_int

```

(End of definition for `\__spintent_spmQ_join_word_str` and others.)

11.32.2 Definition of error messages for `\spmQ`

```

spmQ-zero-denominator
spmQ-not-integer

```

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when the integer part passed in $\langle argument \rangle$ is not actually an integer.

```

2568 \msg_new:nnnn { spintent } { spmQ-zero-denominator }
2569 {
2570   Zero~denominator~in~\token_to_str:N \spmQ~\msg_line_context:.
2571 }
2572 {
2573   The~denominator~of~a~fraction~cannot~be~zero.\
2574 }
2575 \msg_new:nnnn { spintent } { spmQ-not-integer }

```

```

2576 {
2577   The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2578 }
2579 {
2580   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\\
2581 }

```

(End of definition for `spmQ-zero-denominator` and `spmQ-not-integer`.)

11.32.3 Definition of keys for `\spmQ`

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `print-dfrac`.

```

join-word 2582 \keys_define:nn { spintent / spmQ }
use-spnZ   2583 {
wrap-parens 2584   join-word      .choices:nn =
read-parens 2585     { entero , enteros , y }
print-dfrac 2586     {
unknown      2587       \int_case:nn { \l_keys_choice_int }
                2588       {
2589         {1} { \str_set:Nn \l__spintent_spmQ_join_word_str { entero } }
2590         {2} { \str_set:Nn \l__spintent_spmQ_join_word_str { enteros } }
2591         {3} { \str_set:Nn \l__spintent_spmQ_join_word_str { y } }
2592       }
2593     },
2594     join-word      .initial:n = y,
2595     join-word      .value_required:n = true,
2596     join-word/unknown .code:n =
2597       \msg_error:nneee { spintent } { unknown-choice }
2598       { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2599     use-spnZ      .bool_set:N = \l__spintent_spmQ_use_spnZ_bool,
2600     use-spnZ      .initial:n = true,
2601     use-spnZ      .value_required:n = true,
2602     print-dfrac   .bool_set:N = \l__spintent_spmQ_print_dfrac_bool,
2603     print-dfrac   .initial:n = false,
2604     print-dfrac   .value_required:n = true,
2605     wrap-parens   .bool_set:N = \l__spintent_spmQ_wrap_parens_bool,
2606     wrap-parens   .initial:n = false,
2607     wrap-parens   .value_required:n = true,
2608     read-parens   .bool_set:N = \l__spintent_spmQ_read_parens_bool,
2609     read-parens   .initial:n = true,
2610     read-parens   .value_required:n = true,
2611     unknown       .code:n =
2612       {
2613         \str_set:Nn \l__spintent_command_name_str { spmQ }
2614         \__spintent_unknown_keys_cmd:n {#1}
2615       },
2616   }

```

(End of definition for `join-word` and others.)

11.32.4 Internal functions for `\spmQ`

`\__spintent_spmQ_set_wrap_parens_size:` The function `\__spintent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from Lua^T_EX and sets the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `print-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2617 \cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
2618 {
2619   \int_case:nnF { \mathstyle }
2620   {
2621     { 0 }
2622     {
2623       \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2624       \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2625     }
2626     { 1 }
2627     {
2628       \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2629       \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2630     }

```

```

2631     { 2 }
2632     {
2633         \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2634         \tl_set:Nn \l__spinttent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2635     }
2636     { 3 }
2637     {
2638         \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2639         \tl_set:Nn \l__spinttent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2640     }
2641 }
2642 {
2643     \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \lparen }
2644     \tl_set:Nn \l__spinttent_spmQ_wrap_parens_r_tl { \rparen }
2645 }

```

If the key `print-dfrac` is active, we must reset the variables `\l__spinttent_spmQ_wrap_parens_l_tl` and `\l__spinttent_spmQ_wrap_parens_r_tl` to `\Bigl/\Bigr` unconditionally.

```

2646     \bool_if:NT \l__spinttent_spmQ_print_dfrac_bool
2647     {
2648         \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2649         \tl_set:Nn \l__spinttent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2650     }
2651 }

```

(End of definition for `\__spinttent_spmQ_set_wrap_parens_size:`)

```

\__spinttent_spmQ_start_wrap_parens:
\__spinttent_spmQ_stop_wrap_parens:

```

The functions `\__spinttent_spmQ_start_wrap_parens:` and `\__spinttent_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spinttent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spinttent_spmQ_wrap_parens_l_tl` or `\l__spinttent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spinttent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2652 \cs_new_protected:Nn \__spinttent_spmQ_start_wrap_parens:
2653 {
2654     \bool_if:NT \l__spinttent_spmQ_wrap_parens_bool
2655     {
2656         \__spinttent_spmQ_set_wrap_parens_size:
2657         \bool_if:NTF \l__spinttent_spmQ_read_parens_bool
2658         { \l__spinttent_spmQ_wrap_parens_l_tl }
2659         { \MathMLintent { :silent } { \l__spinttent_spmQ_wrap_parens_l_tl } }
2660     }
2661 }
2662 \cs_new_protected:Nn \__spinttent_spmQ_stop_wrap_parens:
2663 {
2664     \bool_if:NT \l__spinttent_spmQ_wrap_parens_bool
2665     {
2666         \__spinttent_spmQ_set_wrap_parens_size:
2667         \bool_if:NTF \l__spinttent_spmQ_read_parens_bool
2668         { \l__spinttent_spmQ_wrap_parens_r_tl }
2669         { \MathMLintent { :silent } { \l__spinttent_spmQ_wrap_parens_r_tl } }
2670     }
2671 }

```

(End of definition for `\__spinttent_spmQ_start_wrap_parens:` and `\__spinttent_spmQ_stop_wrap_parens:`)

```
\__spinttent_spmQ_set_join_word:
```

The function `\__spinttent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spinttent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spinttent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «*entero*», «*enteros*» or «*y*» respectively.

```

2672 \cs_new_protected:Nn \__spinttent_spmQ_set_join_word:
2673 {
2674     \str_case:Nn \l__spinttent_spmQ_join_word_str
2675     {
2676         { entero } { \MathMLintent { _entero } { \c__spinttent_invisible_sep_int } }
2677         { enteros } { \MathMLintent { _enteros } { \c__spinttent_invisible_sep_int } }
2678         { y } { \MathMLintent { _y- } { \c__spinttent_invisible_sep_int } }
2679     }
2680 }

```

(End of definition for `\__spinttent_spmQ_set_join_word:`.)

`\__spinttent_spmQ_print_scale_int:`

The function `\__spinttent_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spinttent_luafun_spmQ_scale_int` it temporarily disables `luamml` by saving the current value of `\__luamml_flag_int` into `\__spinttent_saved_luamml_flag_int` and setting it to `0` (skip completely). This prevents `luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\__spinttent_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\__spinttent_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `print-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```
2681 \cs_new_protected:Nn \__spinttent_spmQ_print_scale_int:
2682 {
2683   \int_set_eq:Nc \__spinttent_saved_luamml_flag_int { \__luamml_flag_int }
2684   \int_set:Nn \__luamml_flag_int { 0 }
2685   \bool_if:NTF \__spinttent_spmQ_print_dfrac_bool
2686     { \__spinttent_luafun_spmQ_scale_int:Vn \__spinttent_luaset_part_int_tl { dfrac } }
2687     { \__spinttent_luafun_spmQ_scale_int:Vn \__spinttent_luaset_part_int_tl { frac } }
2688   \int_set_eq:cN { \__luamml_flag_int } \__spinttent_saved_luamml_flag_int
2689 }
```

(End of definition for `\__spinttent_spmQ_print_scale_int:`.)

`\__spinttent_spmQ_print_scale_int:n`

The function `\__spinttent_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in ‘#1’. It first calls `\__spinttent_luafun_clean_split_and_set:n` on ‘#1’ to parse and populate the shared Lua variables, then checks the status of `\__spinttent_luaset_has_integer_str`.

If the status is `true`, it inspects `\__spinttent_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spinttent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\__spinttent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a `MathML` argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An `\__spinttent_invisible_sep:` call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the `MathML` structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```
2690 \cs_new_protected:Nn \__spinttent_spmQ_print_scale_int:n
2691 {
2692   \__spinttent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2693   \str_if_eq:VnTF \__spinttent_luaset_has_integer_str { true }
2694   {
2695     \tl_if_empty:NTF \__spinttent_luaset_sign_tl
2696     {
2697       \__spinttent_spmQ_print_scale_int:
2698     }
2699     {
2700       \str_if_eq:VnTF \__spinttent_luaset_sign_tl { + }
2701       {
2702         \str_set:Nn
2703           \__spinttent_spmQ_intent_read_sign_str { _más:prefix($e) }
2704       }
2705       {
2706         \str_set:Nn
2707           \__spinttent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2708       }
2709       \MathMLintent { \__spinttent_spmQ_intent_read_sign_str }
2710       {
2711         \__spinttent_luaset_sign_tl \__spinttent_invisible_sep_int % \__spinttent_invisible_sep
2712         \MathMLarg { e } { \__spinttent_spmQ_print_scale_int: }
2713       }
2714     }
2715   }
2716   { \msg_error:nnn { spinttent } { spmQ-not-integer } { #1 } }
2717 }
```

(End of definition for `\__spinttent_spmQ_print_scale_int:n`.)

`\__spinttent_spmQ_print_spmZ_dfrac:nn`

The function `\__spinttent_spmQ_print_spmZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\__spinttent_luafun_clean_split_and_set:`

on ‘#1’ to validate that the numerator is an integer; if `\l__spintent_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l__spintent_spmQ_use_spnZ_bool` and `\l__spintent_spmQ_print_dfrac_bool`, controlled respectively by the keys `use-spnZ` and `print-dfrac`. When `use-spnZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c`: if `print-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2718 \cs_new_protected:Nn \l__spintent_spmQ_print_spnZ_dfrac:nn
2719 {
2720   \l__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2721   \str_if_eq:VnT \l__spintent_luaset_has_integer_str { false }
2722   {
2723     \msg_error:nnn { spintent } { spmQ-not-integer } { #1 }
2724   }
2725   \bool_if:NTF \l__spintent_spmQ_use_spnZ_bool
2726   {
2727     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2728     { \spnZ { #1 } } { \spnZ { #2 } }
2729   }
2730   {
2731     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2732     { #1 } { #2 }
2733   }
2734 }

```

(End of definition for `\l__spintent_spmQ_print_spnZ_dfrac:nn`.)

`\l__spintent_spmQ_print:nnn`

The function `\l__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. It opens the optional parentheses via `\l__spintent_spmQ_start_wrap_parens:`, then calls `\l__spintent_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\l__spintent_spmQ_set_join_word:`. Then it calls `\l__spintent_spmQ_print_spnZ_dfrac:nn` to typeset the fraction using the keys `print-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\l__spintent_spmQ_stop_wrap_parens:`.

```

2735 \cs_new_protected:Nn \l__spintent_spmQ_print:nnn
2736 {
2737   \l__spintent_spmQ_start_wrap_parens:
2738   \l__spintent_spmQ_print_scale_int:n { #1 }
2739   \l__spintent_spmQ_set_join_word:
2740   \l__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2741   \l__spintent_spmQ_stop_wrap_parens:
2742 }

```

(End of definition for `\l__spintent_spmQ_print:nnn`.)

`\l__spintent_spmQ_exec:nnnn`

The internal function `\l__spintent_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then processes the denominator input in ‘#4’ using `\l__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\l__spintent_luaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\l__spintent_spmQ_print:nnn` to format the “mixed number”.

```

2743 \cs_new_protected:Nn \l__spintent_spmQ_exec:nnnn
2744 {
2745   \keys_set:nn { spintent / spmQ } { #1 }
2746   \l__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2747   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2748   {
2749     \int_compare:nNnTF { \l__spintent_luaset_part_int_tl } = { 0 }
2750     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2751     { \l__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2752   }
2753   { \l__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2754 }

```

(End of definition for `\l__spintent_spmQ_exec:nnnn`.)

`\spmQ` The command `\spmQ` is the public interface for formatting “*mixed numbers*”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional *keys* and the *arguments*.

```

2755 \NewDocumentCommand \spmQ { O{} m m m }
2756 {
2757   \mode_if_math:TF
2758   {
2759     \group_begin:
2760     \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2761     \group_end:
2762   }
2763   { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2764 }

```

(End of definition for `\spmQ`. This function is documented on page 17.)

11.33 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\__spintent_geo_luaset_error_str`, `\__spintent_geo_luaset_intent_str`, `\__spintent_geo_luaset_print_tl`, `\__spintent_geo_luaset_sub_str` and `\__spintent_geo_luaset_tipo_str` — constructing a valid MathCAT intent — and a single point is always annotated through `\__spintent_mathmlarg_point:nnn`. Every command also activates `\__spintent_scope_attr_int` within its own group, so the shared queue never claims content typeset outside a geo command in the same formula.

11.33.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

\__spintent_geo_luaset_error_str
\__spintent_geo_luaset_intent_str
\__spintent_geo_luaset_print_tl
\__spintent_geo_luaset_is_name_str
\__spintent_geo_luaset_poly_sym_str
\__spintent_geo_luaset_angulo_case_str
\__spintent_geo_luaset_concept_str
\__spintent_geo_luaset_letra_word_str
\__spintent_geo_luaset_sub_spoken_str
\__spintent_geo_luaset_nombre_es_num_str
\__spintent_geo_luaset_letra_str
\__spintent_geo_luaset_sub_str
\__spintent_geo_luaset_tipo_str

2765 \str_new:N \__spintent_geo_luaset_error_str
2766 \str_new:N \__spintent_geo_luaset_intent_str
2767 \tl_new:N \__spintent_geo_luaset_print_tl
2768 \clist_new:N \__spintent_geo_luaset_points_clist
2769 \str_new:N \__spintent_geo_luaset_is_name_str
2770 \str_new:N \__spintent_geo_luaset_poly_sym_str
2771 \str_new:N \__spintent_geo_luaset_angulo_case_str
2772 \str_new:N \__spintent_geo_luaset_concept_str
2773 \str_new:N \__spintent_geo_luaset_letra_word_str
2774 \str_new:N \__spintent_geo_luaset_sub_spoken_str
2775 \str_new:N \__spintent_geo_luaset_nombre_es_num_str
2776 \str_new:N \__spintent_geo_luaset_letra_str
2777 \str_new:N \__spintent_geo_luaset_sub_str
2778 \str_new:N \__spintent_geo_luaset_tipo_str

```

(End of definition for `\__spintent_geo_luaset_error_str` and others.)

11.34 The command `\spcoord`

The user command `\spcoord` typesets an ordered pair (x, y) , with the comma (or semicolon) between them carrying its own `:infix` intent, and each side wrapped with `:prefix($vp)/:prefix($vn)` when it starts with $+/-$, so the sign is always read as *más/menos* — never *positivo/negativo*, which is not how MathCAT reads a bare sign in this position.

11.34.1 Definition of variables for `\spcoord`

Variables set by the function `\__spintent_luafun_spcoord_parse_and_set:n`. The string variable `\__spintent_spcoord_luaset_sep_str` is `,` or `;`, whichever the argument actually used. For the abscissa and ordinate values, the variables `\__spintent_spcoord_luaset_ #1 _sign_str` is `+`, `-`, or empty; the variables `\__spintent_spcoord_luaset_ #1 _type_str` is `number` or `raw`; the variables `\__spintent_spcoord_luaset_ #1 _value_str` is the value with the sign already removed; and the variables `\__spintent_spcoord_luaset_ #1 _has_frac_str` is `none`, `frac`, or `dfrac`.

```

\__spintent_spcoord_luaset_error_str
\__spintent_spcoord_luaset_sep_str
\__spintent_spcoord_luaset_x_sign_str
\__spintent_spcoord_luaset_x_type_str
\__spintent_spcoord_luaset_x_value_str
\__spintent_spcoord_luaset_x_has_frac_str
\__spintent_spcoord_luaset_y_sign_str
\__spintent_spcoord_luaset_y_type_str
\__spintent_spcoord_luaset_y_value_str
\__spintent_spcoord_luaset_y_has_frac_str

2779 \str_new:N \__spintent_spcoord_luaset_error_str
2780 \str_new:N \__spintent_spcoord_luaset_sep_str
2781 \clist_map_inline:nn { x , y }
2782 {
2783   \str_new:c { \__spintent_spcoord_luaset_ #1 _sign_str }
2784   \str_new:c { \__spintent_spcoord_luaset_ #1 _type_str }
2785   \str_new:c { \__spintent_spcoord_luaset_ #1 _value_str }
2786   \str_new:c { \__spintent_spcoord_luaset_ #1 _has_frac_str }
2787 }

```

(End of definition for `\__spintent_spcoord_luaset_error_str` and others.)

`\l__spintrent_spcoord_raw_arg_tl` holds the $\langle argument \rangle$ exactly as given. The variable `\l__spintrent_spcoord_intent_str` holds the `:infix` word for the separator `_coma:infix` or `_punto-y-coma:infix`. The variables `\l__spintrent_spcoord_wrap_parens_l_tl`/`\l__spintrent_spcoord_wrap_parens_r_tl` hold the opening/closing delimiter, sized by the function `\__spintrent_spcoord_set_wrap_parens_size:`.

```

2788 \tl_new:N \l__spintrent_spcoord_raw_arg_tl
2789 \str_new:N \l__spintrent_spcoord_intent_str
2790 \tl_new:N \l__spintrent_spcoord_wrap_parens_l_tl
2791 \tl_new:N \l__spintrent_spcoord_wrap_parens_r_tl

```

(End of definition for `\l__spintrent_spcoord_raw_arg_tl` and others.)

11.34.2 Definition of error messages for `\spcoord`

`spcoord-invalid-arg`

```

2792 \msg_new:nnn { spintrent } { spcoord-invalid-arg }
2793 {
2794   Invalid~argument~'#1'~for~\token_to_str:N \spcoord.
2795 }

```

(End of definition for `spcoord-invalid-arg`.)

11.34.3 Definition of keys for `\spcoord`

`silent-cmd`

We define the key `silent-cmd`.

```

2796 \keys_define:nn { spintrent / spcoord }
2797 {
2798   silent-cmd .bool_set:N = \l__spintrent_spcoord_silent_cmd_bool,
2799   silent-cmd .initial:n = false,
2800   silent-cmd .value_required:n = true,
2801   unknown .code:n =
2802     {
2803       \str_set:Nn \l__spintrent_command_name_str { spcoord }
2804       \__spintrent_unknown_keys_cmd:n {#1}
2805     },
2806 }

```

(End of definition for `silent-cmd`.)

11.34.4 Internal functions for `\spcoord`

`\__spintrent_spcoord_set_wrap_parens_size:`

With neither side carrying a fraction, the parentheses stay unsized. Otherwise, `\mathstyle` picks `\Bigl/\Bigr` for display, `\bigl/\bigr` for text. When either side is `\dfrac`, the parentheses are forced to `\Bigl/\Bigr` regardless of `\mathstyle`, since `\dfrac` always renders at display height.

```

2807 \cs_new_protected:Nn \__spintrent_spcoord_set_wrap_parens_size:
2808 {
2809   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \lparen }
2810   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \rparen }
2811   \bool_if:nT
2812     {
2813       \bool_lazy_or_p:nn
2814         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_x_has_frac_str { none } } }
2815         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_y_has_frac_str { none } } }
2816     }
2817     {
2818       \int_case:nnF { \mathstyle }
2819       {
2820         { 0 }
2821         {
2822           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2823           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2824         }
2825         { 1 }
2826         {
2827           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2828           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2829         }
2830         { 2 }
2831         {
2832           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2833           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2834         }
2835         { 3 }
2836         {
2837           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }

```

```

2838         \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \bigr \rparen }
2839     }
2840 }
2841 { }
2842 \bool_if:nT
2843 {
2844     \bool_lazy_or_p:nn
2845     { \str_if_eq_p:Vn \l__spintenc_spcoord_luaset_x_has_frac_str { dfraction } }
2846     { \str_if_eq_p:Vn \l__spintenc_spcoord_luaset_y_has_frac_str { dfraction } }
2847 }
2848 {
2849     \tl_set:Nn \l__spintenc_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2850     \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2851 }
2852 }
2853 }

```

(End of definition for `\__spintenc_spcoord_set_wrap_parens_size:`.)

`\__spintenc_spcoord_value:n`

With `\__spintenc_spcoord_luaset_ #1 _type_str` equal to `number`, calls `\spnum` on `\__spintenc_spcoord_value:n`. Otherwise, typesets it as given — a `\frac/\dfrac` or a letter expression, untouched. Receives `x` or `y` in ‘`#1`’.

```

2854 \cs_new_protected:Nn \__spintenc_spcoord_value:n
2855 {
2856     \str_if_eq:eeTF
2857     { \str_use:c { \__spintenc_spcoord_luaset_ #1 _type_str } } { number }
2858     {
2859         \exp_args:Ne \spnum { \str_use:c { \__spintenc_spcoord_luaset_ #1 _value_str } }
2860     }
2861     { \str_use:c { \__spintenc_spcoord_luaset_ #1 _value_str } }
2862 }

```

(End of definition for `\__spintenc_spcoord_value:n`.)

`\__spintenc_spcoord_print:n`

With `\__spintenc_spcoord_luaset_ #1 _sign_str` equal to `+` or `-`, wraps the sign and `\__spintenc_spcoord_value:n` together with `_más:prefix($vp)/_menos:prefix($vn)`, referencing the value through `read as más/menos`. With no sign, delegates directly to `\__spintenc_spcoord_value:n`. Receives `x` or `y` in ‘`#1`’.

```

2863 \cs_new_protected:Nn \__spintenc_spcoord_print:n
2864 {
2865     \str_case:enF { \str_use:c { \__spintenc_spcoord_luaset_ #1 _sign_str } }
2866     {
2867         { + }
2868         {
2869             \MathMLintent { _más:prefix ($vp) }
2870             {
2871                 \str_use:c { \__spintenc_spcoord_luaset_ #1 _sign_str }
2872                 \MathMLarg{ vp } { \__spintenc_spcoord_value:n { #1 } }
2873             }
2874         }
2875         { - }
2876         {
2877             \MathMLintent { _menos :prefix($vn) }
2878             {
2879                 \str_use:c { \__spintenc_spcoord_luaset_ #1 _sign_str }
2880                 \MathMLarg{ vn } { \__spintenc_spcoord_value:n { #1 } }
2881             }
2882         }
2883     }
2884     { \__spintenc_spcoord_value:n { #1 } }
2885 }

```

(End of definition for `\__spintenc_spcoord_print:n`.)

`\__spintenc_spcoord_intent_infix:`

Sets `\__spintenc_spcoord_intent_str` to `_coma:infix` or `_punto-y-coma:infix`, per `\__spintenc_spcoord_intent:n`, then wraps both sides — each referenced through with it, `$x/$y` standing for `\__spintenc_spcoord_print:n`’s own output.

```

2886 \cs_new_protected:Nn \__spintenc_spcoord_intent_infix:
2887 {
2888     \str_if_eq:VnTF \__spintenc_spcoord_luaset_sep_str { ; }
2889     { \str_set:Nn \__spintenc_spcoord_intent_str { _punto-y-coma:infix } }
2890     { \str_set:Nn \__spintenc_spcoord_intent_str { _coma:infix } }
2891     \MathMLintent{ \__spintenc_spcoord_intent_str ($x, $y)}

```

```

2892     {
2893       \MathMLarg{x}{ \__spintenc_spcoord_print:n { x } }
2894       \MathMLintent { :silent } { \__spintenc_spcoord_luaset_sep_str }
2895       \MathMLarg{y}{ \__spintenc_spcoord_print:n { y } }
2896     }
2897   }

```

(End of definition for __spintenc_spcoord_intent_infix:.)

__spintenc_spcoord_mathml: With `silent-cmd=true`, everything — parentheses and argument alike — is wrapped in a single `_-`, reading nothing at all: `\__spintenc_spcoord_raw_arg_tl` is typeset exactly as given, untouched by `\__spintenc_spcoord_print:n/\__spintenc_spcoord_intent_infix:`. Otherwise, the parentheses are silenced individually and `\__spintenc_spcoord_intent_infix:` provides the reading in between.

```

2898 \cs_new_protected:Nn \__spintenc_spcoord_mathml:
2899 {
2900   \bool_if:NTF \__spintenc_spcoord_silent_cmd_bool
2901   {
2902     \MathMLintent { _- }
2903     {
2904       \__spintenc_spcoord_wrap_parens_l_tl
2905       \__spintenc_spcoord_raw_arg_tl
2906       \__spintenc_spcoord_wrap_parens_r_tl
2907     }
2908   }
2909   {
2910     \MathMLintent { :silent } { \__spintenc_spcoord_wrap_parens_l_tl }
2911     \__spintenc_spcoord_intent_infix:
2912     \MathMLintent { :silent } { \__spintenc_spcoord_wrap_parens_r_tl }
2913   }
2914 }

```

(End of definition for __spintenc_spcoord_mathml:.)

__spintenc_spcoord_exec:nn Sets the *(keys)* in ‘#1’ and `\__spintenc_spcoord_raw_arg_tl` to ‘#2’. Calls `\__spintenc_luafun_spcoord_parse_and_set:n` either way — even with `silent-cmd=true`, since `\__spintenc_spcoord_set_wrap_parens_size:` still needs `\__spintenc_spcoord_luaset_ #1 _has_frac_str` — but with `silent-cmd=true`, forces `\__spintenc_spcoord_luaset_error_str` back to `false` rather than reporting it: the raw *{(argument)}* is typeset regardless of whether it parsed. Otherwise, reports `spcoord-invalid-arg` on error. Either way, sizes the parentheses, then delegates to `\__spintenc_spcoord_mathml:` unless there was an error.

```

2915 \cs_new_protected:Nn \__spintenc_spcoord_exec:nn
2916 {
2917   \keys_set:nn { spintenc / spcoord } { #1 }
2918   \tl_set:Nn \__spintenc_spcoord_raw_arg_tl { #2 }
2919   \bool_if:NTF \__spintenc_spcoord_silent_cmd_bool
2920   {
2921     \__spintenc_luafun_spcoord_parse_and_set:n { \exp_not:n { #2 } }
2922     \str_set:Nn \__spintenc_spcoord_luaset_error_str { false }
2923   }
2924   {
2925     \__spintenc_luafun_spcoord_parse_and_set:n { \exp_not:n { #2 } }
2926     \str_if_eq:VnT \__spintenc_spcoord_luaset_error_str { true }
2927     { \msg_error:nnn { spintenc } { spcoord-invalid-arg } { \exp_not:n { #2 } } }
2928   }
2929   \__spintenc_spcoord_set_wrap_parens_size:
2930   \str_if_eq:VnF \__spintenc_spcoord_luaset_error_str { true }
2931   { \__spintenc_spcoord_mathml: }
2932 }

```

(End of definition for __spintenc_spcoord_exec:nn.)

\spcoord The public command enforces *math mode*, opens a local group, and delegates to `\__spintenc_spcoord_exec:nn`.

```

2933 \NewDocumentCommand \spcoord { 0{} m }
2934 {
2935   \mode_if_math:TF
2936   {
2937     \group_begin:
2938     \attribute \__spintenc_scope_attr_int = 1
2939     \__spintenc_spcoord_exec:nn { #1 } { #2 }
2940     \group_end:
2941   }
2942   { \msg_error:nnn { spintenc } { need-math-mode } { \spcoord } }
2943 }

```

(End of definition for `\spcoord`. This function is documented on page 17.)

11.35 The function `\__spintent_mathmlarg_point:nnn`

Scratch variables for `\__spintent_mathmlarg_point:nnn`: the reference name (#1), and the calling command's own `read-cmd/silent-cmd` state, looked up by name (#2).

```

2944 \tl_new:N \__spintent_mathmlarg_point_ref_tl
2945 \str_new:N \__spintent_mathmlarg_point_mode_str
2946 \tl_new:N \__spintent_mathmlarg_point_kv_tl
2947 \bool_new:N \__spintent_mathmlarg_point_silent_bool

```

(End of definition for `\__spintent_mathmlarg_point_ref_tl` and others.)

Annotates one point (#3) with `arg` and, when applicable, a literal `intent`, without wrapping it in its own `<mrow>`. Concept `intent` is passed empty on purpose: the caller applies it externally, via a native `\MathMLintent`.

With `\__spintent_geo_luaset_tipo_str` equal to `sub`, for `read-cmd=school` the letter alone is tagged through `\__spintent_luafun_mrow_intent_arg:nn`, and the subscript is appended separately, silenced. For `read-cmd=mathcat`, letter and subscript are typeset together, tagged as a whole through a native `\MathMLarg`.

```

2948 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
2949 {
2950   \tl_set:Nn \__spintent_mathmlarg_point_ref_tl {#1}
2951   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
2952   \bool_set_eq:Nc \__spintent_mathmlarg_point_silent_bool { \__spintent_ #2 _silent_cmd_bool }
2953   \str_set_eq:Nc \__spintent_mathmlarg_point_mode_str { \__spintent_ #2 _mode_str }
2954   \__spintent_invisible_sep:
2955   \str_case:VnF \__spintent_geo_luaset_tipo_str
2956   {
2957     { sub }
2958     {
2959       \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
2960       {
2961         \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
2962         { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
2963         \__spintent_luafun_mrow_intent_arg:nn
2964         { } { \__spintent_mathmlarg_point_kv_tl }
2965         {
2966           \__spintent_geo_luaset_letra_str
2967           \c_math_subscript_token
2968           { \MathMLintent { :silent } { \__spintent_geo_luaset_sub_str } }
2969         }
2970       }
2971     }
2972     {
2973       \__spintent_inner_sep:
2974       \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
2975       {
2976         {
2977           \__spintent_geo_luaset_letra_str
2978           \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
2979         }
2980       }
2981     }
2982   }
2983 }

```

With `\__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the letter with a literal `intent` and appends the mark unsilenced (a prime is not spoken on the base, so nothing to repeat); `read-cmd=mathcat` tags letter and mark together through a native `\MathMLarg`.

```

2984   { prima }
2985   {
2986     \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
2987     {
2988       \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
2989       { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
2990       \__spintent_luafun_mrow_intent_arg:nn
2991       { } { \__spintent_mathmlarg_point_kv_tl }
2992       { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
2993     }
2994   }
2995   {

```

```

2996         \__spintenc_inner_sep:
2997         \MathMLarg { \__spintenc_mathmlarg_point_ref_tl }
2998         {
2999             { \__spintenc_geo_luaset_letra_str \__spintenc_geo_luaset_sub_str }
3000             }
3001         }
3002     }
3003 }
3004 }

```

Otherwise (no subscript, no prime), the same distinction applies to the combined `\__spintenc_geo_luaset_print_tl` as a whole — the shared queue for `school` (or when `silent-cmd` is `true`), a native `\MathMLarg` for `mathcat`, never mixed within the same formula, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

3005     {
3006         \bool_if:NTF \__spintenc_mathmlarg_point_silent_bool
3007         {
3008             \tl_set:Nc \__spintenc_mathmlarg_point_kv_tl
3009             { arg = "\__spintenc_mathmlarg_point_ref_tl", intent = "-_" }
3010             \__spintenc_luafun_mrow_intent_arg:nn
3011             { } % debe ir vacío
3012             { \__spintenc_mathmlarg_point_kv_tl }
3013             { \__spintenc_geo_luaset_print_tl }
3014         }
3015         {
3016             \str_if_eq:VnTF \__spintenc_mathmlarg_point_mode_str { school }
3017             {
3018                 \tl_set:Nc \__spintenc_mathmlarg_point_kv_tl
3019                 { arg = "\__spintenc_mathmlarg_point_ref_tl", intent = "- \__spintenc_geo_luaset_print_tl" }
3020                 \__spintenc_luafun_mrow_intent_arg:nn
3021                 { } % debe ir vacío
3022                 { \__spintenc_mathmlarg_point_kv_tl }
3023                 { \__spintenc_geo_luaset_print_tl }
3024             }
3025             {
3026                 \MathMLarg { \__spintenc_mathmlarg_point_ref_tl }
3027                 { \__spintenc_geo_luaset_print_tl }
3028             }
3029         }
3030     }
3031 }
3032 \cs_generate_variant:Nn \__spintenc_mathmlarg_point:nnn { nnV }

```

(End of definition for `\__spintenc_mathmlarg_point:nnn`.)

11.36 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintenc.lua`, constructing a valid MathCAT intent.

11.36.1 Definition of variables for `\spPoint`

Internal variables holding the values set by `read-cmd` and `prefix`.

```

\__spintenc_spPoint_mode_str
\__spintenc_spPoint_prefix_tl
\__spintenc_spPoint_intent_arg_tl
3033 \str_new:N \__spintenc_spPoint_mode_str
3034 \tl_new:N \__spintenc_spPoint_prefix_tl
3035 \tl_new:N \__spintenc_spPoint_intent_arg_tl

```

(End of definition for `\__spintenc_spPoint_mode_str`, `\__spintenc_spPoint_prefix_tl`, and `\__spintenc_spPoint_intent_arg_tl`.)

11.36.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point`

The “error” `spPoint-invalid-point` is raised when the argument passed in ‘#1’ is not a single uppercase letter with an optional subscript or prime modifier.

```

3036 \msg_new:nnnn { spintenc } { spPoint-invalid-point }
3037 {
3038     Invalid~point~format~in~\token_to_str:N \spPoint:~'~#1'.
3039 }
3040 {
3041     The~argument~must~be~a~single~uppercase~letter~with~an~optional~
3042     subscript~(\_{...})~or~prime~(').\\
3043 }

```

(End of definition for `spPoint-invalid-point`.)

11.36.3 Definition of keys for \spPoint

We define the *(keys)* `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```

3044 \keys_define:nn { spintent / spPoint }
3045 {
3046   read-cmd          .choices:n = { school , mathcat }
3047   {
3048     \int_case:nn { \l_keys_choice_int }
3049     {
3050       {1} { \str_set:Nn \l__spintent_spPoint_mode_str { school } }
3051       {2} { \str_set:Nn \l__spintent_spPoint_mode_str { mathcat } }
3052     }
3053   },
3054   read-cmd          .initial:n = school,
3055   read-cmd          .value_required:n = true,
3056   read-cmd / unknown .code:n =
3057     \msg_error:nneee { spintent } { unknown-choice }
3058     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3059   prefix            .code:n =
3060     { \__spintent_sanitize_affix:Nn \l__spintent_spPoint_prefix_tl {#1} },
3061   prefix            .initial:n = ,
3062   prefix            .value_required:n = true,
3063   silent-cmd        .bool_set:N = \l__spintent_spPoint_silent_cmd_bool,
3064   silent-cmd        .initial:n = false,
3065   silent-cmd        .value_required:n = true,
3066   concept           .bool_set:N = \l__spintent_spPoint_concept_bool,
3067   concept           .initial:n = true,
3068   concept           .value_required:n = true,
3069   unknown           .code:n =
3070     {
3071       \str_set:Nn \l__spintent_command_name_str { spPoint }
3072       \__spintent_unknown_keys_cmd:n {#1}
3073     },
3074 }

```

(End of definition for `read-cmd` and others.)

11.36.4 Internal functions for \spPoint

`\__spintent_spPoint_print:n` The internal function `\__spintent_spPoint_print:n` processes the *{(argument)}* passed in ‘#1’. Calls `\__spintent_luafun_geo_point_parse_and_set:n`, reporting `spPoint-invalid-point` on error. If `silent-cmd` is `true` or `concept` is `false`, sets `\l__spintent_spPoint_intent_arg_tl` to `\_($pt)` — forced further to `\_-` when `silent-cmd` is `true` and `read-cmd` is not `school`, since `\__spintent_spPoint_mathml:n` then tags through a native `\MathMLarg` that needs the concept silenced too. Otherwise sets it to `punto:prefix($pt)`, prefixed with `prefix` when given. Finally calls `\__spintent_spPoint_mathml:n`.

```

3075 \cs_new_protected:Nn \__spintent_spPoint_print:n
3076 {
3077   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
3078   \str_if_eq:VnT \l__spintent_geo_luasets_error_str { true }
3079   {
3080     \msg_error:nne { spintent } { spPoint-invalid-point } { \exp_not:n {#1} }
3081   }
3082   \bool_if:nTF
3083   {
3084     \bool_lazy_or:p:nn
3085     { \l__spintent_spPoint_silent_cmd_bool }
3086     { ! \l__spintent_spPoint_concept_bool }
3087   }
3088   {
3089     \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { \_($pt) }
3090     \bool_if:NT \l__spintent_spPoint_silent_cmd_bool
3091     {
3092       \str_if_eq:VnF \l__spintent_spPoint_mode_str { school }
3093       {
3094         \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { \_ - }
3095       }
3096     }
3097   }
3098   {
3099     \tl_set:Ne \l__spintent_spPoint_intent_arg_tl
3100     {
3101       \tl_if_empty:NF \l__spintent_spPoint_prefix_tl

```

```

3102         {
3103             \tl_use:N \l__spintent_spPoint_prefix_tl -
3104         }
3105         punto:prefix($pt)
3106     }
3107 }
3108 \__spintent_spPoint_mathml:n { #1 }
3109 }

```

(End of definition for `\__spintent_spPoint_print:n`.)

`\__spintent_spPoint_mathml:n` The function `\__spintent_spPoint_mathml:n` calls `\__spintent_invisible_sep:` first, then wraps `\__spintent_mathmlarg_point:nnn` — which parses ‘#1’, tags it `arg="pt"`, and silences a real subscript or prime on its own — inside a native `\MathMLintent` carrying `\l__spintent_spPoint_intent_arg_tl` as the concept `intent`, with `\__spintent_inner_sep:` first inside that group.

```

3110 \cs_new_protected:Nn \__spintent_spPoint_mathml:n
3111 {
3112     \__spintent_invisible_sep:
3113     \MathMLintent { \l__spintent_spPoint_intent_arg_tl }
3114     {
3115         \__spintent_inner_sep:
3116         \__spintent_mathmlarg_point:nnn { pt } { spPoint } { #1 }
3117     }
3118 }

```

(End of definition for `\__spintent_spPoint_mathml:n`.)

`\__spintent_spPoint_exec:nnn` The function `\__spintent_spPoint_exec:nnn` receives the optional `<keys>` in ‘#1’ and the point `{<argument>}` in ‘#2’, delegating to `\__spintent_spPoint_print:n`. With ‘#3’ given (the optional coordinate), silences the gap between the point and the coordinate first (otherwise read as an implicit product), then temporarily aliases `\l__spintent_spcoord_silent_cmd_bool` to `\l__spintent_spPoint_silent_cmd_bool` and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\__spintent_spcoord_set_wrap_parens_size:` and `\__spintent_spcoord_math`

```

3119 \cs_new_protected:Nn \__spintent_spPoint_exec:nnn
3120 {
3121     \keys_set:nn { spintent / spPoint } { #1 }
3122     \__spintent_spPoint_print:n { #2 }
3123     \tl_if_novalue:nF {#3}
3124     {
3125         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % silencia el "por"
3126         \bool_set_eq:NN \l__spintent_spcoord_silent_cmd_bool \l__spintent_spPoint_silent_cmd_bool
3127         \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#3}
3128         \bool_if:NTF \l__spintent_spcoord_silent_cmd_bool
3129         {
3130             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3131             \str_set:Nn \l__spintent_spcoord_luaseta_error_str { false }
3132         }
3133         {
3134             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3135             \str_if_eq:VnT \l__spintent_spcoord_luaseta_error_str { true }
3136             { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3137         }
3138         \__spintent_spcoord_set_wrap_parens_size:
3139         \str_if_eq:VnF \l__spintent_spcoord_luaseta_error_str { true }
3140         { \__spintent_spcoord_mathml: }
3141     }
3142 }

```

(End of definition for `\__spintent_spPoint_exec:nnn`.)

`\spPoint` The public command `\spPoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nnn`.

```

3143 \NewDocumentCommand \spPoint { 0{} m d() }
3144 {
3145     \mode_if_math:TF
3146     {
3147         \group_begin:
3148         \attribute \l__spintent_scope_attr_int = 1
3149         \__spintent_spPoint_exec:nnn { #1 } { #2 } { #3 }
3150         \group_end:
3151     }

```

```

3152     { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
3153   }

```

(End of definition for `\spPoint`. This function is documented on page 18.)

11.37 The command `\spvec`

The user command `\spvec` supports two types of notation for the input $\langle argument \rangle$: a pair of points (with `\overrightarrow` applied), or a one-letter name (with `\vec` applied instead). Both notations are automatically detected by the Lua module. An optional coordinate, in parentheses, may follow the $\langle argument \rangle$.

11.37.1 Definition of variables for `\spvec`

The variable `\l__spinttent_spvec_mode_str` stores the value set by the key `read-cmd` and the variable `\l__spinttent_spvec_prefix_tl` stores the value set by the key `prefix`.

```

3154 \str_new:N \l__spinttent_spvec_mode_str
3155 \tl_new:N \l__spinttent_spvec_prefix_tl
3156 \tl_new:N \l__spinttent_spvec_intent_arg_tl
3157 \tl_new:N \l__spinttent_spvec_kv_tl
3158 \seq_new:N \l__spinttent_spvec_pt_seq
3159 \tl_new:N \l__spinttent_spvec_refs_tl
3160 \int_new:N \l__spinttent_spvec_pt_idx_int

```

(End of definition for `\l__spinttent_spvec_mode_str` and others.)

11.37.2 Definition of error messages for `\spvec`

`spvec-invalid-arg` The “error” `spvec-invalid-arg` is raised when the argument fails to parse, when a one-letter name carries a prime, or when the point list is not exactly two points.

```

3161 \msg_new:nnn { spintent } { spvec-invalid-arg }
3162 {
3163   Invalid~argument~'#1'~for~\token_to_str:N \spvec.
3164 }

```

(End of definition for `spvec-invalid-arg`.)

11.37.3 Definition of keys for `\spvec`

We define the *(keys)* `silent-cmd`, `concept`, `autofit`, `read-cmd` and `prefix`.

```

3165 \keys_define:nn { spintent / spvec }
3166 {
3167   silent-cmd .bool_set:N = \l__spinttent_spvec_silent_cmd_bool,
3168   silent-cmd .initial:n = false,
3169   silent-cmd .value_required:n = true,
3170   concept .bool_set:N = \l__spinttent_spvec_concept_bool,
3171   concept .initial:n = true,
3172   concept .value_required:n = true,
3173   autofit .bool_set:N = \l__spinttent_spvec_autofit_bool,
3174   autofit .initial:n = false,
3175   autofit .value_required:n = true,
3176   read-cmd .choices:nn = { school , mathcat }
3177   {
3178     \int_case:nn { \l_keys_choice_int }
3179     {
3180       {1} { \str_set:Nn \l__spinttent_spvec_mode_str { school } }
3181       {2} { \str_set:Nn \l__spinttent_spvec_mode_str { mathcat } }
3182     }
3183   },
3184   read-cmd .initial:n = school,
3185   read-cmd .value_required:n = true,
3186   read-cmd / unknown .code:n =
3187     \msg_error:nneee { spintent } { unknown-choice }
3188     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3189   prefix .code:n =
3190     { \__spinttent_sanitizize_affix:Nn \l__spinttent_spvec_prefix_tl {#1} },
3191   prefix .initial:n = { },
3192   prefix .value_required:n = true,
3193   unknown .code:n =
3194     {
3195       \str_set:Nn \l__spinttent_command_name_str { spvec }
3196       \__spinttent_unknown_keys_cmd:n {#1}
3197     },
3198 }

```

(End of definition for `silent-cmd` and others.)

11.37.4 Internal functions for \spvec

__spinttent_spvec_print:n

The internal function `\__spinttent_spvec_print:n` processes the $\langle argument \rangle$ passed in ‘#1’. Calls `\__spinttent_luafun_geo_parse_and_set:n`, reporting `spvec-invalid-arg` on error, on a proper name with `\__spinttent_geo_luaset_tipo_str` equal to `prima` (a one-letter vector cannot carry a prime), or on a point list that is not exactly two points. Otherwise calls `\__spinttent_spvec_mathml:n`.

```

3199 \cs_new_protected:Nn \__spinttent_spvec_print:n
3200 {
3201   \__spinttent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3202   \str_if_eq:VnT \__spinttent_geo_luaset_is_name_str { true }
3203   {
3204     \str_if_eq:VnT \__spinttent_geo_luaset_tipo_str { prima }
3205     { \str_set:Nn \__spinttent_geo_luaset_error_str { true } }
3206   }
3207   \str_if_eq:VnT \__spinttent_geo_luaset_is_name_str { false }
3208   {
3209     \int_compare:nT { \clist_count:N \__spinttent_geo_luaset_points_clist != 2 }
3210     { \str_set:Nn \__spinttent_geo_luaset_error_str { true } }
3211   }
3212   \str_if_eq:VnTF \__spinttent_geo_luaset_error_str { true }
3213   { \msg_error:nne { spinttent } { spvec-invalid-arg } { \exp_not:n {#1} } }
3214   { \__spinttent_spvec_mathml:n { #1 } }
3215 }

```

(End of definition for `\__spinttent_spvec_print:n`.)

__spinttent_spvec_mathml:n

The function `\__spinttent_spvec_mathml:n` branches on `\__spinttent_geo_luaset_is_name_str`. For a proper name (a single letter, no prime), the concept `intent` is `_ $\$pt$` when `silent-cmd` is `true` or `concept` is `false` — forced further to `_-` when `silent-cmd` is `true` and `read-cmd` is not `school`; otherwise `vector:prefix( $\$pt$ )`. With `\__spinttent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags `\vec{letra}` through `\__spinttent_luafun_mrow_intent_arg:nn` with a literal `kv`, the subscript appended and silenced; `read-cmd=mathcat` tags `\vec{...}` through a native `\MathMLarg` instead, the subscript silenced only under `silent-cmd`. `autofit` inserts a `\vphantom{b}` inside `\vec{...}` to normalize its height. Otherwise (no subscript), the same `kv` logic wraps `\vec{print\_tl}` as a whole.

```

3216 \cs_new_protected:Nn \__spinttent_spvec_mathml:n
3217 {
3218   \str_if_eq:VnTF \__spinttent_geo_luaset_is_name_str { true }
3219   {
3220     \bool_if:nTF
3221     {
3222       \bool_lazy_or_p:nn
3223       { \__spinttent_spvec_silent_cmd_bool }
3224       { ! \__spinttent_spvec_concept_bool }
3225     }
3226     {
3227       \tl_set:Nn \__spinttent_spvec_intent_arg_tl { _( $\$pt$ ) }
3228       \bool_if:NT \__spinttent_spvec_silent_cmd_bool
3229       {
3230         \str_if_eq:VnF \__spinttent_spvec_mode_str { school }
3231         { \tl_set:Nn \__spinttent_spvec_intent_arg_tl { _- } }
3232       }
3233     }
3234     {
3235       \tl_set:Nn \__spinttent_spvec_intent_arg_tl { vector:prefix( $\$pt$ ) }
3236     }
3237   }
3238   \str_case:VnF \__spinttent_geo_luaset_tipo_str
3239   {
3240     { sub }
3241     {
3242       \str_if_eq:VnTF \__spinttent_spvec_mode_str { school }
3243       {
3244         \bool_if:NTF \__spinttent_spvec_silent_cmd_bool
3245         { \tl_set:Nn \__spinttent_spvec_kv_tl { arg = "pt", intent = "_-" } }
3246         {
3247           \tl_set:Ne \__spinttent_spvec_kv_tl
3248           { arg = "pt", intent = "_ \__spinttent_geo_luaset_intent_str -" }
3249         }
3250         \__spinttent_invisible_sep:
3251         \MathMLintent { \__spinttent_spvec_intent_arg_tl }
3252         {
3253           \__spinttent_inner_sep:

```

```

3253         \__spintent_luafun_mrow_intent_arg:nn
3254         { } { \__spintent_svec_kv_tl }
3255     {
3256         \vec
3257         {
3258             \bool_if:NT \__spintent_svec_autofit_bool
3259             { \vphantom{ b } }
3260             \__spintent_geo_luaset_letra_str
3261         }
3262         \c_math_subscript_token
3263         { \MathMLint { :silent } { \__spintent_geo_luaset_sub_str } }
3264     }
3265 }
3266 }
3267 {
3268     \__spintent_invisible_sep:
3269     \MathMLint { \__spintent_svec_intent_arg_tl }
3270     {
3271         \__spintent_inner_sep:
3272         \vec
3273         {
3274             \bool_if:NT \__spintent_svec_autofit_bool
3275             { \vphantom{ b } }
3276             \MathMLarg{ pt } { \__spintent_geo_luaset_letra_str }
3277         }
3278         \bool_if:NT \__spintent_svec_silent_cmd_bool
3279         { \c_math_subscript_token { \__spintent_geo_luaset_sub_str } }
3280     }
3281     \bool_if:NF \__spintent_svec_silent_cmd_bool
3282     { \c_math_subscript_token { \__spintent_geo_luaset_sub_str } }
3283 }
3284 }
3285 }
3286 {
3287     \bool_if:NTF \__spintent_svec_silent_cmd_bool
3288     { \tl_set:Nn \__spintent_svec_kv_tl { arg = "pt", intent = "_" } }
3289     {
3290         \tl_set:Ne \__spintent_svec_kv_tl
3291         { arg = "pt", intent = "_" \__spintent_geo_luaset_intent_str -" }
3292     }
3293     \__spintent_invisible_sep:
3294     \MathMLint { \__spintent_svec_intent_arg_tl }
3295     {
3296         \__spintent_inner_sep:
3297         \__spintent_luafun_mrow_intent_arg:nn
3298         { } { \__spintent_svec_kv_tl }
3299     }
3300     \vec
3301     {
3302         \bool_if:NT \__spintent_svec_autofit_bool
3303         { \vphantom{ b } }
3304         \__spintent_geo_luaset_print_tl
3305     }
3306 }
3307 }
3308 }
3309 }

```

For a pair of points, `\__spintent_geo_luaset_points_clist` is split into `\__spintent_svec_pt_seq` and walked to build `\__spintent_svec_refs_tl`, the $\$pts1$, $\$pts2$ reference list. The concept intent is `_` (refs) or vector: `prefix(refs)`, same conditions as above, applied with a native `\MathMLint` wrapping `\overrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

3310     {
3311         \seq_set_from_clist:NN \__spintent_svec_pt_seq \__spintent_geo_luaset_points_clist
3312         \tl_clear:N \__spintent_svec_refs_tl
3313         \int_zero:N \__spintent_svec_pt_idx_int
3314         \seq_map_inline:Nn \__spintent_svec_pt_seq
3315         {
3316             \int_incr:N \__spintent_svec_pt_idx_int
3317             \tl_if_empty:NF \__spintent_svec_refs_tl

```

```

3318         { \tl_put_right:Nn \__spintent_svec_refs_tl { , } }
3319         \tl_put_right:Ne \__spintent_svec_refs_tl { $ pts \int_use:N \__spintent_svec_pt_
3320     }
3321     \bool_if:nTF
3322     {
3323         \bool_lazy_or_p:nn
3324         { \__spintent_svec_silent_cmd_bool }
3325         { ! \__spintent_svec_concept_bool }
3326     }
3327     {
3328         \tl_set:Ne \__spintent_svec_intent_arg_tl { _ ( \__spintent_svec_refs_tl ) }
3329     }
3330     {
3331         \tl_set:Ne \__spintent_svec_intent_arg_tl
3332         {
3333             \tl_if_empty:NF \__spintent_svec_prefix_tl
3334             { \tl_use:N \__spintent_svec_prefix_tl - }
3335             vector:prefix( \__spintent_svec_refs_tl )
3336         }
3337     }
3338     \__spintent_invisible_sep:
3339     \MathMLintent { \__spintent_svec_intent_arg_tl }
3340     {
3341         \__spintent_inner_sep:
3342         \overrightarrow
3343         {
3344             \int_zero:N \__spintent_svec_pt_idx_int
3345             \seq_map_inline:Nn \__spintent_svec_pt_seq
3346             {
3347                 \int_incr:N \__spintent_svec_pt_idx_int
3348                 \__spintent_mathmlarg_point:nnn
3349                 { pts \int_use:N \__spintent_svec_pt_idx_int }
3350                 { svec }
3351                 { ##1 }
3352             }
3353         }
3354     }
3355 }
3356 }

```

(End of definition for `\__spintent_svec_mathml:n`.)

`\__spintent_svec_exec:nnn`

The function `\__spintent_svec_exec:nnn` receives the optional *(keys)* in ‘#1’ and the *{(argument)}* in ‘#2’, delegating to `\__spintent_svec_print:n`. With ‘#3’ given (the optional coordinate), aliases `\__spintent_spcord_silent_cmd_bool` to `\__spintent_svec_silent_cmd_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcord_parse_and_set:n` directly, reporting `spcord-invalid-arg` on error unless silenced, then `\__spintent_spcord_set_wrap_parens_size:` and `\__spintent_spcord_mathml:` — the same rendering functions `\spcord` itself uses.

```

3357 \cs_new_protected:Nn \__spintent_svec_exec:nnn
3358 {
3359     \keys_set:nn { spintent / svec } { #1 }
3360     \__spintent_svec_print:n { #2 }
3361     \tl_if_novalue:nF {#3}
3362     {
3363         \equal % notación de vector =( , )
3364         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % previene lectura de intervalo
3365         \bool_set_eq:NN \__spintent_spcord_silent_cmd_bool \__spintent_svec_silent_cmd_bool
3366         \tl_set:Nn \__spintent_spcord_raw_arg_tl {#3}
3367         \bool_if:NTF \__spintent_svec_silent_cmd_bool
3368         {
3369             \__spintent_luafun_spcord_parse_and_set:n { \exp_not:n {#3} }
3370             \str_set:Nn \__spintent_spcord_luaset_error_str { false }
3371         }
3372         {
3373             \__spintent_luafun_spcord_parse_and_set:n { \exp_not:n {#3} }
3374             \str_if_eq:VnT \__spintent_spcord_luaset_error_str { true }
3375             { \msg_error:nnn { spintent } { spcord-invalid-arg } { \exp_not:n {#3} } }
3376         }
3377         \__spintent_spcord_set_wrap_parens_size:
3378         \str_if_eq:VnF \__spintent_spcord_luaset_error_str { true }
3379         { \__spintent_spcord_mathml: }

```

```

3380     }
3381 }

```

(End of definition for `\__spintent_svec_exec:nnn`.)

\spvec The public command `\spvec` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_svec_exec:nnn`.

```

3382 \NewDocumentCommand \spvec { O{} m d() }
3383 {
3384   \mode_if_math:TF
3385   {
3386     \group_begin:
3387       \attribute \__spintent_scope_attr_int = 1
3388       \__spintent_svec_exec:nnn { #1 } { #2 } { #3 }
3389     \group_end:
3390   }
3391   { \msg_error:nnn { spintent } { need-math-mode } { \spvec } }
3392 }

```

(End of definition for `\spvec`. This function is documented on page 18.)

11.38 The command `\sprecta`

The user command `\sprecta` supports two types of notation for the input $\langle argument \rangle$: when the argument contains two points (separated by an optional comma), `\overlefttrightarrow` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

11.38.1 Definition of variables for `\sprecta`

The variable `\__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix`.

```

3393 \str_new:N \__spintent_sprecta_mode_str
3394 \tl_new:N \__spintent_sprecta_prefix_tl
3395 \seq_new:N \__spintent_sprecta_pt_seq
3396 \tl_new:N \__spintent_sprecta_refs_tl
3397 \tl_new:N \__spintent_sprecta_intent_arg_tl
3398 \tl_new:N \__spintent_sprecta_kv_tl
3399 \int_new:N \__spintent_sprecta_pt_idx_int

```

(End of definition for `\__spintent_sprecta_mode_str` and others.)

11.38.2 Definition of error messages for `\sprecta`

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

3400 \msg_new:nnnn { spintent } { sprecta-invalid-points }
3401 {
3402   Invalid~format~in~\token_to_str:N \sprecta:~'#1'~\msg_line_context:.
3403 }
3404 {
3405   See~the~documentation~of~\token_to_str:N \sprecta~for~the~accepted~argument~forms.
3406 }

```

(End of definition for `sprecta-invalid-points`.)

11.38.3 Definition of keys for `\sprecta`

`read-cmd` We define the $\langle keys \rangle$ `read-cmd`, `prefix` and `silent-cmd`.

```

3407 \keys_define:nn { spintent / sprecta }
3408 {
3409   read-cmd .choices:nn = { school , mathcat }
3410   {
3411     \int_case:nn { \l_keys_choice_int }
3412     {
3413       {1} { \str_set:Nn \__spintent_sprecta_mode_str { school } }
3414       {2} { \str_set:Nn \__spintent_sprecta_mode_str { mathcat } }
3415     }
3416   },
3417   read-cmd .initial:n = school,
3418   read-cmd .value_required:n = true,
3419   read-cmd / unknown .code:n =
3420     \msg_error:nnee { spintent } { unknown-choice }
3421     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3422   prefix .code:n =

```

```

3423     { \__spintent_sanitiz_e_affix:Nn \__spintent_sprecta_prefix_tl {#1} },
3424     prefix                .initial:n = ,
3425     prefix                .value_required:n = true,
3426     silent-cmd            .bool_set:N = \__spintent_sprecta_silent_cmd_bool,
3427     silent-cmd            .initial:n = false,
3428     silent-cmd            .value_required:n = true,
3429     concept               .bool_set:N = \__spintent_sprecta_concept_bool,
3430     concept               .initial:n = true,
3431     concept               .value_required:n = true,
3432     unknown               .code:n =
3433     {
3434         \str_set:Nn \__spintent_command_name_str { sprecta }
3435         \__spintent_unknown_keys_cmd:n {#1}
3436     },
3437 }

```

(End of definition for read-cmd and others.)

11.38.4 Internal functions for \sprecta

__spintent_sprecta_print:n

The internal function `\__spintent_sprecta_print:n` processes the `{⟨argument⟩}` passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting sprecta-invalid-points on error, then `\__spintent_`

```

3438 \cs_new_protected:Nn \__spintent_sprecta_print:n
3439 {
3440     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3441     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3442     {
3443         \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }
3444     }
3445     \__spintent_sprecta_mathml:n { #1 }
3446 }

```

(End of definition for __spintent_sprecta_print:n.)

__spintent_sprecta_mathml:n

The function `\__spintent_sprecta_mathml:n` branches on `\__spintent_geo_luaset_is_name_str`. For a proper name (m, l, ...), the concept `intent` is `:silent($pt)` when `silent-cmd` is `true` or `concept` is `false`; otherwise `recta:prefix($pt)`, prefixed with `prefix` when given.

With `\__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the base through `\__spintent_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags base and subscript together through a native `\MathMLarg` instead.

```

3447 \cs_new_protected:Nn \__spintent_sprecta_mathml:n
3448 {
3449     \str_if_eq:VnTF \__spintent_geo_luaset_is_name_str { true }
3450     {
3451         \__spintent_invisible_sep:
3452         \bool_if:nTF
3453         {
3454             \bool_lazy_or_p:nn
3455             { \__spintent_sprecta_silent_cmd_bool }
3456             { ! \__spintent_sprecta_concept_bool }
3457         }
3458         {
3459             \tl_set:Nn \__spintent_sprecta_intent_arg_tl { :silent($pt) }
3460         }
3461         {
3462             \tl_set:Ne \__spintent_sprecta_intent_arg_tl
3463             {
3464                 \tl_if_empty:NF \__spintent_sprecta_prefix_tl
3465                 { \tl_use:N \__spintent_sprecta_prefix_tl - }
3466                 recta:prefix($pt)
3467             }
3468         }
3469     }
3470     \str_case:VnF \__spintent_geo_luaset_tipo_str
3471     {
3472         { sub }
3473         {
3474             \str_if_eq:VnTF \__spintent_sprecta_mode_str { school }
3475             {
3476                 \tl_set:Ne \__spintent_sprecta_kv_tl
3477                 { arg = "pt", intent = "_ \__spintent_geo_luaset_intent_str -" }
3478                 \MathMLintent { \__spintent_sprecta_intent_arg_tl }
3479             }

```

```

3479         \__spintent_inner_sep:
3480         \__spintent_luafun_mrow_intent_arg:nn
3481         { } { \__spintent_sprecta_kv_tl }
3482     {
3483         \__spintent_geo_luaset_letra_str
3484         \c_math_subscript_token
3485         { \MathMLintent { :silent } { \__spintent_geo_luaset_sub_str } }
3486     }
3487 }
3488 }

```

For `read-cmd=mathcat`, the same sub case tags base and subscript together through a native `\MathMLarg` instead — never mixed with the shared queue, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

3489     {
3490         \MathMLintent { \__spintent_sprecta_intent_arg_tl }
3491     {
3492     {
3493         \__spintent_inner_sep:
3494         \MathMLarg { pt }
3495     {
3496     {
3497         \__spintent_geo_luaset_letra_str
3498         \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
3499     }
3500     }
3501     }
3502     }
3503 }
3504 }

```

With `\__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the base with the literal kv and appends the mark, unsilenced (a prime is not spelled out on the base, so nothing to repeat); `read-cmd=mathcat` tags base and mark together through a native `\MathMLarg`.

```

3505     { prima }
3506     {
3507         \str_if_eq:VnTF \__spintent_sprecta_mode_str { school }
3508     {
3509         \tl_set:Nc \__spintent_sprecta_kv_tl
3510         { arg = "pt", intent = "_ \__spintent_geo_luaset_intent_str -" }
3511         \MathMLintent { \__spintent_sprecta_intent_arg_tl }
3512     {
3513         \__spintent_inner_sep:
3514         \__spintent_luafun_mrow_intent_arg:nn
3515         { } { \__spintent_sprecta_kv_tl }
3516         { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
3517     }
3518     }
3519     {
3520         \MathMLintent { \__spintent_sprecta_intent_arg_tl }
3521     {
3522     {
3523         \__spintent_inner_sep:
3524         \MathMLarg { pt }
3525     {
3526         { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
3527     }
3528     }
3529     }
3530     }
3531 }
3532 }

```

Otherwise (no subscript, no prime), the same kv logic wraps the combined `\__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school`, its own `arg` alone for `mathcat`.

```

3533     {
3534         \bool_if:NTF \__spintent_sprecta_silent_cmd_bool
3535     {
3536         \tl_set:Nn \__spintent_sprecta_kv_tl { arg = "pt", intent = "--" }
3537     }
3538     {
3539         \str_if_eq:VnTF \__spintent_sprecta_mode_str { school }

```

```

3540         {
3541             \tl_set:Nc \l__spintent_sprecta_kv_tl
3542                 { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3543         }
3544         {
3545             \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt" }
3546         }
3547     }
3548     \__spintent_luafun_mrow_intent_arg:nn
3549     { \l__spintent_sprecta_intent_arg_tl }
3550     { \l__spintent_sprecta_kv_tl }
3551     { \l__spintent_geo_luaset_print_tl }
3552 }
3553 }

```

For a list of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sprecta_pt_seq` and walked to build `\l__spintent_sprecta_refs_tl`, the $\$pt_1, \$pt_2, \dots$ reference list. The concept `intent` is `_(refs)` or `recta:prefix(refs)`, same conditions as above, applied with a native `\MathMLintent` wrapping `\overleftrightharrow`. The sequence is walked again, calling `\__spintent_mathmlarg` once per point.

```

3554 {
3555     \seq_set_from_clist:NN \l__spintent_sprecta_pt_seq \l__spintent_geo_luaset_points_clist
3556     \tl_clear:N \l__spintent_sprecta_refs_tl
3557     \int_zero:N \l__spintent_sprecta_pt_idx_int
3558     \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3559     {
3560         \int_incr:N \l__spintent_sprecta_pt_idx_int
3561         \tl_if_empty:NF \l__spintent_sprecta_refs_tl
3562         { \tl_put_right:Nn \l__spintent_sprecta_refs_tl { , } }
3563         \tl_put_right:Nc \l__spintent_sprecta_refs_tl { $ pt \int_use:N \l__spintent_sprecta_
3564     }
3565     \bool_if:nTF
3566     {
3567         \bool_lazy_or_p:nn
3568         { \l__spintent_sprecta_silent_cmd_bool }
3569         { ! \l__spintent_sprecta_concept_bool }
3570     }
3571     {
3572         \tl_set:Nc \l__spintent_sprecta_intent_arg_tl { _ ( \l__spintent_sprecta_refs_tl ) }
3573     }
3574     {
3575         \tl_set:Nc \l__spintent_sprecta_intent_arg_tl
3576         {
3577             \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
3578             { \tl_use:N \l__spintent_sprecta_prefix_tl - }
3579             recta:prefix( \l__spintent_sprecta_refs_tl )
3580         }
3581     }
3582     \__spintent_invisible_sep:
3583     \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3584     {
3585         \__spintent_inner_sep:
3586         \overleftrightharrow
3587         {
3588             \int_zero:N \l__spintent_sprecta_pt_idx_int
3589             \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3590             {
3591                 \int_incr:N \l__spintent_sprecta_pt_idx_int
3592                 \__spintent_mathmlarg_point:nnn
3593                 { pt \int_use:N \l__spintent_sprecta_pt_idx_int }
3594                 { sprecta }
3595                 { ##1 }
3596             }
3597         }
3598     }
3599 }
3600 }

```

(End of definition for `\__spintent_sprecta_mathml:n`)

`\__spintent_sprecta_exec:nn` The function `\__spintent_sprecta_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{ \langle argu-$

`ment\}` passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprecta_print:n`.

```

3601 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
3602 {
3603   \keys_set:nn { spintent / sprecta } { #1 }
3604   \__spintent_sprecta_print:n { #2 }
3605 }

```

(End of definition for `\__spintent_sprecta_exec:nn`.)

`\sprecta` The public command `\sprecta` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

3606 \NewDocumentCommand \sprecta { 0{} m }
3607 {
3608   \mode_if_math:TF
3609   {
3610     \group_begin:
3611     \attribute \__spintent_scope_attr_int = 1
3612     \__spintent_sprecta_exec:nn { #1 } { #2 }
3613     \group_end:
3614   }
3615   { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
3616 }

```

(End of definition for `\sprecta`. This function is documented on page 19.)

11.39 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, *rayo* or *semirrecta*, both rendered identically with `\overrightarrow`.

11.39.1 Definition of variables for `\sprayo`

The variable `\l__spintent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\l__spintent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\l__spintent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

🔗 The variables defined here are assigned using `.choices:n` or `.code:n` within `\keys_define:nn`.

```

3617 \str_new:N \l__spintent_sprayo_mode_str
3618 \tl_new:N \l__spintent_sprayo_prefix_tl
3619 \str_new:N \l__spintent_sprayo_read_sty_str

```

(End of definition for `\l__spintent_sprayo_mode_str`, `\l__spintent_sprayo_prefix_tl`, and `\l__spintent_sprayo_read_sty_str`.)

11.39.2 Definition of error messages for `\sprayo`

`sprayo-invalid-points` Raised when the argument is not a comma-separated pair of uppercase letters.

```

3620 \msg_new:nnnn { spintent } { sprayo-invalid-points }
3621 {
3622   Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'~\msg_line_context:.
3623 }
3624 {
3625   See~the~documentation~of~\token_to_str:N \sprayo~for~the~accepted~point~formats.
3626 }

```

(End of definition for `sprayo-invalid-points`.)

11.39.3 Definition of keys for `\sprayo`

`read-cmd` We define the `\keys` `read-cmd`, `prefix`, `silent-cmd`, `concept` and `read-sty`.

```

3627 \keys_define:nn { spintent / sprayo }
3628 {
3629   read-cmd .choices:nn = { school , mathcat }
3630   {
3631     \int_case:nn { \l_keys_choice_int }
3632     {
3633       {1} { \str_set:Nn \l__spintent_sprayo_mode_str { school } }
3634       {2} { \str_set:Nn \l__spintent_sprayo_mode_str { mathcat } }
3635     }
3636   },
3637   read-cmd .initial:n = school,
3638   read-cmd .value_required:n = true,
3639   read-cmd / unknown .code:n =
3640     \msg_error:nnee { spintent } { unknown-choice }
3641     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3642   prefix .code:n =

```

```

3643     { \__spintent_sanitize_affix:Nn \l__spintent_sprayo_prefix_tl {#1} },
3644     prefix                .initial:n = ,
3645     prefix                .value_required:n = true,
3646     silent-cmd            .bool_set:N = \l__spintent_sprayo_silent_cmd_bool,
3647     silent-cmd            .initial:n = false,
3648     silent-cmd            .value_required:n = true,
3649     concept               .bool_set:N = \l__spintent_sprayo_concept_bool,
3650     concept               .initial:n = true,
3651     concept               .value_required:n = true,
3652     read-sty              .choices:nn = { rayo , semirrecta }
3653     {
3654       \int_case:nn { \l_keys_choice_int }
3655       {
3656         {1} { \str_set:Nn \l__spintent_sprayo_read_sty_str { rayo } }
3657         {2} { \str_set:Nn \l__spintent_sprayo_read_sty_str { semirrecta } }
3658       }
3659     },
3660     read-sty              .initial:n = rayo,
3661     read-sty              .value_required:n = true,
3662     read-sty / unknown    .code:n =
3663     \msg_error:nneee { spintent } { unknown-choice }
3664     { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
3665     unknown               .code:n =
3666     {
3667       \str_set:Nn \l__spintent_command_name_str { sprayo }
3668       \__spintent_unknown_keys_cmd:n {#1}
3669     },
3670   }

```

(End of definition for read-cmd and others.)

11.39.4 Internal functions for \sprayo

Auxiliary variables used by __spintent_sprayo_mathml: to split \l__spintent_geo_luaset_points_clist and build the \$pts1, \$pts2, ... reference list.

```

\l__spintent_sprayo_pt_seq
\l__spintent_sprayo_refs_tl
\l__spintent_sprayo_intent_arg_tl
\l__spintent_sprayo_pt_idx_int
3671 \seq_new:N \l__spintent_sprayo_pt_seq
3672 \tl_new:N \l__spintent_sprayo_refs_tl
3673 \tl_new:N \l__spintent_sprayo_intent_arg_tl
3674 \int_new:N \l__spintent_sprayo_pt_idx_int

```

(End of definition for \l__spintent_sprayo_pt_seq and others.)

__spintent_sprayo_print:n The internal function __spintent_sprayo_print:n process the {<argument>} passed in ‘#1’. It calls __spintent_luafun_geo_parse_and_set:n to process the {<argument>}, returning an “error” if the input is invalid, then calls __spintent_sprayo_mathml:. Unlike \sprecta, \sprayo has no proper-name argument: it always takes a list of points, so no intent is built here.

```

3675 \cs_new_protected:Nn \__spintent_sprayo_print:n
3676 {
3677   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3678   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3679   {
3680     \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
3681   }
3682   \__spintent_sprayo_mathml:
3683 }

```

(End of definition for __spintent_sprayo_print:n.)

__spintent_sprayo_mathml: The function __spintent_sprayo_mathml: splits \l__spintent_geo_luaset_points_clist (set by Lua, one raw point per element) into \l__spintent_sprayo_pt_seq and walks it once to build \l__spintent_sprayo_refs_tl, the comma-separated \$pts1, \$pts2, ... reference list, naming each point by position.

The concept intent is _(refs) when silent-cmd is true or concept is false; otherwise it is \l__spintent_sprayo_read_sty_str:prefix(refs) — the concept word itself (rayo or semirrecta) comes from the read-sty key, not a literal, since \sprayo verbalizes under two names — prefixed with prefix when given. It is applied with a native \MathMLintent wrapping \overrightarrow, preceded by __spintent_inner_sep: (a second sibling so the sole <mrow> does not collapse into <math>). The sequence is walked a second time, calling __spintent_mathmlarg_point:nnn once per point with its ptsn reference name and silent-cmd/read-cmd looked up under sprayo: it alone decides arg/literal intent per point, so concept plays no role at that level.

```

3684 \cs_new_protected:Nn \__spintent_sprayo_mathml:

```

```

3685 {
3686   \seq_set_from_clist:NN \l__spintent_sprayo_pt_seq \l__spintent_geo_luaset_points_clist
3687   \tl_clear:N \l__spintent_sprayo_refs_tl
3688   \int_zero:N \l__spintent_sprayo_pt_idx_int
3689   \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3690     {
3691       \int_incr:N \l__spintent_sprayo_pt_idx_int
3692       \tl_if_empty:NF \l__spintent_sprayo_refs_tl
3693         { \tl_put_right:Nn \l__spintent_sprayo_refs_tl { , } }
3694       \tl_put_right:Ne \l__spintent_sprayo_refs_tl { $ pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3695     }
3696   \bool_if:nTF
3697     {
3698       \bool_lazy_or_p:nn
3699         { \l__spintent_sprayo_silent_cmd_bool }
3700         { ! \l__spintent_sprayo_concept_bool }
3701     }
3702     {
3703       \tl_set:Ne \l__spintent_sprayo_intent_arg_tl { _ ( \l__spintent_sprayo_refs_tl ) }
3704     }
3705     {
3706       \tl_set:Ne \l__spintent_sprayo_intent_arg_tl
3707         {
3708           \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
3709             { \tl_use:N \l__spintent_sprayo_prefix_tl - }
3710           \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
3711         }
3712     }
3713   \__spintent_invisible_sep:
3714   \MathMLint { \l__spintent_sprayo_intent_arg_tl }
3715   {
3716     \__spintent_inner_sep:
3717     \overrightarrow
3718       {
3719         \int_zero:N \l__spintent_sprayo_pt_idx_int
3720         \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3721           {
3722             \int_incr:N \l__spintent_sprayo_pt_idx_int
3723             \__spintent_mathmlarg_point:nnn
3724               { pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3725               { sprayo }
3726               { ##1 }
3727           }
3728       }
3729   }
3730 }

```

(End of definition for __spintent_sprayo_mathml:.)

__spintent_sprayo_exec:nn The function __spintent_sprayo_exec:nn receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{\langle argument \rangle\}$ passed in ‘#2’ delegating the rendering of the latter to __spintent_sprayo_print:n.

```

3731 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
3732 {
3733   \keys_set:nn { spintent / sprayo } { #1 }
3734   \__spintent_sprayo_print:n { #2 }
3735 }

```

(End of definition for __spintent_sprayo_exec:nn.)

\spray The public command \spray enforces *math mode*, opens a local group, and delegates execution to __spintent_sprayo_exec:nn.

```

3736 \NewDocumentCommand \spray { 0{ } m }
3737 {
3738   \mode_if_math:TF
3739     {
3740       \group_begin:
3741         \attribute \l__spintent_scope_attr_int = 1
3742         \__spintent_sprayo_exec:nn { #1 } { #2 }
3743       \group_end:
3744     }
3745     { \msg_error:nnn { spintent } { need-math-mode } { \spray } }
3746 }

```

(End of definition for `\sprayo`. This function is documented on page 19.)

11.40 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — `lado`, `segmento`, or `trazo`, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

11.40.1 Definition of variables for `\spside` and `\spmside`

Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\__spinttent_spside_mode_str` and `\__spinttent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\__spinttent_spside_prefix_tl` and `\__spinttent_spmside_prefix_tl` store the values set by the key `prefix`.

The variables `\__spinttent_spside_read_sty_str` and `\__spinttent_spmside_read_sty_str` stores the value set by the key `read-sty`.

The variables `\__spinttent_spside_read_arg_tl` and `\__spinttent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

• The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```
3747 \clist_map_inline:nn { side , mside }
3748 {
3749   \str_new:c { \__spinttent_sp #1 _mode_str      }
3750   \tl_new:c { \__spinttent_sp #1 _prefix_tl     }
3751   \str_new:c { \__spinttent_sp #1 _read_sty_str  }
3752   \tl_new:c { \__spinttent_sp #1 _read_arg_tl   }
3753 }
```

(End of definition for `\__spinttent_spside_mode_str` and others.)

11.40.2 Definition of error messages for `\spside` and `\spmside`

We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “*point*” in the `{\argument}` passed in ‘#1’ to the command.

```
3754 \clist_map_inline:nn { side , mside }
3755 {
3756   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
3757   {
3758     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
3759   }
3760   {
3761     See~the~documentation~of~##1~for~the~accepted~point~formats.
3762   }
3763 }
```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

11.40.3 Definition of keys for `\spside` and `\spmside`

We define the shared `\keys` `read-cmd`, `prefix`, `silent-cmd`, `concept`, `read-sty`, `read-arg` and `print-m`.

```
read-cmd 3764 \clist_map_inline:nn { side , mside }
prefix   3765 {
silent-cmd 3766   \keys_define:nn { spintent / sp #1 }
concept   3767   {
read-sty  3768     read-cmd          .choices:nn = { school , mathcat }
read-arg  3769     {
print-m   3770       \int_case:nn { \l_keys_choice_int }
unknown   3771       {
3772         {1} { \str_set:cn { \__spinttent_sp #1 _mode_str } { school } }
3773         {2} { \str_set:cn { \__spinttent_sp #1 _mode_str } { mathcat } }
3774       }
3775     },
3776     read-cmd          .initial:n = school,
3777     read-cmd          .value_required:n = true,
3778     read-cmd / unknown .code:n =
3779     \msg_error:nnee { spintent } { unknown-choice }
3780     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3781     prefix            .code:n =
3782     { \__spinttent_sanitize_affix:cn { \__spinttent_sp #1 _prefix_tl } {##1} },
3783     prefix            .initial:n = ,
3784     prefix            .value_required:n = true,
```

```

3785     silent-cmd          .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3786     silent-cmd          .initial:n  = false,
3787     silent-cmd          .value_required:n = true,
3788     concept             .bool_set:c = { l__spintent_sp #1 _concept_bool },
3789     concept             .initial:n  = true,
3790     concept             .value_required:n = true,
3791     read-sty            .choices:nn = { lado , segmento , trazo }
3792     {
3793         \int_case:nn { \l_keys_choice_int }
3794         {
3795             {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { lado      } }
3796             {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { segmento } }
3797             {3} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { trazo     } }
3798         }
3799     },
3800     read-sty            .initial:n  = lado,
3801     read-sty            .value_required:n = true,
3802     read-sty / unknown .code:n =
3803         \msg_error:nnee { spintent } { unknown-choice }
3804         { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
3805     read-arg            .code:n =
3806     {
3807         \__spintent_sanitise_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1}
3808     },
3809     read-arg            .initial:n  = ,
3810     read-arg            .value_required:n = true,
3811     print-m             .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3812     print-m             .initial:n  = false,
3813     print-m             .value_required:n = true,
3814     unknown             .code:n =
3815     {
3816         \str_set:Nn \l__spintent_command_name_str { sp #1 }
3817         \__spintent_unknown_keys_cmd:n {##1}
3818     },
3819 }
3820 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for `read-cmd` and others.)

`\l__spintent_spside_intent_tl` `\l__spintent_spside_pt_seq` `\l__spintent_spside_refs_tl` `\l__spintent_spside_intent_arg_tl` `\l__spintent_spside_pt_idx_int` `\l__spintent_spside_intent_tl` holds the concept word (`read-arg` or `read-sty`). The rest are scratch variables shared by both commands (the function itself is shared) to split `\l__spintent_geo_luaset_points_clist` and build the `$pts1`, `$pts2`, ... reference list.

```

3821 \tl_new:N \l__spintent_spside_intent_tl
3822 \seq_new:N \l__spintent_spside_pt_seq
3823 \tl_new:N \l__spintent_spside_refs_tl
3824 \tl_new:N \l__spintent_spside_intent_arg_tl
3825 \int_new:N \l__spintent_spside_pt_idx_int

```

(End of definition for `\l__spintent_spside_intent_tl` and others.)

`\__spintent_spside_process:nn` The internal function `\__spintent_spside_process:nn` receives the short name of the command (`side` or `mside`) in ‘#1’ and the ‘{*argument*}’ in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then builds `\l__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘medida-del-’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘medida-del-’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_math` builds after splitting the points.

```

3826 \cs_new_protected:Nn \__spintent_spside_process:nn
3827 {
3828     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3829     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3830     {
3831         \msg_error:nnee { spintent } { sp #1 -invalid-points }
3832         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3833     }
3834     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
3835     {
3836         \tl_set:Nc \l__spintent_spside_intent_tl { \str_use:c { l__spintent_sp #1 _read_sty_str }
3837         \str_if_eq:eeT {#1} { mside }

```

```

3838         {
3839             \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3840         }
3841     }
3842     {
3843         \tl_set:Ne \l__spintent_spside_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
3844         \bool_if:cT { l__spintent_sp #1 _concept_bool }
3845         {
3846             \str_if_eq:eeT {#1} { mside }
3847             {
3848                 \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3849             }
3850         }
3851     }
3852     \l__spintent_spside_mathml:n { #1 }
3853 }

```

(End of definition for `\l__spintent_spside_process:nn`.)

`\l__spintent_spside_mathml:n`

The function `\l__spintent_spside_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spside_pt_seq` and walks it once to build `\l__spintent_spside_refs_tl`, the comma-separated `$pts1,$pts2,...` reference list, naming each point by position.

```

3854 \cs_new_protected:Nn \l__spintent_spside_mathml:n
3855 {
3856     \seq_set_from_clist:NN \l__spintent_spside_pt_seq \l__spintent_geo_luaset_points_clist
3857     \tl_clear:N \l__spintent_spside_refs_tl
3858     \int_zero:N \l__spintent_spside_pt_idx_int
3859     \seq_map_inline:Nn \l__spintent_spside_pt_seq
3860     {
3861         \int_incr:N \l__spintent_spside_pt_idx_int
3862         \tl_if_empty:NF \l__spintent_spside_refs_tl
3863         { \tl_put_right:Nn \l__spintent_spside_refs_tl { , } }
3864         \tl_put_right:Ne \l__spintent_spside_refs_tl { $ pts \int_use:N \l__spintent_spside_pt_idx_int }
3865     }

```

The concept `intent` is `_(refs)` when `silent-cmd` is `true`; otherwise, if `read-arg` is empty, it is `_(refs)` when `concept` is `false` or `\l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read-arg` is given, it is always `\l__spintent_spside_intent_tl:prefix(refs)` — `concept` never silences a user-given `read-arg` (see `\l__spintent_spside_process:nn`). The `:prefix(...)` form is prefixed with `prefix` when given.

```

3866     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3867     {
3868         \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3869     }
3870     {
3871         \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
3872         {
3873             \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3874             {
3875                 \tl_set:Ne \l__spintent_spside_intent_arg_tl
3876                 {
3877                     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3878                     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3879                     \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3880                 }
3881             }
3882             {
3883                 \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3884             }
3885         }
3886         {
3887             \tl_set:Ne \l__spintent_spside_intent_arg_tl
3888             {
3889                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3890                 { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3891                 \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3892             }
3893         }
3894     }

```

Applied with a native `\MathMLintent`, preceded by `\__spinttent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is `true` a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\__spinttent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent-cmd/read-cmd` looked up under `sp#1`: it alone decides `arg`/literal `intent` per point, so `concept` plays no role at that level.

```

3895   \__spinttent_invisible_sep: % (luamml issues #26)
3896   \MathMLintent { \__spinttent_spside_intent_arg_tl }
3897   {
3898     \__spinttent_inner_sep:
3899     \str_if_eq:eeTF {#1} { side }
3900     {
3901       \overline { \__spinttent_spside_points_loop:n {#1} }
3902     }
3903     {
3904       \bool_if:cTF { l__spinttent_sp #1 _print_m_bool }
3905       {
3906         \__spinttent_luafun_mrow_intent_arg:nn
3907         { } { intent = ":silent" }
3908         { \operatorname { m } }
3909         \overline { \__spinttent_spside_points_loop:n {#1} }
3910       }
3911       {
3912         \__spinttent_spside_points_loop:n {#1}
3913       }
3914     }
3915   }
3916 }

```

(End of definition for `\__spinttent_spside_mathml:n`.)

`\__spinttent_spside_points_loop:n`

Walks `\__spinttent_spside_pt_seq` a second time, calling `\__spinttent_mathmlarg_point:nnn` once per point. Factored out of `\__spinttent_spside_mathml:n` since it is needed identically in three places there (`side`, `mside` with and without `print-m`); it takes the short command name in `#1` again since a separate macro does not inherit its caller's own `#1`.

```

3917 \cs_new_protected:Nn \__spinttent_spside_points_loop:n
3918 {
3919   \int_zero:N \__spinttent_spside_pt_idx_int
3920   \seq_map_inline:Nn \__spinttent_spside_pt_seq
3921   {
3922     \int_incr:N \__spinttent_spside_pt_idx_int
3923     \__spinttent_mathmlarg_point:nnn
3924     { pts \int_use:N \__spinttent_spside_pt_idx_int }
3925     { sp #1 }
3926     { ##1 }
3927   }
3928 }

```

(End of definition for `\__spinttent_spside_points_loop:n`.)

`\__spinttent_spside_exec:nnn`

The function `\__spinttent_spside_exec:nnn` receives the *short name* of the command passed in `#1`, the optional `<keys>` passed in `#2` and the `<argument>` passed in `#3`, delegating the rendering to `\__spinttent_spside_process:nn`.

```

3929 \cs_new_protected:Nn \__spinttent_spside_exec:nnn
3930 {
3931   \keys_set:nn { spinttent / sp #1 } { #2 }
3932   \__spinttent_spside_process:nn { #1 } { #3 }
3933 }

```

(End of definition for `\__spinttent_spside_exec:nnn`.)

`\spside`
`\spmside`

Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spinttent_spside_exec:nnn`.

```

3934 \NewDocumentCommand \spside { O{} m }
3935 {
3936   \mode_if_math:TF
3937   {

```

```

3938     \group_begin:
3939         \attribute \l__spintent_scope_attr_int = 1
3940         \l__spintent_spside_exec:nnn { side } { #1 } { #2 }
3941     \group_end:
3942 }
3943 { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
3944 }
3945 \NewDocumentCommand \spmside { 0{} m }
3946 {
3947     \mode_if_math:TF
3948     {
3949         \group_begin:
3950             \attribute \l__spintent_scope_attr_int = 1
3951             \l__spintent_spside_exec:nnn { mside } { #1 } { #2 }
3952         \group_end:
3953     }
3954     { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
3955 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 20.)

11.41 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside/\spmside`. The key `print-m` chooses between the `\angle/\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle/\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.41.1 Definition of variables for `\spangle` and `\spmangle`

```

\l__spintent_spangle_mode_str
\l__spintent_spmangle_mode_str
\l__spintent_spangle_prefix_tl
\l__spintent_spmangle_prefix_tl

```

Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\l__spintent_spangle_mode_str` and `\l__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintent_spangle_prefix_tl` and `\l__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

• The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

3956 \clist_map_inline:nn { angle , mangle }
3957 {
3958     \str_new:c { l__spintent_sp #1 _mode_str }
3959     \tl_new:c { l__spintent_sp #1 _prefix_tl }
3960 }

```

(End of definition for `\l__spintent_spangle_mode_str` and others.)

11.41.2 Definition of error messages for `\spangle` and `\spmangle`

```

spangle-invalid-arg
spmangle-invalid-arg

```

We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a `invalid` {`\langle argument \rangle`} passed in ‘#1’ to the command.

```

3961 \clist_map_inline:nn { angle , mangle }
3962 {
3963     \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3964     {
3965         Invalid~format~in~##1:~'##2'~\msg_line_context:.
3966     }
3967     {
3968         See~the~documentation~of~##1~for~the~accepted~argument~forms.
3969     }
3970 }

```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

11.41.3 Definition of keys for `\spangle` and `\spmangle`

```

read-cmd
prefix
recto
print-m
silent-cmd
concept
unknown

```

We define the shared `\keys` `read-cmd`, `prefix`, `recto`, `print-m`, `silent-cmd` and `concept`,

```

3971 \clist_map_inline:nn { angle , mangle }
3972 {
3973     \keys_define:nn { spintent / sp #1 }
3974     {
3975         read-cmd .choices:nn = { school , mathcat }
3976         {
3977             \int_case:nn { \l_keys_choice_int }
3978             {
3979                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
3980                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }

```

```

3981         }
3982     },
3983     read-cmd          .initial:n = school,
3984     read-cmd          .value_required:n = true,
3985     read-cmd / unknown .code:n =
3986     \msg_error:nneee { spintent } { unknown-choice }
3987     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3988     prefix            .code:n =
3989     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
3990     prefix            .initial:n = ,
3991     prefix            .value_required:n = true,
3992     recto             .bool_set:c = { l__spintent_sp #1 _recto_bool },
3993     recto             .initial:n = false,
3994     recto             .value_required:n = true,
3995     print-m          .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3996     print-m          .initial:n = false,
3997     print-m          .value_required:n = true,
3998     silent-cmd       .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3999     silent-cmd       .initial:n = false,
4000     silent-cmd       .value_required:n = true,
4001     concept          .bool_set:c = { l__spintent_sp #1 _concept_bool },
4002     concept          .initial:n = true,
4003     concept          .value_required:n = true,
4004     unknown          .code:n =
4005     {
4006         \str_set:Nn \l__spintent_command_name_str { sp #1 }
4007         \__spintent_unknown_keys_cmd:n {##1}
4008     },
4009 }
4010 }

```

🔗 The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for `read-cmd` and others.)

11.41.4 Internal functions for `\spangle` and `\spmangle`

`\__spintent_spangle_sym:n`

The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightanglesqr` when `print-m` is true; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

4011 \cs_new_protected:Nn \__spintent_spangle_sym:n
4012 {
4013     \str_if_eq:eeTF {#1} { mangle }
4014     {
4015         \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
4016         {
4017             \bool_if:cTF { l__spintent_sp #1 _recto_bool }
4018             { \__spintent_right_angle_sqr: } { \angle }
4019         }
4020         {
4021             \bool_if:cTF { l__spintent_sp #1 _recto_bool }
4022             { \measuredrightangle } { \measuredangle }
4023         }
4024     }
4025     {
4026         \bool_if:cTF { l__spintent_sp #1 _recto_bool }
4027         { \__spintent_right_angle_sqr: } { \angle }
4028     }
4029 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

`\l__spintent_spangle_intent_tl`

`\l__spintent_spangle_intent_tl` holds the concept word (ángulo or medida-del-ángulo, ‘-recto’- suffixed when `recto` is true). The rest are scratch variables shared by both commands to split `\l__spintent_geo_lua` and build the reference list — used only in the tres-puntos and un-punto cases (see `\__spintent_spangle_mathml:`

`\l__spintent_spangle_pt_seq`
`\l__spintent_spangle_refs_tl`
`\l__spintent_spangle_intent_arg_tl`
`\l__spintent_spangle_pt_idx_int`
`\l__spintent_spangle_print_m_now_bool`

```

4030 \tl_new:N \l__spintent_spangle_intent_tl
4031 \seq_new:N \l__spintent_spangle_pt_seq
4032 \tl_new:N \l__spintent_spangle_refs_tl
4033 \tl_new:N \l__spintent_spangle_intent_arg_tl
4034 \int_new:N \l__spintent_spangle_pt_idx_int
4035 \str_new:N \l__spintent_spangle_kv_tl
4036 \bool_new:N \l__spintent_spangle_print_m_now_bool

```

(End of definition for `\__spintent_spangle_intent_tl` and others.)

`\__spintent_spangle_process:nn`

The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in ‘#1’ and the $\{\langle argument \rangle\}$ in ‘#2’. Calls `\__spintent_luafun_geo_angulo_parse_and_set:n`, reporting `sp#1-invalid-arg` on error. For a proper name that is a single letter (not numeric), also reports the same error when that letter is lowercase — `angle` names take an uppercase letter, number, or Greek letter, never a lowercase Latin one; the length check keeps a Greek letter command such as `\alpha` out of this comparison. Then sets `\__spintent_spangle_intent_tl` to `ángulo` (or `medida-del-ángulo` for `\spmangle`), suffixed with `-recto` when `recto` is `true`, and calls `\__spintent_spangle_mathml:n`.

```

4937 \cs_new_protected:Nn \__spintent_spangle_process:nn
4938 {
4939   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
4940   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4941   {
4942     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4943     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4944   }
4945   \str_if_eq:VnT \__spintent_geo_luaset_angulo_case_str { nombre }
4946   {
4947     \str_if_eq:VnF \__spintent_geo_luaset_nombre_es_num_str { true }
4948     {
4949       \int_compare:nT { \str_count:N \__spintent_geo_luaset_letra_str = 1 }
4950       {
4951         \str_if_eq:eeF { \__spintent_geo_luaset_letra_str } { \str_uppercase:n { \__spintent_geo_luaset_letra_str } }
4952         {
4953           \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4954           { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4955         }
4956       }
4957     }
4958   }
4959   \str_if_eq:eeTF {#1} { mangle }
4960   {
4961     \tl_set:Nn \__spintent_spangle_intent_tl { medida-del-ángulo }
4962   }
4963   {
4964     \tl_set:Nn \__spintent_spangle_intent_tl { ángulo }
4965   }
4966   \bool_if:cT { l__spintent_sp #1 _recto_bool }
4967   {
4968     \tl_put_right:Nn \__spintent_spangle_intent_tl { -recto }
4969   }
4970   \__spintent_spangle_mathml:n {#1}
4971 }

```

(End of definition for `\__spintent_spangle_process:nn`.)

`\__spintent_spangle_mathml:n`

The function `\__spintent_spangle_mathml:n` branches on `\__spintent_geo_luaset_angulo_case_str`. For `nombre`, the symbol (`\__spintent_spangle_sym:n`) is tagged through `\__spintent_luafun_mrow_intent_arg_tl` on its own, with `\__spintent_spangle_intent_tl:prefix` (no $\$$ -reference: `nombre` never refers to more than one thing) — `_` — when `silent-cmd` is `true` or `concept` is `false`. With `silent-cmd` `true`, the name that follows is silenced as a whole.

```

4972 \cs_new_protected:Nn \__spintent_spangle_mathml:n
4973 {
4974   \__spintent_invisible_sep: % (luamml issues #26)
4975   \str_if_eq:VnTF \__spintent_geo_luaset_angulo_case_str { nombre }
4976   {
4977     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
4978     {
4979       \tl_set:Nn \__spintent_spangle_intent_arg_tl { _- }
4980     }
4981     {
4982       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4983       {
4984         \tl_set:Nn \__spintent_spangle_intent_arg_tl
4985         {
4986           \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4987           { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4988           \__spintent_spangle_intent_tl :prefix
4989         }

```

```

4090     }
4091     {
4092         \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4093     }
4094 }
4095 \tl_set:Ne \l__spintent_spangle_kv_tl
4096 { intent = "\l__spintent_spangle_intent_arg_tl" }
4097 \__spintent_luafun_mrow_intent_arg:nn
4098 { } { \l__spintent_spangle_kv_tl }
4099 { \__spintent_spangle_sym:n {#1} }
4100 \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4101 {
4102     \__spintent_luafun_mrow_intent_arg:nn { } { intent = "_-" }
4103     { \l__spintent_geo_luaset_print_tl }
4104 }

```

Otherwise, a numeric name is tagged as a whole, through `\__spintent_luafun_mrow_intent_arg:nn`, with a literal `intent` built from `\l__spintent_geo_luaset_intent_str`.

```

4105     {
4106         \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
4107         {
4108             \tl_set:Ne \l__spintent_spangle_kv_tl
4109             { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4110             \__spintent_luafun_mrow_intent_arg:nn
4111             { } { \l__spintent_spangle_kv_tl }
4112             { \l__spintent_geo_luaset_print_tl }
4113         }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the letter through the same function, with a literal `intent`, and the subscript is appended and silenced; `read-cmd=mathcat` leaves both letter and subscript untouched, for MathCAT to read the whole `<msub>` natively (no `arg` is set in either case: there is no `$`-reference to resolve).

```

4114     {
4115         \str_case:VnF \l__spintent_geo_luaset_tipo_str
4116         {
4117             { sub }
4118             {
4119                 \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
4120                 {
4121                     \tl_set:Ne \l__spintent_spangle_kv_tl
4122                     { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4123                     \__spintent_luafun_mrow_intent_arg:nn
4124                     { } { \l__spintent_spangle_kv_tl }
4125                     {
4126                         \l__spintent_geo_luaset_letra_str
4127                         \c_math_subscript_token
4128                         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4129                     }
4130                 }
4131                 {
4132                     \l__spintent_geo_luaset_letra_str
4133                     \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
4134                 }
4135             }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the prime is always read — spoken as part of the literal for `read-cmd=school`, native for `read-cmd=mathcat` — since a prime is not a repeated word to silence.

```

4136         { prima }
4137         {
4138             \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
4139             {
4140                 \tl_set:Ne \l__spintent_spangle_kv_tl
4141                 { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4142                 \__spintent_luafun_mrow_intent_arg:nn
4143                 { } { \l__spintent_spangle_kv_tl }
4144                 { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4145             }
4146             {
4147                 \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str
4148             }
4149         }

```

```

4150         }
4151         { \l__spintent_geo_luaset_print_tl }
4152     }
4153 }
4154 }

```

For tres-puntos/un-punto, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spangle_pt` and walked to build `\l__spintent_spangle_refs_tl`, the `$pt1, $pt2, ...` reference list. A single native `\MathMLintent` wraps the symbol and the points together, with `\l__spintent_spangle_intent_tl:prefix(refs — _-` when `silent-cmd` is `true`, `_(refs)` when `concept` is `false` alone. For `\spmangle` with `print-m true`, a silent `\operatorname{m}` precedes `\l__spintent_spangle_sym:n` inside the same `\MathMLintent`. The sequence is walked again, calling `\l__spintent_mathmlarg_point:nnn` once per point.

```

4155     {
4156       \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
4157       \tl_clear:N \l__spintent_spangle_refs_tl
4158       \int_zero:N \l__spintent_spangle_pt_idx_int
4159       \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4160       {
4161         \int_incr:N \l__spintent_spangle_pt_idx_int
4162         \tl_if_empty:NF \l__spintent_spangle_refs_tl
4163         { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
4164         \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_
4165       }
4166       \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4167       {
4168         \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4169       }
4170       {
4171         \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4172         {
4173           \tl_set:Ne \l__spintent_spangle_intent_arg_tl
4174           {
4175             \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4176             { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4177             \l__spintent_spangle_intent_tl :prefix( \l__spintent_spangle_refs_tl )
4178           }
4179         }
4180         {
4181           \tl_set:Ne \l__spintent_spangle_intent_arg_tl { _( \l__spintent_spangle_refs_tl )
4182         }
4183       }
4184       \bool_set_false:N \l__spintent_spangle_print_m_now_bool
4185       \str_if_eq:eeT { #1 } { mangle }
4186       {
4187         \bool_if:cT { \l__spintent_sp #1 _print_m_bool }
4188         { \bool_set_true:N \l__spintent_spangle_print_m_now_bool }
4189       }
4190       \MathMLintent { \l__spintent_spangle_intent_arg_tl }
4191       {
4192         \l__spintent_inner_sep:
4193         \bool_if:NT \l__spintent_spangle_print_m_now_bool
4194         {
4195           \l__spintent_luafun_mrow_intent_arg:nn
4196           { } { intent = ":silent" }
4197           { \operatorname { m } } }
4198         }
4199         \l__spintent_luafun_mrow_intent_arg:nn
4200         { } { intent = ":silent" }
4201         { \l__spintent_spangle_sym:n { #1 } }
4202         \int_zero:N \l__spintent_spangle_pt_idx_int
4203         \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4204         {
4205           \int_incr:N \l__spintent_spangle_pt_idx_int
4206           \l__spintent_mathmlarg_point:nnn
4207           { pt \int_use:N \l__spintent_spangle_pt_idx_int }
4208           { sp#1 }
4209           { ##1 }
4210         }
4211       }
4212     }
4213 }

```

(End of definition for `\__spintent_spangle_mathml:n`.)

`\__spintent_spangle_exec:nnn`

The function `\__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spangle_process:nn`.

```
4214 \cs_new_protected:Nn \__spintent_spangle_exec:nnn
4215 {
4216   \keys_set:nn { spintent / sp #1 } { #2 }
4217   \__spintent_spangle_process:nn { #1 } { #3 }
4218 }
```

(End of definition for `\__spintent_spangle_exec:nnn`.)

`\spangle`
`\spmangle`

Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

```
4219 \NewDocumentCommand \spangle { 0{} m }
4220 {
4221   \mode_if_math:TF
4222   {
4223     \group_begin:
4224     \attribute \__spintent_scope_attr_int = 1
4225     \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
4226     \group_end:
4227   }
4228   { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }
4229 }
4230 \NewDocumentCommand \spmangle { 0{} m }
4231 {
4232   \mode_if_math:TF
4233   {
4234     \group_begin:
4235     \attribute \__spintent_scope_attr_int = 1
4236     \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
4237     \group_end:
4238   }
4239   { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
4240 }
```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 22.)

11.42 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

11.42.1 Definition of variables for `\sparc` and `\spmarc`

`\__spintent_sparc_mode_str`
`\__spintent_spmarc_mode_str`
`\__spintent_sparc_prefix_tl`
`\__spintent_spmarc_prefix_tl`

Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\__spintent_sparc_mode_str` and `\__spintent_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_sparc_prefix_tl` and `\__spintent_spmarc_prefix_tl` store the values set by the key `prefix`.

```
4241 \clist_map_inline:nn { arc , marc }
4242 {
4243   \str_new:c { \__spintent_sp #1 _mode_str }
4244   \tl_new:c { \__spintent_sp #1 _prefix_tl }
4245 }
```

(End of definition for `\__spintent_sparc_mode_str` and others.)

11.42.2 Definition of error messages for `\sparc` and `\spmarc`

`sparc-invalid-points`
`spmarc-invalid-points`

We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid *{argument}* passed in ‘#1’ to the command.

```
4246 \clist_map_inline:nn { arc , marc }
4247 {
4248   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4249   {
4250     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4251   }
4252   {
4253     See~the~documentation~of~##1~for~the~accepted~point~formats.
4254   }
4255 }
```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

11.42.3 Definition of keys for \sparc and \spmarc

We define the shared *(keys)* `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```

read-cmd 256 \clist_map_inline:nn { arc , marc }
prefix   257 {
silent-cmd 258 \keys_define:nn { spintent / sp #1 }
concept   259 {
unknown   260   read-cmd           .choices:nn = { school , mathcat }
          261   {
          262     \int_case:nn { \l_keys_choice_int }
          263     {
          264       {1} { \str_set:cn { \l__spintent_sp #1 _mode_str } { school } }
          265       {2} { \str_set:cn { \l__spintent_sp #1 _mode_str } { mathcat } }
          266     }
          267   },
          268   read-cmd           .initial:n = school,
          269   read-cmd           .value_required:n = true,
          270   read-cmd / unknown .code:n =
          271     \msg_error:nneee { spintent } { unknown-choice }
          272     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
          273   prefix           .code:n =
          274     { \l__spintent_sanitize_affix:cn { \l__spintent_sp #1 _prefix_tl } {##1} },
          275   prefix           .initial:n = ,
          276   prefix           .value_required:n = true,
          277   silent-cmd        .bool_set:c = { \l__spintent_sp #1 _silent_cmd_bool },
          278   silent-cmd        .initial:n = false,
          279   silent-cmd        .value_required:n = true,
          280   concept           .bool_set:c = { \l__spintent_sp #1 _concept_bool },
          281   concept           .initial:n = true,
          282   concept           .value_required:n = true,
          283   unknown           .code:n =
          284     {
          285       \str_set:Nn \l__spintent_command_name_str { sp #1 }
          286       \l__spintent_unknown_keys_cmd:n {##1}
          287     },
          288   }
          289 }

```

(End of definition for `read-cmd` and others.)

11.42.4 Internal functions for \sparc and \spmarc

`\__spintent_sparc_wide_arc:n` The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of TeX-SX.

```

4290 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
4291 {
4292   \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
4293 }

```

(End of definition for `\__spintent_sparc_wide_arc:n`)

`\__spintent_sparc_print_arc:n` The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

4294 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
4295 {
4296   \cs_if_exist:NTF \widearc
4297   { \widearc {#1} }
4298   { \__spintent_sparc_wide_arc:n {#1} }
4299 }

```

(End of definition for `\__spintent_sparc_print_arc:n`)

`\l__spintent_sparc_intent_tl` `\l__spintent_sparc_pt_seq` `\l__spintent_sparc_refs_tl` `\l__spintent_sparc_intent_arg_tl` `\l__spintent_sparc_pt_idx_int` `\l__spintent_sparc_intent_tl` holds the concept word (arco or medida-del-arco). The rest are scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1`, `$pt2`, ... reference list used in the concept intent.

```

4300 \tl_new:N \l__spintent_sparc_intent_tl
4301 \seq_new:N \l__spintent_sparc_pt_seq
4302 \tl_new:N \l__spintent_sparc_refs_tl
4303 \tl_new:N \l__spintent_sparc_intent_arg_tl
4304 \int_new:N \l__spintent_sparc_pt_idx_int

```

(End of definition for `\l__spintent_sparc_intent_tl` and others.)

`\__spintent_sparc_process:nn`

The internal function `\__spintent_sparc_process:nn` receives the short name of the command (arc or marc) in ‘#1’ and the `{(argument)}` in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then sets `\l__spintent_sparc_intent_tl` to arco (or medida-del-arco for `\spmarc`). No `intent` is built here: it needs the reference list, which `\__spintent_sparc_mathml` builds after splitting the points.

```

4305 \cs_new_protected:Nn \__spintent_sparc_process:nn
4306 {
4307   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
4308   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4309   {
4310     \msg_error:nnee { spintent } { sp #1 -invalid-points }
4311     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4312   }
4313   \str_if_eq:eeTF {#1} { marc }
4314   { \tl_set:Nn \l__spintent_sparc_intent_tl { medida-del-arco } }
4315   { \tl_set:Nn \l__spintent_sparc_intent_tl { arco } }
4316   \__spintent_sparc_mathml:n {#1}
4317 }

```

(End of definition for `\__spintent_sparc_process:nn`.)

`\__spintent_sparc_mathml:n`

The function `\__spintent_sparc_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_` into `\l__spintent_sparc_pt_seq` and walks it once to build `\l__spintent_sparc_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each point by position.

```

4318 \cs_new_protected:Nn \__spintent_sparc_mathml:n
4319 {
4320   \seq_set_from_clist:NN \l__spintent_sparc_pt_seq \l__spintent_geo_luaset_points_clist
4321
4322   \tl_clear:N \l__spintent_sparc_refs_tl
4323   \int_zero:N \l__spintent_sparc_pt_idx_int
4324   \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4325   {
4326     \int_incr:N \l__spintent_sparc_pt_idx_int
4327     \tl_if_empty:NF \l__spintent_sparc_refs_tl
4328     { \tl_put_right:Nn \l__spintent_sparc_refs_tl { , } }
4329     \tl_put_right:Ne \l__spintent_sparc_refs_tl { $ pt \int_use:N \l__spintent_sparc_pt_idx_int }
4330   }

```

The concept `intent` is `_` - when `silent-cmd` is `true` or `concept` is `false`; otherwise `\l__spintent_sparc_intent_` prefixed with `prefix` when given.

```

4331 \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4332 {
4333   \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4334 }
4335 {
4336   \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4337   {
4338     \tl_set:Ne \l__spintent_sparc_intent_arg_tl
4339     {
4340       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4341       { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4342       \l__spintent_sparc_intent_tl :prefix( \l__spintent_sparc_refs_tl )
4343     }
4344   }
4345   {
4346     \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4347   }
4348 }

```

It is applied with a single native `\MathMLintent` wrapping `\__spintent_sparc_print_arc:n` directly (same shape as `\__spintent_spside_mathml:n`), preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26). A silent `\operatorname{m}` precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

4349 \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
4350 \MathMLintent { \l__spintent_sparc_intent_arg_tl }
4351 {
4352   \__spintent_inner_sep:
4353   \str_if_eq:eeT {#1} { marc }

```

```

4354     {
4355       \__spintent_luafun_mrow_intent_arg:nn
4356       { } { intent = ":silent" }
4357       { \operatorname { m } }
4358     }
4359     \__spintent_sparc_print_arc:n
4360     {
4361       \int_zero:N \__spintent_sparc_pt_idx_int
4362       \seq_map_inline:Nn \__spintent_sparc_pt_seq
4363       {
4364         \int_incr:N \__spintent_sparc_pt_idx_int
4365         \__spintent_mathmlarg_point:nnn
4366         { pt \int_use:N \__spintent_sparc_pt_idx_int }
4367         { sp #1 }
4368         { ##1 }
4369       }
4370     }
4371   }
4372 }

```

(End of definition for __spintent_sparc_mathml:n.)

__spintent_sparc_exec:nnn

The function `\__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *argument* passed in ‘#3’, delegating the rendering to `\__spintent_sparc_process:nn`.

```

4373 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
4374 {
4375   \keys_set:nn { spintent / sp #1 } { #2 }
4376   \__spintent_sparc_process:nn { #1 } { #3 }
4377 }

```

(End of definition for __spintent_sparc_exec:nnn.)

\sparc
\spmarc

Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sparc_exec:nnn`.

```

4378 \NewDocumentCommand \sparc { O{} m }
4379 {
4380   \mode_if_math:TF
4381   {
4382     \group_begin:
4383     \attribute \__spintent_scope_attr_int = 1
4384     \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
4385     \group_end:
4386   }
4387   { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
4388 }
4389 \NewDocumentCommand \spmarc { O{} m }
4390 {
4391   \mode_if_math:TF
4392   {
4393     \group_begin:
4394     \attribute \__spintent_scope_attr_int = 1
4395     \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
4396     \group_end:
4397   }
4398   { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
4399 }

```

(End of definition for \sparc and \spmarc. These functions are documented on page 23.)

11.43 The command \spPoly

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\__spintent_geo_luaseta_intent_str` (the full *intent*, with the figure name).

11.43.1 Definition of variables for \spPoly

The variable `\__spintent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```

4400 \str_new:N \__spintent_spPoly_mode_str
4401 \tl_new:N \__spintent_spPoly_prefix_tl

```

(End of definition for __spintent_spPoly_mode_str and __spintent_spPoly_prefix_tl.)

11.43.2 Definition of error messages for \spPoly

spPoly-invalid-points Raised when the argument is not a comma-separated list of at least three uppercase letters.

```

4402 \msg_new:nnnn { spintent } { spPoly-invalid-points }
4403 {
4404   Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'~\msg_line_context:.
4405 }
4406 {
4407   See~the~documentation~of~\token_to_str:N \spPoly~for~the~accepted~point~formats.
4408 }

```

(End of definition for spPoly-invalid-points.)

11.43.3 Definition of keys for \spPoly

read-cmd We define the *(keys)* read-cmd, prefix, print-sym, silent-cmd and concept.

```

4409 \keys_define:nn { spintent / spPoly }
4410 {
4411   read-cmd          .choices:n = { school , mathcat }
4412   {
4413     \int_case:nn { \l_keys_choice_int }
4414     {
4415       {1} { \str_set:Nn \l__spintent_spPoly_mode_str { school } }
4416       {2} { \str_set:Nn \l__spintent_spPoly_mode_str { mathcat } }
4417     }
4418   },
4419   read-cmd          .initial:n = school,
4420   read-cmd          .value_required:n = true,
4421   read-cmd / unknown .code:n =
4422     \msg_error:nneee { spintent } { unknown-choice }
4423     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4424   prefix            .code:n =
4425     { \__spintent_sanitize_affix:Nn \l__spintent_spPoly_prefix_tl {#1} },
4426   prefix            .initial:n = ,
4427   prefix            .value_required:n = true,
4428   print-sym         .bool_set:N = \l__spintent_spPoly_print_sym_bool,
4429   print-sym         .initial:n = true,
4430   print-sym         .value_required:n = true,
4431   silent-cmd        .bool_set:N = \l__spintent_spPoly_silent_cmd_bool,
4432   silent-cmd        .initial:n = false,
4433   silent-cmd        .value_required:n = true,
4434   concept           .bool_set:N = \l__spintent_spPoly_concept_bool,
4435   concept           .initial:n = true,
4436   concept           .value_required:n = true,
4437   unknown           .code:n =
4438     {
4439       \str_set:Nn \l__spintent_command_name_str { spPoly }
4440       \__spintent_unknown_keys_cmd:n {#1}
4441     },
4442 }

```

(End of definition for read-cmd and others.)

11.43.4 Internal functions for \spPoly

\l__spintent_spPoly_pt_seq Scratch variables to split \l__spintent_geo_luaset_points_clist and build the \$pt1, \$pt2, ... reference list, and to hold the final concept intent once built.

```

\l__spintent_spPoly_refs_tl
\l__spintent_spPoly_intent_arg_tl
\l__spintent_spPoly_pt_idx_int
4443 \seq_new:N \l__spintent_spPoly_pt_seq
4444 \tl_new:N \l__spintent_spPoly_refs_tl
4445 \tl_new:N \l__spintent_spPoly_intent_arg_tl
4446 \int_new:N \l__spintent_spPoly_pt_idx_int

```

(End of definition for \l__spintent_spPoly_pt_seq and others.)

__spintent_spPoly_sym: Unchanged: reads \l__spintent_geo_luaset_poly_sym_str and typesets the corresponding symbol.

```

4447 \cs_new_protected:Nn \__spintent_spPoly_sym:
4448 {
4449   \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4450   {
4451     { triangle } { \triangle }
4452     { square } { \mdlgwhtsquare }
4453   }
4454 }

```

(End of definition for `\__spintent_spPoly_sym:`.)

`\__spintent_spPoly_build_refs:` Splits `\__spintent_geo_luaset_points_clist` into `\__spintent_spPoly_pt_seq` and walks it once to build `\__spintent_spPoly_refs_tl`, the comma-separated `$pt1, $pt2, ...` reference list, naming each vertex by position.

```

4455 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
4456 {
4457   \seq_set_from_clist:NN \__spintent_spPoly_pt_seq \__spintent_geo_luaset_points_clist
4458   \tl_clear:N \__spintent_spPoly_refs_tl
4459   \int_zero:N \__spintent_spPoly_pt_idx_int
4460   \seq_map_inline:Nn \__spintent_spPoly_pt_seq
4461   {
4462     \int_incr:N \__spintent_spPoly_pt_idx_int
4463     \tl_if_empty:NF \__spintent_spPoly_refs_tl
4464     { \tl_put_right:Nn \__spintent_spPoly_refs_tl { , } }
4465     \tl_put_right:Ne \__spintent_spPoly_refs_tl { $ pt \int_use:N \__spintent_spPoly_pt_idx.
4466   }
4467 }
```

(End of definition for `\__spintent_spPoly_build_refs:`.)

`\__spintent_spPoly_build_concept:` Builds `\__spintent_spPoly_intent_arg_tl`, independently of `print-sym: _`(refs) when `silent-cmd` is `true` or `concept` is `false`; otherwise `\__spintent_geo_luaset_concept_str:prefix(refs)`, prefixed with `prefix` when given. Must be called after `\__spintent_spPoly_build_refs:`.

```

4468 \cs_new_protected:Nn \__spintent_spPoly_build_concept:
4469 {
4470   \bool_if:NTF \__spintent_spPoly_silent_cmd_bool
4471   {
4472     \tl_set:Ne \__spintent_spPoly_intent_arg_tl { _ ( \__spintent_spPoly_refs_tl ) }
4473   }
4474   {
4475     \bool_if:NTF \__spintent_spPoly_concept_bool
4476     {
4477       \tl_set:Ne \__spintent_spPoly_intent_arg_tl
4478       {
4479         \tl_if_empty:NF \__spintent_spPoly_prefix_tl
4480         { \tl_use:N \__spintent_spPoly_prefix_tl - }
4481         \__spintent_geo_luaset_concept_str :prefix( \__spintent_spPoly_refs_tl )
4482       }
4483     }
4484     {
4485       \tl_set:Ne \__spintent_spPoly_intent_arg_tl { _ ( \__spintent_spPoly_refs_tl ) }
4486     }
4487   }
4488 }
```

(End of definition for `\__spintent_spPoly_build_concept:`.)

`\__spintent_spPoly_points_loop:` Walks `\__spintent_spPoly_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `spPoly:` it alone decides `arg/literal intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute/sub-sup` limitation already found and worked around for `\spPoint`).

```

4489 \cs_new_protected:Nn \__spintent_spPoly_points_loop:
4490 {
4491   \int_zero:N \__spintent_spPoly_pt_idx_int
4492   \seq_map_inline:Nn \__spintent_spPoly_pt_seq
4493   {
4494     \int_incr:N \__spintent_spPoly_pt_idx_int
4495     \__spintent_mathmlarg_point:nnn
4496     { pt \int_use:N \__spintent_spPoly_pt_idx_int }
4497     { spPoly }
4498     { ##1 }
4499   }
4500 }
```

(End of definition for `\__spintent_spPoly_points_loop:`.)

`\__spintent_spPoly_process:n` Calls `\__spintent_luafun_geo_poly_parse_and_set:n`, reports `spPoly-invalid-points` on failure, then delegates to `\__spintent_spPoly_mathml:`. No `intent` is built here: it needs the reference list, which `\__spintent_spPoly_mathml:` builds after splitting the vertices.

```

4501 \cs_new_protected:Nn \__spintent_spPoly_process:n
```

```

4502 {
4503   \__spintenc_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
4504   \str_if_eq:VnT \l__spintenc_geo_luaset_error_str { true }
4505   {
4506     \msg_error:nne { spintenc } { spPoly-invalid-points }
4507     { \exp_not:n {#1} }
4508   }
4509   \__spintenc_spPoly_mathml:
4510 }

```

(End of definition for __spintenc_spPoly_process:n.)

__spintenc_spPoly_mathml: Builds the reference list and the concept intent(__spintenc_spPoly_build_refs:, __spintenc_spPoly_build_refs:) then wraps everything in a single native \MathMLintent, preceded by __spintenc_inner_sep: (a second sibling so the sole <mrow> does not collapse into <math>, luamml issue #26).

print-sym and concept/silent-cmd are independent: the former only decides whether the glyph (__spintenc_spPoly_sym:, silenced with __spintenc_luafun_mrow_intent_arg:nn) is attempted — it is skipped anyway when \l__spintenc_geo_luaset_poly_sym_str is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

4511 \cs_new_protected:Nn \__spintenc_spPoly_mathml:
4512 {
4513   \__spintenc_invisible_sep: % (luamml issues #26)
4514   \__spintenc_spPoly_build_refs:
4515   \__spintenc_spPoly_build_concept:
4516   \MathMLintent { \l__spintenc_spPoly_intent_arg_tl }
4517   {
4518     \__spintenc_inner_sep:
4519     \bool_if:NT \l__spintenc_spPoly_print_sym_bool
4520     {
4521       \str_if_empty:NF \l__spintenc_geo_luaset_poly_sym_str
4522       {
4523         \__spintenc_luafun_mrow_intent_arg:nn
4524         { } { intent = ":silent" }
4525         { \__spintenc_spPoly_sym: }
4526       }
4527     }
4528     \__spintenc_spPoly_points_loop:
4529   }
4530 }

```

(End of definition for __spintenc_spPoly_mathml:.)

__spintenc_spPoly_exec:nn The function __spintenc_spPoly_exec:nn receives the optional keys in #1 and the argument in #2, delegating the rendering to __spintenc_spPoly_process:n.

```

4531 \cs_new_protected:Nn \__spintenc_spPoly_exec:nn
4532 {
4533   \keys_set:nn { spintenc / spPoly } { #1 }
4534   \__spintenc_spPoly_process:n { #2 }
4535 }

```

(End of definition for __spintenc_spPoly_exec:nn.)

\spPoly The public command \spPoly enforces *math mode*, opens a local group, and delegates execution to __spintenc_spPoly_exec:nn.

```

4536 \NewDocumentCommand \spPoly { 0{} m }
4537 {
4538   \mode_if_math:TF
4539   {
4540     \group_begin:
4541     \attribute \l__spintenc_scope_attr_int = 1
4542     \__spintenc_spPoly_exec:nn { #1 } { #2 }
4543     \group_end:
4544   }
4545   { \msg_error:nnn { spintenc } { need-math-mode } { \spPoly } }
4546 }

```

(End of definition for \spPoly. This function is documented on page 24.)

11.44 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()` already established by `\spmcd/\spmcm`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\l__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintent{(:silent)}` — the same mechanism already used by `\spPoly`.

11.44.1 Definition of variables for `\spAfig` and `\spPfig`

```
\l__spintent_geo_final_print_tl
```

Scratch variables used by `\__spintent_spfig_process:nn` to assemble the visual and the `intent`. `\l__spintent_geo_final_print_tl` holds the final visual placed under `\mathcal`’s subscript. The final literal `intent` and the concept forwarded when `read-cmd` is `mathcat`.

```
4547 \tl_new:N \l__spintent_geo_final_print_tl
```

(End of definition for `\l__spintent_geo_final_print_tl`.)

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

```
4548 \clist_map_inline:nn { Afig , Pfig }
```

```
4549 {
```

```
4550   \str_new:c { \l__spintent_sp #1 _mode_str      }
```

```
4551   \str_new:c { \l__spintent_sp #1 _print_sym_str }
```

```
4552   \tl_new:c { \l__spintent_sp #1 _prefix_tl     }
```

```
4553   \tl_new:c { \l__spintent_sp #1 _read_arg_tl   }
```

```
4554 }
```

(End of definition for `\l__spintent_spAfig_mode_str` and others.)

11.44.2 Definition of error messages for `\spAfig` and `\spPfig`

All eight messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside`. `sp #1-invalid-points` is currently unused by `\__spintent_spfig_process:nn`, which raises only `sp #1-invalid-arg` for any parse failure, regardless of whether a comma was present. `sp #1-sym-letra-conflict` and `sp #1-sym-cardinality-conflict` are raised when `print-sym` is forced to a value incompatible with the actual argument.

```
4555 \clist_map_inline:nn { Afig , Pfig }
```

```
4556 {
```

```
4557   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
```

```
4558   {
```

```
4559     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
```

```
4560   }
```

```
4561   {
```

```
4562     See~the~documentation~of~##1~for~the~accepted~point~formats.
```

```
4563   }
```

```
4564   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
```

```
4565   {
```

```
4566     Invalid~argument~format~in~##1:~'##2'~\msg_line_context:.
```

```
4567   }
```

```
4568   {
```

```
4569     See~the~documentation~of~##1~for~the~accepted~argument~forms.
```

```
4570   }
```

```
4571   \msg_new:nnnn { spintent } { sp #1 -sym-letra-conflict }
```

```
4572   {
```

```
4573     print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'~\msg_line_context:.
```

```
4574   }
```

```
4575   {
```

```
4576     See~the~documentation~of~##1~for~details~on~print-sym.
```

```
4577   }
```

```
4578   \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
```

```
4579   {
```

```
4580     print-sym~does~not~match~the~vertex~count~in~'##1'~\msg_line_context:.
```

```
4581   }
```

```
4582   {
```

```
4583     See~the~documentation~of~##1~for~details~on~print-sym.
```

```
4584   }
```

```
4585   \msg_new:nnnn { spintent } { sp #1 -read-arg-key }
```

```
4586   {
```

```
4587     read-arg~has~no~effect~with~a~single~argument~in~'##1'~\msg_line_context:.
```

```
4588   }
```

```
4589   {
```

```
4590     See~the~documentation~of~##1~for~details~on~read-arg.
```

```

4591     }
4592 }

```

(End of definition for `spAfig-invalid-points` and others.)

11.44.3 Definition of keys for `\spAfig` and `\spPfig`

The key `print-sym` chooses the visual symbol: `auto` (default) infers it from the vertex count (three points give `\triangle`, four give `\mdlgwhtsquare`, five or more give none); `triangle/square` force one, requiring the vertex count to match; `none` omits it, even with vertices. With a bare number/letter argument, `auto` and `none` behave the same (no symbol); `triangle/square` nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key `read-arg` overrides the spoken figure name for real vertices, independently of `print-sym`. The key `read-sub` controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

4593 \clist_map_inline:nn { Afig , Pfig }
4594 {
4595   \keys_define:nn { spintent / sp #1 }
4596   {
4597     read-cmd      .choices:nn = { school , mathcat }
4598     {
4599       \int_case:nn { \l_keys_choice_int }
4600       {
4601         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4602         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4603       }
4604     },
4605     read-cmd      .initial:n = school,
4606     read-cmd      .value_required:n = true,
4607     read-cmd / unknown .code:n =
4608       \msg_error:nneee { spintent } { unknown-choice }
4609       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4610     prefix        .code:n =
4611       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4612     prefix        .initial:n = ,
4613     prefix        .value_required:n = true,
4614     print-sym     .choices:nn = { auto , triangle , square , none }
4615     {
4616       \int_case:nn { \l_keys_choice_int }
4617       {
4618         {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto } }
4619         {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
4620         {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square } }
4621         {4} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { none } }
4622       }
4623     },
4624     print-sym     .initial:n = auto,
4625     print-sym     .value_required:n = true,
4626     print-sym / unknown .code:n =
4627       \msg_error:nneee { spintent } { unknown-choice }
4628       { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
4629     read-arg      .code:n =
4630       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4631     read-arg      .initial:n = ,
4632     read-arg      .value_required:n = true,
4633     read-sub      .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
4634     read-sub      .initial:n = false,
4635     read-sub      .value_required:n = true,
4636     silent-cmd    .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
4637     silent-cmd    .initial:n = false,
4638     silent-cmd    .value_required:n = true,
4639     concept       .bool_set:c = { l__spintent_sp #1 _concept_bool },
4640     concept       .initial:n = true,
4641     concept       .value_required:n = true,
4642     concept*      .meta:n =
4643     {
4644       concept = false,
4645       read-arg = { \str_if_eq:eeTF {#1} { Afig } { _área } { _perímetro } },
4646     },
4647     concept*      .value_forbidden:n = true,
4648     unknown       .code:n =
4649     {
4650       \str_set:Nn \l__spintent_command_name_str { sp #1 }

```

```

4651         \__spintent_unknown_keys_cmd:n {##1}
4652     },
4653 }
4654 }

```

(End of definition for read-cmd and others.)

11.44.4 Internal functions for \spAfig and \spPfig

`\__spintent_spfig_sym:` The function `\__spintent_spfig_sym:` reads `\__spintent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```

4655 \cs_new_protected:Nn \__spintent_spfig_sym:
4656 {
4657     \str_case:Nn \__spintent_geo_print_sym_str
4658     {
4659         { triangle } { \triangle }
4660         { square } { \mdlgwhtsquare }
4661     }
4662 }

```

(End of definition for `\__spintent_spfig_sym:`.)

`\__spintent_spfig_op_tl` `\__spintent_spfig_op_tl` holds `área` or `perímetro`. `\__spintent_spfig_intent_tl` holds the concept word for the vertices case (`read-arg` or the auto-detected figure name). The rest are scratch variables to split `\__spintent_geo_luaset_points_clist` and build the `$pt1`, `$pt2`, ... reference list.

```

\__spintent_spfig_op_tl
\__spintent_spfig_intent_tl
\__spintent_spfig_symbol_tl
\__spintent_spfig_pt_seq
\__spintent_spfig_refs_tl
\__spintent_spfig_intent_arg_tl
\__spintent_spfig_pt_idx_int
\__spintent_geo_print_sym_str
4663 \tl_new:N \__spintent_spfig_op_tl
4664 \tl_new:N \__spintent_spfig_intent_tl
4665 \tl_new:N \__spintent_spfig_desc_tl
4666 \tl_new:N \__spintent_spfig_symbol_tl
4667 \seq_new:N \__spintent_spfig_pt_seq
4668 \tl_new:N \__spintent_spfig_refs_tl
4669 \tl_new:N \__spintent_spfig_intent_arg_tl
4670 \int_new:N \__spintent_spfig_pt_idx_int
4671 \str_new:N \__spintent_geo_print_sym_str

```

(End of definition for `\__spintent_spfig_op_tl` and others.)

`\__spintent_spfig_process:nn` Receives the short command name (Afig or Pfig) in ‘#1’ and the `{\argument}` in ‘#2’. Calls `\__spintent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, and `sp#1-read-arg` when `read-arg` is given for a bare number/letter argument (no vertices), where it has no effect. Sets `\__spintent_spfig_op_tl` to `área`/`perímetro`, then delegates to `\__spintent_spfig_mathml:n`.

```

4672 \cs_new_protected:Nn \__spintent_spfig_process:nn
4673 {
4674     \str_if_eq:eeTF {#1} { Afig }
4675     { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
4676     { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
4677     \str_if_eq:NnT \__spintent_geo_luaset_error_str { true }
4678     {
4679         \msg_error:nnee { spintent } { sp #1 -invalid-arg }
4680         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4681     }
4682     \clist_if_empty:NT \__spintent_geo_luaset_points_clist
4683     {
4684         \tl_if_empty:cF { \__spintent_sp #1 _read_arg_tl }
4685         {
4686             \msg_error:nne { spintent } { sp #1 -read-arg-key }
4687             { \c_backslash_str sp #1 }
4688         }
4689     }
4690     \str_if_eq:eeTF {#1} { Afig }
4691     { \tl_set:Nn \__spintent_spfig_op_tl { área } }
4692     { \tl_set:Nn \__spintent_spfig_op_tl { perímetro } }
4693     \str_set:Ne \__spintent_geo_print_sym_str { \str_use:c { \__spintent_sp #1 _print_sym_str } }
4694     \__spintent_spfig_mathml:n {#1}
4695 }

```

(End of definition for `\__spintent_spfig_process:nn`.)

`\__spintent_spfig_build_simple:n` Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds `\__spintent_geo_final_print` (the visual placed under `\mathcal`’s subscript) and `\__spintent_spfig_desc_tl` (the descriptive part, without `área`/`perímetro`). Receives the short command name in ‘#1’.

With an empty argument, both are cleared for `print-sym auto/ none`; a forced `print-sym` instead places the symbol alone, with its own name as the description.

```

4696 \cs_new_protected:Nn \__spintent_spfig_build_simple:n
4697 {
4698   \tl_if_empty:NTF { \__spintent_geo_luaset_print_tl }
4699   {
4700     \str_case:VnF \__spintent_geo_print_sym_str
4701     {
4702       { auto }
4703       {
4704         \tl_clear:N \__spintent_geo_final_print_tl
4705         \tl_clear:N \__spintent_spfig_desc_tl
4706       }
4707       { none }
4708       {
4709         \tl_clear:N \__spintent_geo_final_print_tl
4710         \tl_clear:N \__spintent_spfig_desc_tl
4711       }
4712     }
4713   }
4714   \tl_set:Nn \__spintent_geo_final_print_tl { \__spintent_spfig_sym: }
4715   \tl_set:Ne \__spintent_spfig_desc_tl
4716   {
4717     \str_if_eq:VnTF \__spintent_geo_print_sym_str { triángulo }
4718     { triángulo } { cuadrado }
4719   }
4720 }
4721 }
```

With a bare number/letter (empty `\__spintent_geo_luaset_letra_word_str`), the value itself becomes the visual for `print-sym auto/none`, or its own subscript under a forced symbol otherwise; the description is the spoken value, prefixed with «sub-» when `read-sub` is `true` (further prefixed with the symbol's own name for a forced `print-sym`).

```

4722 {
4723   \str_if_empty:NTF \__spintent_geo_luaset_letra_word_str
4724   {
4725     \str_case:VnF \__spintent_geo_print_sym_str
4726     {
4727       { auto } { \tl_set:Nn \__spintent_geo_final_print_tl { \__spintent_geo_luaset_p
4728       { none } { \tl_set:Nn \__spintent_geo_final_print_tl { \__spintent_geo_luaset_p
4729     }
4730   }
4731   \tl_set:Nn \__spintent_geo_final_print_tl
4732   {
4733     \__spintent_spfig_sym:
4734     \c_math_subscript_token { \__spintent_geo_luaset_print_tl }
4735   }
4736 }
4737 \str_case:VnF \__spintent_geo_print_sym_str
4738 {
4739   { auto }
4740   {
4741     \tl_set:Ne \__spintent_spfig_desc_tl
4742     {
4743       \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
4744       \__spintent_geo_luaset_sub_spoken_str
4745     }
4746   }
4747   { none }
4748   {
4749     \tl_set:Ne \__spintent_spfig_desc_tl
4750     {
4751       \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
4752       \__spintent_geo_luaset_sub_spoken_str
4753     }
4754   }
4755 }
4756 {
4757   \tl_set:Ne \__spintent_spfig_desc_tl
4758   {
4759     del-
```

```

4760         \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4761         { triángulo } { cuadrado }
4762     -
4763     \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4764     \l__spintent_geo_luaset_sub_spoken_str
4765 }
4766 }
4767 }

```

With a table letter (F/P/B/L/T, non-empty `\l__spintent_geo_luaset_letra_word_str`), a forced `print-sym` raises `sp#1-sym-letra-conflict` — a table letter is never nested under a symbol. The visual is the value as-is; the description is its own word, connected as `\l__spintent_geo_luaset_letra_conector_str` requires, with the same spoken subscript as above when present.

```

4768     {
4769     \str_case:VnF \l__spintent_geo_print_sym_str
4770     {
4771         { auto } { }
4772         { none } { }
4773     }
4774     {
4775         \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
4776         { \c_backslash_str sp #1 }
4777     }
4778     \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
4779     \tl_set:Ne \l__spintent_spfig_desc_tl
4780     {
4781         \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
4782         { \l__spintent_geo_luaset_letra_conector_str - }
4783         \l__spintent_geo_luaset_letra_word_str
4784         \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
4785         {
4786             -
4787             \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4788             \l__spintent_geo_luaset_sub_spoken_str
4789         }
4790     }
4791 }
4792 }

```

The variable `\l__spintent_spfig_intent_tl` is always the `_word-` shape: `\l__spintent_spfig_op_tl` alone when `\l__spintent_spfig_desc_tl` is empty, both joined by a dash otherwise. `silent-cmd` overrides `\l__spintent_spfig_intent_arg_tl` with `_-`; with `silent-cmd=false` and `concept false`, it becomes `_ \l__spintent_spfig_desc_tl-` (the descriptive part alone, without *área/perímetro*); with `concept true` it is `\l__spintent_spfig_intent_tl`.

```

4793     \tl_set:Ne \l__spintent_spfig_intent_tl
4794     {
4795         \tl_if_empty:NTF \l__spintent_spfig_desc_tl
4796         { _ \l__spintent_spfig_op_tl - }
4797         { _ \l__spintent_spfig_op_tl - \l__spintent_spfig_desc_tl - }
4798     }
4799     \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4800     { \tl_set:Nn \l__spintent_spfig_intent_arg_tl { _- } }
4801     {
4802         \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4803         { \tl_set:Ne \l__spintent_spfig_intent_arg_tl { \l__spintent_spfig_intent_tl } }
4804         { \tl_set:Ne \l__spintent_spfig_intent_arg_tl { _ \l__spintent_spfig_desc_tl - } }
4805     }
4806 }

```

(End of definition for `\l__spintent_spfig_build_simple:n`)

`\l__spintent_spfig_build_refs:`

Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spfig_pt_seq` and walks it once to build `\l__spintent_spfig_refs_tl`, the comma-separated `$pt1, $pt2, ...` reference list, naming each vertex by position.

```

4807 \cs_new_protected:Nn \l__spintent_spfig_build_refs:
4808 {
4809     \seq_set_from_clist:NN \l__spintent_spfig_pt_seq \l__spintent_geo_luaset_points_clist
4810     \tl_clear:N \l__spintent_spfig_refs_tl
4811     \int_zero:N \l__spintent_spfig_pt_idx_int
4812     \seq_map_inline:Nn \l__spintent_spfig_pt_seq
4813     {

```

```

4814         \int_incr:N \l__spintent_sfig_pt_idx_int
4815         \tl_if_empty:NF \l__spintent_sfig_refs_tl
4816         { \tl_put_right:Nn \l__spintent_sfig_refs_tl { , } }
4817         \tl_put_right:Ne \l__spintent_sfig_refs_tl { $ pt \int_use:N \l__spintent_sfig_pt_idx_int
4818     }
4819 }

```

(End of definition for `\l__spintent_sfig_build_refs:`.)

`\l__spintent_sfig_build_concept_vertices:n` Handles the vertices case. Receives the short command name in ‘#1’. Must be called after `\l__spintent_sfig_build_refs:n`. `read-arg`, when given, overrides the auto-detected figure name in `\l__spintent_sfig_intent_tl`: with `concept true` it is still prefixed with `área-del-/perímetro-del-`, with `concept false` it is used verbatim. With `read-arg` empty, `\l__spintent_sfig_intent_tl` is simply `\l__spintent_geo_luaset_concept_str`.

```

4820 \cs_new_protected:Nn \l__spintent_sfig_build_concept_vertices:n
4821 {
4822     \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
4823     { \tl_set:Ne \l__spintent_sfig_intent_tl { \l__spintent_geo_luaset_concept_str } }
4824     {
4825         \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4826         {
4827             \tl_set:Ne \l__spintent_sfig_intent_tl
4828             {
4829                 \l__spintent_sfig_op_tl -del-
4830                 \tl_use:c { \l__spintent_sp #1 _read_arg_tl }
4831             }
4832         }
4833         {
4834             \tl_set:Ne \l__spintent_sfig_intent_tl { \tl_use:c { \l__spintent_sp #1 _read_arg_tl } }
4835         }
4836     }

```

Determines `\l__spintent_sfig_symbol_tl` (empty when none applies): for `print-sym=auto`, the symbol matching `\l__spintent_geo_luaset_poly_sym_str` (`\triangle`/`\mdlgwhtsquare`), or none when that is empty; `print-sym=none` always clears it; a forced `print-sym` requires it to match the actual vertex count, reporting `sp#1-sym-cardinality-conflict` otherwise.

```

4837     \str_case:VnF \l__spintent_geo_print_sym_str
4838     {
4839         { auto }
4840         {
4841             \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4842             {
4843                 { triangle } { \tl_set:Nn \l__spintent_sfig_symbol_tl { \triangle } }
4844                 { square } { \tl_set:Nn \l__spintent_sfig_symbol_tl { \mdlgwhtsquare } }
4845             }
4846             \str_if_empty:NT \l__spintent_geo_luaset_poly_sym_str
4847             { \tl_clear:N \l__spintent_sfig_symbol_tl }
4848         }
4849         { none }
4850         { \tl_clear:N \l__spintent_sfig_symbol_tl }
4851     }
4852     {
4853         \str_if_eq:VVTf \l__spintent_geo_print_sym_str \l__spintent_geo_luaset_poly_sym_str
4854         { \tl_set:Nn \l__spintent_sfig_symbol_tl { \l__spintent_sfig_sym: } }
4855         {
4856             \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
4857             { \c_backslash_str sp #1 }
4858         }
4859     }

```

`silent-cmd` sets `\l__spintent_sfig_intent_arg_tl` to `_(refs)`. Otherwise, with `read-arg` empty, `concept` chooses between `\l__spintent_sfig_intent_tl:prefix(refs)` (prefixed with `prefix` when given) and `_(refs)` alone; with `read-arg` given, it is always `\l__spintent_sfig_intent_tl:prefix(refs)` — `read-arg` is never silenced by `concept` alone, only by `silent-cmd`.

```

4860     \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4861     {
4862         \tl_set:Ne \l__spintent_sfig_intent_arg_tl { _ ( \l__spintent_sfig_refs_tl ) }
4863     }
4864     {
4865         \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
4866         {
4867             \bool_if:cTF { \l__spintent_sp #1 _concept_bool }

```

```

4868         {
4869             \tl_set:Ne \l__spintent_spfig_intent_arg_tl
4870             {
4871                 \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4872                 { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4873                 \l__spintent_spfig_intent_tl :prefix( \l__spintent_spfig_refs_tl )
4874             }
4875         }
4876         {
4877             \tl_set:Ne \l__spintent_spfig_intent_arg_tl { _( \l__spintent_spfig_refs_tl ) }
4878         }
4879     }
4880     {
4881         \tl_set:Ne \l__spintent_spfig_intent_arg_tl
4882         {
4883             \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4884             { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4885             \l__spintent_spfig_intent_tl :prefix( \l__spintent_spfig_refs_tl )
4886         }
4887     }
4888 }
4889 }

```

(End of definition for `\l__spintent_spfig_build_concept_vertices:n`)

`\l__spintent_spfig_points_loop:n`

Walks `\l__spintent_spfig_pt_seq` a second time, calling `\l__spintent_mathmlarg_point:nnn` once per vertex with its `pt` n reference name, looked up under `sp` $\#1$. Receives the short command name in `'#1'`.

```

4890 \cs_new_protected:Nn \l__spintent_spfig_points_loop:n
4891 {
4892     \int_zero:N \l__spintent_spfig_pt_idx_int
4893     \seq_map_inline:Nn \l__spintent_spfig_pt_seq
4894     {
4895         \int_incr:N \l__spintent_spfig_pt_idx_int
4896         \l__spintent_mathmlarg_point:nnn
4897         { pt \int_use:N \l__spintent_spfig_pt_idx_int }
4898         { sp #1 }
4899         { ##1 }
4900     }
4901 }

```

(End of definition for `\l__spintent_spfig_points_loop:n`)

`\l__spintent_spfig_mathml:n`

Receives the short command name in `'#1'`. Branches on whether `\l__spintent_geo_luaseta_points_clist` is empty.

Without vertices, `\l__spintent_spfig_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` — with `concept=false` the content is typeset directly, with no `intent` at all. With vertices, `\l__spintent_spfig_build_refs:` then `\l__spintent_spfig_build_concept_vertices:n` build the reference list and the concept `intent`, applied with a single native `\MathMLintent`. Inside, `\l__spintent_mathcal:n{⟨A | P⟩}` is silenced with `\l__spintent_luafun_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`'s own symbol); `\l__spintent_spfig_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\l__spintent_spfig_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback's recursive lookup through a noad's sub field, not just its nucleus.

`\l__spintent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`).

```

4902 \cs_new_protected:Nn \l__spintent_spfig_mathml:n
4903 {
4904     \l__spintent_invisible_sep: % (luamml issues #26)
4905     \clist_if_empty:NNTF \l__spintent_geo_luaseta_points_clist
4906     {
4907         \l__spintent_spfig_build_simple:n {#1}
4908         \MathMLintent { \l__spintent_spfig_intent_arg_tl }
4909         {
4910             \l__spintent_inner_sep:
4911             \str_if_eq:eeTF {#1} { Afig } { \l__spintent_mathcal:n { A } } { \l__spintent_mathcal:n
4912             \tl_if_empty:NF \l__spintent_geo_final_print_tl
4913             {
4914                 \c_math_subscript_token { \l__spintent_geo_final_print_tl }
4915             }

```

```

4916     }
4917   }
4918   {
4919     \__spintent_spfig_build_refs:
4920     \__spintent_spfig_build_concept_vertices:n {#1}
4921     \MathMLintent { \__spintent_spfig_intent_arg_tl }
4922     {
4923       \__spintent_inner_sep:
4924       \__spintent_luafun_mrow_intent_arg:nn
4925       { } { intent = ":silent" }
4926       {
4927         \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal
4928       }
4929       \c_math_subscript_token
4930       {
4931         \tl_if_empty:NF \__spintent_spfig_symbol_tl
4932         {
4933           \__spintent_luafun_mrow_intent_arg:nn
4934           { } { intent = ":silent" }
4935           { \tl_use:N \__spintent_spfig_symbol_tl }
4936         }
4937         \__spintent_spfig_points_loop:n {#1}
4938       }
4939     }
4940   }
4941 }

```

(End of definition for __spintent_spfig_mathml:n.)

__spintent_spfig_exec:nnn The function __spintent_spfig_exec:nnn receives the short name of the command in #1, the optional keys in #2, and the argument in #3, delegating the rendering to __spintent_spfig_process:nn.

```

4942 \cs_new_protected:Nn \__spintent_spfig_exec:nnn
4943 {
4944   \keys_set:nn { spintent / sp #1 } { #2 }
4945   \__spintent_spfig_process:nn { #1 } { #3 }
4946 }

```

(End of definition for __spintent_spfig_exec:nnn.)

\spAfig Both public commands \spAfig and \spPfig enforce math mode, open a local group, and delegate execution to __spintent_spfig_exec:nnn.

```

4947 \NewDocumentCommand \spAfig { 0{ } d{ } }
4948 {
4949   \mode_if_math:TF
4950   {
4951     \group_begin:
4952     \attribute \__spintent_scope_attr_int = 1
4953     \tl_if_novalue:nTF {#2}
4954     { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }
4955     { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
4956     \group_end:
4957   }
4958   { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
4959 }
4960 \NewDocumentCommand \spPfig { 0{ } d{ } }
4961 {
4962   \mode_if_math:TF
4963   {
4964     \group_begin:
4965     \attribute \__spintent_scope_attr_int = 1
4966     \tl_if_novalue:nTF {#2}
4967     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
4968     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
4969     \group_end:
4970   }
4971   { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
4972 }

```

(End of definition for \spAfig and \spPfig. These functions are documented on page 25.)

11.45 The commands \sprad and \spdiam

The user-level commands `\sprad` and `\spdiam` provide a semantic interface for the radius and diameter of a circle. They delegate argument validation to the Lua module `spintent.lua`, thereby constructing a valid intent for MathCAT.

11.45.1 Definition of variables for \sprad and \spdiam

Each pair of variables is declared once for both `\sprad` and `\spdiam` via `\clist_map_inline:nn`.

```
4973 \clist_map_inline:nn { rad , diam }
4974 {
4975   \str_new:c { \__spintent_sp #1 _mode_str }
4976   \str_new:c { \__spintent_sp #1 _sub_index_str }
4977   \tl_new:c { \__spintent_sp #1 _read_arg_tl }
4978 }
```

(End of definition for `\__spintent_sprad_mode_str` and others.)

Scratch variables used by the function `\__spintent_sp_rad_diam_print:n`.

```
4979 \tl_new:N \__spintent_sp_rad_diam_letra_tl
4980 \tl_new:N \__spintent_sp_rad_diam_word_tl
```

(End of definition for `\__spintent_sp_rad_diam_letra_tl` and `\__spintent_sp_rad_diam_word_tl`.)

11.45.2 Definition of keys for \sprad and \spdiam

We define the shared `<keys>` `read-cmd`, `upper-case`, `sub-index`, `read-sub`, `read-arg`, `concept`, `silent-cmd` and `print-sym`.

```

read-cmd
upper-case
sub-index
read-sub
read-arg
concept
silent-cmd
print-sym
unknown
4981 \clist_map_inline:nn { rad , diam }
4982 {
4983   \keys_define:nn { spintent / sp #1 }
4984   {
4985     read-cmd      .choices:nn = { school , mathcat }
4986     {
4987       \int_case:nn { \_keys_choice_int }
4988       {
4989         {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
4990         {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
4991       }
4992     },
4993     read-cmd      .initial:n = school,
4994     read-cmd      .value_required:n = true,
4995     read-cmd / unknown .code:n =
4996       \msg_error:nneee { spintent } { unknown-choice }
4997       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4998     upper-case    .bool_set:c = { \__spintent_sp #1 _upper_case_bool },
4999     upper-case    .initial:n = false,
5000     upper-case    .value_required:n = true,
5001     sub-index     .code:n =
5002       { \str_set:cn { \__spintent_sp #1 _sub_index_str } {##1} },
5003     sub-index     .initial:n = ,
5004     sub-index     .value_required:n = true,
5005     read-sub      .bool_set:c = { \__spintent_sp #1 _read_sub_bool },
5006     read-sub      .initial:n = false,
5007     read-sub      .value_required:n = true,
5008     read-arg      .code:n =
5009       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _read_arg_tl } {##1} },
5010     read-arg      .initial:n = ,
5011     read-arg      .value_required:n = true,
5012     concept       .bool_set:c = { \__spintent_sp #1 _concept_bool },
5013     concept       .initial:n = true,
5014     concept       .value_required:n = true,
5015     silent-cmd    .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
5016     silent-cmd    .initial:n = false,
5017     silent-cmd    .value_required:n = true,
5018     print-sym     .bool_set:c = { \__spintent_sp #1 _print_sym_bool },
5019     print-sym     .initial:n = false,
5020     print-sym     .value_required:n = true,
5021     unknown       .code:n =
5022     {
5023       \str_set:Nn \__spintent_command_name_str { sp #1 }
5024       \__spintent_unknown_keys_cmd:n {##1}
5025     },
5026   }

```

```
5027 }
```

🌱 The keys `print-sym` are defined for both commands for simplicity, but has no effect on `\sprad`.

(End of definition for `read-cmd` and others.)

11.45.3 Internal functions for `\sprad` and `\spdiam`

```
\__spintent_sp_rad_diam_print:n
```

The function `\__spintent_sp_rad_diam_print:n` sets `\__spintent_sp_rad_diam_letra_tl` to the bare letter — `r/R` for `\sprad`, `d/D` for `\spdiam` — per `upper-case`, and, for `\spdiam` with `print-sym true`, silently typesets `\diameter` first. Receives the short command name in `#1` (`rad/diam`).

```
5028 \cs_new_protected:Nn \__spintent_sp_rad_diam_print:n
5029 {
5030   \bool_if:cTF { \__spintent_sp #1 _upper_case_bool }
5031   {
5032     \str_if_eq:eeTF {#1} { rad }
5033     { \tl_set:Nn \__spintent_sp_rad_diam_letra_tl { R } }
5034     { \tl_set:Nn \__spintent_sp_rad_diam_letra_tl { D } }
5035   }
5036   {
5037     \str_if_eq:eeTF {#1} { rad }
5038     { \tl_set:Nn \__spintent_sp_rad_diam_letra_tl { r } }
5039     { \tl_set:Nn \__spintent_sp_rad_diam_letra_tl { d } }
5040   }
5041   \__spintent_invisible_sep:
5042   \str_if_eq:eeT {#1} { diam }
5043   {
5044     \bool_if:NT \__spintent_spdiam_print_sym_bool
5045     { \MathMLintent { :silent } { \diameter } }
5046   }
```

With `silent-cmd true`, the intent is `_` — the letter alone through the shared queue when there is no `sub-index`, or letter and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```
5047   \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
5048   {
5049     \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
5050     {
5051       \__spintent_luafun_mrow_intent_arg:nn
5052       { } { intent = "_-" }
5053       { \__spintent_sp_rad_diam_letra_tl }
5054     }
5055     {
5056       \MathMLintent { _- }
5057       {
5058         \__spintent_inner_sep:
5059         \__spintent_sp_rad_diam_letra_tl
5060         \c_math_subscript_token
5061         {
5062           \MathMLintent { :silent }
5063           { \str_use:c { \__spintent_sp #1 _sub_index_str } }
5064         }
5065       }
5066     }
5067   }
```

Otherwise, with `concept true` (the default), it verbalizes `radio/diámetro` (`read-arg` replaces this word when given), with `sub-index`, if present, appended right after (`read-sub` choosing `sub-⟨value⟩` or `⟨value⟩` alone) — the letter itself is never read here. With no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent` for `read-cmd=school` (subscript silenced, already spoken), or the shared queue tags the letter alone with the subscript left bare for `read-cmd=mathcat`.

```
5068   {
5069     \bool_if:cTF { \__spintent_sp #1 _concept_bool }
5070     {
5071       \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
5072       {
5073         \str_if_eq:eeTF {#1} { rad }
5074         { \tl_set:Nn \__spintent_sp_rad_diam_word_tl { radio } }
5075         { \tl_set:Nn \__spintent_sp_rad_diam_word_tl { diámetro } }
5076       }
5077       \tl_set:Ne \__spintent_sp_rad_diam_word_tl { \tl_use:c { \__spintent_sp #1 _read_
5078       \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
5079       {
```

```

5080     \__spintent_luafun_mrow_intent_arg:nn
5081     { } { intent = "_ \__spintent_sp_rad_diam_word_tl - :prefix" }
5082   { \__spintent_sp_rad_diam_letra_tl }
5083 }
5084 {
5085   \str_if_eq:eeTF
5086   { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
5087   {
5088     \MathMLintent
5089     {
5090       _ \__spintent_sp_rad_diam_word_tl -
5091       \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
5092       \str_use:c { \__spintent_sp #1 _sub_index_str }
5093       :prefix
5094     }
5095     {
5096       \__spintent_inner_sep:
5097       \__spintent_sp_rad_diam_letra_tl
5098       \c_math_subscript_token
5099       {
5100         \MathMLintent { :silent }
5101         { \str_use:c { \__spintent_sp #1 _sub_index_str } }
5102       }
5103     }
5104   }
5105   {
5106     \__spintent_luafun_mrow_intent_arg:nn
5107     { } { intent = "_ \__spintent_sp_rad_diam_word_tl - :prefix" }
5108     { \__spintent_sp_rad_diam_letra_tl }
5109     \c_math_subscript_token
5110     { \str_use:c { \__spintent_sp #1 _sub_index_str } }
5111   }
5112 }
5113 }

```

With `concept false`, the letter itself is read instead — but only when `read-cmd` is `school`, built the same way from the letter (with `sub-index`, letter and subscript wrapped together, subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the letter and its subscript, unmarked by `us`.

```

5114   {
5115     \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
5116     {
5117       \str_if_eq:eeTF
5118       { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
5119       {
5120         \__spintent_luafun_mrow_intent_arg:nn
5121         { } { intent = "_ \__spintent_sp_rad_diam_letra_tl - :prefix" }
5122         { \__spintent_sp_rad_diam_letra_tl }
5123       }
5124       { \__spintent_sp_rad_diam_letra_tl }
5125     }
5126     {
5127       \str_if_eq:eeTF
5128       { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
5129       {
5130         \MathMLintent
5131         {
5132           _ \__spintent_sp_rad_diam_letra_tl -
5133           \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
5134           \str_use:c { \__spintent_sp #1 _sub_index_str }
5135           :prefix
5136         }
5137         {
5138           \__spintent_inner_sep:
5139           \__spintent_sp_rad_diam_letra_tl
5140           \c_math_subscript_token
5141           {
5142             \MathMLintent { :silent }
5143             { \str_use:c { \__spintent_sp #1 _sub_index_str } }
5144           }
5145         }
5146       }
5147     }
5148   }

```

```

5146         }
5147     {
5148         \l__spinttent_sp_rad_diam_letra_tl
5149         \c_math_subscript_token
5150         { \str_use:c { \l__spinttent_sp #1 _sub_index_str } }
5151     }
5152 }
5153 }
5154 }
5155 }

```

(End of definition for \l__spinttent_sp_rad_diam_print:n.)

\l__spinttent_sp_rad_diam_exec:nn

The function `\l__spinttent_sp_rad_diam_exec:nn` receives the *short name* of the command passed in ‘#1’ and the optional (*keys*) passed in ‘#2’ then delegating the rendering to `\l__spinttent_sp_rad_diam_print:n`.

```

5156 \cs_new_protected:Nn \l__spinttent_sp_rad_diam_exec:nn
5157 {
5158     \keys_set:nn { spinttent / sp #1 } {#2}
5159     \l__spinttent_sp_rad_diam_print:n {#1}
5160 }

```

(End of definition for \l__spinttent_sp_rad_diam_exec:nn.)

\sprad
\spdiam

Both public commands `\sprad` and `\spdiam` enforces *math mode*, opens a local group, and delegates execution to `\l__spinttent_sp_rad_diam_exec:nn`.

```

5161 \NewDocumentCommand \sprad { 0{} }
5162 {
5163     \mode_if_math:TF
5164     {
5165         \group_begin:
5166         \l__spinttent_sp_rad_diam_exec:nn { rad } {#1}
5167         \group_end:
5168     }
5169     { \msg_error:nnn { spinttent } { need-math-mode } { \sprad } }
5170 }
5171 \NewDocumentCommand \spdiam { 0{} }
5172 {
5173     \mode_if_math:TF
5174     {
5175         \group_begin:
5176         \l__spinttent_sp_rad_diam_exec:nn { diam } {#1}
5177         \group_end:
5178     }
5179     { \msg_error:nnn { spinttent } { need-math-mode } { \spdiam } }
5180 }

```

(End of definition for \sprad and \spdiam. These functions are documented on page 28.)

11.46 The command \spcirc

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()` already established by `\spmcd/\spmcm` and `\spAfig/\spPfig`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside/\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.46.1 Definition of variables for \spcirc

\l__spinttent_geo_luaset_circ_radio_type_str
\l__spinttent_geo_luaset_circ_radio_raw_str

Shared outputs of `\l__spinttent_luafun_geo_circ_parse_and_set:n`. `\l__spinttent_geo_luaset_circ_radio_type_str` holds one of *number*, *measure*, *label*, *segment*, or an empty string when no radius was given. `\l__spinttent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```

5181 \str_new:N \l__spinttent_geo_luaset_circ_radio_type_str
5182 \str_new:N \l__spinttent_geo_luaset_circ_radio_raw_str

```

(End of definition for \l__spinttent_geo_luaset_circ_radio_type_str and \l__spinttent_geo_luaset_circ_radio_raw_str.)

\l__spinttent_spcirc_mode_str
\l__spinttent_spcirc_prefix_tl
\l__spinttent_spcirc_print_sym_str
\l__spinttent_spcirc_read_sty_str
\l__spinttent_spcirc_radio_sty_str
\l__spinttent_spcirc_sub_index_str
\l__spinttent_spcirc_articulo_str

The variable `\l__spinttent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\l__spinttent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\l__spinttent_spcirc_print_sym_str` stores the value set by the key `print-sym` (*usual* or *odot*). The variable `\l__spinttent_spcirc_read_sty_str` stores the value set by the key `read-sty` (*circun* or *circle*). The variable `\l__spinttent_spcirc_radio_sty_str` stores the value set by the key `radio-sty` (*radius* or *circumference*).

stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\l__spintent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\l__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to `la` or `el`, matching the grammatical gender of the chosen word.

```

5183 \str_new:N \l__spintent_spcirc_mode_str
5184 \tl_new:N \l__spintent_spcirc_prefix_tl
5185 \str_new:N \l__spintent_spcirc_print_sym_str
5186 \str_new:N \l__spintent_spcirc_read_sty_str
5187 \str_new:N \l__spintent_spcirc_radio_sty_str
5188 \str_new:N \l__spintent_spcirc_sub_index_str
5189 \str_new:N \l__spintent_spcirc_articulo_str
5190 \tl_new:N \l__spintent_spcirc_read_arg_tl

```

(End of definition for `\l__spintent_spcirc_mode_str` and others.)

`\l__spintent_spcirc_word_tl` Scratch variables used by `\__spintent_spcirc_process:n` to assemble the chosen word (circunferencia/círculo or the `read-arg` override) and the final literal `intent` string.

```

5191 \tl_new:N \l__spintent_spcirc_word_tl

```

(End of definition for `\l__spintent_spcirc_word_tl`.)

11.46.2 Definition of error messages for `\spcirc`

`spcirc-invalid-arg` Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```

5192 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
5193 {
5194   Invalid~argument~format~in~\token_to_str:N \spcirc:~'!#1'~\msg_line_context:.
5195 }
5196 {
5197   See~the~documentation~of~\token_to_str:N \spcirc~for~the~accepted~argument~forms.
5198 }

```

(End of definition for `spcirc-invalid-arg`.)

11.46.3 Definition of keys for `\spcirc`

The key `print-sym` chooses the visual symbol: `usual` for a plain C, `odot` (default) for `\odot`. The key `read-sty` chooses the verbalized word and its article: `circun` (default) for «*la circunferencia*», `circle` for «*el círculo*». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (`side`, via `\spside`) or its *measure* (`mside`, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```

5199 \keys_define:nn { spintent / spcirc }
5200 {
5201   read-cmd      .choices:nn = { school , mathcat }
5202   {
5203     \int_case:nn { \l_keys_choice_int }
5204     {
5205       {1}
5206       {
5207         \str_set:Nn \l__spintent_spcirc_mode_str { school }
5208         \keys_set:nn { spintent / spside } { read-cmd = school }
5209         \keys_set:nn { spintent / spmside } { read-cmd = school }
5210       }
5211       {2}
5212       {
5213         \str_set:Nn \l__spintent_spcirc_mode_str { mathcat }
5214         \keys_set:nn { spintent / spside } { read-cmd = mathcat }
5215         \keys_set:nn { spintent / spmside } { read-cmd = mathcat }
5216       }
5217     }
5218   },
5219   read-cmd      .initial:n = school,
5220   read-cmd      .value_required:n = true,
5221   prefix        .code:n =
5222   { \__spintent_sanitizé:affix:Nn \l__spintent_spcirc_prefix_tl {#1} },
5223   prefix        .initial:n = ,
5224   prefix        .value_required:n = true,
5225   print-sym     .choices:nn = { usual , odot }
5226   {
5227     \int_case:nn { \l_keys_choice_int }
5228     {

```

```

5229         {1} { \str_set:Nn \l__spintent_spcirc_print_sym_str { usual } }
5230         {2} { \str_set:Nn \l__spintent_spcirc_print_sym_str { odot } }
5231     }
5232 },
5233 print-sym .initial:n = odot,
5234 print-sym .value_required:n = true,
5235 read-sty .choices:nn = { circun , circle }
5236 {
5237     \int_case:nn { \l_keys_choice_int }
5238     {
5239         {1} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circun } }
5240         {2} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circle } }
5241     }
5242 },
5243 read-sty .initial:n = circun,
5244 read-sty .value_required:n = true,
5245 diameter .bool_set:N = \l__spintent_spcirc_diameter_bool,
5246 diameter .initial:n = false,
5247 diameter .value_required:n = true,
5248 radio-sty .choices:nn = { side , mside }
5249 {
5250     \int_case:nn { \l_keys_choice_int }
5251     {
5252         {1} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { side } }
5253         {2} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { mside } }
5254     }
5255 },
5256 radio-sty .initial:n = mside,
5257 radio-sty .value_required:n = true,
5258 sub-index .code:n =
5259 { \str_set:Nn \l__spintent_spcirc_sub_index_str {#1} },
5260 sub-index .initial:n = ,
5261 sub-index .value_required:n = true,
5262 read-sub .bool_set:N = \l__spintent_spcirc_read_sub_bool,
5263 read-sub .initial:n = false,
5264 read-sub .value_required:n = true,
5265 read-arg .code:n =
5266 { \__spintent_sanitizet_affix:Nn \l__spintent_spcirc_read_arg_tl {#1} },
5267 read-arg .initial:n = ,
5268 read-arg .value_required:n = true,
5269 silent-cmd .bool_set:N = \l__spintent_spcirc_silent_cmd_bool,
5270 silent-cmd .initial:n = false,
5271 silent-cmd .value_required:n = true,
5272 concept .bool_set:N = \l__spintent_spcirc_concept_bool,
5273 concept .initial:n = true,
5274 concept .value_required:n = true,
5275 spside .meta:nn = { spintent / spside } { #1 },
5276 spside .value_required:n = true,
5277 spmside .meta:nn = { spintent / spmside } { #1 },
5278 spmside .value_required:n = true,
5279 unknown .code:n =
5280 {
5281     \str_set:Nn \l__spintent_command_name_str { spcirc }
5282     \__spintent_unknown_keys_cmd:n {#1}
5283 },
5284 }

```

(End of definition for read-cmd and others.)

11.46.4 Internal functions for \spcirc

`\l__spintent_spcirc_has_arg_bool` `\l__spintent_spcirc_intent_tl` `\l__spintent_spcirc_intent_arg_tl` `\l__spintent_spcirc_connector_tl` `\l__spintent_spcirc_connector_arg_tl` `\l__spintent_spcirc_radio_arg_tl`

`\l__spintent_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no `(...)` at all has neither a center nor a radius). `\l__spintent_spcirc_intent_tl` holds the word (circunferencia or círculo, with an optional `sub-index` suffix) paired with the symbol. `\l__spintent_spcirc_connector_tl` holds `de-centro-en`, paired with the (center) group; the center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. `O_{1}`), as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word's final wrapped form; a third, `\l__spintent_spcirc_radio_arg_tl`, is spoken directly on the , delimiter after the center, replacing its own reading rather than pairing with a group.

```

5285 \bool_new:N \l__spintent_spcirc_has_arg_bool
5286 \tl_new:N \l__spintent_spcirc_intent_tl
5287 \tl_new:N \l__spintent_spcirc_intent_arg_tl

```

```

5288 \tl_new:N \l__spintent_spcirc_connector_tl
5289 \tl_new:N \l__spintent_spcirc_connector_arg_tl
5290 \tl_new:N \l__spintent_spcirc_radio_arg_tl

```

(End of definition for `\l__spintent_spcirc_has_arg_bool` and others.)

`\__spintent_spcirc_sym:` Typesets `\bigodot` or the plain letter C, according to `print-sym`.

```

5291 \cs_new_protected:Nn \__spintent_spcirc_sym:
5292 {
5293   \str_if_eq:VnTF \l__spintent_spcirc_print_sym_str { odot }
5294   {
5295     \group_begin:
5296     \textstyle\bigodot
5297     \group_end:
5298   }
5299   { C }
5300 }

```

(End of definition for `\__spintent_spcirc_sym:`.)

`\__spintent_spcirc_word_and_article:` Sets `\l__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```

5301 \cs_new_protected:Nn \__spintent_spcirc_word_and_article:
5302 {
5303   \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
5304   {
5305     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5306     {
5307       \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia }
5308       \str_set:Nn \l__spintent_spcirc_articulo_str { la }
5309     }
5310     {
5311       \tl_set:Nn \l__spintent_spcirc_word_tl { círculo }
5312       \str_set:Nn \l__spintent_spcirc_articulo_str { el }
5313     }
5314   }
5315   {
5316     \tl_set:Nn \l__spintent_spcirc_word_tl { \l__spintent_spcirc_read_arg_tl }
5317     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5318     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
5319     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5320   }
5321 }

```

(End of definition for `\__spintent_spcirc_word_and_article:`.)

`\__spintent_spcirc_radio_print:` Dispatches on `\l__spintent_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```

5322 \cs_new_protected:Nn \__spintent_spcirc_radio_print:
5323 {
5324   \str_case:Vn \l__spintent_geo_luaset_circ_radio_type_str
5325   {
5326     { number } { \exp_args:Ne \spnum { \l__spintent_geo_luaset_circ_radio_raw_str } }
5327     { measure } { \exp_args:Ne \spunit { \l__spintent_geo_luaset_circ_radio_raw_str } }
5328     { label } { \l__spintent_geo_luaset_circ_radio_raw_str }
5329     { segment }
5330     {
5331       \str_if_eq:VnTF \l__spintent_spcirc_radio_sty_str { side }
5332       { \exp_args:Ne \spside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5333       { \exp_args:Ne \spmside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5334     }
5335   }
5336 }

```

(End of definition for `\__spintent_spcirc_radio_print:`.)

`\__spintent_spcirc_build_concept:` Builds two words: `\l__spintent_spcirc_intent_tl` (via `\__spintent_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\l__spintent_spcirc_connector_tl` (de-centro-en), paired with the (center) group — built even when `\l__spintent_spcirc_has_arg_bool` is `false`, though `\__spintent_spcirc_mathml:` never uses it in that case. Neither carries the center's own

text: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. $O_{\{1\}}$, as in the Euler circle) is never an issue.

```

5337 \cs_new_protected:Nn \__spintent_spcirc_build_concept:
5338 {
5339   \__spintent_spcirc_word_and_article:
5340   \tl_set:Nc \__spintent_spcirc_intent_tl { \__spintent_spcirc_word_tl }
5341   \str_if_empty:NF \__spintent_spcirc_sub_index_str
5342   {
5343     \tl_put_right:Nc \__spintent_spcirc_intent_tl
5344     {
5345       -
5346       \bool_if:NT \__spintent_spcirc_read_sub_bool { sub - }
5347       \__spintent_spcirc_sub_index_str
5348     }
5349   }
5350   \tl_set:Nn \__spintent_spcirc_connector_tl { de-centro-en }

```

`silent-cmd true` overrides `\__spintent_spcirc_intent_arg_tl`/`\__spintent_spcirc_radio_arg_tl` with `_-` and `\__spintent_spcirc_connector_arg_tl` with `_($pt)` (the center is always read), regardless of `read-arg`.

```

5351   \bool_if:NTF \__spintent_spcirc_silent_cmd_bool
5352   {
5353     \tl_set:Nn \__spintent_spcirc_intent_arg_tl { _- }
5354     \tl_set:Nc \__spintent_spcirc_connector_arg_tl { _($ pt ) }
5355     \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _- }
5356   }

```

With `concept true`, `\__spintent_spcirc_intent_arg_tl` is `\__spintent_spcirc_intent_tl:prefix` (bare — the group it wraps has nothing to reference, only a silenced symbol), prefixed with `prefix` when given; `\__spintent_spcirc_connector_arg_tl` is `\__spintent_spcirc_connector_tl:prefix($pt)`; and `\__spintent_spcirc_radio_arg_tl` — spoken directly on the , delimiter, replacing its own reading rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`, read as its own phrase right before `\__spintent_spcirc_radio_print:`'s independent reading follows as the next sibling.

```

5357   {
5358     \bool_if:NTF \__spintent_spcirc_concept_bool
5359     {
5360       \tl_set:Nc \__spintent_spcirc_intent_arg_tl
5361       {
5362         \tl_if_empty:NF \__spintent_spcirc_prefix_tl
5363         { \tl_use:N \__spintent_spcirc_prefix_tl - }
5364         \__spintent_spcirc_intent_tl :prefix
5365       }
5366       \tl_set:Nc \__spintent_spcirc_connector_arg_tl
5367       { \__spintent_spcirc_connector_tl :prefix( $ pt ) }
5368       \bool_if:NTF \__spintent_spcirc_diameter_bool
5369       { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5370       { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-radio- } }
5371     }

```

With `concept false` and `read-arg` empty, the same `_/_($pt)` silencing applies as `silent-cmd`; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio/y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\__spintent_spside_process:nn`).

```

5372   {
5373     \tl_if_empty:NTF \__spintent_spcirc_read_arg_tl
5374     {
5375       \tl_set:Nn \__spintent_spcirc_intent_arg_tl { _- }
5376       \tl_set:Nc \__spintent_spcirc_connector_arg_tl { _($ pt ) }
5377       \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _- }
5378     }
5379     {
5380       \tl_set:Nc \__spintent_spcirc_intent_arg_tl { \__spintent_spcirc_intent_tl }
5381       \tl_set:Nc \__spintent_spcirc_connector_arg_tl
5382       { \__spintent_spcirc_connector_tl :prefix( $ pt ) }
5383       \bool_if:NTF \__spintent_spcirc_diameter_bool
5384       { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5385       { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-radio- } }
5386     }
5387   }
5388 }

```

```
5389 }
```

(End of definition for `\__spintent_spcirc_build_concept:`)

`\__spintent_spcirc_process:n` Receives the `d()` argument in ‘#1’, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\__spintent_spcirc_exec:nn`) — `\__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\__spintent_luafun_geo_circ_parse_and_set:n` never called at all — there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\__spintent_spcirc_mathml:`, which calls `\__spintent_spcirc_build_concept:` itself.

```
5390 \cs_new_protected:Nn \__spintent_spcirc_process:n
5391 {
5392   \tl_if_empty:nTF {#1}
5393   {
5394     \bool_set_false:N \__spintent_spcirc_has_arg_bool
5395   }
5396   {
5397     \bool_set_true:N \__spintent_spcirc_has_arg_bool
5398     \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
5399     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5400     {
5401       \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
5402     }
5403   }
5404   \__spintent_spcirc_mathml:
5405 }
```

(End of definition for `\__spintent_spcirc_process:n`)

`\__spintent_spcirc_mathml:` Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius).

Otherwise, the `(center)` group (`\__spintent_spcirc_connector_arg_tl`) follows as a separate sibling, never nested inside the symbol’s own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>/<msub>` instead, the same problem already found for `\spangle`’s symbol and `\sparc`’s accent. When a radius is present, the `,` delimiter after the center carries `\__spintent_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\__spintent_spcirc_radio_print:` follows as an ordinary sibling right after — wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum/\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\__spintent_luafun_mrow_intent_arg_tl` *separately* — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` callback’s recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `luamml` only keeps a delimiter’s own math class (giving `<mo>` instead of `<mi>`, needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\__spintent_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{:silent}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`.

```
5406 \cs_new_protected:Nn \__spintent_spcirc_mathml:
5407 {
5408   \__spintent_invisible_sep: % (luamml issues #26)
5409   \__spintent_spcirc_build_concept:
5410   \MathMLintent { \__spintent_spcirc_intent_arg_tl }
5411   {
5412     \__spintent_inner_sep:
5413     \__spintent_luafun_mrow_intent_arg:nn
5414     { } { intent = ":silent" }
5415     { \__spintent_spcirc_sym: }
5416     \str_if_empty:NF \__spintent_spcirc_sub_index_str
5417     {
5418       \c_math_subscript_token
5419       {
5420         \__spintent_luafun_mrow_intent_arg:nn
5421         { } { intent = ":silent" }
5422         { \__spintent_spcirc_sub_index_str }
5423       }
5424     }
5425   }
5426 }
```

```

5424     }
5425   }
5426   \bool_if:NT \l__spintent_spcirc_has_arg_bool
5427   {
5428     \str_if_empty:NTF \l__spintent_geo_luaset_circ_radio_type_str
5429     {
5430       \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
5431       {
5432         \__spintent_inner_sep:
5433         \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5434         \__spintent_mathmlarg_point:nnV { pt } { spcirc } \l__spintent_geo_luaset_points_
5435         \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } )
5436       }
5437     }
5438     {
5439       \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
5440       {
5441         \__spintent_inner_sep:
5442         \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5443         \__spintent_mathmlarg_point:nnV { pt } { spcirc } \l__spintent_geo_luaset_points_
5444       }
5445       \MathMLintent { \l__spintent_spcirc_radio_arg_tl } { , }
5446       \__spintent_spcirc_radio_print:
5447       \MathMLintent { :silent } { \rparen }
5448     }
5449   }
5450 }

```

(End of definition for `\__spintent_spcirc_mathml:`.)

`\__spintent_spcirc_exec:nn` The function `\__spintent_spcirc_exec:nn` receives the optional keys in #1 and the argument in #2, delegating the rendering to `\__spintent_spcirc_print:n`.

```

5451 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
5452 {
5453   \keys_set:nn { spintent / spcirc } { #1 }
5454   \__spintent_spcirc_process:n { #2 }
5455 }

```

(End of definition for `\__spintent_spcirc_exec:nn`.)

`\spcirc` The public command `\spcirc` enforces math mode, opens a local group, and delegates execution to `\__spintent_spcirc_exec:nn`. It follows the same `d()`-argument pattern as `\spmcld/\spmcm` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the -NoValue- marker.

```

5456 \NewDocumentCommand \spcirc { 0{} d() }
5457 {
5458   \mode_if_math:TF
5459   {
5460     \group_begin:
5461     \attribute \l__spintent_scope_attr_int = 1
5462     \tl_if_novalue:NTF {#2}
5463     { \__spintent_spcirc_exec:nn { #1 } { } }
5464     { \__spintent_spcirc_exec:nn { #1 } { #2 } }
5465     \group_end:
5466   }
5467   { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
5468 }

```

(End of definition for `\spcirc`. This function is documented on page 28.)

11.47 The commands `\spAcirc` and `\spPcirc`

The user commands `\spAcirc` and `\spPcirc` typeset the area and the perimeter of a circle or a circumference. The argument follows `\spcirc`'s own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as `\spAfig/\spPfig`. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

11.47.1 Definition of variables for \spAcirc and \spPcirc

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```
\__spinttent_spAcirc_mode_str
\__spinttent_spPcirc_mode_str
\__spinttent_spAcirc_print_sym_str
\__spinttent_spPcirc_print_sym_str
\__spinttent_spAcirc_read_sty_str
\__spinttent_spPcirc_read_sty_str
\__spinttent_spAcirc_radio_sty_str
\__spinttent_spPcirc_radio_sty_str
\__spinttent_spAcirc_sub_index_str
\__spinttent_spPcirc_sub_index_str
\__spinttent_spAcirc_prefix_tl
\__spinttent_spPcirc_prefix_tl
\__spinttent_spAcirc_read_arg_tl
\__spinttent_spPcirc_read_arg_tl
\__spinttent_geo_circfig_mode_str
```

(End of definition for __spinttent_spAcirc_mode_str and others.)

The variable `\__spinttent_geo_circfig_mode_str` stores whether the argument was resolved as a real circle (`circle`) or as the bare number/letter shortcut (`bare`).

```
\str_new:N \__spinttent_geo_circfig_mode_str
```

(End of definition for __spinttent_geo_circfig_mode_str.)

11.47.2 Definition of error messages for \spAcirc and \spPcirc

```
spAcirc-invalid-arg
spPcirc-invalid-arg
```

The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```
\clist_map_inline:nn { Acirc , Pcirc }
{
  \msg_new:nnnn { spinttent } { sp #1 -invalid-arg }
  {
    Invalid~argument~in~##1:~'##2'~\msg_line_context:.
  }
  {
    See~the~documentation~of~##1~for~the~accepted~argument.
  }
}
```

(End of definition for `spAcirc-invalid-arg` and `spPcirc-invalid-arg`.)

11.47.3 Definition of keys for \spAcirc and \spPcirc

```
read-cmd
prefix
print-sym
read-sty
concept
sub-index
read-sub
read-arg
silent-cmd
unknown
```

We define the shared (*keys*) `read-cmd`, `prefix`, `print-sym`, `read-sty`, `concept`, `sub-index`, `read-sub`, `read-arg` and `silent-cmd`.

```
\keys_define:nn { spinttent / sp #1 }
{
  read-cmd .choices:nn = { school , mathcat }
  {
    \int_case:nn { \l_keys_choice_int }
    {
      {1} { \str_set:cn { \__spinttent_sp #1 _mode_str } { school } }
      {2} { \str_set:cn { \__spinttent_sp #1 _mode_str } { mathcat } }
    }
  },
  read-cmd .initial:n = school,
  read-cmd .value_required:n = true,
  read-cmd / unknown .code:n =
    \msg_error:nneee { spinttent } { unknown-choice }
    { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
  prefix .code:n =
    { \__spinttent_sanitize_affix:cn { \__spinttent_sp #1 _prefix_tl } {##1} },
  prefix .initial:n = ,
  prefix .value_required:n = true,
  print-sym .choices:nn = { usual , odot }
  {
    \int_case:nn { \l_keys_choice_int }
    {
      {1} { \str_set:cn { \__spinttent_sp #1 _print_sym_str } { usual } }
      {2} { \str_set:cn { \__spinttent_sp #1 _print_sym_str } { odot } }
    }
  },
  print-sym .initial:n = odot,
  print-sym .value_required:n = true,
  print-sym / unknown .code:n =
    \msg_error:nneee { spinttent } { unknown-choice }
```

```

5520         { print-sym } { usual,~odot } { \exp_not:n {##1} },
5521 read-sty                .choices:nn = { circun , circle }
5522     {
5523         \int_case:nn { \l_keys_choice_int }
5524         {
5525             {1} { \str_set:cn { \l__spintent_sp #1 _read_sty_str } { circun } }
5526             {2} { \str_set:cn { \l__spintent_sp #1 _read_sty_str } { circle } }
5527         }
5528     },
5529 read-sty                .initial:n = circun,
5530 read-sty                .value_required:n = true,
5531 concept                .bool_set:c = { \l__spintent_sp #1 _concept_bool },
5532 concept                .initial:n = true,
5533 concept                .value_required:n = true,
5534 sub-index              .code:n =
5535     { \str_set:cn { \l__spintent_sp #1 _sub_index_str } {##1} },
5536 sub-index              .initial:n = ,
5537 sub-index              .value_required:n = true,
5538 read-sub               .bool_set:c = { \l__spintent_sp #1 _read_sub_bool },
5539 read-sub               .initial:n = false,
5540 read-sub               .value_required:n = true,
5541 read-arg               .code:n =
5542     { \l__spintent_sanitize_affix:cn { \l__spintent_sp #1 _read_arg_tl } {##1} },
5543 read-arg               .initial:n = ,
5544 read-arg               .value_required:n = true,
5545 silent-cmd             .bool_set:c = { \l__spintent_sp #1 _silent_cmd_bool },
5546 silent-cmd             .initial:n = false,
5547 silent-cmd             .value_required:n = true,
5548 unknown                .code:n =
5549     {
5550         \str_set:Nn \l__spintent_command_name_str { sp #1 }
5551         \l__spintent_unknown_keys_cmd:n {##1}
5552     },
5553 }
5554 }

```

(End of definition for read-cmd and others.)

`\l__spintent_spcircfig_short_bool` `\l__spintent_spcircfig_op_tl` `\l__spintent_spcircfig_intent_tl` `\l__spintent_spcircfig_intent_arg_tl`

`\l__spintent_spcircfig_short_bool` records whether the argument resolved as the short number/letter form (via `\l__spintent_luafun_geo_afig_parse_and_set:n/\l__spintent_luafun_geo_pfig_parse_and_set:n`) the same entry point `\spAfig/\spPfig` use) rather than a real circle. `\l__spintent_spcircfig_op_tl` holds área or perímetro. `\l__spintent_spcircfig_intent_tl` holds the full spoken phrase: `\l__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg`), an optional `sub-index` suffix, and, when a center is given, `-con-centro-en` — the center itself is never concatenated into it: it is read separately, by reference, through `\l__spintent_mathmlarg_point:nnn`, so a subscripted or primed center (e.g. $O_{\{1\}}$, as in the Euler circle) is never an issue.

```

5555 \bool_new:N \l__spintent_spcircfig_short_bool
5556 \tl_new:N \l__spintent_spcircfig_op_tl
5557 \tl_new:N \l__spintent_spcircfig_del_word_tl
5558 \tl_new:N \l__spintent_spcircfig_intent_tl
5559 \tl_new:N \l__spintent_spcircfig_intent_arg_tl

```

(End of definition for `\l__spintent_spcircfig_short_bool` and others.)

`\l__spintent_spcircfig_sym:n` Typesets `\bigodot` or the plain letter C, according to `print-sym`. Receives the short command name in ‘#1’.

```

5560 \cs_new_protected:Nn \l__spintent_spcircfig_sym:n
5561 {
5562     \str_if_eq:vnTF { \l__spintent_sp #1 _print_sym_str } { odot }
5563     { \bigodot }
5564     { C }
5565 }

```

(End of definition for `\l__spintent_spcircfig_sym:n`.)

`\l__spintent_spcircfig_word_and_article:n` Sets `\l__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word. Receives the short command name in ‘#1’.

```

5566 \cs_new_protected:Nn \l__spintent_spcircfig_word_and_article:n
5567 {
5568     \str_if_eq:vnTF { \l__spintent_sp #1 _read_sty_str } { circun }
5569     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }

```

```

5570     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5571   \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
5572   {
5573     \str_if_eq:vnTF { l__spintent_sp #1 _read_sty_str } { circun }
5574     { \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia } }
5575     { \tl_set:Nn \l__spintent_spcirc_word_tl { círculo } }
5576   }
5577   {
5578     \tl_set:Ne \l__spintent_spcirc_word_tl
5579     {
5580       \tl_use:c { l__spintent_sp #1 _read_arg_tl }
5581     }
5582   }
5583 }

```

(End of definition for `\__spintent_spcircfig_word_and_article:n`.)

`\__spintent_spcircfig_del_word:n` Sets `\__spintent_spcircfig_del_word_tl` to `\__spintent_spcircfig_word_and_article:n`'s word, prefixed with `del-` or `de-la-` — contracted, not `de-el-`, since a screen reader does not contract that on its own. Receives the short command name in `'#1'`.

```

5584 \cs_new_protected:Nn \__spintent_spcircfig_del_word:n
5585 {
5586   \__spintent_spcircfig_word_and_article:n {#1}
5587   \str_if_eq:VnTF \l__spintent_spcirc_articulo_str { el }
5588   {
5589     \tl_set:Ne \l__spintent_spcircfig_del_word_tl { del- \l__spintent_spcirc_word_tl }
5590   }
5591   {
5592     \tl_set:Ne \l__spintent_spcircfig_del_word_tl { de-la- \l__spintent_spcirc_word_tl }
5593   }
5594 }

```

(End of definition for `\__spintent_spcircfig_del_word:n`.)

`\__spintent_spcircfig_build_short:n` Handles the short number/letter case: builds `\l__spintent_spcircfig_intent_arg_tl` as `\l__spintent_spcircfig_intent_arg_tl` (*ber word*), wrapped in bare `:prefix` (no reference — there is nothing to point to), or `_-` when `silent-cmd` is `true` or `concept` is `false`. Receives the short command name in `'#1'`.

```

5595 \cs_new_protected:Nn \__spintent_spcircfig_build_short:n
5596 {
5597   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
5598   {
5599     \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
5600   }
5601   {
5602     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5603     {
5604       \__spintent_spcircfig_del_word:n {#1}
5605       \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5606       {
5607         \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl -
5608         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5609         \l__spintent_geo_luaset_sub_spoken_str :prefix
5610       }
5611     }
5612     {
5613       \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
5614     }
5615   }
5616 }

```

(End of definition for `\__spintent_spcircfig_build_short:n`.)

`\__spintent_spcircfig_finish_intent:nn` Appends an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` to `\l__spintent_spcircfig_intent_arg_tl` — shared by both branches of `\__spintent_spcircfig_build_concept:nn`, after either one has set the word itself. Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5617 \cs_new_protected:Nn \__spintent_spcircfig_finish_intent:nn
5618 {
5619   \str_if_empty:cF { l__spintent_sp #1 _sub_index_str }
5620   {
5621     \tl_put_right:Ne \l__spintent_spcircfig_intent_tl
5622     {

```

```

5623         -
5624         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5625         \str_use:c { l__spintent_sp #1 _sub_index_str }
5626     }
5627 }
5628 \tl_if_empty:nF {#2}
5629 {
5630     \tl_put_right:Nn \l__spintent_spcircfig_intent_tl { -con-centro-en }
5631 }
5632 }

```

(End of definition for `\l__spintent_spcircfig_finish_intent:nn`.)

`\l__spintent_spcircfig_build_concept:nn`

Builds `\l__spintent_spcircfig_intent_arg_tl`. `_-` when `silent-cmd` is `true`, or when `false` and `concept` is `false` with `read-arg` empty (`_( $pt )` instead when `#2` is non-empty — the center is still read either way). Otherwise `\l__spintent_spcircfig_intent_tl:prefix` — with `( $pt )` appended when `#2` is non-empty — prefixed with `prefix` when given, where `\l__spintent_spcircfig_intent_tl` is: with `concept` `true`, `\l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl` (via `\l__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg` the same way); with `false` and `read-arg` given, `read-arg` verbatim instead, with `\l__spintent_spcircfig_op_tl` dropped entirely — `área/ perímetro` is itself the concept `concept` toggles, not the `-con-centro-en` that follows, which always stays (it identifies which figure, not the concept). No automatic article either — the word (and any article it needs) is the user's to write, same convention as `\l__spintent_spcirc_process:n`. Either way, an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` follow. Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5633 \cs_new_protected:Nn \l__spintent_spcircfig_build_concept:nn
5634 {
5635     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
5636     {
5637         \tl_if_empty:nTF {#2}
5638         { \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- } }
5639         { \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5640     }
5641     {
5642         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5643         {
5644             \l__spintent_spcircfig_del_word:n {#1}
5645             \tl_set:Ne \l__spintent_spcircfig_intent_tl
5646             {
5647                 \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl
5648             }
5649             \l__spintent_spcircfig_finish_intent:nn {#1} {#2}
5650             \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5651             {
5652                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5653                 {
5654                     \tl_use:c { l__spintent_sp #1 _prefix_tl } -
5655                 }
5656                 \l__spintent_spcircfig_intent_tl :prefix
5657                 \tl_if_empty:nF {#2} { ( $ pt ) }
5658             }
5659         }
5660         {
5661             \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
5662             {
5663                 \tl_if_empty:nTF {#2}
5664                 { \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- } }
5665                 { \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5666             }
5667             {
5668                 \tl_set:Ne \l__spintent_spcircfig_intent_tl
5669                 { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
5670                 \l__spintent_spcircfig_finish_intent:nn {#1} {#2}
5671                 \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
5672                 {
5673                     \l__spintent_spcircfig_intent_tl :prefix
5674                     \tl_if_empty:nF {#2} { ( $ pt ) }
5675                 }
5676             }
5677         }
5678     }

```

```
5679 }
```

(End of definition for `\__spintent_spcircfig_build_concept:nn`.)

```
\__spintent_spcircfig_process:nn
```

Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’. Sets `\__spintent_spcircfig_op_tl` to `área/perímetro`. With `#2` empty, `\__spintent_spcircfig_short_bool` is set `false` and nothing else is parsed. With a comma in `#2`, reports `sp#1-invalid-arg` immediately — `\@@_luafun_geo_circ_parse_and_set:` (shared with `\spcirc`) still accepts a second, comma-separated part as a generic `label`-type radius, matching almost any text, so it would never itself report an error for a comma-containing argument here, where a comma is never meaningful at all. Without a comma, calls `\__spintent_luafun_geo_circ_parse_and_set:n`; on error, retries through `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_parse_and_set:n` (the same entry point `\spAfig/\spPfig` use) for the short number/letter form, rejecting a table-letter match (not meaningful for a circle) back to an error. Reports `sp#1-invalid-arg` on failure either way, then delegates to `\__spintent_spcircfig_build_short:n/\__spintent_spcircfig_build_concept:nn` and `\__spintent_spcircfig_mathml:nn`.

- Unlike `\spcirc`, there is no radius/diameter here at all: a formula’s left-hand side (`A_0`) names the figure, it does not carry the values needed to compute it — those belong on the right-hand side, elsewhere in the document.

```
5680 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
5681 {
5682   \str_if_eq:eeTF {#1} { Acirc }
5683   { \tl_set:Nn \__spintent_spcircfig_op_tl { área } }
5684   { \tl_set:Nn \__spintent_spcircfig_op_tl { perímetro } }
5685   \tl_if_empty:nTF {#2}
5686   {
5687     \bool_set_false:N \__spintent_spcircfig_short_bool
5688   }
5689   {
5690     \bool_set_false:N \__spintent_spcircfig_short_bool
5691     \tl_if_in:nnTF {#2} { , }
5692     {
5693       \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5694       { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5695     }
5696     {
5697       \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
5698       \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5699       {
5700         \str_if_eq:eeTF {#1} { Acirc }
5701         { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
5702         { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
5703         \str_if_eq:VnF \__spintent_geo_luaset_error_str { true }
5704         {
5705           \str_if_empty:NTF \__spintent_geo_luaset_letra_word_str
5706           { \bool_set_true:N \__spintent_spcircfig_short_bool }
5707           { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
5708         }
5709       }
5710       \bool_if:NF \__spintent_spcircfig_short_bool
5711       {
5712         \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5713         {
5714           \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5715           { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5716         }
5717       }
5718     }
5719   }
5720   \bool_if:NTF \__spintent_spcircfig_short_bool
5721   { \__spintent_spcircfig_build_short:n {#1} }
5722   { \__spintent_spcircfig_build_concept:nn {#1} {#2} }
5723   \__spintent_spcircfig_mathml:nn { #1 } { #2 }
5724 }
```

(End of definition for `\__spintent_spcircfig_process:nn`.)

```
\__spintent_spcircfig_mathml:nn
```

Renders `\__spintent_spcircfig_intent_arg_tl`, already built by `\__spintent_spcircfig_process:nn`. Three cases: the short case prints the symbol with the short value as its own subscript; an empty `#2` prints the symbol alone; a real center prints the symbol (plus optional `sub-index` subscript) and the center between parentheses. In every case a single native `\MathMLintent` wraps `\__spintent_mathcal:n{⟨A|P⟩}` and the whole subscript together — splitting the symbol and the center into separate `\MathMLintent` siblings

inside the same subscript was tried and reads as `<msub>`'s own native structural narration instead. The center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt` (same convention as `\spcirc`). Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5725 \cs_new_protected:Nn \__spintent_spcircfig_mathml:nn
5726 {
5727   \__spintent_invisible_sep: % (luamml issues #26)
5728   \MathMLintent { \__spintent_spcircfig_intent_arg_tl }
5729   {
5730     \__spintent_inner_sep:
5731     \__spintent_luafun_mrow_intent_arg:nn
5732     { } { intent = ":silent" }
5733     {
5734       \str_if_eq:eeTF {#1} { Acirc } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
5735     }
5736     \bool_if:NTF \__spintent_spcircfig_short_bool
5737     {
5738       \c_math_subscript_token
5739       {
5740         \__spintent_spcircfig_sym:n {#1}
5741         \c_math_subscript_token { \__spintent_geo_luaset_print_tl }
5742       }
5743     }
5744     {
5745       \c_math_subscript_token
5746       {
5747         \MathMLintent { :silent } { \__spintent_spcircfig_sym:n {#1} }
5748         \str_if_empty:cF { \__spintent_sp #1 _sub_index_str }
5749         {
5750           \c_math_subscript_token
5751           {
5752             \MathMLintent { :silent }
5753             { \str_use:c { \__spintent_sp #1 _sub_index_str } }
5754           }
5755         }
5756         \tl_if_empty:nF {#2}
5757         {
5758           \MathMLintent { :silent } { \lparen }
5759           \__spintent_mathmlarg_point:nnV { pt } { sp #1 } \__spintent_geo_luaset_poin
5760           \MathMLintent { :silent } { \rparen }
5761         }
5762       }
5763     }
5764   }
5765 }

```

(End of definition for `\__spintent_spcircfig_mathml:nn`.)

`\__spintent_spcircfig_exec:nnn`

The function `\__spintent_spcircfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintent_spcircfig_process:nn`.

```

5766 \cs_new_protected:Nn \__spintent_spcircfig_exec:nnn
5767 {
5768   \keys_set:nn { spintent / sp #1 } { #2 }
5769   \__spintent_spcircfig_process:nn { #1 } { #3 }
5770 }

```

(End of definition for `\__spintent_spcircfig_exec:nnn`.)

`\spAcirc`
`\spPcirc`

Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintent_spcircfig_exec:nnn`.

```

5771 \NewDocumentCommand \spAcirc { 0{} d() }
5772 {
5773   \mode_if_math:TF
5774   {
5775     \group_begin:
5776     \attribute \l__spintent_scope_attr_int = 1
5777     \tl_if_novalue:nTF {#2}
5778     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { } }
5779     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
5780     \group_end:
5781   }
5782   { \msg_error:nnn { spintent } { need-math-mode } { \spAcirc } }

```

```

5783   }
5784   \NewDocumentCommand \spPcirc { 0{} d() }
5785   {
5786     \mode_if_math:TF
5787     {
5788       \group_begin:
5789       \attribute \l__spintent_scope_attr_int = 1
5790       \tl_if_novalue:nTF {#2}
5791       { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { } }
5792       { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
5793       \group_end:
5794     }
5795     { \msg_error:nnn { spintent } { need-math-mode } { \spPcirc } }
5796   }

```

(End of definition for \spAcirc and \spPcirc. These functions are documented on page 30.)

11.48 Finish package

Finish package implementation.

```

5797 \file_input_stop:
5798 \endpackage

```


Keys for \spmcmm provide by spintent :		Keys for \spPcirc provide by spintent :	
concept	81	concept	150
prefix	81	prefix	150
read-cmd	81	print-sym	150, 151
result	81	radio-sty	150
silent-cmd	81	read-arg	150
upper-case	81	read-cmd	150
Keys for \spmoney provide by spintent :		read-sty	150
cifras-sep	57	read-sub	150
currency	57	silent-cmd	150
dolar-math	57	sub-index	150
sub-units	57	Keys for \spPfig provide by spintent :	
Keys for \spmQ provide by spintent :		prefix	132
join-word	92-94	print-sym	132, 133
print-dfrac	93-96	read-arg	132-134
read-parens	93, 94	read-cmd	132
use-spnZ	93, 96	read-sub	133
wrap-parens	92-94	Keys for \spPoint provide by spintent :	
Keys for \spmside provide by spintent :		concept	103
concept	116	prefix	102, 103
prefix	116	read-cmd	102, 103
print-m	116	silent-cmd	103
print-sym	140	Keys for \spPoly provide by spintent :	
read-arg	116	concept	129
read-cmd	116, 119	prefix	128, 129
read-sty	116	print-sym	129
silent-cmd	116, 119	read-cmd	128, 129
Keys for \spmsym provide by spintent :		silent-cmd	129
not-print	76	Keys for \sprad provide by spintent :	
prefix	76	concept	140
read-sym	76	read-arg	140
set-sym	76	read-cmd	140
suffix	76	read-sub	140
Keys for \spmul provide by spintent :		silent-cmd	140
cifras-sep	79	sub-index	140
notation-sym	79	upper-case	140
read-parens	79	Keys for \sprayo provide by spintent :	
read-period-sep	79	concept	113
read-sign	79	prefix	113
read-sym	79	read-cmd	113, 114
set-sym	79	read-sty	113
wrap-parens	79	silent-cmd	113, 114
Keys for \spnD provide by spintent :		Keys for \sprazon provide by spintent :	
concept	85	inline	88
prefix	85	silent-cmd	88
result-num	85	use-dfrac	88
result	85	Keys for \spread provide by spintent :	
silent-cmd	85	read-arg	74
Keys for \spnM provide by spintent :		silent-cmd	74
concept	85	Keys for \sprecta provide by spintent :	
prefix	85	prefix	109
result-num	85	read-cmd	109
result	85	silent-cmd	109
silent-cmd	85	Keys for \spshort provide by spintent :	
Keys for \spnum provide by spintent :		read-arg	63
cifras-sep	45-47	small-caps	63
notation-sym	46	Keys for \spside provide by spintent :	
read-period-sep	46, 48	concept	116
read-sign	46	prefix	116
Keys for \spnZ provide by spintent :		read-arg	116
cifras-sep	91	read-cmd	116, 119
read-parens	91	read-sty	116
read-sign	91	silent-cmd	116, 119
wrap-parens	91	Keys for \spsiglo provide by spintent :	
		print-era	66, 67

Keys for \spunit provide by [spintent](#):

cifras-sep	52
notation-sym	52
print-cdot	52
read-cdot	52
read-period-sep	52
read-slash	52

Keys for \spusep provide by [spintent](#):

silent	71
size	71

Keys for \spvec provide by [spintent](#):

autofit	105
concept	105
prefix	105
read-cmd	105
silent-cmd	105

M

mathcal-fallback	80
\MathMLintent	124
mc-invalid-arguments	2023
md-invalid-argument	2236

Lua module provide by [spintent](#):

spintent.lua	38, 39, 41, 44-46, 51, 55-58, 62, 63, 65, 67, 68, 70, 81, 84, 89, 92, 97, 102, 140
--------------	--

N

not-print	1794
notation-sym	284, 569, 1868, 1939

P

Packages:

expl3	44, 46
lua-unicode-math	38
luamml	41, 92, 95
tagpdf	62, 67
unicode-math	38, 72
prefix	1724, 1794, 2030, 2210, 3044, 3165, 3407, 3627, 3764, 3971, 4256, 4409, 4593, 5199, 5489
print-dfrac	2582
print-era	1239
print-m	3764, 3971
print-sym	4409, 4593, 4981, 5199, 5489

R

radio-sty	5199
read-arg	1109, 1648, 3764, 4593, 4981, 5199, 5489
read-cdot	569
read-cmd ..	2030, 3044, 3165, 3407, 3627, 3764, 3971, 4256, 4409, 4593, 4981, 5199, 5489
read-parens	1868, 1939, 2504, 2582
read-period-sep	284, 569, 1868, 1939
read-sign	284, 1868, 1939, 2504
read-slash	569
read-sty	3627, 3764, 5199, 5489
read-sub	4593, 4981, 5199, 5489
read-sym	1724, 1794, 1868, 1939
recto	3971
result	2030, 2210
result-num	2210
right-angle-sqr-fallback	80
\rightangle	121

S

seq commands:

\seq_set_from_clist:Nn	62
------------------------------	----

set-sym	1724, 1794, 1868, 1939
\setspintent	3, 42, 191
silent	1509
silent-cmd	1648, 2030, 2210, 2366, 2796, 3044, 3165, 3407, 3627, 3764, 3971, 4256, 4409, 4593, 4981, 5199, 5489
size	1509
small-caps	1109
\spAcirc	30, 149, 5771
spAcirc-invalid-arg	5480
\spAfig	25, 132, 4947
spAfig-invalid-arg	4555
spAfig-invalid-points	4555
spAfig-read-arg-key	4555
spAfig-sym-cardinality-conflict	4555
spAfig-sym-letra-conflict	4555
\spang	23, 89, 2494
\spangle	22, 120, 4219
spangle-invalid-arg	3961
\sparc	23, 125, 4378
sparc-invalid-points	4246
\spcirc	28, 143, 5456
spcirc-invalid-arg	5192
\spcoord	17, 97, 2933
spcoord-invalid-arg	2792
\spdate	11, 70, 1485
\spdiam	28, 140, 5161
\spdiv	13, 78, 1929
spdiv-empty-argument	1888
\spdsym	12, 75, 1771

spintent internal commands:

_spintent_collapse_hyphens:N	128
\l_spintent_command_name_str	203
\l_spintent_geo_circfig_mode_str	5479
\l_spintent_geo_final_print_tl	4547
\l_spintent_geo_luaset_angulo_case_-str	2765
\l_spintent_geo_luaset_circ_radio_raw_-str	5181
\l_spintent_geo_luaset_circ_radio_type_-str	5181
\l_spintent_geo_luaset_concept_str ...	2765
\l_spintent_geo_luaset_error_str	2765
\l_spintent_geo_luaset_intent_str ...	2765
\l_spintent_geo_luaset_letra_str	2765
\l_spintent_geo_luaset_letra_word_str	2765
\l_spintent_geo_luaset_nombre_es_num_-str	2765
\l_spintent_geo_luaset_points_-clist _{uuuu} \l_spintent_geo_luaset_is_-name_str	2765
\l_spintent_geo_luaset_poly_sym_str ..	2765
\l_spintent_geo_luaset_print_tl	2765
\l_spintent_geo_luaset_sub_spoken_str	2765
\l_spintent_geo_luaset_sub_str	2765
\l_spintent_geo_luaset_tipo_str	2765
\l_spintent_geo_print_sym_str	4663
_spintent_inner_sep:	120
_spintent_invisible_sep:	116
\c_spintent_invisible_sep_int	124
\c_spintent_invisible_sep_tl	94
_spintent_luafun_clean_split_and_set:n	76, 241

<code>__spintent_luafun_mrow_intent_arg:nn</code> .	116	<code>__spintent_setspintent_parse_args:nn</code> .	174
<code>__spintent_luafun_set_debug_ghost:n</code> ...	71	<code>__spintent_setspintent_set_keys:nn</code> ...	163
<code>__spintent_luafun_spang_parse:n</code>	90	<code>\l__spintent_smparc_prefix_tl</code>	4241
<code>__spintent_luafun_sptime_parse:n</code>	70	<code>__spintent_sp_rad_diam_exec:nn</code>	5156
<code>__spintent_luafun_spmoney_lookup_data:n</code>	76	<code>\l__spintent_sp_rad_diam_letra_tl</code>	4979
<code>__spintent_luafun_spmQ_scale_int:nn</code> ...	76	<code>__spintent_sp_rad_diam_print:n</code>	5028
<code>__spintent_luafun_sptime_parse:n</code>	69	<code>\l__spintent_sp_rad_diam_word_tl</code>	4979
<code>__spintent_luafun_spunit_lookup_alias:n</code>	76	<code>\l__spintent_spAcirc_mode_str</code>	5469
<code>\l__spintent_luaset_arg_above_tl</code>	260	<code>\l__spintent_spAcirc_prefix_tl</code>	5469
<code>\l__spintent_luaset_arg_below_tl</code>	260	<code>\l__spintent_spAcirc_print_sym_str</code> ...	5469
<code>\l__spintent_luaset_cal_fam_ok_str</code>	67	<code>\l__spintent_spAcirc_radio_sty_str</code> ...	5469
<code>\l__spintent_luaset_dec_is_all_zeros_-</code> <code>str</code>	253	<code>\l__spintent_spAcirc_read_arg_tl</code>	5469
<code>\l__spintent_luaset_dec_sep_tl</code>	241	<code>\l__spintent_spAcirc_read_sty_str</code>	5469
<code>\l__spintent_luaset_decimal_period_str</code>	253	<code>\l__spintent_spAcirc_sub_index_str</code> ...	5469
<code>\l__spintent_luaset_denom_has_number_-</code> <code>str</code>	260	<code>\l__spintent_sPAfig_mode_str</code>	4548
<code>\l__spintent_luaset_exponent_tl</code>	253	<code>\l__spintent_sPAfig_prefix_tl</code>	4548
<code>\l__spintent_luaset_format_part_dec_str</code>	248	<code>\l__spintent_sPAfig_print_sym_str</code>	4548
<code>\l__spintent_luaset_format_part_int_str</code>	248	<code>\l__spintent_sPAfig_read_arg_tl</code>	4548
<code>\l__spintent_luaset_has_exponent_str</code> ..	253	<code>\l__spintent_spang_luaset_error_str</code> ...	2450
<code>\l__spintent_luaset_has_integer_str</code> ..	95, 96, 241	<code>\l__spintent_spang_luaset_grado_str</code> ...	2450
<code>\l__spintent_luaset_has_natural_str</code> ...	241	<code>\l__spintent_spang_luaset_minuto_str</code> ..	2450
<code>\l__spintent_luaset_has_units_str</code>	260	<code>\l__spintent_spang_luaset_segundo_str</code> .	2450
<code>\l__spintent_luaset_millions_str</code>	253	<code>__spintent_spang_process:n</code>	2459
<code>\l__spintent_luaset_not_decimal_not_-</code> <code>period_str</code>	253	<code>__spintent_spangle_exec:nnn</code>	4214
<code>\l__spintent_luaset_not_decimal_period_-</code> <code>str</code>	253	<code>\l__spintent_spangle_intent_arg_tl</code> ...	4030
<code>\l__spintent_luaset_only_part_dec_str</code> .	248	<code>\l__spintent_spangle_intent_tl</code>	4030
<code>\l__spintent_luaset_only_part_int_str</code> .	248	<code>__spintent_spangle_mathml:n</code>	4072
<code>\l__spintent_luaset_only_part_period_-</code> <code>str</code>	248	<code>\l__spintent_spangle_mode_str</code>	3956
<code>\l__spintent_luaset_part_dec_tl</code>	241	<code>\l__spintent_spangle_prefix_tl</code>	3956
<code>\l__spintent_luaset_part_int_tl</code>	95, 241	<code>\l__spintent_spangle_print_m_now_bool</code> .	4030
<code>\l__spintent_luaset_part_period_tl</code> ...	241	<code>__spintent_spangle_process:nn</code>	4037
<code>\l__spintent_luaset_scope_attr_str</code>	67	<code>\l__spintent_spangle_pt_idx_int</code>	4030
<code>\l__spintent_luaset_sign_tl</code>	95, 241	<code>\l__spintent_spangle_pt_seq</code>	4030
<code>\l__spintent_luaset_status_str</code>	260	<code>\l__spintent_spangle_refs_tl</code>	4030
<code>\l__spintent_luaset_sym_fam_ok_str</code>	67	<code>__spintent_spangle_sym:n</code>	4011
<code>__spintent_mathcal:n</code>	94	<code>__spintent_sparc_exec:nnn</code>	4373
<code>__spintent_mathmlarg_point:nnn</code>	2948	<code>\l__spintent_sparc_intent_arg_tl</code>	4300
<code>\l__spintent_mathmlarg_point_kv_tl</code> ...	2944	<code>\l__spintent_sparc_intent_tl</code>	4300
<code>\l__spintent_mathmlarg_point_mode_str</code> .	2944	<code>__spintent_sparc_mathml:n</code>	4318
<code>\l__spintent_mathmlarg_point_ref_tl</code> ...	2944	<code>\l__spintent_sparc_mode_str</code>	4241
<code>\l__spintent_mathmlarg_point_silent_-</code> <code>bool</code>	2944	<code>\l__spintent_sparc_prefix_tl</code>	4241
<code>\l__spintent_mc_args_seq</code>	2010	<code>__spintent_sparc_print_arc:n</code>	4294
<code>\l__spintent_mc_luaset_all_numeric_str</code>	2010	<code>__spintent_sparc_process:nn</code>	4305
<code>\l__spintent_mc_luaset_types_str</code>	2010	<code>\l__spintent_sparc_pt_idx_int</code>	4300
<code>__spintent_mc_process_args:nn</code>	2071	<code>\l__spintent_sparc_pt_seq</code>	4300
<code>\l__spintent_mc_raw_args_seq</code>	2010	<code>\l__spintent_sparc_refs_tl</code>	4300
<code>\l__spintent_mc_types_seq_{uuu}\l__spintent_-</code> <code>spmcd_print_result_bool</code>	2010	<code>__spintent_sparc_wide_arc:n</code>	4290
<code>\l__spintent_md_out_seq</code>	2201	<code>\l__spintent_spcirc_articulo_str</code>	5183
<code>__spintent_print_spunit_hierarchy:</code> ...	670	<code>__spintent_spcirc_build_concept:</code>	5337
<code>__spintent_right_angle_sqr:</code>	107	<code>\l__spintent_spcirc_connector_arg_tl</code> ..	5285
<code>__spintent_sanitize_affix:Nn</code>	136	<code>\l__spintent_spcirc_connector_tl</code>	5285
<code>\l__spintent_saved_luamml_flag_int</code>	95, 2563	<code>__spintent_spcirc_exec:nn</code>	5451
<code>\l__spintent_scope_attr_int</code>	74	<code>\l__spintent_spcirc_has_arg_bool</code>	5285
<code>\l__spintent_setspintent_input_arg_cmd_-</code> <code>tl</code>	154	<code>\l__spintent_spcirc_intent_arg_tl</code>	5285
		<code>\l__spintent_spcirc_intent_tl</code>	5285
		<code>__spintent_spcirc_mathml:</code>	5406
		<code>\l__spintent_spcirc_mode_str</code>	5183
		<code>\l__spintent_spcirc_prefix_tl</code>	5183
		<code>\l__spintent_spcirc_print_sym_str</code>	5183
		<code>__spintent_spcirc_process:n</code>	5390
		<code>\l__spintent_spcirc_radio_arg_tl</code>	5285

<code>__spintent_spcirc_radio_print:</code>	5322	<code>__spintent_spcirc_build_refs:</code>	4807
<code>\l__spintent_spcirc_radio_sty_str</code>	5183	<code>__spintent_spcirc_build_simple:n</code>	4696
<code>\l__spintent_spcirc_read_sty_str</code>	5183	<code>__spintent_spcirc_exec:nnn</code>	4942
<code>\l__spintent_spcirc_sub_index_str</code>	5183	<code>\l__spintent_spcirc_intent_arg_tl</code>	4663
<code>__spintent_spcirc_sym:</code>	5291	<code>\l__spintent_spcirc_intent_tl</code>	4663
<code>__spintent_spcirc_word_and_article:</code>	5301	<code>__spintent_spcirc_mathml:n</code>	4902
<code>\l__spintent_spcirc_word_tl</code>	5191	<code>\l__spintent_spcirc_op_tl</code>	4663
<code>__spintent_spcircfig_build_concept:nn</code>	5633	<code>__spintent_spcirc_points_loop:n</code>	4890
<code>__spintent_spcircfig_build_short:n</code>	5595	<code>__spintent_spcirc_process:nn</code>	4672
<code>__spintent_spcircfig_del_word:n</code>	5584	<code>\l__spintent_spcirc_pt_idx_int</code>	4663
<code>__spintent_spcircfig_exec:nnn</code>	5766	<code>\l__spintent_spcirc_pt_seq</code>	4663
<code>__spintent_spcircfig_finish_intent:nn</code>	5617	<code>\l__spintent_spcirc_refs_tl</code>	4663
<code>\l__spintent_spcircfig_intent_arg_tl</code>	5555	<code>__spintent_spcirc_sym:</code>	4655
<code>\l__spintent_spcircfig_intent_tl</code>	5555	<code>\l__spintent_spcirc_symbol_tl</code>	4663
<code>__spintent_spcircfig_mathml:nn</code>	5725	<code>\l__spintent_spcirc_angle_mode_str</code>	3956
<code>\l__spintent_spcircfig_op_tl</code>	5555	<code>\l__spintent_spcirc_angle_prefix_tl</code>	3956
<code>__spintent_spcircfig_process:nn</code>	5680	<code>\l__spintent_spcirc_marc_mode_str</code>	4241
<code>\l__spintent_spcircfig_short_bool</code>	5555	<code>__spintent_spcirc_process:nnnn</code>	2115
<code>__spintent_spcircfig_sym:n</code>	5560	<code>\l__spintent_spcirc_mcd_prefix_tl</code>	2010
<code>__spintent_spcircfig_word_and_-</code>		<code>\l__spintent_spcirc_mcd_print_upper_bool</code>	2010
<code> article:n</code>	5566	<code>\l__spintent_spcirc_mcd_read_terse_bool</code>	2010
<code>__spintent_spcoord_exec:nn</code>	2915	<code>\l__spintent_spcirc_mcd_suffix_tl</code>	2010
<code>__spintent_spcoord_intent_infix:</code>	2886	<code>\l__spintent_spcirc_mcm_prefix_tl</code>	2010
<code>\l__spintent_spcoord_intent_str</code>	2788	<code>\l__spintent_spcirc_mcm_print_result_bool</code>	2010
<code>\l__spintent_spcoord_luaset_error_str</code>	2779	<code>\l__spintent_spcirc_mcm_print_upper_bool</code>	2010
<code>\l__spintent_spcoord_luaset_sep_str</code>	2779	<code>\l__spintent_spcirc_mcm_read_terse_bool</code>	2010
<code>\l__spintent_spcoord_luaset_x_has_frac_-</code>		<code>\l__spintent_spcirc_mcm_spmcd_luaset_error_-</code>	
<code> str</code>	2779	<code> str</code>	2010
<code>\l__spintent_spcoord_luaset_x_sign_str</code>	2779	<code>\l__spintent_spcirc_mcm_suffix_tl</code>	2010
<code>\l__spintent_spcoord_luaset_x_type_str</code>	2779	<code>__spintent_spcirc_money_build_tree:</code>	1021
<code>\l__spintent_spcoord_luaset_x_value_-</code>		<code>__spintent_spcirc_money_check_sub_units:</code>	974
<code> str</code>	2779	<code>\l__spintent_spcirc_money_extracted_curr_str</code>	803
<code>\l__spintent_spcoord_luaset_y_has_frac_-</code>		<code>\l__spintent_spcirc_money_extracted_num_tl</code>	803
<code> str</code>	2779	<code>__spintent_spcirc_money_intent_int:</code>	939
<code>\l__spintent_spcoord_luaset_y_sign_str</code>	2779	<code>\l__spintent_spcirc_money_intent_money_str</code>	803
<code>\l__spintent_spcoord_luaset_y_type_str</code>	2779	<code>\l__spintent_spcirc_money_intent_sign_str</code>	803
<code>\l__spintent_spcoord_luaset_y_value_-</code>		<code>\l__spintent_spcirc_money_intent_sub_unit_-</code>	
<code> str</code>	2779	<code> str</code>	803
<code>__spintent_spcoord_mathml:</code>	2898	<code>\l__spintent_spcirc_money_luaset_conde_str</code>	793
<code>__spintent_spcoord_print:n</code>	2863	<code>\l__spintent_spcirc_money_luaset_currency_arg_-</code>	
<code>\l__spintent_spcoord_raw_arg_tl</code>	2788	<code> str</code>	793
<code>__spintent_spcoord_set_wrap_parens_-</code>		<code>\l__spintent_spcirc_money_luaset_plur_str</code>	793
<code> size:</code>	2807	<code>\l__spintent_spcirc_money_luaset_position_-</code>	
<code>__spintent_spcoord_value:n</code>	2854	<code> str</code>	793
<code>\l__spintent_spcoord_wrap_parens_l_tl</code>	2788	<code>\l__spintent_spcirc_money_luaset_print_iso_-</code>	
<code>\l__spintent_spcoord_wrap_parens_r_tl</code>	2788	<code> str</code>	793
<code>\l__spintent_spcdate_luaset_error_str</code>	1459	<code>\l__spintent_spcirc_money_luaset_sing_str</code>	793
<code>\l__spintent_spcdate_luaset_output_str</code>	1459	<code>\l__spintent_spcirc_money_luaset_status_str</code>	793
<code>__spintent_spcdate_process:n</code>	71, 1470	<code>\l__spintent_spcirc_money_luaset_subplur_str</code>	793
<code>\l__spintent_spcdiam_mode_str</code>	4973	<code>\l__spintent_spcirc_money_luaset_subsing_str</code>	793
<code>\l__spintent_spcdiam_sub_index_str</code>	4973	<code>\l__spintent_spcirc_money_luaset_symbol_str</code>	793
<code>\l__spintent_spcdiam_upper_case_bool</code>	4973	<code>__spintent_spcirc_money_normalize_key:n</code>	828
<code>__spintent_spcdiv_process:nnn</code>	1914	<code>__spintent_spcirc_money_parse_and_route:n</code>	1062
<code>__spintent_spcdiv_render_op:nn</code>	1893	<code>__spintent_spcirc_money_print_dec_sub_-</code>	
<code>__spintent_spcdiv_render_parens:nn</code>	1902	<code> unit:</code>	1021
<code>\l__spintent_spcdsym_prefix_tl</code>	1720	<code>__spintent_spcirc_money_print_int:</code>	910
<code>\l__spintent_spcdsym_read_sym_tl</code>	1720	<code>__spintent_spcirc_money_print_sep_con:</code>	1021
<code>\l__spintent_spcdsym_suffix_tl</code>	1720	<code>__spintent_spcirc_money_print_symbol:</code>	876
<code>\l__spintent_spcdsym_symbol_tl</code>	1720	<code>__spintent_spcirc_money_render_layout:</code>	889
<code>__spintent_spcfig_build_concept_-</code>		<code>__spintent_spcirc_money_render_number_node:</code>	901
<code> vertices:n</code>	4820	<code>__spintent_spcirc_money_render_only_integer_-</code>	
		<code> node:</code>	901

__spintent_spmoney_res_main_voice_str	803	__spintent_spnun_print_number_core: ..	446
__spintent_spmoney_set_and_print_sub-		__spintent_spnun_print_part_dec:	425
units:	974	__spintent_spnun_print_part_period: ..	425
__spintent_spmoney_set_currency:n	828	__spintent_spnun_print_period_bar: ...	354
__spintent_spmoney_sub_units_bool	803	__spintent_spnun_read_period_sep_str	46,
__spintent_spmoney_subnum_int	803	279	
__spintent_spmoney_validate_money: ...	860	__spintent_spnun_read_terse_bool	46, 279
__spintent_spmQ_exec:nnnn	2743	__spintent_spnun_render_dec:	337
__spintent_spmQ_join_word_str	94, 2563	__spintent_spnun_render_decimal_sep: ..	391
__spintent_spmQ_print:nnn	2735	__spintent_spnun_render_int:	337
__spintent_spmQ_print_dfrac_bool	96	__spintent_spnun_render_only_period: ..	35
__spintent_spmQ_print_scale_int:	2681	__spintent_spnun_render_part:n	337
__spintent_spmQ_print_scale_int:n	2690	__spintent_spnun_render_period_sep: ..	354
__spintent_spmQ_print_spnZ_dfrac:nn	2718	__spintent_spnz_process:nn	2537
__spintent_spmQ_set_join_word:	2672	__spintent_spnz_render_parens:	2525
__spintent_spmQ_set_wrap_parens_size:	2617	__spintent_spPcirc_mode_str	5469
__spintent_spmQ_start_wrap_parens: ...	2652	__spintent_spPcirc_prefix_tl	5469
__spintent_spmQ_stop_wrap_parens: ...	2652	__spintent_spPcirc_print_sym_str	5469
__spintent_spmQ_use_spnZ_bool	96	__spintent_spPcirc_radio_sty_str	5469
__spintent_spm_side_mode_str	3747	__spintent_spPcirc_read_arg_tl	5469
__spintent_spm_side_prefix_tl	3747	__spintent_spPcirc_read_sty_str	5469
__spintent_spm_side_read_arg_tl	3747	__spintent_spPcirc_sub_index_str	5469
__spintent_spm_side_read_sty_str	3747	__spintent_spPfig_mode_str	4548
__spintent_spm_sym_prefix_tl	1790	__spintent_spPfig_prefix_tl	4548
__spintent_spm_sym_read_sym_tl	1790	__spintent_spPfig_print_sym_str	4548
__spintent_spm_sym_suffix_tl	1790	__spintent_spPfig_read_arg_tl	4548
__spintent_spm_sym_symbol_tl	1790	__spintent_spPoint_exec:nnn	3119
__spintent_spmul_process:nnn	1985	__spintent_spPoint_intent_arg_tl	3033
__spintent_spmul_render_op:nn	1964	__spintent_spPoint_mathml:n	3110
__spintent_spmul_render_parens:nn	1973	__spintent_spPoint_mode_str	3033
__spintent_spnD_luaset_is_truncated-		__spintent_spPoint_prefix_tl	3033
str	2201	__spintent_spPoint_print:n	3075
__spintent_spnD_luaset_result_tl	2201	__spintent_spPoly_build_concept:	4468
__spintent_spnD_prefix_tl	2201	__spintent_spPoly_build_refs:	4455
__spintent_spnD_print_result_num_tl	2201	__spintent_spPoly_exec:nn	4531
__spintent_spnM_luaset_is_truncated-		__spintent_spPoly_intent_arg_tl	4443
str	2201	__spintent_spPoly_mathml:	4511
__spintent_spnM_luaset_result_tl	2201	__spintent_spPoly_mode_str	4400
__spintent_spnM_prefix_tl	2201	__spintent_spPoly_points_loop:	4489
__spintent_spnM_print_result_num_tl	2201	__spintent_spPoly_prefix_tl	4400
__spintent_spnM_spnD_exec:nnnn	2296	__spintent_spPoly_process:n	4501
__spintent_spnM_spnD_intent:nn	2248	__spintent_spPoly_pt_idx_int	4443
__spintent_spnM_spnD_luaset_error-		__spintent_spPoly_pt_seq	4443
str	2201	__spintent_spPoly_refs_tl	4443
__spintent_spnM_spnD_process:nnnnn	2243	__spintent_spPoly_sym:	4447
__spintent_spnM_spnD_result:nn	2264	__spintent_sprad_mode_str	4973
__spintent_spnM_spnD_silent:nn	2256	__spintent_sprad_sub_index_str	4973
__spintent_spnmd_luaset_is_number-		__spintent_sprad_upper_case_bool	4973
str	2201	__spintent_sprayo_exec:nn	3731
__spintent_spnQ_intent_read_sign_str	2563	__spintent_sprayo_intent_arg_tl	3671
__spintent_spnQ_read_parens_bool	94	__spintent_sprayo_mathml:	3684
__spintent_spnQ_wrap_parens_l_tl	93, 94,	__spintent_sprayo_mode_str	3617
2563		__spintent_sprayo_prefix_tl	3617
__spintent_spnQ_wrap_parens_r_tl	93, 94,	__spintent_sprayo_print:n	3675
2563		__spintent_sprayo_pt_idx_int	3671
__spintent_spnun__notation_sym_tl	46	__spintent_sprayo_pt_seq	3671
__spintent_spnun_intent_read_dec_sep-		__spintent_sprayo_read_sty_str	3617
str	46, 48, 279	__spintent_sprayo_refs_tl	3671
__spintent_spnun_intent_read_sign_str	46,	__spintent_sprazon_exec:nnn	2383
279		__spintent_spread_exec:nn	1663
__spintent_spnun_notation_sym_tl	279	__spintent_spread_luaset_reading_str	1645
__spintent_spnun_print_integer:	446		
__spintent_spnun_print_num_start: ...	446		

\l__spintent_spread_luaset_status_str	1645	\l__spintent_spunit_exp_seq	540
\l__spintent_spread_read_arg_tl	1645	\l__spintent_spunit_exponent_tl	540
__spintent_sprecta_exec:nn	3601	__spintent_spunit_format_canonical:	603
\l__spintent_sprecta_intent_arg_tl	3393	\l__spintent_spunit_is_mult_bool	540
\l__spintent_sprecta_kv_tl	3393	\l__spintent_spunit_luaset_canonical_-	
__spintent_sprecta_mathml:n	3447	str	533
\l__spintent_sprecta_mode_str	3393	\l__spintent_spunit_luaset_compact_exp_-	
\l__spintent_sprecta_prefix_tl	3393	str	533
__spintent_sprecta_print:n	3438	\l__spintent_spunit_luaset_is_compact_-	
\l__spintent_sprecta_pt_idx_int	3393	str	533
\l__spintent_sprecta_pt_seq	3393	\l__spintent_spunit_luaset_is_-	
\l__spintent_sprecta_refs_tl	3393	sexagesimal_str	533
__spintent_spsshort_execute_layout:	1170	\l__spintent_spunit_luaset_lookup_status_-	
\l__spintent_spsshort_luaset_base_str	1087	str	533
\l__spintent_spsshort_luaset_expanded_-		\l__spintent_spunit_luaset_read_str	533
str	1087	\l__spintent_spunit_luaset_status_str	533
\l__spintent_spsshort_luaset_layout_str	1087	\l__spintent_spunit_mult_seq	540
\l__spintent_spsshort_luaset_output_str	1087	__spintent_spunit_process_base_exp:n	611
\l__spintent_spsshort_luaset_status_str	1087	__spintent_spunit_process_mult:n	663
\l__spintent_spsshort_luaset_suffix_str	1087	\l__spintent_spunit_read_cdot_bool	540
__spintent_spsshort_print:nn	1123	\l__spintent_spunit_read_slash_str	540
__spintent_spsshort_process_and_-		__spintent_spunit_safe_exec:n	682
render:n	1135	\l__spintent_spunit_unit_name_tl	540
__spintent_spsshort_render_bypass:n	1146	\l__spintent_spusep_arg_tl	71, 72, 1506
__spintent_spsshort_render_error:n	1146	__spintent_spusep_convert:n	1568
__spintent_spsshort_render_fallback:	1146	\l__spintent_spusep_left_tl	71, 72, 1506
__spintent_spsshort_render_success:	1146	\l__spintent_spusep_right_tl	71, 72, 1506
__spintent_spsshort_tag_span:nn	1096	__spintent_spusep_set_parens:nn	1557
__spintent_spside_exec:nnn	3929	\l__spintent_spusep_silent_bool	72
\l__spintent_spside_intent_arg_tl	3821	__spintent_spmvec_exec:nnn	3357
\l__spintent_spside_intent_tl	3821	\l__spintent_spmvec_intent_arg_tl	3154
__spintent_spside_mathml:n	3854	\l__spintent_spmvec_kv_tl	3154
\l__spintent_spside_mode_str	3747	__spintent_spmvec_mathml:n	3216
__spintent_spside_points_loop:n	3917	\l__spintent_spmvec_mode_str	3154
\l__spintent_spside_prefix_tl	3747	\l__spintent_spmvec_prefix_tl	3154
__spintent_spside_process:nn	3826	__spintent_spmvec_print:n	3199
\l__spintent_spside_pt_idx_int	3821	\l__spintent_spmvec_pt_idx_int	3154
\l__spintent_spside_pt_seq	3821	\l__spintent_spmvec_pt_seq	3154
\l__spintent_spside_read_arg_tl	3747	\l__spintent_spmvec_refs_tl	3154
\l__spintent_spside_read_sty_str	3747	__spintent_unknown_keys:n	230
\l__spintent_spside_refs_tl	3821	__spintent_unknown_keys:nn	230
\l__spintent_spsiglo_luaset_arabic_str	1227	spintent.lua	70
\l__spintent_spsiglo_luaset_error_str	1227	\spmangle	22, 120, 4219
\l__spintent_spsiglo_luaset_roman_min_-		spmangle-invalid-arg	3961
str	1227	\spmarc	24, 125, 4378
\l__spintent_spsiglo_print_era_str	1227	spmarc-invalid-points	4246
__spintent_spsiglo_print_math:n	1311	\spmcd	15, 81, 2181
__spintent_spsiglo_print_text:n	1311	\spmcm	16, 81, 2181
__spintent_spsiglo_render_simple:nn	1262	\spmoney	6, 56, 1076
__spintent_spsiglo_render_with_-		\spmQ	17, 92, 2755
era:nnn	1262	spmQ-not-integer	2568
__spintent_spsiglo_tag_span:nn	1298	spmQ-zero-denominator	2568
__spintent_sptime_build_seconds:nn	1391	\spmside	21, 116, 3934
__spintent_sptime_build_tree:	70, 1409	spmside	5199
\l__spintent_sptime_luaset_base_str	1377	spmside-invalid-points	3754
\l__spintent_sptime_luaset_error_str	1377	\spmssym	13, 76, 1843
\l__spintent_sptime_luaset_has_seconds_-		\spmul	13, 79, 2000
str	1377	spmul-empty-argument	1959
\l__spintent_sptime_luaset_is_fraction_-		\spnD	14, 84, 2346
str	1377	\spnewread	11, 1702
\l__spintent_sptime_luaset_seconds_str	1377	spnewread-duplicate	1617

spnewread-empty-args	1617	spside-invalid-points	3754
\spnewunit	6, 55, 761	\spsiglo	10, 65, 1364
spnewunit-duplicate	740	\sptime	11, 68, 1441
\spnM	14, 84, 2346	\spunit	4, 51, 52, 54, 728
\spnum	3, 46, 47, 515	spunit-invalid-unit	547
spnum-has-units	501	spunit-no-units	547
spnum-no-digits	501	\spunitalias	6, 55, 775
\spnZ	16, 91, 2553	spunitalias-duplicate	740
spnz-not-integer	2520	spunitalias-invalid-expr	740
\spPcirc	31, 149, 5771	\spusep	12, 71, 1606
spPcirc-invalid-arg	5480	spusep-empty-arg	1495
\spPfig	26, 132, 4947	spusep-invalid-arg	1495
spPfig-invalid-arg	4555	\spvec	18, 105, 3382
spPfig-invalid-points	4555	spvec-invalid-arg	3161
spPfig-read-arg-key	4555	str commands:	
spPfig-sym-cardinality-conflict	4555	\str_if_eq:nnTF	62
spPfig-sym-letra-conflict	4555	\str_lowercase:n	62
\spPoint	18, 102, 3143	\str_uppercase:n	62
spPoint-invalid-point	3036	sub-index	4981, 5199, 5489
\spPoly	24, 128, 4536	sub-units	842
spPoly-invalid-points	4402	suffix	1724, 1794
\spProp	89, 2434		
\sprad	28, 140, 5161	U	
\sprayo	19, 113, 3736	unknown	1509, 1648, 1724, 1794, 2030, 2210, 2366, 2582, 3044, 3165, 3407, 3627, 3764, 3971, 4256, 4409, 4593, 4981, 5199, 5489
sprayo-invalid-points	3620	unknown-abbreviation	1093
\sprazon	88, 2424	unknown-choice	203
\spread	11, 73, 1692	unknown-command-setup	154
spread-unknown-sym	1617	upper-case	2030, 4981
\sprecta	19, 109, 3606	use-dfrac	2366
sprecta-invalid-points	3400	use-spnZ	2582
\spshort	8, 62, 1214		
\spside	20, 116, 3934	W	
spside	5199	wrap-parens	1868, 1939, 2504, 2582